

# 有界测向误差下干扰源的连续几何规划与清除

## 摘 要

无线电干扰源的快速清除需要在有界测向误差和未知接收条件下，同时完成目标发现、位置收缩与路径调度。本文以集合几何统一表示观测信息，以连续覆盖证书约束发现流程，并将移动、换频、检测与清除纳入同一时间目标。

**针对问题一**，将前向角锥写成半平面交，通过凸多边形顶点求直径和最小包围圆。证明直径不超过 40 米不足以保证单点清除，建立最小包围圆半径不超过 20 米的判据，并推导可供路径优化的清除邻域。

**针对问题二**，利用首次接收提供的半径信息，推导由四个圆盘相交构成的完整保证接收域。通过有理区间认证，得到一次补测最坏后验半径的候选上界 55.739 米、物理量化模型全域下界 55.6528 米，证明完整首扇区的 35 米保证定位域为空；边界裁剪后的区域按更新后的集合重新求解。

**针对问题三**，构建 1124 米七点发现环，结合有限动作规划、集合测向定位与光学后备，实现全向源的发现和清除。默认方法 78 在独立 200 局中全部清除，总均时 3003.37 秒；相对关闭安全提前停车的同一控制器，平均节省 1.485 秒，计时分解表明节省来自移动。三次官方演练全部清除 40 个源，加权均时 218.309 秒/个。

**针对问题四**，以历史全频道扫描点与剩余发现点的联合覆盖为不变量，采用向外舍入区间算术认证 21 点双环及连续点替换，并以自适应光学条带完成终端清除。默认方法 78 在独立 200 局中全部清除，总均时 5733.40 秒；三次官方演练全部清除 42 个源，加权均时 370.688 秒/个。连续点替换候选 84 相对默认方法的平均节省为 28.41 秒，其 95% 区间为  $[-5.92, 62.75]$  秒，按预设规则保留默认配置。

六次官方演练合计清除 82/82 个源；六次正式测试合计清除 74 个，问题三、四加权均时分别为 227.107 和 480.837 秒/个，程序均主动退出、运行时间低于 6 秒。正式案例不公开源总数，清除比例以平台判定为准。另以每间 200 局冻结新样本验证源数与单源均时关系，验证  $R^2$  分别为 0.746 和 0.906。本文形成了从几何判据、控制决策到逐局记录的完整验证链条。

**关键词：**有界误差；主动测向；最小包围圆；剩余覆盖；邻域巡游

# 1 问题分析与统一目标

## 1.1 物理约束及时间结构

目标位置固定在  $\Omega = \{z : \|z\| \leq 1800\}$ ，总数为 10 至 16，每个源独占一个频道。机器人从原点、频道 1 出发，移动速度 5 米/秒。每次换频 1 秒、检测 5 秒、失败光学 3 秒、成功光学与清除共 5 秒。移动时不能检测；在同一地点重复测向不能消除固定误差。真实接收半径在 1000 至 1500 米之间，定向源另有未知闭辐射半平面。合法源距可能小于 1000 米，因此接收半径下界并不等于源距下界。

用  $L, N_s, N_m, N_f, N_c$  分别表示移动距离、换频数、检测数、失败清除数和成功清除数，任务总时间严格为

$$T = L/5 + N_s + 5N_m + 3N_f + 5N_c. \quad (1)$$

研究目标是在全部清除约束下减小  $T$ 。正式题设另要求报告  $T/N_c$ ，本文同时提供该指标，比较策略时坚持同种子配对的**总时间均值**。先对每局计算  $T/N_c$  再取平均，不等于总体总时间除以总体源数。

## 1.2 文献的可迁移内容

主动测向研究强调测点的信息价值与运动成本。Kaya[1] 采用 Fisher 信息目标，Vander Hook 等 [2] 在高斯估计框架下综合移动与测量代价；本文吸收其主动观测思路，以有界角锥交和最坏包围半径对应本题固定误差与清除阈值。

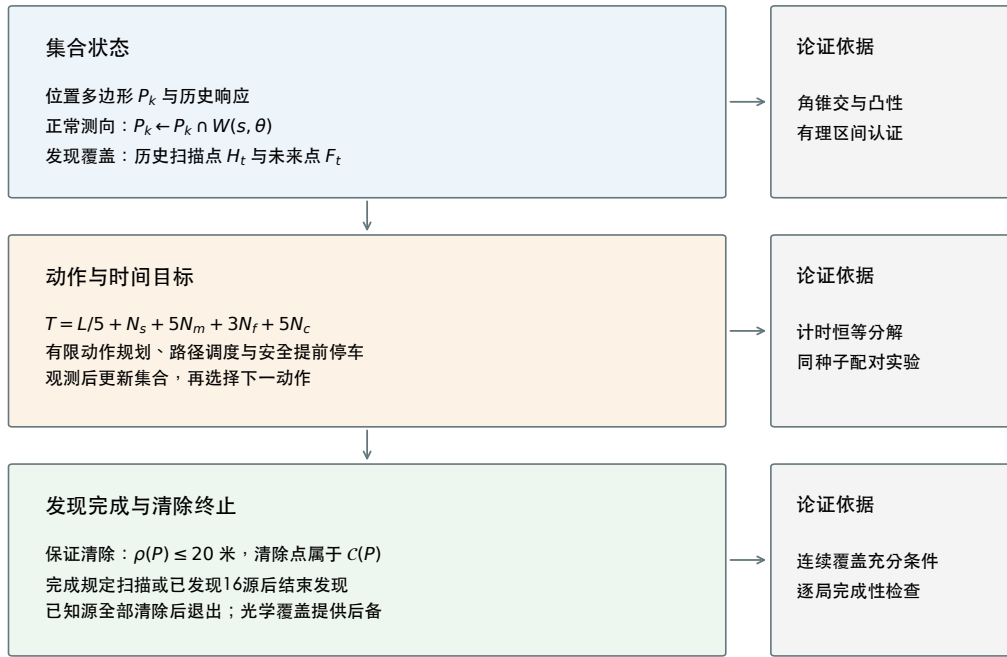
Bayram 等 [3] 联合设计精度约束下的测点与巡游，Dumitrescu–Mitchell[4] 及 Peng 等 [5] 提供邻域巡游的建模基础。本文据此将清除位置表示为可行邻域，再由本题接收与覆盖条件确定其边界。

Hoffmann 等 [6] 通过策略续跑评价当前动作，本文将其用于单源控制器的候选筛选。连续观测规划中的状态表示还需控制粒子退化，Sunberg–Kochenderfer[7] 提出 POMCPOW 与 PFT-DPW 处理这一问题；本文使用有限动作规划和集合证书，未实现完整 POMDP 搜索树。

## 1.3 系统总览、实现边界与方法编号

四问共用一套“集合估计–主动观测–邻域访问–安全终止”闭环：不确定量一律用集合表示（角锥交、圆盘交、剩余覆盖集），决策只依赖集合与已用代价，退出条件由连续覆盖证书给出。图1给出该闭环的状态、动作、终止条件与验证指标总览，后文各节可据此定位。

本文对每一项结论都区分三个层级，凡跨层使用均在原处标明，避免把工程实现当成数学证明，或把有限实验当成连续命题。表1汇总三层的含义、成



**图 1:** 系统总览。三行自上而下依次为集合状态、允许动作及其代价、终止条件，每行右侧给出该层对应的证据类型。问题三以七点环处理全向源发现，问题四以 21 点双环及剩余覆盖条件处理未知朝向。

立范围与本文对应要求：理论保证层给出可证明命题，浮点实现层只保证程序行为，模拟验证层只说明这批场景下如此。

表 1: 三层证据边界：理论保证、浮点实现与模拟验证

| 层级   | 含义与成立范围                                                        | 本文中的对应要求                                        |
|------|----------------------------------------------------------------|-------------------------------------------------|
| 理论保证 | 在题设精确语义（实数坐标、闭半平面朝向、 $\pm 1^\circ$ 区间误差）下的可证明命题，用有理区间算术或凸性论证支撑 | 问题二全域下界与“35 米保证域为空”、问题三七点环覆盖界、问题四剩余覆盖不变量与固定点数下界 |
| 浮点实现 | 在线程序用 <code>double</code> 加容差与预算运行；除明确标注“向外舍入认证”的模块外，不构成形式化证明  | 多边形裁剪、最小包围圆、角区间二分、局部候选搜索                        |
| 模拟验证 | 在公开协议内核、压力场景与官方模拟器上的有限次观测；只说明“这批场景下如此”，不作因果或全称结论               | 开发 / 独立 / 压力 / 冻结新样本四层配对，以及六次演练与六次正式测试记录        |

为便于复核，表2列出全文出现的全部方法编号及其配置与继承关系。编号只是配置索引，不按优劣排序；默认推荐为方法 78，实验候选为方法 84。

## 2 集合状态及问题一的求解

### 2.1 前向角锥与多边形交

设检测点  $s$ 、示向度  $\theta$ ，令  $u = (\cos \theta, \sin \theta)$ ， $v = (-\sin \theta, \cos \theta)$ 。半角为  $\bar{\varepsilon}$  的前向角锥为

$$W(s, \theta) = \{z : |v^\top(z - s)| \leq \tan \bar{\varepsilon} u^\top(z - s)\}. \quad (2)$$

两个线性不等式还隐含  $u^\top(z - s) \geq 0$ ，因此必须保留射线的前向性。理论物理误差为  $\pm 1^\circ$ ；本地公开协议模拟器保留 0.01 度读数，在线采用  $\bar{\varepsilon} = 1.005^\circ$  包含四舍五入误差。若实际接口不量化，使用这一角度仍为保守外包。

每个频道保存正常示向交形成的凸多边形  $P_k$  及全部历史响应。初始化包含  $\Omega$  的外切多边形；每次正常示向与  $W(s, \theta)$ 、接收半径 1500 米的外切多边形相交，并加轴向上界。无信号不直接裁掉定向源的位置多边形，其位置与朝向耦合保留在历史状态和剩余覆盖逻辑中。有限假设用于排序，不能作为全清除的集合证据。

若仅给出半平面、不额外给出场地边界，先判断空集、无界集或有界集。非空无界集直径为无穷大；空集提示观测不一致。退化点和线段分别按零直径和端点距处理。有界多边形可以用逐半平面裁剪得到，本文小规模实现扫

表 2: 方法配置与继承关系

| 编号    | 改动        | 说明                                                                                                            |
|-------|-----------|---------------------------------------------------------------------------------------------------------------|
| 56    | 基线        | 联合控制，关闭安全提前停车，作为基础对照；新增模块另与默认方法 78 比较                                                                         |
| 67    | 连续测点候选细化  | 继承 56，启用 1 档局部细化，总评价预算 80 次                                                                                   |
| 68    | 细化与假设加密   | 继承 67，将 Q3 规划假设上限由 32 增至 63                                                                                   |
| 69    | 连续测点候选细化  | 继承 56，启用 2 档局部细化，总评价预算 160 次                                                                                  |
| 70-73 | 测向分组档位    | 基线测向结果的分组宽度由 2 度改为 4 / 3 / 6 / 8 度 (4 / 6 / 8 比 2 度更粗，用于检验分组的必要性)                                             |
| 74-77 | 光学组合      | Q4 轮换使用不同数量的光学清除子块组合                                                                                          |
| 78    | 安全提前停车    | 仅把已验证可成功的清除动作沿当前路段提前；Q3 与 Q4 的推荐默认                                                                            |
| 79    | 最坏半径与行走门槛 | 启用最坏剩余半径目标与式 (14) 的启发式门槛                                                                                      |
| 80    | 最近保证第二测   | 启用最近保证域搜索选择第二测点                                                                                               |
| 81    | 连续发现点替换   | Q4 按剩余覆盖证书把发现点移向更近的可行位置                                                                                       |
| 82    | 单源续跑筛选    | 把决策树叶端估价替换为“同一单源控制器续跑至实际清除”的筛选（源码记作 <code>robust_mode=3</code> ，在规划入口以 0 调用后由 <code>rollout_choice</code> 接管） |
| 83    | 邻域巡游      | 对已满足 $\rho \leq 20$ 的源联合调整清除位置                                                                                |
| 84    | 冻结候选组合    | Q4 = 方法 81 + 安全提前停车；Q3 与方法 78 逐局相同                                                                            |
| 85    | 连续替换及机会删点 | 在 81 基础上尝试删除一个未来发现点（开发集未成功）                                                                                   |
| 86    | 可变覆盖及邻域巡游 | 83 的可变宽度条带覆盖版本                                                                                                |

描全部顶点对，计算

$$D(P) = \max_{i,j} \|p_i - p_j\|. \quad (3)$$

凸函数在多面体上的最大值必在极点取得，故枚举顶点对即已覆盖区域内任意点对的距离最大值。该实现耗时  $O(m^2)$ ；当顶点规模增大时，对已按边界排序的凸多边形可用旋转卡壳法在  $O(m)$  内求解；若输入为无序点集，另需  $O(m \log m)$  的凸包预处理。

## 2.2 直径不足以判断单点清除

定义

$$\rho(P) = \min_c \max_{z \in P} \|z - c\|. \quad (4)$$

等边三角形边长 40 米时， $D = 40$  米而  $\rho = 40/\sqrt{3} \approx 23.094$  米。因此半径  $D/2$  的圆未必能覆盖定位区域，不能以  $D \leq 40$  替代  $\rho \leq 20$ 。最小包围圆由至多三个顶点支持：一个点、两个点的中点圆，或三个点的外接圆。通用几何库计算圆心后再次取全部顶点距离的最大值，返回的是普通浮点估计，并不构成严格向外舍入半径。本文所处理的定位区域  $P$  均为有界多边形，记  $V(P)$  为其顶点集合， $\overline{B}(c, r) = \{z : \|z - c\| \leq r\}$  为闭圆盘。连续候选模块另加数值余量，保证清除检查使用阈值裕量；这些工程处理不包含全部坐标裁剪误差的形式证明。凡采用精确有理数认证之处，均在相应位置单独标明。

保证清除区域实际上是

$$\mathcal{C}(P) = \bigcap_{p_i \in V(P)} \overline{B}(p_i, 20), \quad (5)$$

其非空当且仅当  $\rho(P) \leq 20$ 。若最小包围圆为  $(c, r)$ ，则  $\overline{B}(c, 20 - r) \subseteq \mathcal{C}(P)$ 。这给出可供路径优化的邻域，而不是必须访问的唯一圆心。

## 3 问题二的保证接收域与第二测的能力边界

### 3.1 四圆盘完整接收域

把首检测点和示向方向分别归一化为原点、 $x$  轴，源位置写成  $z = ru_\phi$ ，其中  $5 < r \leq 1500$ 、 $|\phi| \leq \bar{\varepsilon}$ 。首次收到信号意味着其未知真实接收半径满足  $R_s \geq \max(1000, r)$ 。第二测点  $q$  必能收到的充要条件是

$$\|q - ru_\phi\| \leq \max(1000, r), \quad 5 < r \leq 1500, |\phi| \leq \bar{\varepsilon}. \quad (6)$$

对  $r \leq 1000$ ，距离平方关于  $r$  凸，只需检查闭包端点 5、1000。对  $r \geq 1000$ ，约束化为  $\|q\|^2 \leq 2rq^\top u_\phi$ ，在  $r = 1000$  成立后随  $r$  增大不再加严。由角端点

取最坏方向，得到

$$G = \bigcap_{r \in \{5, 1000\}, \sigma \in \{-1, 1\}} \overline{B}(r(\cos \bar{\varepsilon}, \sigma \sin \bar{\varepsilon}), 1000). \quad (7)$$

令  $q = (a, b)$ 、 $d = a \cos \bar{\varepsilon} - |b| \sin \bar{\varepsilon}$ ，等价代数式为

$$a^2 + b^2 \leq 2000d, \quad a^2 + b^2 \leq 999975 + 10d. \quad (8)$$

此前经验点  $(750, \pm 300)$  具有良好接收裕量；它不是最优定位点。完整扇区的已证候选为  $(843.1178, \pm 545.4003)$ ，复核精确有理数脚本后得到表3。上界用

**表 3:** 完整首扇区的第二测最坏半径证书，单位米

| 模型             | 全域下界     | 候选上界   | 差距上界    |
|----------------|----------|--------|---------|
| 连续保守半角 1.005 度 | 55.73823 | 55.739 | 0.00077 |
| 物理误差及 0.01 度量化 | 55.6528  | 55.739 | 0.0862  |

第二读数区间的加宽角锥外包，并对外包顶点精确核验包围圆。下界分别用近共线区固定双点见证和其余区域的远端不可区分双点见证覆盖完整  $G$ 。相关三角函数以有理区间界计算，接受判断不依赖浮点搜索结果。脚本与输出随支撑材料给出。

全域下界的双点构造如下。由对称性只考虑  $q = (a, b)$ 、 $b \geq 0$ 。当  $b \leq 30$  时，首轴上的  $(1340, 0)$  与  $(1500, 0)$  可产生相同的两次正常示向读数，给出 80 米半径下界。其余区域取  $A = 1500u_{-e_1}$ ，从  $q$  指向  $A$  的射线顺时针转  $2e_2$  后与首扇区上边界相交于  $B = tu_{e_1}$ 。记  $k = \sin(e_1 + 2e_2)$ 、 $h = \cos(e_1 + 2e_2)$ 、 $s = \sin(2e_2)$ 、 $l = \sin(2e_1 + 2e_2)$ ，则

$$t = \frac{1500(ka + hb) - s(a^2 + b^2)}{1500l - ka + hb}. \quad (9)$$

证书先核验分母正、 $t > 5$ 、两次读数均不会触发近距离响应，再将  $t \leq T_e$  化成凹二次式在包含  $G$  的 1000 米圆盘上的最大值非正。两模型的  $T_e$  分别为 1400.796371、1400.99，故不可区分双点的半距离至少为

$$\frac{1}{2} \sqrt{1500^2 + T^2 - 3000T \cos(2e_1)}. \quad (10)$$

取  $e_1 = 1.00499999^\circ$ ，连续模型的  $e_2 = 1.00499999^\circ$ 、物理量化模型的  $e_2 = 0.99999999^\circ$ ，保留严格读数余量，得到表中的两项下界。构造和区间余量均由有理数脚本逐项检查，而非只抽样验证  $q$ 。

### 3.2 最近保证定位区域可以为空

欲以最近点代替固定坐标，可求

$$\min_{q \in G(P)} \|q - x\|, \quad R(P, q) := \sup_{\theta \in \Theta(P, q)} \rho(P \cap W(q, \theta)) \leq \tau. \quad (11)$$

阈值取 20 米时保证下一次正常示向后可清除；取 35 米时只允许尝试清除，不能保证成功。表3表明，完整首扇区在这两个阈值下均无可行点。边界裁剪或已有更多测向可能使  $P$  变小，此时应重新求域；不能把完整域下界搬到裁剪域，也不能假定保证区域永远非空。

在线局部搜索使用更强的充分接收条件  $\max_{z \in P} \|q - z\| \leq 950.7$  米，以保持原经验接收余量。它并非式(7)给出的完整保证接收域。最近保证域实验中，检查当前多边形的 35 米阈值时未找到被门槛采纳的替换动作；这一结果仅说明本次搜索未触发，不证明每个裁剪实例的完整域都为空。

## 4 可独立复现的联合控制流程与实现边界

方法编号只是配置索引，索引见表2。默认策略由“有限深度动作规划、发现与定位联合调度、有限光学后备、安全提前停车”组成，比较基线为关闭提前停车的同一控制器。表4列出决定其行为的参数。

表 4: 默认控制器的主要参数与终止门槛

| 参数            | Q3                            | Q4         |
|---------------|-------------------------------|------------|
| 发现点           | 原点及 1124 米六等分环<br>(六点加原点，共七点) | 认证 21 点双环  |
| 单源决策深度/节点预算   | 2 层/6000                      | 2 层/6000   |
| 状态假设上限/局部动作上限 | 32/6                          | 63/6       |
| 机会补测上限        | 每源 4 次                        | 关闭         |
| 基线补测偏移        | 首示向坐标 (750, $\pm 300$ )       | 动作菜单中保留同坐标 |
| 主动阶段后备        | 至多 3 轮中心试探后光学                 | 直接自适应条带光学  |
| 尝试清除/保证清除     | 35 米/20 米                     | 按规划动作/20 米 |
| 路由精确解阈值       | 节点数不超过 12                     | 节点数不超过 12  |

1. 初始化 20 个频道的未知状态与发现点集。从当前频道开始循环扫描未知频道；正常示向建立保守多边形，近距响应立即在原地清除。成功后永久退出该频道。
2. 对每个已发现未清除源，从与全部历史响应一致的有限位置、半径和朝向假设构造动作表。测点包括当前位置、包围圆心、首示向偏移点、长轴与横轴偏移点；清除动作包括圆心和长轴上的试探点，Q3 还加入保证清除内域。排除历史同地点同类动作。

3. 在深度 2 的有限状态树上比较“行走、换频、测量、成功或失败清除”总秒数。节点上限 6000，叶端是有限假设的 28 米蛇形期望代价；它仅作基线排序，不提供覆盖或最优性证明。基线测向结果仍按 2 度分组，本文不对这项旧代理进行连续加密。任何时候真实集合均用完整角锥更新。
4. 把未访问发现点与各已知源的下一动作同时放入开放巡游问题。Q3 允许同一源的若干候选服务入口，Q4 使用选中的单个动作点。节点不超过 12 时用子集动态规划，否则多起点最近邻、2-opt 和重定位，辅以最小生成树及指派松弛下界；下界只约束这一次路由代理。每次仅执行首个任务，响应后重建全部节点。
5. 每源最多执行 6 次局部动作。Q3 随后允许保证域中的第二测及至多 3 轮中心附近试探，35 米半径时可以试清，失败继续；Q4 转入自适应条带覆盖。光学把区域切成可被半径 20 米圆覆盖的有限子块，按有限假设命中时间选取巡游顺序；逐点发出清除命令，直到成功。
6. Q3 在已停车点可为其它已知源补测：每源最多 4 次，要求离估计圆心不超过 1000 米、相对首示向横向间距至少 50 米且未重复。它只增加信息，不参与全清除证明。发现数达到 16 后可跳过剩余发现点；否则完成证书要求的未知频道扫描，再清除所有已知源才退出。

方法 78 在该控制器外包一层安全提前停车适配器：仅把已验证可成功的清除动作沿当前路段提前，控制器仍保留原路标，所有测向位置保持不变。实验候选 84 在 Q3 与 78 相同，在 Q4 才启用后文的连续发现点证书替换。局部动作和机会补测数都有上限，光学子块数有限；在理想几何、接口按题设响应且无传输故障时，全发现证书与逐块清除共同给出有限完成性。程序异常或超时会报告失败，不把它当成全清除。

## 5 问题三的最坏剩余半径规划

### 5.1 对全部读数取最坏而非仅枚举无噪声顶点

若以  $z \in V(P)$  的真实方向直接切一次  $\pm 1^\circ$  角锥，既遗漏测量误差偏移，也没有证明极值只在这些方向出现。本文覆盖全部可能示向区间  $\Theta(P, q)$ ：当  $q$  在  $P$  外时，用顶点方位的最大圆周空隙确定可见角区间，再向两端扩张  $\bar{\varepsilon}$ ；当  $q \in P$  时使用整个圆周。

把读数区间分成  $[\theta_j - \delta_j, \theta_j + \delta_j]$ 。只要  $\bar{\varepsilon} + \delta_j < 90^\circ$ ，有

$$\bigcup_{|\theta - \theta_j| \leq \delta_j} W_{\bar{\varepsilon}}(q, \theta) \subseteq W_{\bar{\varepsilon} + \delta_j}(q, \theta_j). \quad (12)$$

故

$$R(P, q) \leq \max_j \rho(P \cap W_{\bar{\varepsilon} + \delta_j}(q, \theta_j)). \quad (13)$$

优先二分当前半径上界最大的区间，直到预算或上下界间距阈值满足。近距响应的真实不确定度至多 5 米；将这些几何方向一并纳入角锥外包只会更保守。在线半平面与圆运算仍采用浮点加容差；角区间的连续覆盖论证不等于整个浮点几何库已获形式化验证。离散检查（25005 个密集读数）只能作为下界方向的旁证，连续覆盖性由式 (12) 的角区间外包保证，二者不可互相替代；关键理论阈值另由问题二的有理数证书复核。

候选点用确定性单纯形局部搜索，接收域由凸盘交投影构造。新候选仅保留一个；不使用旧 2 度粒子分箱目标，也不再进行八方向加密。半径目标的非凸性意味着这是局部候选搜索，而不是测点全域最优证书。

## 5.2 把支配门槛与时间保证分开

设原动作后半径上界为  $r_0$ ，原动作点为  $q_0$ ，当前位置为  $x$ ；新点为  $q$ ，其上界为  $r_q$ 。筛选条件为

$$\frac{2(r_0 - r_q)}{5} > \frac{\max(0, \|x - q\| - \|x - q_0\|)}{5} + 5, \quad r_q < 0.85\rho(P). \quad (14)$$

左侧是把直径收缩换算成行走秒数的启发式，右侧包含额外移动和一次检测。需要强调，这不是后续真实总时间的支配定理：半径变小可能增加入口代价、改变联合调度，或延后一次本可成功的试清。开发实验的负结果证实仅靠该门槛不足以消除退步。

## 5.3 用当前单源控制器续跑筛选首动作

对通过几何门槛的连续候选与原动作，构造与已观测历史一致的有限场景，包括内部假设、可行边界点和  $\pm 1^\circ$  误差。每个首动作均由同一单源控制器继续运行，直至真实模拟清除或执行实际的光学覆盖函数；没有把 28 米旧蛇形估价作为停止叶端。

续跑时保留当前位置、频道、多边形、历史和已用动作预算，同地点响应复用原值。仅当场景平均剩余时间至少下降 5 秒且最大场景退步不超过 5 秒时才替换。这里的“最坏”只针对列出的有限续跑场景，不是对连续源位置的时间证明。它是单源控制器续跑，未模拟其它源的后续联合调度或机会补测，因此不能等同整局策略改进定理。该模块（方法 82）在开发集 Q3 上共执行 34624 次场景续跑，仅采纳 22 次动作替换，配对总时间只改善 1.10 秒；候选 84 未启用该模块，其 Q4 改走连续发现点替换通道。

续跑中的“真实”专指调用同一单源控制器直至实际清除成功，未来观测仍由有限假设生成：每个场景固定取  $+1$  或  $-1$  度误差，不执行 0.01 度量化。它不复用评测内核完整的位置相关误差场；同位置历史读数则重放保存值。因此有限场景的门槛不能升级为连续误差场下的鲁棒保证，方法 82 没有推广。

## 5.4 全向发现与有限清除机制

七点集由原点和半径 1124 米的六等分环构成。原点覆盖半径 999.6 米圆域；其余源的方位距离最近环点角差至多 30 度。距离平方

$$d^2(r) = r^2 + 1124^2 - 2r \cdot 1124 \cos 30^\circ \quad (15)$$

关于  $r$  凸，在  $[999.6, 1800]$  端点检查即可证明小于  $999.6^2$ 。有理区间核验  $\sqrt{3}$  和实际二进制坐标，保证真实最小接收半径 1000 米下覆盖整个场地。问题三的干扰源全部为全向源（题设给定），接收只取决于距离，故上述覆盖结论不需要附加朝向条件；问题四含定向源，必须按 §6.1 另行处理，两者不可互相套用。

每个正常发现源最多采用有限次局部试探；Q3 随后进入原单源定位后备，Q4 直接进入有限光学扫掠。每一光学子区域都有 20 米覆盖证据，因此逐个访问必有一个成功。若已发现 16 个源，可停止发现并处理已知剩余源；否则必须完成覆盖证书规定的各频道扫描。对 10 至 15 个源的局，不得以“已出现大量空频道”为由提前终止发现阶段，因为未扫描频道中仍可能存在源。

## 6 问题四的负信息与剩余覆盖

### 6.1 未知朝向必须进入可能状态

未发现频道的状态应写为  $\omega = (z, R, h, d)$ ，其中  $d$  表示是否定向， $h$  为单位朝向。令  $C(q)$  表示测点  $q$  可收到的状态集合，则无信号更新为

$$\Omega_k^{t+1} = \Omega_k^t \setminus C(q_t). \quad (16)$$

在不知道  $R$  的情况下，只能保证  $R \geq 1000$ ；定向源的必收状态还需  $h^\top(q-z) \geq 0$ 。单纯在位置平面切一个 1000 米圆盘对 Q4 不成立。

一次正响应也不足以给出不同位置的普遍必收点。固定一个可能源位置  $z$ ，已知  $h^\top(s-z) \geq 0$ ，若要求对所有这样的  $h$  都满足  $h^\top(q-z) \geq 0$ ，则  $q-z$  必须是  $s-z$  的非负倍数。当首角锥包含两个与  $s$  不共线的可能源位置时，两个必要射线的共同测点只有  $s$ 。所以不能不加条件地把 Q3 保证第二测推广到 Q4。多个正测点的凸包给出一个共同必收充分域，但它仍可能很小。

### 6.2 剩余覆盖不变量及证明

记  $H_t$  为截至当前对所有仍未发现频道都已完整扫描的实际点集， $F_t$  为计划保留的未来发现点。对源位置  $z$ ，定义保证距离内点集

$$S_t(z) = \{q \in H_t \cup F_t : \|q - z\| \leq 1000\}. \quad (17)$$

**命题 1.** 若对所有  $z \in \Omega$  有  $z \in \text{conv } S_t(z)$ , 则每个仍未发现频道的任意合法源状态都能在  $F_t$  中至少一个点被收到。

证明. 对任意单位朝向  $h$ , 若所有  $q \in S_t(z)$  都满足  $h^\top(q - z) < 0$ , 则其任何凸组合也具有负投影, 与  $z \in \text{conv } S_t(z)$  矛盾。因此至少一个点在距离 1000 米以内且投影非负, 能接收该源。若这个点属于  $H_t$ , 此前完整频道扫描就会发现它, 与“仍未发现”矛盾, 故必有可接收点在  $F_t$ 。全向源不需要投影条件, 结论同样成立。  $\square$

该命题等价于用连续几何充分条件认证  $\Omega_k^t \subseteq \bigcup_{q \in F_t} C(q)$ , 不需要把四维状态离散成粒子。所有未发现频道共享完整扫描历史; 频道一旦发现, 退出剩余发现证书, 转入已知源定位。假如将来加入只扫部分频道的机会扫描, 必须分别保存  $H_{k,t}$ , 不能套用共享历史。

初始  $F_0$  使用已认证 21 点双环布局。每次欲删除一个点或将其移向当前位置, 都先重新验证命题1; 若验证失败或计算预算耗尽, 则保留原布局, 既不删除也不移动。一次阴性响应只是把某点从未来集合移入历史集合,  $H_t \cup F_t$  并未变化, 因此不会自动让另一个本来必需的静态点变成冗余点。真正的删除需要额外实测位置、已证覆盖冗余或其它合法先验。所以把 21 点减至 15 点在本题局内并非必然可实现的省空间。

### 6.3 连续单元与向外舍入认证

从包围场地的正方形递归二分矩形单元  $B$ 。若单元与场地无交则略去; 否则令

$$I_B = \{q : \max_{v \in V(B)} \|q - v\| \leq 999.999\}. \quad (18)$$

当  $V(B) \subseteq \text{conv } I_B$  时, 距离凸性保证  $I_B$  中的每一点对整个  $B$  都在 1000 米以内, 凸包条件也延伸到整个  $B$ 。未通过则沿长边二分, 最多 30 层、100000 次检查; 未决即拒绝该修改。

每个距离平方及叉积均使用向外舍入区间: 只有距离上界小于阈值、叉积下界非负时才接受。凸包构造用于提出见证, 其每条边及凸性再用区间检查; 它本身的浮点排序不作为证明。实际二进制坐标通过 17 位有效数字传入接口, 保证认证坐标与发送坐标一致。初始布局认证 4552 个叶单元; 一个实际接受的 3.595 米点替换产生 4564 个叶单元, 独立 Python 有理数程序同时核验单元几何及整个场地分区无缺口。

图2给出该认证的空间形态: 左图为整个场地的单元细分, 细分最密处即为距离或凸包边界最紧的区域; 右图为放大后的可行位移范围, 可见内环点的可动区间很窄。

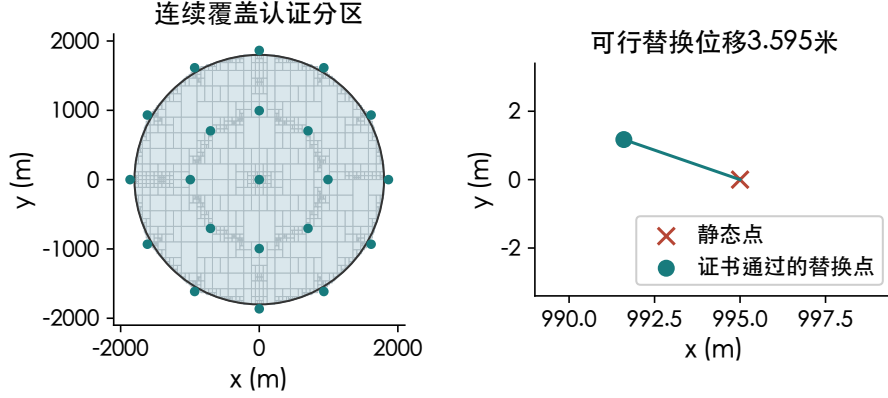


图 2: 固定 21 点及一个连续替换的认证空间单元。细分密度反映距离和凸包边界的紧约束；放大图显示内环点允许移动的范围很小。

#### 6.4 实施范围及固定点数下界

方法 81 在已选中的下一发现点到当前位置的线段上寻找通过证书的较近点。七次二分是有可行性复核的局部搜索，不宣称该约束沿线单调，也不宣称找到了完整可行域的最近点。方法 85 进一步尝试在现有定位位置扫描后删除一个未来点，仍逐次认证；开发集没有成功删除整点，候选 84 关闭这一额外计算。

固定通用布点下界的适用对象是出发前固定、对任意位置和朝向保证接收的有限点集，不包括适应性信息和光学扫描。下面补出面积与边界弧证书之间的分析连接。

**命题 2.** 对半径  $R = 1800$  米场地及最小接收半径  $L = 1000$  米，任意固定通用定向发现点集至少含 15 点。

证明. 设点  $q_i$  到原点距离为  $r_i$ ，其接收圆盘在场地内的面积为  $A_i$ 。全朝向可接收等价于源位置属于其距离  $L$  内测点的凸包；若仅有两个测点，则源必须落在两点连线上。有限条连线的面积为零，因此几乎处处至少有三重圆盘覆盖，积分得  $\sum_i A_i \geq 3\pi R^2$ 。

再考虑位于场地边界且径向向外发射的源。内点  $r_i < R$  不可能接收， $r_i = R$  只可能覆盖零长弧， $r_i > R + L$  完全无效。其余点能覆盖的边界弧半角为

$$\delta_i = \min \left\{ \arccos(R/r_i), \arccos \frac{R^2 + r_i^2 - L^2}{2Rr_i} \right\}. \quad (19)$$

圆周全部覆盖要求  $\sum_i \delta_i \geq \pi$ 。定义面积损失  $d_i = 1 - A_i/(\pi L^2)$ 。对  $r_i \geq R$ ,

两圆交面积为

$$A(r_i) = 2 \int_0^L [\sqrt{R^2 - y^2} + \sqrt{L^2 - y^2} - r_i]_+ dy. \quad (20)$$

被积函数随  $y$  单调下降，1 米步长左矩形和给出面积上界。脚本在径向分段  $[1800, 1900, 1950, 2000, 2025, 2050, 2800]$  上，以有理数平方根上界和余弦泰勒界逐段认证  $d_i \geq K\delta_i$ ，其中  $K = 1.38$ 。内点的  $\delta_i = 0$  且  $d_i \geq 0$ ，外于  $R + L$  的点也满足。因此

$$N = \sum_i \frac{A_i}{\pi L^2} + \sum_i d_i \geq \frac{3R^2}{L^2} + K\pi > 14.0553 > 14, \quad (21)$$

由整数性得到  $N \geq 15$ 。每段最小余量及有理数运算输出均在支撑材料中。□

由命题2，当前关系为  $15 \leq N^* \leq 21$ ，其中  $N^*$  为“出发前固定、对场地内任意位置与朝向都保证接收”的最少通用点数。该区间既不证明 21 点最优，也不证明动态方案可省 6 次扫描。

## 7 光学覆盖和邻域巡游的严格转换

Fekete 等 [8] 的割草模型要求连续路径上的刀具覆盖区域；本题光学必须显式发出清除命令，不能把沿线经过算成自动光学成功。先将  $P$  分成有限凸子区域  $P_i$ ，再为每一块选择清除位置，才能保留覆盖保证。

若  $P_i \subseteq \overline{B}(c_i, r_i)$  且  $r_i \leq 20$ ，可行清除点是式(5)给出的  $\mathcal{C}(P_i)$ ，其内含  $\overline{B}(c_i, 20 - r_i)$ 。不能仅要求路径碰到  $\overline{B}(c_i, 20)$ ：例如源在该圆右端、机器人在左端时，二者相距 40 米，仍无法完成清除。本文对每块全部顶点执行 20 米距离复核，交替优化访问坐标与 2-opt 顺序，并在整条扫掠长度和有限假设平均发现时间都改善时接受。

原条带覆盖、可变条带覆盖和连续邻域巡游分别保留为方法 83、86 的消融。对多个已满足  $\rho \leq 20$  的源，实验实现沿全局访问顺序联合调整清除位置；该实现并未求解所有排列上的全局 CETSP。某一固定扫掠全长缩短，也不保证每个未知源的首次命中时间缩短。开发集 Q4 仍出现净退步，因而没有启用这些模块。

推荐默认方法 78 保留的安全提前停车只修改已经保证成功的清除动作，在原路段上求最早满足  $\mathcal{C}(P)$  的停靠位置，未来测向路标保持不变。其路程不变量为  $L + \|x - a\| \leq L_0$ ，其中  $a$  为原计划路标、 $x$  为真实位置。由线段关系与三角不等式可归纳证明。该机制本身不改变路标集合，因此本文不把它计为结构性规划收益。

## 8 配对实验及策略选择

### 8.1 开发与独立验证严格隔离

开发使用种子 2110001 至 2110040，每问 40 局。历史 CSV 在本机同种子标签下未能逐行重现，因此本文重新编译原始未修改方法 56，并与新代码重跑的 80 行开发基线逐行核对一致；所有比较均使用同一编译器和标准库。历史 CSV 不参与配对或合并，不能据种子标签相同声称跨平台场景相同。冻结后使用 2610001 至 2610200，每问 200 局，比较方法 56、既有默认 78 和候选 84。压力五组分别为边界外向、共线端点、近重合、相关角误差、分区端点误差，各 40 局/问，起始种子 2620001 至 2660001，步长 10000。独立集不用于继续调整参数。

本地各局独立生成 10 至 16 个源及不重复频道；局内位置在圆内按面积均匀提议，但拒绝任意两源距离小于 40 米的位置，故位置间不独立。接收半径在 1000 至 1500 米、朝向在整圆均匀提议；Q4 定向源数在 1 至源数减 1 之间生成。默认误差由厘米舍入位置和种子的散列映射到  $[-1,1]$  度，再把读数量化至 0.01 度；同位置误差固定。统计均值仅适用于这一公开的本地场景分布。不同方法可走不同位置，配对共享环境和误差场，而非强行共享一串独立测量噪声。压力组合用于暴露失效结构，其分布与 IID 不同，不与 IID 合并估计总体均值。

表 5: 开发 40 局每问的配对总时间，正值表示节省；未混入独立集

| 方法 | 变化        | Q3 节省时间/s | Q4 节省时间/s |
|----|-----------|-----------|-----------|
| 79 | 最坏半径与行走门槛 | -165.79   | 0.00      |
| 80 | 最近保证第二测   | 0.00      | 0.00      |
| 81 | 连续发现点替换   | 0.00      | 31.83     |
| 82 | 几何门槛及单源续跑 | 1.10      | 0.00      |
| 83 | 原覆盖的邻域巡游  | 1.72      | -5.21     |
| 85 | 连续替换及机会删点 | 0.00      | 31.83     |
| 86 | 可变覆盖及邻域巡游 | 1.72      | -4.09     |
| 84 | 冻结候选组合    | 1.91      | 32.67     |

表5给出各几何模块在开发集上的配对结果。方法 79 的 Q3 表现为 7 局更快、33 局更慢，明显偏向不利；因跨平台历史样本不可逐行对齐，不把这一胜负结构与旧 67-69 作严格同场景比较。不能据半径目标更贴物理，就断言时间必然改善。方法 82 用真实单源续跑后将平均退步消除，但仅节省 1.10 秒；Q3 最后选择方法 78 的安全执行优化。Q4 开发平均节省约 32.67 秒的候选 84 由连续发现点替换和提前停车组成，其它模块关闭。方法 81 与 85 的动作结果相同，后者仅增加无效认证计算。

## 8.2 独立同种子配对结果

表6按两组对照分栏给出独立 200 局的配对结果：(a) 栏以方法 56 为基线，(b) 栏以既有默认方法 78 为基线，(c) 栏为运行开销与完成性。对每局配对

**表 6:** 独立 200 局每局的配对结果。上半部分为候选 84 与方法 56（基线）的比较，下半部分为与方法 78（既有默认）的比较；时间单位为秒

| 指标                       | Q3          | Q4            |
|--------------------------|-------------|---------------|
| (a) 候选 84 相对方法 56 (基线)   |             |               |
| 方法 56 总均时                | 3004.85     | 5734.60       |
| 默认 78 总均时                | 3003.37     | 5733.40       |
| 默认 78 单源均时               | 235.00      | 452.52        |
| 候选 84 总均时                | 3003.37     | 5704.99       |
| 候选 84 单源均时               | 235.00      | 450.62        |
| 总均时节省                    | 1.48        | 29.62         |
| 节省率/%                    | 0.049       | 0.516         |
| 配对 95% 区间                | [1.21,1.76] | [-4.69,63.92] |
| 更快/持平/更慢                 | 185/15/0    | 119/0/81      |
| 最大单局退步                   | 0.00        | 707.85        |
| (b) 候选 84 相对方法 78 (既有默认) |             |               |
| 84 对 78 新增节省             | 0.00        | 28.41         |
| 新增节省率/%                  | 0.000       | 0.496         |
| 配对 95% 区间                | [0.00,0.00] | [-5.92,62.75] |
| 更快/持平/更慢                 | 0/200/0     | 114/0/86      |
| 最大单局退步                   | 0.00        | 708.96        |
| (c) 运行开销与完成性             |             |               |
| CPU 平均时间                 | 0.10        | 1.48          |
| CPU 最大时间                 | 0.20        | 2.68          |
| 全部清除                     | 200/200     | 200/200       |

差  $d_i = T_{56,i} - T_{84,i}$ ，以  $\bar{d} \pm t_{0.975,n-1} s_d / \sqrt{n}$  报告均值区间。收益率是  $100\bar{d}/\bar{T}_{56}$ ，不是各局百分比的平均值。退步局以  $10^{-4}$  秒容差判定，避开微秒级累计舍入。以上数字来自完整 CSV，保持失败或不利结果可追踪。

图3逐局给出配对差：问题三的差值集中在零附近且全部非负，问题四则同时存在约 1600 秒的正收益与约 -700 秒的退步，正是其均值区间跨零、不能据此推广候选 84 的直接原因。

Q3 与方法 78 逐局相同，因此其对方法 56 的 1.48 秒收益全部来自既有的安全提前停车，不能记为候选 84 带来的新增规划收益。Q4 相对方法 78 平均节省 28.41 秒、0.496%，95% 区间为 [-5.92,62.75] 秒，86 局变慢；相对方法 56 则为 29.62 秒、0.516%，区间 [-4.69,63.92] 秒，81 局变慢。两个对照组（56 与 78）在 Q4 的区间均跨零，按预先声明的推广规则不推广候选 84，默认仍保留方法 78；表中凡涉及区间的行均已在 (a)(b) 两栏分别标明对照组。最大退步出现在同一个 16 源环境（种子 2610038），候选 84 比方法 78 多耗 708.96

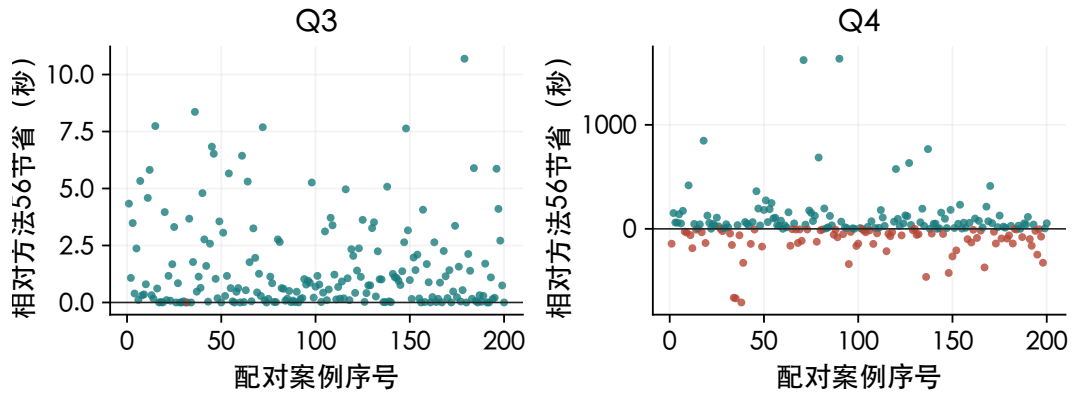


图 3: 冻结独立集每问 200 个环境的配对时间差。正值为候选 84 节省，横线为零；异常值与负收益全部保留。

秒、比方法 56 多耗 707.85 秒；最大收益也出现在同一环境下两法相同（种子 2610090，1635.82 秒）。这说明 Q4 的收益与退步都由少数高源数环境主导，均值不足以代表逐局表现。实验候选可由 geometric 策略名显式运行。

### 8.3 成本分解及压力测试

式(1)使配对节省分成移动、测向、换频和失败清除四项；同局成功清除数相同，其成本相消。图4按这一恒等式分解平均节省：候选 84 相对方法 56 在问题三平均节省 1.485 秒，全部来自移动；问题四平均节省 29.615 秒，其中移动贡献 27.810 秒，测向、换频和失败清除合计贡献 1.805 秒。这是计时恒等分解，不是各模块的独立因果贡献。表7显示，五组均全部清除并不能

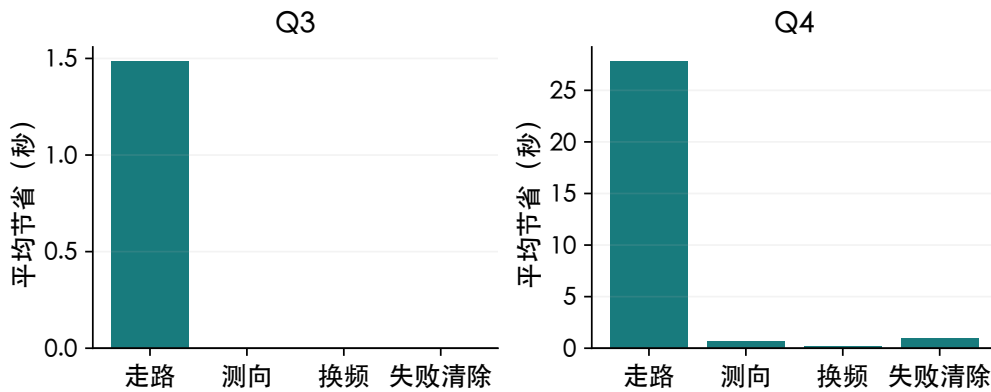


图 4: 独立集候选 84 相对方法 56 平均节省的计时分解。几何点的小幅移动可改变后续响应和控制分支，不能把全部差异直接归为移动量。

推出每组都更快：Q4 在边界外向、共线端点和近重合三组出现净退步。最大 CPU 时间是本机本地计算观测值，不能据此证明官方网络下 20 分钟最坏墙钟约束；桥接器另设真实截止计时并在请求结果不确定时停止发送新动作。

表 7: 五组压力测试, 每组每问 40 局, 候选 84 相对 56; 全部清除

| 组别     | Q3 节省时间/s | Q4 节省时间/s | Q3 更慢局 | Q4 更慢局 |
|--------|-----------|-----------|--------|--------|
| 边界外向   | 11.89     | -23.98    | 0      | 15     |
| 共线端点   | 7.45      | -8.02     | 0      | 17     |
| 近重合    | 0.17      | -0.28     | 0      | 18     |
| 相关角误差  | 2.20      | 14.45     | 0      | 13     |
| 分区端点误差 | 4.29      | 60.23     | 0      | 15     |

#### 8.4 信号源数量与单源平均清除时长

设信号源数量为  $N$ , 全部清除总时长为  $T$ , 单源平均时长为  $\tau = T/N$ 。以下研究默认方法 78 在  $10 \leq N \leq 16$  内的条件均值, 不将源数量视为决定单局时长的唯一因素。

总行走距离、换频数、测向数和失败清除数分别记为  $L, S, M, F$ , 它们与式(1)的记号一一对应:  $S = N_s$ 、 $M = N_m$ 、 $F = N_f$ , 成功清除数仍记  $N = N_c$ 。计时规则严格给出

$$T = L/5 + S + 5M + 3F + 5N. \quad (22)$$

记实际发现点数为  $m$ , 第  $k$  个源首次被发现的发现点序号为  $J_k$ 。空频道在每一点检测, 有源频道检测至首次发现, 故

$$M_{\text{disc}} = m(20 - N) + \sum_{k=1}^N J_k. \quad (23)$$

每一发现点的首次测量至多节省一次换频。记  $d = M_{\text{disc}} - S_{\text{disc}}$ , 则  $0 \leq d \leq m$ ; 其它检测和换频数记为  $M_{\text{loc}}, S_{\text{loc}}$ 。由此得到

$$\begin{aligned} \tau = \frac{L}{5N} + 6m \left( \frac{20}{N} - 1 \right) + 6\bar{J} + 5 \\ + \frac{5M_{\text{loc}} + S_{\text{loc}} + 3F - d}{N}, \quad \bar{J} = \frac{1}{N} \sum_k J_k. \end{aligned} \quad (24)$$

源数增加会减少空频道重复扫描, 并分摊共同开销。 $N < 16$  时, Q3 与 Q4 分别需完成 7 个和 21 个发现点;  $N = 16$  时, 已知数量上界允许提前结束发现。

据此在有限数量范围内建立均值近似

$$\mathbb{E}[T \mid N] \approx A + BN + C\mathbf{1}_{\{N=16\}}, \quad \mu(N) := \mathbb{E}[\tau \mid N] \approx B + \frac{A}{N} + \frac{C}{N}\mathbf{1}_{\{N=16\}}. \quad (25)$$

原独立集每问 200 局用于事后关系建模; 冻结公式后, 新增种子 2710001 至

2710200、每问 200 局作验证，不用新样本重估参数。总时长最小二乘拟合得到（单位：秒）

$$\hat{\mu}_3(N) = 60.01 + \frac{2240.54}{N} - \frac{158.67}{N} \mathbf{1}_{\{N=16\}}, \quad (26)$$

$$\hat{\mu}_4(N) = -31.60 + \frac{6206.85}{N} - \frac{513.96}{N} \mathbf{1}_{\{N=16\}}. \quad (27)$$

HC3 异方差稳健标准误给出两问  $A$  的 95% 近似区间分别为 [2014.89, 2466.19]、[5889.35, 6524.36]，均为正； $C$  的相应区间均为负。因而对  $N = 10, \dots, 14$ ，模型预测的相邻变化为

$$\hat{\mu}(N+1) - \hat{\mu}(N) = -\frac{\hat{A}}{N(N+1)} < 0. \quad (28)$$

从 15 到 16 还包含  $\hat{C}/16$  的修正。 $B$  是减少空频道检测后的净增量系数，不是独立单源的操作费用，因此可以为负。

表 8：冻结的新 200 局验证，逐局计算误差

| 指标          | Q3    | Q4    |
|-------------|-------|-------|
| 单源均时 RMSE/秒 | 17.20 | 28.93 |
| 单源均时 $R^2$  | 0.746 | 0.906 |
| 总时长 $R^2$   | 0.203 | 0.364 |

表8表明公式主要解释每源平均时长的下降趋势（ $R^2$  为 0.746 与 0.906），而总时长的解释力较低（0.203 与 0.364）。原 400 行结果逐行重现，800 个剖析案例均通过计时及发现次数恒等式。空间位置、接收半径、朝向和误差仍造成较大变动，尤其 16 源分组存在可见预测误差，因此不能把公式当作单局精确预测或所有布局的单调性定理。结论限于含 40 米位置排斥规则的本地场景分布，不外推官方成绩或题设数量范围之外。

图5给出拟合曲线、原样本分组均值与新样本分组均值的对照：曲线描述单源均时的总体下降趋势，若干新样本分组均值位于拟合均值区间之外。图中的区间描述拟合均值的不确定性，并非新观测的预测区间；逐局预测表现由本节表格中的冻结验证指标评价。

## 8.5 源数量与平均清除时长的一般关系

§8.4 的拟合式是在本地场景分布下的经验结果。本节补出与具体控制器无关的严格关系，用以界定该经验式的适用范围。

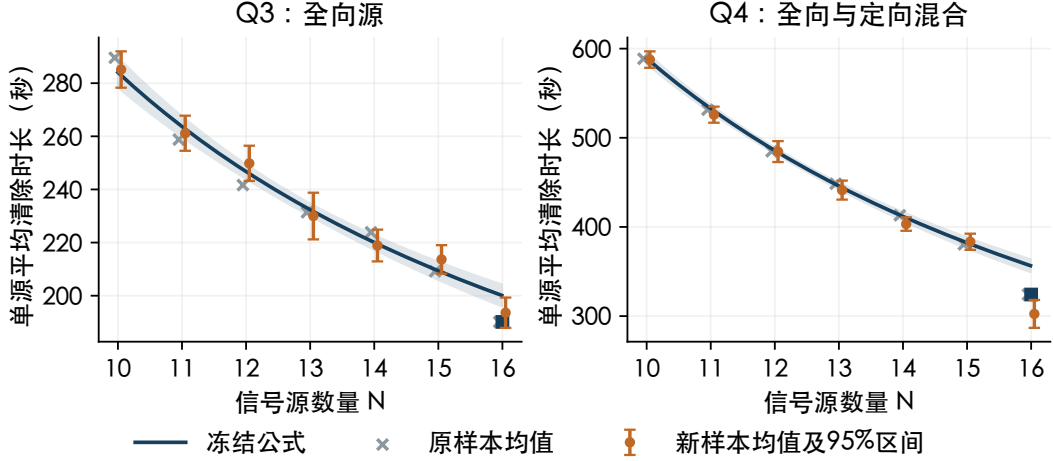


图 5: 源数量与每源均时。阴影为拟合均值 HC3 近似 95% 区间, 误差棒为新样本分组均值区间; 16 处方点包含停止规则修正。

对任意合法策略  $\pi$ , 题设计费规则给出的计时恒等式可写为

$$T_\pi = \frac{L_\pi}{5} + S_\pi + 5M_\pi + 3F_\pi + 5N, \quad (29)$$

其中  $L_\pi$  为米制路程,  $S_\pi, M_\pi, F_\pi$  分别为换频次数、测向次数和失败光学尝试次数。固定场景生成口径, 令  $g_\pi(n) = \mathbb{E}[T_\pi | N = n]$ 、 $\mu_\pi(n) = g_\pi(n)/n$ , 其中  $N$  为随机源数,  $n \in \{10, \dots, 16\}$  为其取值, 则

$$\mu_\pi(n+1) - \mu_\pi(n) = \frac{n\{g_\pi(n+1) - g_\pi(n)\} - g_\pi(n)}{n(n+1)}. \quad (30)$$

式(30)是一个代数恒等式, 等价地说明: 每源均时下降, 当且仅当新增一源带来的期望总时间增量低于原来的每源均时。总时间与每源均时因此可以同时上升或同时下降, 源数量本身不能唯一决定二者。

在固定有界区域、目标位置近似独立同分布、主要行走可由近最优点巡游描述的附加条件下, Beardwood–Halton–Hammersley 定理 [9] 给出最短点巡游长度的主要项  $L_n^* \sim \beta_2 \sqrt{n} \int_D \sqrt{f(z)} dz$ ; 均匀分布时为  $\beta_2 \sqrt{A_D n}$ , 其中  $f$  为位置密度,  $A_D$  为区域面积,  $\beta_2$  为二维 BHH 常数。再加上共同搜索费用近似稳定、每源局部定位增量近似稳定, 得到有机制依据的结构模型

$$\mathbb{E}[T | N = n] \approx c + an + b\sqrt{n}, \quad \mu(n) \approx a + \frac{b}{\sqrt{n}} + \frac{c}{n}, \quad (31)$$

三项依次对应每源增量、共享巡游与共同搜索费用。本文 §8.4 所用的  $A/N + C/N \cdot \mathbf{1}_{\{N=16\}}$  形式与  $\mu \approx a + b/\sqrt{n} + c/n$  并不相同: 前者把共享项整体按

$1/N$  处理，只是对共同成本的近似记账，因此才需要用竞争模型作独立检验，而不能仅凭训练拟合优度认定两个系数可辨识。在本题  $10 \leq N \leq 16$  的窄区间内， $1/N$  与  $1/\sqrt{n}$  高度共线，这一点尤其需要注意。

本题可在 20 米邻域内发出清除命令，因此优化对象是邻域旅行商问题而非点巡游 [4]。记同一起点、无需返回的最短点路径长度为  $L_{\text{pt}}^*$  (米)、最短圆盘路径长度为  $L_{\text{disk}}^*$  (米)，机器人速度  $v = 5$  米/秒，则

$$\max\{0, L_{\text{pt}}^* - 2rn\} \leq L_{\text{disk}}^* \leq L_{\text{pt}}^*, \quad r = 20. \quad (32)$$

右侧来自“访问圆心必然访问圆盘”；左侧由在每个圆盘访问处插入一次圆心往返（至多增加  $2r$ ）得到。由于任何全清除策略都必须在每个清除邻域内至少出现一次、且每次成功清除另计 5 秒，得到与策略无关的下界

$$T_{\pi} \geq 5n + \frac{L_{\text{disk}}^*}{v} \geq 5n + \frac{\max\{0, L_{\text{pt}}^* - 2rn\}}{v}. \quad (33)$$

该下界允许事先知道全部源位置，因而只作理论参照，不能当作可实现的搜索成绩。

最后需说明两条外推边界。其一，固定区域与固定正半径  $r$  时，取该区域的有限  $r$  网并访问全部网点即可与任意以区内位置为圆心的  $r$  圆盘相交，故  $L_{\text{disk}}^*$  存在与  $n$  无关的上界；因此不能把点 TSP 的  $n^{-1/2}$  当作固定 20 米邻域的无限规模规律，本题源数上限为 16，更不应外推。其二，本题未声明源位置独立均匀（本地场景还含 40 米位置排斥），而 Arlotto-Steele 给出过单点边缘均匀、过程平稳遍历却不满足经典 BHH 结论的反例 [10]，故上述结构模型只作为待检验的条件模型。本文的一般结论仅为式(30)的代数判据与圆盘巡游的几何下界，跨策略经验比较仍留作后续工作。

## 8.6 公开协议内核和 HTTP 验证

候选 84 通过 10 个本地 Python 物理核案例和 6 个本地 HTTP 案例，逐局检查清除数。该内核由队内按照公开协议实现，并非官方模拟器。另执行 27 个 C++ 几何、规划与控制回归程序、有理数剩余覆盖复核及传输超时测试。问题二的精确证书、问题三七点覆盖证书和问题四固定点数下界均在每次重建时重新运行。在线机器人只链接传感器接口，不链接快速模拟器的真值结构。

## 8.7 官方演练测试统计

题面要求用演练测试的两个统计量检验策略有效性：被清除干扰源个数的比例（清除数/总数）与平均定位清除时间（定位清除总时间/清除数）。演练模块会公布干扰源总数，因此两个统计量都可计算。表9给出问题三、四各

三次官方演练测试的记录；该批演练在模拟器中以方法 78（策略名 joint、规划深度 2）完成，其代码快照早于 §8.2 的独立集冻结，故仅用于检验策略有效性，不与正式测试成绩或本地配对结果合并统计。

**表 9：**问题三、问题四各三次官方演练测试结果（“源数”为该案例干扰源总数，“均时”为平均定位清除时间）

| 问题   | 案例编码                | 源数 | 清除数 | 比例   | 均时/s    |
|------|---------------------|----|-----|------|---------|
| Q3-1 | 39NJ-R5R7-2A6U-5FR9 | 10 | 10  | 100% | 259.843 |
| Q3-2 | AJBD-EBV4-FDJB-VKYS | 15 | 15  | 100% | 206.029 |
| Q3-3 | BYVK-XANU-TXZZ-JQZP | 15 | 15  | 100% | 202.899 |
| Q4-1 | NMYG-86T3-QHTZ-A76F | 10 | 10  | 100% | 563.302 |
| Q4-2 | SQPP-9EYR-3PES-S566 | 16 | 16  | 100% | 367.577 |
| Q4-3 | RQ3Z-P24P-NUUH-BMFC | 16 | 16  | 100% | 253.416 |

表9中六个案例全部清除，合计清除 82 个干扰源，整体清除比例为 100%；问题三合计 40 个、总定位清除时间 8732.347 秒、加权平均 218.309 秒/个，问题四合计 42 个、总时间 15568.901 秒、加权平均 370.688 秒/个。采用相同的“总时间之和/清除数之和”口径，本地独立集默认方法 78 的对应值为 230.496 与 440.016 秒/个。演练与本地场景未配对，分别报告其统计量，不由每问三局的差异推断总体分布或策略优劣。

## 8.8 官方正式测试与实施验证

表10给出问题三、四各三次正式测试结果。清除数由程序日志中/clear 成功响应计数，总时间取退出响应的虚拟时刻，平均定位清除时间为二者之比；程序运行时间由首末请求的本机时间戳之差计算。六局均收到/exit 的 user\_exit 响应，HTTP 错误数为 0。正式测试不公开案例中的干扰源总数，因而不能由这些记录计算清除比例，也不把“主动退出”表述为已知真值下的 100% 清除。

**表 10：**问题三、问题四官方正式测试结果

| 问题   | 测试案例编码              | 清除数 | 平均定位清除时间/s | 程序运行时间/s |
|------|---------------------|-----|------------|----------|
| Q3-1 | 42PD-EFKH-MB35-HUGM | 11  | 254.474    | 2.074    |
| Q3-2 | 64KQ-SEGG-VQCH-M8C8 | 14  | 215.121    | 1.624    |
| Q3-3 | B4WR-YVMD-JQQV-XFTP | 14  | 217.592    | 2.089    |
| Q4-1 | ZHZN-TEBH-HMAG-FFDG | 15  | 366.344    | 4.742    |
| Q4-2 | WMJX-EKTD-KPRD-786W | 10  | 575.192    | 5.361    |
| Q4-3 | MTHU-N87U-YWR2-3RRK | 10  | 558.222    | 4.908    |

问题三三局合计清除 39 个，定位清除总时间 8857.186 秒，加权平均 227.107 秒/个；问题四三局合计清除 35 个，总时间 16829.312 秒，加权平均 480.837 秒/个。表10中合计与加权均值按全精度日志值计算，与按表内三位

小数反算的结果相差不超过 0.02 秒。这些加权均值与前节本地合并口径一致；正式测试、本地独立集和演练案例分别统计，不进行跨样本的策略增益归因。六份官方加密日志保持导出文件名不变，列入支撑材料。

## 9 模型评价与结论

### 9.1 模型的优点

第一，定位状态采用有界角锥交而非高斯近似，能够直接对应题设的  $\pm 1^\circ$  固定误差，并以最小包围圆半径给出 20 米保证清除判据。第二，问题三的七点环和问题四的剩余覆盖不变量把“发现所有频道”写成可复核的连续几何充分条件，避免仅凭有限粒子或有限网格声称全覆盖。第三，控制器显式计入移动、换频、检测及清除成本，并用同种子配对、独立集、压力场景和新的冻结样本区分开发与验证。第四，在线程序只使用公开 HTTP 响应，六次演练与六次正式测试均无 HTTP 错误且主动退出，说明算法与工程实现能够闭环运行。第五，官方演练与正式记录分别报告清除数和加权均时，将可计算的演练清除比例与正式案例未公开的真值区分开来。

### 9.2 适用边界与改进方向

问题四的固定通用布点满足  $15 \leq N^* \leq 21$ ，21 点布局提供可认证的可行上界；进一步缩小区间可结合列生成与半无限规划。连续选点与单源续跑使用有限场景代理，后续可在整局控制器上引入区间分支定界和分层鲁棒续跑。点巡游的渐近关系需要相应分布与尺度假设，本题的有限源数分析以冻结新样本检验为依据。

候选 84 的 Q4 独立配对区间跨零，现有样本未确认其总体平均节省为正，因此默认配置保持方法 78。该比较评价新增连续点替换的时间收益，不用于估计整个覆盖证书体系的价值。证书给出几何充分条件，漏清风险的统计变化仍需预先设计的对照实验。官方每问三局的记录用于接口与执行验证，推广到其他场景分布时应采用预先冻结的多分布验证集。

### 9.3 模型的推广

有界角锥交、最小包围圆终止准则和“历史点加未来点”的剩余覆盖证书可推广到无人机搜救、声源定位及移动传感器部署。在重新核验接收集合、运动与动作约束后，可构造“集合估计-主动观测-邻域访问-安全终止”的通用闭环；对于三维场景，可将二维角锥和圆盘分别替换为空间锥与球覆盖，沿用证书化思想并重新证明空间覆盖条件。

## 9.4 结论

问题一给出了角锥半平面交、直径和最小包围圆算法，并证明直径圆不一定覆盖定位区域。问题二推导四圆盘保证接收域，证明完整首扇区一次补测的最坏后验半径下界超过 55.65 米，故 35 米保证区域为空。问题三以七点发现环和有限光学兜底完成闭环；官方演练三次均清除全部干扰源，正式测试清除 39 个、加权均时 227.107 秒/个。问题四以 21 点双环、剩余覆盖证书和自适应条带处理未知定向源；演练三次清除全部 42 个干扰源，正式测试清除 35 个、加权均时 480.837 秒/个。六次演练合计清除 82 个、清除比例 100%，六次正式测试合计清除 74 个；六次正式测试中程序均正常执行并主动退出。由于正式案例真值不公开，正式测试的清除比例仍以官方平台的后续判定为准。

## AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码调试、数值检验和论文语言严谨性润色，详细使用情况见支撑材料。所用工具为 Kimi K3、GLM 5.3 与 DeepSeek V4.1 Flash；主体建模工作与全部编程实现由参赛队员人工完成，AI 工具仅起辅助作用。

## 参考文献

- [1] Kaya C Y. Observer path planning for maximum information[J]. Optimization, 2022, 71(4):1097–1116. DOI:10.1080/02331934.2021.2011868. <https://arxiv.org/abs/2103.02059>.
- [2] Vander Hook J, Tokekar P, Isler V. Cautious greedy strategy for bearing-only active localization: Analysis and field experiments[J]. Journal of Field Robotics, 2014, 31(2):296–318. DOI:10.1002/rob.21499.
- [3] Bayram H, Vander Hook J, Isler V. Gathering bearing data for target localization[J]. IEEE Robotics and Automation Letters, 2016, 1(1):369–374. DOI:10.1109/LRA.2016.2521387.
- [4] Dumitrescu A, Mitchell J S B. Approximation algorithms for TSP with neighborhoods in the plane[J]. Journal of Algorithms, 2003, 48(1):135–159. DOI:10.1016/S0196-6774(03)00047-6.
- [5] Peng C, Wei M, Isler V. Stochastic traveling salesperson problem with neighborhoods for object detection[C]//2023 IEEE International Conference on Robotics and Automation (ICRA). Piscataway: IEEE, 2023:3607–3613. DOI:10.1109/ICRA48891.2023.10161120.
- [6] Hoffmann F, Schily H, Krestel M, et al. Non-myopic sensor path planning for emitter localization with a UAV[C]//2023 26th International Conference on Information Fusion (FUSION). Piscataway: IEEE, 2023:1–8. DOI:10.23919/FUSION52260.2023.10224174.
- [7] Sunberg Z, Kochenderfer M. Online algorithms for POMDPs with continuous state, action, and observation spaces[C]//Proceedings of the International Conference

- on Automated Planning and Scheduling (ICAPS). Palo Alto: AAAI Press, 2018, 28:259–263.
- [8] Fekete S P, Krupke D, Perk M, et al. The lawn mowing problem: From algebra to algorithms[C]//ESA 2023. LIPIcs 274. Dagstuhl: Schloss Dagstuhl, 2023:45:1–45:18. DOI:10.4230/LIPIcs.ESA.2023.45.
  - [9] Beardwood J, Halton J H, Hammersley J M. The shortest path through many points[J]. Mathematical Proceedings of the Cambridge Philosophical Society, 1959, 55(4):299–327. DOI:10.1017/S0305004100034095.
  - [10] Arlotto A, Steele J M. Beardwood–Halton–Hammersley theorem for stationary ergodic sequences: A counterexample[J]. The Annals of Applied Probability, 2016, 26(4):2141–2168. DOI:10.1214/15-AAP1142.

## A 支撑材料与复现

| 文件位置                                            | 内容                                                            |
|-------------------------------------------------|---------------------------------------------------------------|
| src/ 与 review/                                  | 可编译 C++ 在线机器人、几何、规划、覆盖证书及验证脚本                                 |
| tests/                                          | 27 个 C++ 回归程序与 6 个 Python 核验脚本                                |
| results/                                        | 每局开发、独立和压力 CSV 及匿名化实验汇总                                       |
| supplement/count_time/                          | 源数-单源均时附加分析的源码、四份逐局 CSV、统计结果及图表                               |
| supplement/<br>count_time_theory/<br>artifacts/ | 源数量-平均时长一般关系的推导、有理数代数核验脚本与文献核验<br>冻结配置、执行命令、哈希、区间统计、几何证书及复核结果 |
| evidence/<br>paper/ 与 figures/                  | 文献核验、旧版本哈希与原数学推导<br>本文 LaTeX 源码、自动生成数表与独立科研图                  |
| vendor/mocksim/                                 | 来自所给压缩包的非官方公开协议模拟内核                                           |
| official-logs/                                  | 六次官方正式测试导出的加密行为日志，文件名保持不变                                     |
| AI 工具使用详情.pdf                                   | AI 工具、用途、主要提示方式及人工核验说明                                        |
| run_all.py                                      | 重新编译、运行测试并重建实验汇总                                              |

编译使用 C++17 与优化级别 O2，不允许 fast-math；该选项会破坏向外舍入认证的前提。新会话由原点开始；非零位置误差采用旧后备策略。支撑材料不包含含队号的明文结果 JSON、数据库或程序自记 HTTP 日志；仅按题目要求保留六份官方导出的加密日志原件。附录 B 列出全部源码文件（共 103 个），与支撑包内文件逐项对应。

## B 完整计算源程序

此附录列出在线机器人、实验内核、消融模块、几何证书与附加分析脚本的完整源码；共 103 个文件，与支撑包内文件逐项对应。历史方法分支仅供复核，不表示默认启用。模块标题与支撑包同名。

## review/certified\_layout21\_hex.hpp

```
1 // Exactly the binary64 coordinates independently certified by the interval-arithmetic
2 // producer. The certified leaf-box witness ships as artifacts/residual-witness.json
3 // (21 sites, 4564 certified leaf boxes) and is replayed by tests/replay_residual_fraction.py.
4 #pragma once
5 #include <array>
6 namespace certified_layout21 {
7 struct Point { double x,y; };
8 inline constexpr std::array<Point,21> points = {{
9     {0x0.0p+0, 0x0.0p+0},
10    {0x1.f180000000000p+9, 0x0.0p+0},
11    {0x1.5fc91ea18215dp+9, 0x1.5fc91ea18215dp+9},
12    {0x0.0p+0, 0x1.f180000000000p+9},
13    {-0x1.5fc91ea18215dp+9, 0x1.5fc91ea18215dp+9},
14    {-0x1.f180000000000p+9, 0x0.0p+0},
15    {-0x1.5fc91ea18215dp+9, -0x1.5fc91ea18215dp+9},
16    {0x0.0p+0, -0x1.f180000000000p+9},
17    {0x1.5fc91ea18215dp+9, -0x1.5fc91ea18215dp+9},
18    {0x1.d200000000000p+10, 0x0.0p+0},
19    {0x1.93915dd785dc9p+10, 0x1.d200000000000p+9},
20    {0x1.d200000000000p+9, 0x1.93915dd785dc9p+10},
21    {0x0.0p+0, 0x1.d200000000000p+10},
22    {-0x1.d200000000000p+9, 0x1.93915dd785dc9p+10},
23    {-0x1.93915dd785dc9p+10, 0x1.d200000000000p+9},
24    {-0x1.d200000000000p+10, 0x0.0p+0},
25    {-0x1.93915dd785dc9p+10, -0x1.d1fffffffffff7p+9},
26    {-0x1.d200000000000p+9, -0x1.93915dd785dc4p+10},
27    {0x0.0p+0, -0x1.d200000000000p+10},
28    {0x1.d200000000000p+9, -0x1.93915dd785dc9p+10},
29    {0x1.93915dd785dc4p+10, -0x1.d200000000000p+9},
30 }};
31 }
```

## run\_all.py

```
1 """Portable frozen reproduction entry point. Never starts official tests."""
2 from pathlib import Path
3 import subprocess,sys,argparse,shutil,json,time,os
4 R=Path(__file__).resolve().parent
5 p=argparse.ArgumentParser();p.add_argument('--full',action='store_true');p.add_argument('--
    paper',action='store_true');a=p.parse_args()
6 cc=os.environ.get('CXX') or shutil.which('clang++') or shutil.which('g++');assert cc,'C++17
    compiler required'
7 (R/'bin').mkdir(exist_ok=True);records=[]
8 def run(cmd,cwd=R):
9     begin=time.time()
10    # Explicit UTF-8: subprocess text mode otherwise decodes child output with the host
11    # locale (e.g. GBK on Chinese Windows) and crashes on UTF-8 tracebacks.
12    out=subprocess.run(cmd,cwd=cwd,capture_output=True,text=True,encoding='utf-8',errors='replace
        ');rec={'command':cmd,'cwd':str(cwd.relative_to(R)) if cwd.is_relative_to(R) else str(cwd)
        , 'exit_code':out.returncode,'runtime_s':time.time()-begin,'stdout':out.stdout,'stderr':out
        .stderr};records.append(rec)
13    # UTF-8 on write too: the default locale encoding (cp936 on Chinese Windows) would
14    # produce a log that is not valid UTF-8 and breaks downstream readers.
15    (R/'artifacts/reproduction-log.json').write_text(json.dumps(records,ensure_ascii=False,indent
        =2),encoding='utf-8')
16    if out.returncode:raise RuntimeError(rec)
17    print('PASS',cmd[0],cmd[-1],flush=True)
18    run([cc,'-O2','-std=c++17','src/robot.cpp','-o','robot.exe'])
19    run([cc,'-O2','-std=c++17','src/benchmark.cpp','-o','bin/benchmark_final'])
20    run([sys.executable,'src/build_overview_figure.py'])
21    run([sys.executable,'tests/run_checks.py'])
22    run([cc,'-O2','-std=c++17','tests/test_q3_radius.cpp','-o','bin/test_q3_radius']);run([str(R/'
        bin/test_q3_radius')]);run([sys.executable,'tests/q3_coverage_bound.py'])
23    run([sys.executable,'src/q2_full_domain_certificate.py','artifacts/q2-certificate.json'])
24    run([sys.executable,'src/q4_area_arc_certificate.py','artifacts/q4-lower-bound.json'])
25    if a.full:
26        run([sys.executable,'src/run_experiments.py','--suite','iid'])
27        run([sys.executable,'src/run_experiments.py','--suite','stress'])
28    run([sys.executable,'src/analyze_revision.py'])
29    if a.paper:
30        run([sys.executable,'src/build_paper_assets.py'])
31        for document in ['main.tex','AI工具使用详情.tex']:
32            for _ in range(2):run(['xelatex','-interaction=nonstopmode','-halt-on-error',document],R/'
                paper')
33    shutil.copy2(R/'paper/AI工具使用详情.pdf',R/'AI工具使用详情.pdf')
34    print('Core checks and frozen-data analysis completed; full simulations run only with --full.')
```

## src/active.hpp

```
1 #pragma once
2 #include "geometry.hpp"
3 namespace active {
4 // Envelope over bearing intervals; only a ranking surrogate, not a certified
5 // time bound. Real updates continue using the original conservative polygon.
6 inline double posterior_radius(const geo::Poly&p,geo::Pt q){
7     auto c=geo::mec(p);double reference=atan2(c.c.y-q.y,c.c.x-q.x)*180/geo::pi;
8     double lo=1e99,hi=-1e99;
9     for(auto v:p){double a=atan2(v.y-q.y,v.x-q.x)*180/geo::pi-reference;a=std::remainder(a,360.);
        lo=std::min(lo,a);hi=std::max(hi,a);}
```

```

10 if(hi-lo>=179)return c.r;
11 lo-=1.005;hi+=1.005;double step=(hi-lo)/24,worst=0;
12 for(int j=0;j<24;++j){auto post=geo::observe(p,q,reference+lo+(j+.5)*step,1.005+step/2);if(!
    post.empty())worst=std::max(worst,geo::mec(post).r);}
13 return std::min(c.r,worst);
14 }
15 inline geo::Pt choose(const geo::Poly&p,geo::Pt pos,geo::Pt first,double angle,double A,double
    B,int mode){
16 auto c=geo::mec(p);auto u=geo::polar(1,angle);geo::Pt v{-u.y,u.x};
17 geo::Pt q1=first+u*A+v*B,q2=first+u*A-v*B;
18 geo::Pt best=geo::dist(pos,q1)<geo::dist(pos,q2)?q1:q2;
19 // Longest chord is an uncertainty-axis estimate, not a stochastic covariance.
20 double longest=0;geo::Pt axis=u;
21 for(auto a:p)for(auto b:p){double dd=geo::dist(a,b);if(dd>longest){longest=dd;axis=(a-b)/dd;}}
22 geo::Pt transverse{-axis.y,axis.x};
23 geo::Poly candidates{q1,q2};
24 for(double along:{-.2,0.,.2})for(double side:{-1.,1.})for(double d:{50.,100.,200.,300.})
    candidates.push_back(c.c+axis*(along*longest)+transverse*(side*d));
25 double bestscore=1e100;double weight=mode==1?1.:3.;
26 for(auto q:candidates){
27 if(geo::contains(p,q)||geo::norm(q)>2671.)continue;
28 double nearest=1e100,farthest=0;
29 for(size_t i=0;i<p.size();++i){nearest=std::min(nearest,geo::segment_distance(q,p[i],p[(i+1)%
    p.size()]));farthest=std::max(farthest,geo::dist(q,p[i]));}
30 if(nearest<10||farthest>999.9)continue;
31 double r=posterior_radius(p,q);
32 double score=geo::dist(pos,q)/5+6+geo::dist(q,c.c)/5+weight*r/5;
33 if(score<bestscore){bestscore=score;best=q;}
34 }return best;
35 }
36 }

```

## src/adaptive\_optical.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 #include <stdexcept>
4 namespace adaptive_optical {
5 #define B_VARIABLE_COVER 1
6 inline geo::Poly variable_cover(const geo::Poly&p,geo::Pt first,double angle,std::vector<geo::
    Poly>*certificate=nullptr){
7 if(certificate)certificate->clear();if(p.empty())return {};
8 auto whole=geo::mec(p);if(whole.r<20-1e-4){if(certificate)certificate->push_back(p);return {
    whole.c};}
9 auto u=geo::polar(1,angle);geo::Pt v{-u.y,u.x};geo::Poly local;
10 double xmin=INFINITY,xmax=-INFINITY,ymin=INFINITY,ymax=-INFINITY;
11 for(auto z:p){auto d=z-first;local.push_back({geo::dot(d,u),geo::dot(d,v)});xmin=std::min(xmin
    ,local.back().x);xmax=std::max(xmax,local.back().x);ymin=std::min(ymin,local.back().y);
    ymax=std::max(ymax,local.back().y);}
12 struct Packing {bool valid=true;geo::Poly centers;std::vector<geo::Poly>cells;};
13 auto pack=[&](double left,double right,int rows){Packing result;
14 auto strip=geo::clip(geo::clip(local,{-1,0},-left+1e-6),{1,0},right+1e-6);
15 if(strip.empty()){result.valid=false;return result;}
16 double bottom=INFINITY,top=-INFINITY;for(auto z:strip){bottom=std::min(bottom,z.y);top=std::
    max(top,z.y);}
17 for(int j=0;j<rows;++j){double a=bottom+(top-bottom)*j/rows,b=bottom+(top-bottom)*(j+1)/rows;
18 auto cell=geo::clip(geo::clip(strip,{0,-1},-a+1e-6),{0,1},b+1e-6);if(cell.empty())continue;

```

```

19  auto circle=geo::mec(cell);if(!(circle.r<20-1e-4)){result.valid=false;return result;}
20  result.centers.push_back(circle.c);result.cells.push_back(std::move(cell));
21  }return result;
22  };
23  geo::Poly out;double left=xmin;int strips=0;
24  do{
25  double remaining=xmax-left,chosen_width=-1,efficiency=-1;Packing chosen;
26  std::vector<int> row_counts{1,2};int fallback_rows=std::max(1,int(ceil((ymax-ymin)/28)));
27  if(fallback_rows>2)row_counts.push_back(fallback_rows);
28  for(int rows:row_counts){double low=0,high=std::min(39.99,std::max(0.,remaining));
29  auto start=pack(left,left,rows);if(!start.valid)continue;
30  auto full=pack(left,left+high,rows);
31  if(full.valid)low=high;else for(int iteration=0;iteration<14;++iteration){double mid=(low+
    high)/2;if(pack(left,left+mid,rows).valid)low=mid;else high=mid;}
32  auto candidate=pack(left,left+low,rows);if(!candidate.valid||candidate.centers.empty())
    continue;
33  double score=(remaining<=1e-7?1.:low)/candidate.centers.size();
34  if(score>efficiency){efficiency=score;chosen_width=low;chosen=std::move(candidate);}
35  }
36  if(chosen_width<0||((remaining>1e-7&&chosen_width<1e-7))throw std::runtime_error("variable
    cover made no progress");
37  if(strips%2){std::reverse(chosen.centers.begin(),chosen.centers.end());std::reverse(chosen.
    cells.begin(),chosen.cells.end());}
38  for(size_t i=0;i<chosen.centers.size();++i){auto c=chosen.centers[i];out.push_back(first+u*c.
    x+v*c.y);
39  if(certificate){geo::Poly cell;for(auto z:chosen.cells[i])cell.push_back(first+u*z.x+v*z.y);
    certificate->push_back(std::move(cell));}
40  }
41  left+=chosen_width;if(++strips>10000)throw std::runtime_error("variable cover exceeds strip
    budget");
42  }while(left<xmax-1e-7);
43  return out;
44  }
45  inline geo::Poly cover(const geo::Poly&p,geo::Pt first,double angle,std::vector<geo::Poly>*&
    certificate=nullptr){
46  if(certificate)certificate->clear();
47  if(p.empty())return {};auto u=geo::polar(1,angle);geo::Pt v{-u.y,u.x};geo::Poly local;
48  double xmin=1e99,xmax=-1e99;for(auto z:p){auto d=z-first;local.push_back({geo::dot(d,u),geo::
    dot(d,v)});xmin=std::min(xmin,local.back().x);xmax=std::max(xmax,local.back().x);}
49  int nx=std::max(1,int(ceil((xmax-xmin)/28)));geo::Poly out;
50  for(int i=0;i<nx;++i){double left=xmin+i*(xmax-xmin)/nx,right=xmin+(i+1)*(xmax-xmin)/nx;auto
    cell=geo::clip(geo::clip(local,{-1,0},{-left+1e-5},{1,0},right+1e-5);if(cell.empty())
    continue;
51  auto c=geo::mec(cell);if(c.r<20-1e-5){out.push_back(first+u*c.x+v*c.y);if(certificate){
    geo::Poly global;for(auto z:cell)global.push_back(first+u*z.x+v*z.y);certificate->
    push_back(global);}continue;}
52  double bottom=1e99,top=-1e99;for(auto z:cell){bottom=std::min(bottom,z.y);top=std::max(top,z.
    y);}int ny=std::max(1,int(ceil((top-bottom)/28)));
53  for(int jj=0;jj<ny;++jj){int j=i%2?ny-1-jj:jj;double a=bottom+j*(top-bottom)/ny,b=bottom+(j
    +1)*(top-bottom)/ny;
54  auto part=geo::clip(geo::clip(cell,{0,-1},{-a+1e-5},{0,1},b+1e-5);if(part.empty())continue;
    auto circle=geo::mec(part);
55  if(circle.r>=20-1e-5)throw std::runtime_error("adaptive optical subcell radius exceeds20");
    out.push_back(first+u*circle.c.x+v*circle.c.y);if(certificate){geo::Poly global;for(auto z
    :part)global.push_back(first+u*z.x+v*z.y);certificate->push_back(global);}
56  }
57  }return out;
58  }

```

## src/analyze\_joint.py

```

1 import csv,json,math
2 from pathlib import Path
3 import numpy as np
4 from scipy.stats import t
5 ROOT=Path(__file__).resolve().parents[1]
6 def interval(a):
7     mean=float(a.mean());half=float(t.ppf(.975,len(a)-1)*a.std(ddof=1)/math.sqrt(len(a)))
8     return [mean-half,mean+half]
9 def paired(basefile,newfile,start,count,final_method=38):
10     groups=[]
11     for filename,method in [(basefile,0),(newfile,final_method)]:
12         rows=list(csv.DictReader((ROOT/'results'/filename).open(encoding='utf-8-sig'))))
13         assert len(rows)==2*count
14         data={}
15         for r in rows:
16             p,seed=int(r['problem']),int(r['seed']);assert int(r['method'])==method
17             assert int(r['n'])==int(r['cleared']) and r['certificate']!='incomplete'
18             assert int(r['dp_actions'])<=6*int(r['n']) and int(r['aux_calls'])<=4*int(r['n'])
19             cost=float(r['move_m'])/5+float(r['switches'])+5*float(r['measures'])+3*float(r['
miss'])+5*float(r['cleared'])
20             assert abs(cost-float(r['time_s']))<.001
21             assert abs(float(r['avg_s'])-float(r['time_s'])/int(r['n']))<1e-6
22             assert (p,seed) not in data;data[p,seed]=r
23             assert set(data)=={(p,s) for p in [3,4] for s in range(start,start+count)}
24             groups.append(data)
25     result={}
26     for p in [3,4]:
27         arr=lambda g,key:np.array([float(groups[g][p,s][key]) for s in range(start,start+count)
28 ])
29         assert np.array_equal(arr(0,'n'),arr(1,'n'))
30         b,n=arr(0,'time_s'),arr(1,'time_s');d=b-n;over=.9*b-n;idx=int(np.argmin(d))
31         result[p]={ 'cases':count, 'all_cleared':True, 'baseline_total_s':float(b.mean()), '
new_total_s':float(n.mean()), 'baseline_single_s':float(arr(0,'avg_s').mean()), '
new_single_s':float(arr(1,'avg_s').mean()),
32         'saving_percent':float(100*d.mean()/b.mean()), 'saving_mean_s':float(d.mean()), '
paired_ci95_s':interval(d), 'excess_over_10pct_s':float(over.mean()), '
excess_over_10pct_ci95_s':interval(over),
33         'wins':int(sum(d>1e-6)), 'losses':int(sum(d<-1e-6)), 'ties':int(sum(abs(d)<=1e-6)), '
max_slowdown_s':float(max(0,-min(d))), 'worst_seed':start+idx,
34         'baseline_p95_s':float(np.quantile(b,.95)), 'new_p95_s':float(np.quantile(n,.95)),
35         'baseline_cpu_mean_s':float(arr(0,'runtime_s').mean()), 'new_cpu_mean_s':float(arr
(1,'runtime_s').mean()), 'new_cpu_max_s':float(arr(1,'runtime_s').max()), 'budget_hits':int(
arr(1,'dp_budget_hits').sum()),
36         'dp_calls_mean':float(arr(1,'dp_calls').mean()), 'aux_calls_mean':float(arr(1,'
aux_calls').mean()),
37         'cost_savings_s':{ 'move':float((arr(0,'move_m')-arr(1,'move_m')).mean()/5), 'measure
':float((arr(0,'measures')-arr(1,'measures')).mean()*5), 'switch':float((arr(0,'switches')-
arr(1,'switches')).mean()), 'failed_clear':float((arr(0,'miss')-arr(1,'miss')).mean()*3)}
38     return result
39 def main():
40     report={'baseline':'previous round default two-step DP', 'iid':paired('iid_baseline.csv', '
iid_final.csv',1510001,200), 'stress':{}}
```

```

40     for k,start in enumerate([1520001,1530001,1540001,1550001,1560001],1):report['stress'][k]=
        paired(f'stress{k}_0.csv',f'stress{k}_38.csv',start,40)
41     (ROOT/'results/summary.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),
        encoding='utf-8')
42     print(json.dumps(report,ensure_ascii=False,indent=2))
43 if __name__=='__main__':main()

```

## src/analyze\_revision.py

```

1 import csv,json,math,statistics as st
2 from pathlib import Path
3 R=Path(__file__).resolve().parents[1]
4 def rows(name):
5     p=R/'results'/name
6     return list(csv.DictReader(p.open()))
7 def paired(a,b,q):
8     a={int(x['seed']):x for x in a if int(x['problem'])==q};b={int(x['seed']):x for x in b if int(
        x['problem'])==q}
9     assert set(a)==set(b) and a
10    for k in a:
11        assert a[k]['n']==b[k]['n']==a[k]['cleared']==b[k]['cleared']
12    x=[float(a[k]['time_s']) for k in sorted(a)];y=[float(b[k]['time_s']) for k in sorted(a)];d=[i
        -j for i,j in zip(x,y)];n=len(d);mean=st.mean(d);se=st.stdev(d)/math.sqrt(n) if n>1 else 0
13    from scipy.stats import t
14    critical=float(t.ppf(.975,n-1))
15    out={'n':n,'baseline_mean_s':st.mean(x),'candidate_mean_s':st.mean(y),'saving_s':mean,
        'saving_pct':100*mean/st.mean(x),'ci95_s':[mean-critical*se,mean+critical*se],'faster':sum(
            v>1e-4 for v in d),'slower':sum(v<-1e-4 for v in d),'ties':sum(abs(v)<=1e-4 for v in d),'
        max_regression_s':-min(d),'max_saving_s':max(d),'all_cleared':True,'mean_source_s':st.mean(
            float(i['avg_s']) for i in b.values()),'cpu_mean_s':st.mean(float(i['runtime_s']) for i
            in b.values()),'cpu_max_s':max(float(i['runtime_s']) for i in b.values()),'
        baseline_source_s':st.mean(float(i['avg_s']) for i in a.values())}
16    out['decomposition_s']={k:st.mean((float(a[s][field])-float(b[s][field]))*factor for s in a)
        for k,field,factor in [('move','move_m',.2),('measure','measures',5),('switch','switches
            ',1),('failed_clear','miss',3)]}
17    out['candidate_counts']={field:st.mean(float(i.get(field,0)) for i in b.values()) for field in
        ['move_m','measures','switches','miss','fallbacks','residual_replaced','residual_retired
            ','rollout_calls','rollout_replaced']}
18    assert abs(sum(out['decomposition_s'].values())-mean)<.001
19    out['pairs']=[{'seed':s,'baseline_s':float(a[s]['time_s']),'candidate_s':float(b[s]['time_s'])
        ,'saving_s':float(a[s]['time_s'])-float(b[s]['time_s'])} for s in sorted(a)]
20    return out
21 if __name__=='__main__':
22     development={};base=rows('dev56.csv')
23     for method,name in [(79,'dev79.csv'),(80,'dev80.csv'),(81,'dev81b.csv'),(82,'dev82.csv'),(83,'
        dev83.csv'),(84,'dev84.csv'),(85,'dev85.csv'),(86,'dev86.csv')]:
24         development[str(method)]={'q':paired(base,rows(name),q) for q in [3,4]}
25     out={'development':development,'iid':{},'stress':{}}
26     for q in [3,4]:
27         out['iid'][str(q)]={'vs56':paired(rows('iid56.csv'),rows('iid84.csv'),q),'vs78':paired(rows('
            iid78.csv'),rows('iid84.csv'),q)}
28     for group in range(1,6):out['stress'][str(group)]={'q':paired(rows(f'stress{group}_56.csv')
        ,rows(f'stress{group}_84.csv'),q) for q in [3,4]}
29     for q in ['3','4']:
30         assert out['iid'][q]['vs56']['n']==200
31         for group in out['stress']:assert out['stress'][group][q]['n']==40
32     (R/'artifacts/summary.json').write_text(json.dumps(out,ensure_ascii=False,indent=2))

```

```

33 print(json.dumps({q:{k:{f:v for f,v in d.items() if f not in ['pairs']] for k,d in m.items()}
    for q,m in out['iid'].items()},ensure_ascii=False,indent=2))

```

### src/analyze\_stage2.py

```

1 import json
2 from analyze_joint import ROOT,paired
3 result={'baseline':'previous default two-step DP','final_method':39,'iid':paired('iid2_baseline
    .csv','iid2_final.csv',1710001,200,39),'stress':{}}
4 for k,start in enumerate([1720001,1730001,1740001,1750001,1760001],1):result['stress'][k]=
    paired(f'stress2_{k}_0.csv',f'stress2_{k}_39.csv',start,40,39)
5 (ROOT/'results/summary_stage2.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),
    encoding='utf-8')
6 print(json.dumps(result,ensure_ascii=False,indent=2))

```

### src/analyze\_stage3.py

```

1 import json
2 from analyze_joint import ROOT,paired
3 result={'baseline':'previous default two-step DP','final_method':56,'iid':paired('iid3_baseline
    .csv','iid3_final.csv',1910001,200,56),'stress':{}}
4 for k,start in enumerate([1920001,1930001,1940001,1950001,1960001],1):result['stress'][k]=
    paired(f'stress3_{k}_0.csv',f'stress3_{k}_56.csv',start,40,56)
5 (ROOT/'results/summary_stage3.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),
    encoding='utf-8')
6 print(json.dumps(result,ensure_ascii=False,indent=2))

```

### src/bearing\_types.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 namespace dynamic_selection {
4 struct Event {geo::Pt q;int kind;double angle=0;};
5 struct Hypothesis {geo::Pt z;double radius;bool directional;double heading;double weight;};
6 }

```

### src/belief\_dp.hpp

```

1 #pragma once
2 #define B_CONTEXT_TERMINAL 1
3 #include <vector>
4 #include <functional>
5 #include <cstdint>
6 #include <stdexcept>
7 #include <map>
8 #include <tuple>
9 #include <cmath>
10 #include <algorithm>
11 namespace belief_dp {
12 struct Action {bool measure;std::vector<int>outcome;std::vector<double>seconds;};
13 struct Model {std::vector<double>weights;std::vector<Action>actions;std::vector<std::vector<
    double>>travel;std::function<double(uint64_t,int)>terminal;std::function<int(int,int,int)>
    advance;std::function<double(uint64_t,int,int)>terminal_context;};
14 struct Result {int action=-1;double value=0;int expanded=0;bool exact=false,budget_hit=false;
    std::vector<double>action_values;};

```

```

15 inline Result solve(const Model&m,int depth,bool tuned,int limit){
16   const int n=m.weights.size(),a=m.actions.size();
17   if(n<1||n>63||depth<0||depth>5||limit<0||!m.terminal||m.travel.size()!=size_t(a+1))throw std::
      invalid_argument("invalid DP dimensions");
18   for(double w:m.weights)if(!(w>0)||!std::isfinite(w))throw std::invalid_argument("invalid
      hypothesis weight");
19   for(const auto &row:m.travel){if(row.size()!=size_t(a+1))throw std::invalid_argument("invalid
      travel matrix");for(double x:row)if(x<0||!std::isfinite(x))throw std::invalid_argument("
      invalid travel cost");}
20   for(const auto &x:m.actions){if(x.outcome.size()!=size_t(n)||x.seconds.size()!=size_t(n))throw
      std::invalid_argument("invalid action dimensions");for(double c:x.seconds)if(c<0||!std::
      isfinite(c))throw std::invalid_argument("invalid action cost");}
21   if(bool(m.advance)!=bool(m.terminal_context))throw std::invalid_argument("both context
      callbacks required");
22   Result result;result.action_values.assign(a,INFINITY);std::map<std::tuple<uint64_t,int,int,
      bool,int>,std::pair<double,int>>memo;
23   std::map<std::tuple<uint64_t,int,int>,double>leaves;std::map<uint64_t,double>masses;
24   auto mass=[&](uint64_t mask){auto it=masses.find(mask);if(it!=masses.end())return it->second;
      double w=0;for(int i=0;i<n;++i)if(mask>>i&1)w+=m.weights[i];masses[mask]=w;return w;};
25   auto terminal=[&](uint64_t mask,int at,int context){auto key=std::make_tuple(mask,at,context);
      auto it=leaves.find(key);if(it!=leaves.end())return it->second;double v=m.terminal_context
      ?m.terminal_context(mask,at,context):m.terminal(mask,at);if(v<0||!std::isfinite(v))throw
      std::invalid_argument("invalid terminal policy cost");leaves[key]=v;return v;};
26   std::function<std::pair<double,int>(uint64_t,int,int,bool,int)>value;
27   value=[&](uint64_t mask,int at,int h,bool tunedNow,int context){
28     auto key=std::make_tuple(mask,at,h,tunedNow,context);auto it=memo.find(key);if(it!=memo.end())
      return it->second;
29     std::pair<double,int>best{terminal(mask,at,context),-1};if(h==0)return best;
30     if(result.expanded>=limit){result.budget_hit=true;return best;result.expanded;
31     double total=mass(mask);
32     for(int j=0;j<a;++j){const auto &act=m.actions[j];std::map<int,uint64_t>groups;
33     double cost=m.travel[at][j]+(act.measure&&!tunedNow?1.0.);
34     for(int i=0;i<n;++i)if(mask>>i&1){cost+=m.weights[i]/total*act.seconds[i];groups[act.outcome
      [i]]|=uint64_t(1)<<i;}}
35     // No state reduction and no terminal success: extra action is uninformative.
36     if(!m.advance&&groups.size()==1&&groups.begin()->first>=0)continue;
37     for(auto [out,sub]:groups)if(out>=0)cost+=mass(sub)/total*value(sub,j,h-1,tunedNow|act.
      measure,m.advance?m.advance(context,j,out):0).first;
38     if(at==a&&h==depth)result.action_values[j]=cost;
39     if(cost<best.first-1e-10)best={cost,j};
40   }
41   memo[key]=best;return best;
42 };
43 auto best=value((uint64_t(1)<<n)-1,a,depth,tuned,0);result.value=best.first;result.action=best
      .second;result.exact=!result.budget_hit;return result;
44 }
45 }

```

## src/benchmark.cpp

```

1 #include "simulator.hpp"
2 #include "configurations.hpp"
3 #include "joint_config.hpp"
4 #include "shortcut_sensor.hpp"
5 #include <chrono>
6 #include <iostream>
7 Options benchmark_options(int method,int problem){return joint_configuration(method,problem);}

```

```

8 int main(int argc, char**argv){
9   if(argc<4){std::cerr<<"benchmark OUTPUT START COUNT [STRESS] [LAST_METHOD]\n";return 2;}
10  int start=atoi(argv[2]),count=atoi(argv[3]),stress=argc>4?atoi(argv[4]):0,last=argc>5?atoi(
    argv[5]):6;
11  int first=argc>6?atoi(argv[6]):0;
12  if(count<1||start<1||first<0||last<first||last>86||stress<0||stress>5)return 2;
13  std::ofstream f(argv[1]);if(!f)return 2;
14  f<<"problem,seed,method,stress,n,cleared,time_s,avg_s,move_m,measures,switches,miss,fallbacks,
    dp_calls,dp_actions,dp_fallbacks,dp_expanded,dp_budget_hits,aux_calls,aux_positive,
    joint_steps,runtime_s,certificate,shortcuts,shortcut_saved_m,residual_calls,
    residual_replaced,residual_retired,rollout_calls,rollout_replaced\n";
15  for(int prob:{3,4})for(int seed=start;seed<start+count;++seed){auto environment=make_case(prob
    ,seed,stress);
16    for(int method=first;method<=last;++method){auto options=benchmark_options(method,prob);Sim
        sim(environment,seed,stress==2?2:stress==4?3:stress==5?4:0);ShortcutSensor sensor(sim,
            options.shortcut&&options.eta==0);Policy bot(sensor,prob,options);
17    auto begin=std::chrono::steady_clock::now();
18    try{bot.run();if(sim.success!=sim.total())throw std::runtime_error("incomplete");if(bot.
        dynamic_actions>options.dynamic_limit*sim.total())throw std::runtime_error("dynamic action
            limit");
19    double cost=sim.length/5+sim.switches+5*sim.measures+3*sim.miss+5*sim.success;if(std::abs(
        sim.time-cost)>.001)throw std::runtime_error("accounting");}
20    catch(const std::exception&e){std::cerr<<"FAIL problem="<<prob<<" seed="<<seed<<" method="<<
        method<<" "<<e.what()<<std::endl;return 1;}
21    double seconds=std::chrono::duration<double>(std::chrono::steady_clock::now()-begin).count()
        ;
22    f<<std::setprecision(13)<<prob<<","<<seed<<","<<method<<","<<stress<<","<<sim.total()<<","<<
        sim.success<<","<<sim.time<<","<<sim.time/sim.total()<<","<<sim.length<<","<<sim.measures
        <<","<<sim.switches<<","<<sim.miss<<","<<bot.fallbacks<<","<<bot.dynamic_plans<<","<<bot.
        dynamic_actions<<","<<bot.dynamic_fallbacks<<","<<bot.dp_expanded<<","<<bot.dp_budget_hits
        <<","<<bot.auxiliary_calls<<","<<bot.auxiliary_positive<<","<<bot.joint_steps<<","<<
        seconds<<","<<bot.certificate<<","<<sensor.shortcuts<<","<<sensor.saved_distance()<<","<<
        bot.residual_calls<<","<<bot.residual_replaced<<","<<bot.residual_retired<<","<<bot.
        rollout_calls<<","<<bot.rollout_replaced<<"\n";
23  }
24  if((seed-start+1)%10==0){f.flush();std::cout<<"p="<<prob<<" cases="<<seed-start+1<<std::endl
        ;}
25  }
26  }

```

## src/bridge.py

```

1  """Serial HTTP transport only. Model and decisions reside in robot.exe."""
2  import argparse
3  import json
4  import queue
5  import subprocess
6  import sys
7  import threading
8  import time
9  from pathlib import Path
10
11  ROOT = Path(__file__).resolve().parents[1]
12  sys.path.insert(0, str(ROOT / 'vendor'))
13  from mocksim.client import RobotClient, Rejected, TransportFailure, ProtocolError
14
15

```

```

16 def _until(fn, deadline_at):
17     """Bound the whole operation, including a stalled HTTP body or child pipe.
18
19     A timed-out worker may still be receiving its old response. The caller must
20     abort the run without sending any new action (including /exit) in that case.
21     """
22     remaining = deadline_at - time.perf_counter()
23     if remaining <= 0:
24         raise TimeoutError('real deadline: incomplete run')
25     result = queue.Queue(maxsize=1)
26     def work():
27         try:
28             result.put((True, fn()))
29         except Exception as exc:
30             result.put((False, exc))
31     threading.Thread(target=work, daemon=True).start()
32     try:
33         ok, value = result.get(timeout=max(0, deadline_at-time.perf_counter()))
34     except queue.Empty:
35         raise TimeoutError('real deadline during operation: incomplete run') from None
36     if not ok:
37         raise value
38     return value
39
40
41 class DeadlineClient(RobotClient):
42     """Reuse original serialization/logging, but cap every attempt and retry.
43
44     New IDs are still generated by RobotClient action methods. All retries call
45     its post() with the identical payload. Only one request is in flight.
46     """
47     deadline_at = float('inf')
48
49     def __init__(self, **kwargs):
50         super().__init__(**kwargs)
51         self.retry_limit = self.retries
52         self.retries = 0 # Remain zero even if an old worker outlives its deadline.
53         self.aborted = False
54
55     def _position_action(self, kind, x, y, channel):
56         # Preserve the exact binary64 coordinate received from robot stdout.
57         # Original RobotClient._num rounds to 1e-6 m, which can invalidate a
58         # closed angular boundary in a strict set-membership calculation.
59         payload = self._base(kind)
60         payload.update(position={'x':float(x), 'y':float(y)}, channel=int(channel))
61         return self.post('/'+kind, payload)
62
63     def measure(self, x, y, channel):
64         return self._position_action('measure', x, y, channel)
65
66     def clear(self, x, y, channel):
67         return self._position_action('clear', x, y, channel)
68
69     def post(self, path, payload):
70         if self.aborted:
71             raise TimeoutError('run already aborted with an uncertain request')
72         timeout, retry_limit = self.timeout_s, self.retry_limit
73         try:

```

```

74         for attempt in range(retry_limit + 1):
75             remaining = self.deadline_at-time.perf_counter()
76             if remaining <= 0:
77                 raise TimeoutError('real deadline before request: incomplete run')
78             self.timeout_s = min(timeout, remaining)
79             try:
80                 if self.deadline_at == float('inf'):
81                     response = super().post(path, payload)
82                 else:
83                     response = _until(lambda: super(DeadlineClient, self).post(path,
payload), self.deadline_at)
84                     status = self.log[-1]['http_status']
85                     if status != 200:
86                         raise ProtocolError(status, response)
87                     return response
88             except TransportFailure:
89                 if attempt == retry_limit:
90                     raise
91                 delay = self.retry_backoff_s*(attempt+1)
92                 remaining = self.deadline_at-time.perf_counter()
93                 if remaining <= delay:
94                     raise TimeoutError('real deadline before retry: incomplete run') from
None
95                 self.stats.retries += 1
96                 time.sleep(delay)
97             except TimeoutError:
98                 self.aborted = True
99                 raise
100         finally:
101             self.timeout_s = timeout
102
103
104 def drive(problem, client, early=35, deadline=1200, position_error=0.0, planner_depth=2,
strategy='joint'):
105     if strategy not in ('geometric', 'joint', 'previous', 'stage1', 'stage2', 'baseline'):
106         raise ValueError('strategy must be geometric, joint, previous, stage1, stage2 or
baseline')
107     if type(planner_depth) is not int or not 0 <= planner_depth <= 3:
108         raise ValueError('planner depth must be an integer in [0,3]')
109     if not 0 <= position_error <= .4:
110         raise ValueError('position error must be in [0, 0.4] metres')
111     start = time.perf_counter()
112     hard_stop = start + deadline
113     def call(fn, cutoff):
114         if isinstance(client, DeadlineClient):
115             client.deadline_at = cutoff
116             return fn()
117         return _until(fn, cutoff)
118     enter = call(client.enter, hard_stop)
119     if enter.get('accepted') is not True:
120         raise Rejected(enter)
121     # Start-based accounting is conservative about time spent receiving /enter.
122     hard_stop = start + min(deadline, enter['remaining_real_duration_s'])
123     remaining = hard_stop-time.perf_counter()
124     if remaining <= 0:
125         raise TimeoutError('no real duration remains after enter: incomplete run')
126     action_stop = hard_stop-min(2.0, remaining*.1)

```

```

127 proc = subprocess.Popen([str(ROOT / 'robot.exe'), str(problem), str(early), str(
    position_error), str(planner_depth), strategy],
128                         stdin=subprocess.PIPE, stdout=subprocess.PIPE,
129                         stderr=subprocess.PIPE, text=True, bufsize=1)
130 virtual, final = 0, None
131 lines = queue.Queue()
132 errors = []
133 def read_output():
134     try:
135         for line in proc.stdout:
136             lines.put(line)
137     finally:
138         lines.put(None)
139 def read_errors():
140     for line in proc.stderr:
141         errors.append(line)
142 reader = threading.Thread(target=read_output, daemon=True)
143 err_reader = threading.Thread(target=read_errors, daemon=True)
144 reader.start(); err_reader.start()
145 try:
146     while True:
147         # Safe /exit only between completed actions, never after an uncertain
148         # timed-out request, which might still be registered on the server.
149         remaining = action_stop-time.perf_counter()
150         if remaining <= 0:
151             final = call(client.exit, hard_stop)
152             raise TimeoutError('real deadline guard: incomplete run')
153         try:
154             line = lines.get(timeout=remaining)
155         except queue.Empty:
156             final = call(client.exit, hard_stop)
157             raise TimeoutError('solver output deadline: incomplete run') from None
158         if line is None:
159             raise RuntimeError('solver exited without certificate: ' + ''.join(errors))
160         if virtual >= 359900:
161             final = call(client.exit, hard_stop)
162             raise TimeoutError('virtual deadline guard: incomplete run')
163         fields = line.split()
164         if fields[0] == 'exit':
165             final = call(client.exit, hard_stop)
166             if final.get('accepted') is not True:
167                 raise Rejected(final)
168             return dict(cleared=int(fields[1]), certificate=fields[2],
169                       planner_depth=planner_depth if position_error == 0 else 0,
170                       strategy=strategy if position_error==0 and planner_depth>0 else '
legacy',
171                       virtual_time_s=final['virtual_time_s'],
172                       runtime_s=time.perf_counter()-start, response=final)
173         action, channel, x, y = fields
174         if action not in ('measure', 'clear'):
175             raise ValueError('unknown solver action')
176         response = call(lambda: getattr(client, action)(float(x), float(y), int(channel)),
action_stop)
177         if response.get('accepted') is not True:
178             raise Rejected(response)
179         virtual = response['virtual_time_s']
180         if action == 'measure':
181             kind = {'no_signal': 0, 'near': 1, 'direction': 2}[response['measure_result']]

```

```

182         reply = f"{kind} {response.get('svd_deg', 0)}\n"
183     else:
184         reply = f"{int(response['clear_result'] == 'success')}\n"
185     proc.stdin.write(reply)
186     proc.stdin.flush()
187 finally:
188     if proc.poll() is None:
189         proc.terminate()
190     proc.wait(timeout=5)
191     reader.join(timeout=1)
192     err_reader.join(timeout=1)
193     for stream in (proc.stdin, proc.stdout, proc.stderr):
194         stream.close()
195
196
197 def main():
198     p = argparse.ArgumentParser()
199     p.add_argument('--problem', type=int, choices=[3,4], required=True)
200     p.add_argument('--team-id', required=True)
201     p.add_argument('--url', default='http://127.0.0.1:2026')
202     p.add_argument('--log', required=True)
203     p.add_argument('--early', type=float, default=35)
204     p.add_argument('--position-error', type=float, default=0.0)
205     p.add_argument('--strategy', choices=['geometric', 'joint', 'previous', 'stage1', 'stage2', 'baseline'], default='joint')
206     p.add_argument('--planner-depth', type=int, choices=[0,1,2,3], default=2,
207                    help='0: previous controller; 1/2/3: receding-horizon decision depth (
208                        default 2)')
209     a = p.parse_args()
210     # This program never launches or purchases a formal test session.
211     with open(a.log, 'x', encoding='utf-8') as f:
212         def log(row):
213             f.write(json.dumps(row, ensure_ascii=False) + '\n')
214             f.flush()
215         client = DeadlineClient(base_url=a.url, robot_id=a.team_id, log_sink=log)
216         result = drive(a.problem, client, a.early, position_error=a.position_error,
217                       planner_depth=a.planner_depth, strategy=a.strategy)
218         print(json.dumps(result, ensure_ascii=False, indent=2))
219
220 if __name__ == '__main__':
221     main()

```

## src/build\_ledger.py

```

1 """Audit delivered execution records and bind paper numbers to machine artifacts."""
2 import csv,hashlib,json,math,platform,statistics,sys,time
3 from pathlib import Path
4 R=Path(__file__).resolve().parents[1]
5 def get(p):return json.loads((R/p).read_text())
6 def put(p,v):(R/p).write_text(json.dumps(v,ensure_ascii=False,indent=2))
7 def sha(p):return hashlib.sha256((R/p).read_bytes()).hexdigest()
8 def rows(p):return list(csv.DictReader((R/p).open()))
9 S=get('artifacts/summary.json');F=get('validation_frozen.json')
10 files=['results/dev56.csv']+['results/dev{m}.csv' if m!=81 else 'results/dev81b.csv' for m in
11     [79,80,81,82,83,84,85,86]]
11 files +=['results/iid{m}.csv' for m in [56,78,84]]+['results/stress{g}_{m}.csv' for g in
12     range(1,6) for m in [56,84]]

```

```

12 record=[];max_residual=0;unique=set();seeds={k:set() for k in ['development','iid','stress']}
13 for f in files:
14 a=rows(f);keys=set();group='development' if 'dev' in f else 'iid' if 'iid' in f else 'stress'
15 for z in a:
16 key=(int(z['problem']),int(z['seed']));assert key not in keys;keys.add(key);seeds[group].add(
    key[1])
17 assert z['n']==z['cleared'] and 10<=int(z['n'])<=16
18 assert all(v!='' for v in z.values())
19 for k,v in z.items():
20 if k!='certificate':assert math.isfinite(float(v))
21 err=abs(float(z['time_s'])-(float(z['move_m'])/5+float(z['switches'])+5*float(z['measures'])
    +3*float(z['miss'])+5*float(z['cleared'])))
22 max_residual=max(max_residual,err);assert err<.001
23 if group!='development':unique.add(key)
24 record.append({'path':f,'sha256':sha(f),'rows':len(a),'group':group,'all_cleared':True})
25 assert all(not seeds[a]&seeds[b] for a,b in [('development','iid'),('development','stress'),('
    iid','stress']))
26 for suite in ['iid','stress']:
27 e=get(f'artifacts/execution-{suite}.json')
28 for z in e['records']:assert z['exit_code']==0 and sha(z['output'])==z['output_sha256']
29 assert get('artifacts/execution-iid.json')['binary_sha256']==sha('bin/benchmark_final')
30 a=rows('results/baseline_native56.csv');b=rows('results/dev56.csv')
31 assert len(a)==len(b)==80
32 for x,y in zip(a,b):
33 for k in ['problem','seed','n','cleared','time_s','move_m','measures','switches','miss']:
    assert abs(float(x[k])-float(y[k]))<.001
34 http={}
35 for strategy in ['joint','geometric']:
36 data=get(f'results/original_mock_{strategy}_depth2.json');assert len(data)==16 and all(z.get('
    cleared_count',z.get('cleared'))==z.get('jammer_total',z.get('total')) for z in data)
37 http[strategy]={ 'all_cleared':True,'kernel_cases':sum(z['layer']=='original_kernel' for z in
    data),'http_cases':sum(z['layer']!='original_kernel' for z in data)}
38 reg=get('artifacts/regression-tests.json');assert reg['status']=='PASS' and len(reg['records'])
    ==11
39 witness=get('artifacts/residual-fraction-replay.json');assert witness['status']=='PASS' and
    witness['witness_sha256']==sha('artifacts/residual-witness.json')==reg['records'][0]['
    fresh_witness_sha256']
40 checks=[{'name':n,'status':'pass','evidence':e} for n,e in [
41 ('schema_missing_finiteness_duplicates','Every reported CSV row has a unique problem/seed key
    within its method file and finite complete fields.'),
42 ('units_and_accounting',f'Seconds/metres identity checked, max residual {max_residual:.10g} s
    < 0.001.'),
43 ('seed_separation','Development, IID and pressure seed sets are disjoint.'),
44 ('execution_hashes','All frozen IID/pressure output hashes and benchmark binary match executed
    records.'),
45 ('baseline_native_replay','80 rows of recompiled unmodified original baseline match current
    method56.'),
46 ('coverage_and_completion','All included local runs clear their actual source count; six
    official formal runs completed with user_exit and zero HTTP errors.'),
47 ('outliers_retained','No performance outlier or negative pair was discarded.'),
48 ('certificate_freshness','Producer hash and newly generated witness hash match the Fraction
    consumer.'))]
49 put('artifacts/data-audit.json',{'schema_version':1,'status':'completed','checks':checks,'files
    ':record,'max_time_identity_residual_s':max_residual,'independent_unique_environments':len(
    unique),'independent_method_runs':sum(z['rows'] for z in record if z['group']!='
    development'),'iid_cases_per_question':200,'stress_cases_per_group_question':40,'
    development_cases_per_question':40,'http':http,'regression_programs':len(reg['records']),
    'limitations':['Official formal tests do not disclose jammer counts or sampling

```

```

distribution.', 'IID source positions use 40 m rejection spacing; this is not arbitrary
official distribution.', 'Same seed labels are not cross-platform environment identities;
historical CSV not pooled.', 'All clear in finite local tests is not by itself a continuous
proof.'])
50 ids=['q3_total_mean_s', 'q4_total_mean_s', 'q3_per_source_mean_s', 'q4_per_source_mean_s']
51 base=dict(zip(ids, [S['iid']['3']['vs56']['baseline_mean_s'], S['iid']['4']['vs56']['
baseline_mean_s'], S['iid']['3']['vs56']['baseline_source_s'], S['iid']['4']['vs56']['
baseline_source_s']]))
52 selected=dict(zip(ids, [S['iid']['3']['vs78']['baseline_mean_s'], S['iid']['4']['vs78']['
baseline_mean_s'], S['iid']['3']['vs78']['baseline_source_s'], S['iid']['4']['vs78']['
baseline_source_s']]))
53 put('artifacts/baseline-metrics.json', {'method': 56, **base, 'source': 'results/iid56.csv'})
54 put('artifacts/metrics.json', {'method': 78, **selected, 'source': 'results/iid78.csv',
experimental84': {q: {k: v for k, v in S['iid'][q]['vs56'].items() if k != 'pairs'} for q in
['3', '4']}}))
55 # Register all summary fields except individual pairs (kept in the raw artifact).
56 entries=[]
57 def walk(obj, source, path='', prefix=''):
58     if isinstance(obj, dict):
59         for k, v in obj.items():
60             if k not in {'pairs', 'leaves', 'pi_lower', 'exact_area_upper', 'command', 'stdout', 'stderr',
runtime_s', 'source', 'output_sha256'}: walk(v, source, path+'/'+k.replace('~', '~0').replace
('/', '~1'), prefix+'.'+k)
61     elif isinstance(obj, list):
62         for i, v in enumerate(obj): walk(v, source, path+'/'+str(i), prefix+'.'+str(i))
63     elif isinstance(obj, (int, float, bool)) or (isinstance(obj, str) and any(c.isdigit() for c in obj
) and len(obj)<80):
64         entries.append({'id': prefix.lstrip('.'), 'value': obj, 'source_artifact': source, 'source_path':
path, 'status': 'verified'})
65 for f, prefix in [('artifacts/summary.json', 'experiment'), ('artifacts/q2-certificate.json', 'q2'),
('artifacts/q4-lower-bound.json', 'q4lower'), ('artifacts/geometry-tests.json', 'geometry'),
('artifacts/residual-fraction-replay.json', 'fraction'), ('results/q3_coverage_bound.json',
'q3cover'), ('artifacts/baseline-metrics.json', 'baseline'), ('artifacts/metrics.json',
recommended')]: walk(get(f), f, prefix=prefix)
66 put('artifacts/result-registry.json', {'schema_version': 1, 'results': entries})
67 put('paper/result-claims.json', {'schema_version': 1, 'note': 'Exact source values are recorded
here; displayed tables round to the stated precision. Model dimensions, thresholds and
analytic constants are in the supplied statement, frozen configuration or proof scripts.
Entries cover paper, solution and numeric table dependencies.', 'claims': [{ 'result_id': z[
'id'], 'value': z['value'], 'location': 'paper/main.tex and generated tables; B题几何策略优化_题
解.md'} for z in entries]})
68 m=get('modeling-manifest.json'); m['project'].update(stage='delivery', status='completed')
69 inputs=['inputs/B题.pdf', 'inputs/论文格式规范.pdf']; ih={p: sha(p) for p in inputs}
70 m['source'].update(statement_files=[inputs[0]], data_files=[inputs[1]], input_hashes=ih)
71 m['source']['official_rules']={'status': 'verified', 'url': 'inputs/论文格式规范.pdf', 'version':
用户随题提供的2026竞赛题面与论文格式文件，按SHA256绑定', 'reason': 'Read supplied PDFs
directly; six official formal records are tracked separately from local experiments.'}
72 m['framing'].update(objectives=['在全部源清除约束下最小化配对总任务时间，并报告题设每源均时。'],
decision_variables=['下一测向/清除/发现点的连续位置', '未知频道扫描集合', '定位与发现任务次序'],
constraints=['场地半径1800米，源数10至16，独占1至20频道', '接收半径1000至1500米，Q4未知闭半平
面方向', '角误差正负1度；本地0.01度量化保守外包', '速度5米每秒，换频1秒，测量5秒，光学失败3秒成功
5秒', '光学20米清除；发现布局修改必须通过连续证书'], ambiguities=['官方正式案例不公开干扰源总数
和抽样分布', '不以历史跨平台CSV当作当前种子场景'], assumptions=['理论几何取题设精确位置；生产浮
点几何边界明确披露', '统计结论限于已记录的40米排斥本地场景分布', '续跑为有限固定端点误差的单源模
型'])
73 families=[('默认联合控制78', '有限动作树和路由；有界后备清除', '保留原控制器，安全提前停车', '已有有限
叶端启发式与浮点误差', '原方法56配对、全清除与路程不变量'), ('最坏半径与最近保证区79/80', '角锥交
的最坏MEC及最近可行点', '连续读数外包和强接收充分域', '半径不等于剩余时间；完整首扇区35米域空',

```

开发40局退步、Q2有理下界'),('剩余覆盖候选84','覆盖证书下连续发现点替换','历史完整扫描与未来点联合凸包覆盖','局部可行位移小,整点未能删除','冻结IID与五组压力、独立Fraction复发'),('真实单源续跑82','同控制器执行到清除的首动作比较','有限源假设及固定符号误差','非完整多源树且不复用完整误差场','开发消融与真实光学终止'),('光学邻域巡游83/86','覆盖子区域清除域中的路径','全部顶点20米交集','全程更短未必更早命中真实目标','开发配对及子块覆盖检查')]

```

74 m['models']['candidates']=[{'name':n,'target_output':o,'assumptions':[a],'data_requirements':['
    实时传感器历史及保守多边形; 比较用本地生成场景'],'failure_mode':f,'diagnostic':d} for n,o,a,f
    ,d in families]
75 m['models']['selected']=families[0][0];m['models']['selection_rationale']='{objective_fit':'直接
    比较总任务秒数; 不以FIM或半径代理冒充最终目标','constraint_fit':'发现与光学覆盖保留几何证书
    ','data_fit':'预冻结独立本地场景, 开发不合并','baseline':'在同一编译器环境重跑56和既有78','
    risks':['Q4候选84对78收益区间跨零','官方案例真值与抽样分布不公开','一般浮点裁剪未形式验证'],'
    fallback_trigger':'预声明95%均值节省区间不全为正则不推广84; 默认保留78'}
76 e=get('artifacts/execution-iid.json');b=e['records'][0];c=e['records'][1]
77 command='python3 src/run_experiments.py --suite iid --workers 3'
78 m['models']['baseline'].update(status='completed',name='method56',command=' '.join(b['command
    ']),exit_code=b['exit_code'],runtime_seconds=b['runtime_s'],metric_ids=ids)
79 figures=[f'figures/{x}.pdf' for x in ['residual_geometry','paired_results','cost_components']]
80 m['execution'].update(status='completed',command=command,exit_code=0,runtime_seconds=max(z['
    started_unix']+z['runtime_s'] for z in e['records'])-min(z['started_unix'] for z in e['
    records']),environment={'platform':e['environment'],'compiler':'clang++ 21, -O2 -std=c
    ++17, no fast-math','python':sys.version,'benchmark_sha256':e['binary_sha256']},seed={'iid
    ':F['iid_start'],'development':F['development_seeds'],'stress':F['stress_starts']},
    metric_ids=ids,figures=figures)
81 m['validation'].update(status='completed',checks=checks,sensitivity=[{'artifact':'artifacts/
    summary.json','scope':'Development79-86 vary objective, rollout and optical/residual
    modules; no parameter tuning on IID.'}],robustness=[{'artifact':'artifacts/execution-
    stress.json','scope':'Five groups x 40 per question; all clear, losses retained.'}],
    falsification=[{'artifact':'artifacts/summary.json','result':'Radius-only79 strongly
    worsens Q3; candidate84 CI crosses zero.'},{ 'artifact':'artifacts/regression-tests.json','
    result':'Deleting an essential site is rejected; invalid witness output must fail.'}])
82 m['results']['paper_file']='B题几何策略优化_论文.pdf'
83 reviewers='/root/judge_paper; /root/judge_evidence; /root/judge_math'
84 m['review'].update(status='completed',independent=True,reviewer_id=reviewers)
85 m['pending']=[]
86 m['violations']=[]
87 put('modeling-manifest.json',m)
88 outputs=['artifacts/data-audit.json','artifacts/baseline-metrics.json','artifacts/metrics.json
    ','artifacts/result-registry.json','paper/result-claims.json',*figures,'B题几何策略优化_论
    文.pdf','B题几何策略优化_题解.pdf']+z['path'] for z in record]
89 put('artifacts/reproducibility.json',{'schema_version':1,'command':command,'seed':m['execution
    ']['seed'],'runtime':m['execution']['environment'],'input_hashes':ih,'output_hashes':{p:
    sha(p) for p in outputs},'execution_records':[ 'artifacts/execution-iid.json','artifacts/
    execution-stress.json','artifacts/reproduction-log.json'],'rerun_entry':'python3 run_all.
    py --full','limitations':m['pending']})
90 print(json.dumps({'status':'AUDIT_PASS_WITH_SIX_OFFICIAL_RUNS','csv_files':len(record),'
    independent_runs':sum(z['rows'] for z in record if z['group']!='development'),'
    independent_environments':len(unique),'registered_results':len(entries)},ensure_ascii=
    False))

```

## src/build\_overview\_figure.py

```

1 """Render the set-state/action/termination overview with mathematical labels."""
2 from pathlib import Path
3 import matplotlib.pyplot as plt
4 from matplotlib.patches import FancyBboxPatch
5 R = Path(__file__).resolve().parents[1]

```

```

6 from matplotlib import font_manager
7 available_fonts = {font.name for font in font_manager.fontManager.ttflist}
8 figure_font = next((name for name in ['Heiti TC', 'Microsoft YaHei',
9     'Noto Sans CJK SC', 'SimHei', 'FandolHei'] if name in available_fonts), None)
10 if figure_font is None:
11     import subprocess
12     font_file = subprocess.check_output(['kpsewhich', 'FandolHei-Regular.otf'],
13         text=True).strip()
14     if not font_file:
15         raise RuntimeError('Install a Chinese font or TeX Fandol fonts.')
16     font_manager.fontManager.addfont(font_file)
17     figure_font = font_manager.FontProperties(fname=font_file).get_name()
18
19 plt.rcParams.update({'font.family':figure_font,'font.size':10,
20     'axes.unicode_minus':False,'pdf.fonttype':42})
21 fig, ax = plt.subplots(figsize=(8.4, 5.6))
22 fig.subplots_adjust(left=0,right=1,bottom=0,top=1)
23 ax.set(xlim=(0,100),ylim=(0,100)); ax.axis('off')
24 rows = [
25     (70, '集合状态',
26         r'位置多边形  $P_k$  与历史响应'+'\n'+
27         r'正常测向:  $P_k \leftarrow P_k \cap W(s, \theta)$ '+'\n'+
28         r'发现覆盖: 历史扫描点  $H_t$  与未来点  $F_t$ ',
29         '角锥交与凸性\n有理区间认证', '#edf4fa'),
30     (38, '动作与时间目标',
31         r' $T=L/5+N_s+5N_m+3N_f+5N_c$ '+'\n'+
32         '有限动作规划、路径调度与安全提前停车\n观测后更新集合, 再选择下一动作',
33         '计时恒等分解\n同种子配对实验', '#fcf2e5'),
34     (6, '发现完成与清除终止',
35         r'保证清除:  $\rho(P) \leq 20$  米, 清除点属于  $\mathcal{C}(P)$ '+'\n'+
36         '完成规定扫描或已发现16源后结束发现\n已知源全部清除后退出; 光学覆盖提供后备',
37         '连续覆盖充分条件\n逐局完成性检查', '#edf6ef')]
38 for y,title,body,evidence,color in rows:
39     for x,w,c in [(5,66,color),(76,22,'#f4f4f4')]:
40         ax.add_patch(FancyBboxPatch((x,y),w,26,boxstyle='round,pad=.4',
41             facecolor=c,edgecolor='#63767c',linewidth=.9))
42         ax.text(8,y+22,title,weight='bold',va='top',fontsize=12)
43         ax.text(8,y+15,body,va='top',linespacing=1.65,fontsize=10)
44         ax.text(78,y+22,'论证依据',va='top',weight='bold',fontsize=11)
45         ax.text(78,y+14,evidence,va='top',linespacing=1.9,fontsize=10)
46         ax.annotate('',(75.5,y+13),(71.5,y+13),arrowprops={'arrowstyle':'->','color': '#63767c'})
47 for upper,lower in [(70,64),(38,32)]:
48     ax.annotate('',(38,lower+.4),(38,upper-.4),arrowprops={'arrowstyle':'->','color': '#63767c'})
49 for extension in ['pdf','png']:
50     fig.savefig(R/'figures'/f'system_overview.{extension}',dpi=180)
51 plt.close(fig)

```

## src/build\_paper\_assets.py

```

1 """Rebuild figures and full source listing; preserve reviewed manuscript tables.
2
3 Numerical inputs are frozen CSVs. This entry does not select models or rerun
4 experiments. Historical table generators must not overwrite reviewed labels.
5 """
6 from pathlib import Path
7 import subprocess
8 import sys

```

```

9 R = Path(__file__).resolve().parents[1]
10 for script in ['build_overview_figure.py', 'build_readable_figures.py',
11               'build_small_figures.py', 'build_source_appendix.py']:
12     subprocess.run([sys.executable, str(R / 'src' / script)], cwd=R, check=True)

```

## src/build\_readable\_figures.py

```

1 """Regenerate the two remaining small-font figures at a legible effective size.
2
3 * fig: paired_results      -- saved by src/build_paper_assets.py at figsize=(10,3.5)
4 * fig: count_time_relation -- saved by supplement/count_time/src/build_count_report.py at
   (10,4.4)
5
6 Both are drawn on a ~25.4 cm canvas but included at <=14.2 cm, i.e. ~56 % scale, so a
7 10-11 pt base font lands at ~5.6 pt on the page. We redraw them on a 16.51 cm (6.5 in)
8 canvas with >=11 pt fonts, giving >=8 pt on the page.
9 """
10 import json
11 from pathlib import Path
12
13 import math
14
15 import matplotlib
16 matplotlib.use("Agg")
17 import matplotlib.pyplot as plt
18 import numpy as np
19
20
21 def t_ppf(p, df):
22     """Student-t quantile via bisection on the regularised incomplete beta function
23     (avoids a scipy dependency for figure regeneration)."""
24     def betainc(a, b, x, n=4000):
25         # continued fraction (Lentz) for the regularised incomplete beta
26         if x <= 0:
27             return 0.0
28         if x >= 1:
29             return 1.0
30         lbeta = math.lgamma(a) + math.lgamma(b) - math.lgamma(a + b)
31         front = math.exp(math.log(x) * a + math.log(1 - x) * b - lbeta) / a
32         f, c, d = 1.0, 1.0, 0.0
33         for i in range(n):
34             m = i // 2
35             if i == 0:
36                 num = 1.0
37             elif i % 2 == 0:
38                 num = (m * (b - m) * x) / ((a + 2 * m - 1) * (a + 2 * m))
39             else:
40                 num = -((a + m) * (a + b + m) * x) / ((a + 2 * m) * (a + 2 * m + 1))
41             d = 1.0 + num * d
42             d = 1e-30 if abs(d) < 1e-30 else d
43             d = 1.0 / d
44             c = 1.0 + num / c
45             c = 1e-30 if abs(c) < 1e-30 else c
46             f *= c * d
47             if abs(1 - c * d) < 1e-14:
48                 break
49     return front * (f - 1)

```

```

50
51 def cdf(x):
52     ib = betainc(df / 2.0, 0.5, df / (df + x * x)) / 2.0
53     return 1 - ib if x > 0 else ib
54
55 lo, hi = 0.0, 100.0
56 for _ in range(200):
57     mid = (lo + hi) / 2
58     if cdf(mid) < p:
59         lo = mid
60     else:
61         hi = mid
62 return (lo + hi) / 2
63
64
65 R = Path(__file__).resolve().parents[1]
66
67 from matplotlib import font_manager
68 available_fonts = {font.name for font in font_manager.fontManager.ttflist}
69 figure_font = next((name for name in ['Heiti TC', 'Microsoft YaHei',
70     'Noto Sans CJK SC', 'SimHei', 'FandolHei'] if name in available_fonts), None)
71 if figure_font is None:
72     import subprocess
73     font_file = subprocess.check_output(['kpsewhich', 'FandolHei-Regular.otf'],
74         text=True).strip()
75     if not font_file:
76         raise RuntimeError('Install a Chinese font or TeX Fandol fonts.')
77     font_manager.fontManager.addfont(font_file)
78     figure_font = font_manager.FontProperties(fname=font_file).get_name()
79
80 plt.rcParams.update({
81     "font.family": figure_font,
82     "font.size": 12,
83     "axes.unicode_minus": False,
84     "axes.spines.top": False,
85     "axes.spines.right": False,
86     "pdf.fonttype": 42,
87 })
88
89 # ----- paired_results
90 S = json.loads((R / "artifacts/summary.json").read_text(encoding="utf-8"))
91 fig, axes = plt.subplots(1, 2, figsize=(6.5, 2.5))
92 for ax, q in zip(axes, ["3", "4"]):
93     pairs = S["iid"][q]["vs56"]["pairs"]
94     x = np.arange(1, len(pairs) + 1)
95     y = np.array([v["saving_s"] for v in pairs])
96     ax.scatter(x, y, s=6, c=np.where(y >= 0, "#187b7d", "#b64635"), alpha=.8)
97     ax.axhline(0, color="#222", lw=.7)
98     ax.set(xlabel="配对案例序号", ylabel="相对方法56节省 (秒)", title="Q" + q)
99     ax.title.set_fontsize(13)
100    ax.xaxis.label.set_fontsize(12)
101    ax.yaxis.label.set_fontsize(12)
102    ax.tick_params(labelsize=11)
103    ax.grid(alpha=.15)
104 fig.tight_layout(pad=0.6)
105 fig.savefig(R / "figures/paired_results.pdf")
106 fig.savefig(R / "figures/paired_results.png", dpi=200)
107 plt.close(fig)

```

```

108 print("paired_results regenerated")
109
110 # ----- count_time_relation
111 D = json.loads((R / "supplement/count_time/artifacts/count_time_results.json").read_text(
    encoding="utf-8"))
112 Q = D["Q"]
113 fig, axes = plt.subplots(1, 2, figsize=(6.5, 3.0))
114 handles = []
115 for ax, q in zip(axes, ["3", "4"]):
116     d = Q[q]
117     f = d["models"]["affine_step16"]
118     beta = np.array(f["coefficients"])
119     cov = np.array(f["HC3_covariance"])
120     x = np.linspace(10, 15.99, 150)
121     g = np.column_stack([1 / x, np.ones(len(x)), np.zeros(len(x))])
122     y = g @ beta
123     margin = t_ppf(.975, 197) * np.sqrt(np.einsum("ij,jk,ik->i", g, cov, g))
124     line, = ax.plot(x, y, color="#163e5d", lw=1.6, label="冻结公式")
125     ax.fill_between(x, y - margin, y + margin, color="#163e5d", alpha=.13)
126     point = d["predictions"][-1]
127     ax.plot([16], [point["predicted_avg_s"]], marker="s", color="#163e5d", ms=5)
128     a = d["training_groups"]
129     old = ax.scatter([z["N"] - .05 for z in a], [z["avg_mean_s"] for z in a],
130                    color="#8c979e", marker="x", s=22, label="原样本均值")
131     b = d["validation_groups"]
132     new = ax.errorbar([z["N"] + .05 for z in b], [z["avg_mean_s"] for z in b],
133                     yerr=[z["avg_mean_s"] - z["avg_mean_t95_s"][0] for z in b],
134                           [z["avg_mean_t95_s"][1] - z["avg_mean_s"] for z in b]],
135                     fmt="o", ms=3.5, capsize=2.5, color="#c06927",
136                     label="新样本均值及95%区间")
137     ax.set(xlabel="信号源数量 N", ylabel="单源平均清除时长 (秒)",
138           title="Q3: 全向源 if q == '3' else 'Q4: 全向与定向混合'",
139           xticks=range(10, 17), xlim=(9.7, 16.3))
140     ax.title.set_fontsize(12)
141     ax.xaxis.label.set_fontsize(11)
142     ax.yaxis.label.set_fontsize(11)
143     ax.tick_params(labelsize=10.5)
144     ax.grid(alpha=.16)
145     handles = [line, old, new]
146 fig.legend(handles=handles, loc="lower center", ncol=3, bbox_to_anchor=(.5, .005),
147           frameon=False, fontsize=11)
148 fig.tight_layout(rect=(0, .10, 1, 1), pad=0.6)
149 fig.savefig(R / "supplement/count_time/figures/count_time_relation.pdf")
150 fig.savefig(R / "supplement/count_time/figures/count_time_relation.png", dpi=200)
151 fig.savefig(R / "figures/count_time_relation.pdf")
152 fig.savefig(R / "figures/count_time_relation.png", dpi=200)
153 plt.close(fig)
154 print("count_time_relation regenerated")

```

## src/build\_small\_figures.py

```

1 """Regenerate the two small-font figures at a legible effective size.
2
3 Included at 0.88 and 0.95 of a 14.20 cm text block, so a figure drawn at 22.86 cm
4 wide is scaled to ~55-59 %. Base 10 pt therefore renders at ~5.5 pt. We redraw both
5 figures at 16.51 cm (6.5 in) wide with explicit font sizes so that the *effective*
6 on-page size is >= 6 pt (labels/legends) and >= 7 pt (axis labels, titles).

```

```

7 """
8 import json
9 from pathlib import Path
10
11 import matplotlib
12 matplotlib.use("Agg")
13 import matplotlib.pyplot as plt
14 import numpy as np
15 from matplotlib.patches import Circle, Rectangle
16 from matplotlib.collections import PatchCollection
17
18 R = Path(__file__).resolve().parents[1]
19 S = json.loads((R / "artifacts/summary.json").read_text(encoding="utf-8"))
20 from matplotlib import font_manager
21 available_fonts = {font.name for font in font_manager.fontManager.ttflist}
22 figure_font = next((name for name in ['Heiti TC', 'Microsoft YaHei',
23   'Noto Sans CJK SC', 'SimHei', 'FandolHei'] if name in available_fonts), None)
24 if figure_font is None:
25     import subprocess
26     font_file = subprocess.check_output(['kpsewhich', 'FandolHei-Regular.otf'],
27       text=True).strip()
28     if not font_file:
29         raise RuntimeError('Install a Chinese font or TeX Fandol fonts.')
30     font_manager.fontManager.addfont(font_file)
31     figure_font = font_manager.FontProperties(fname=font_file).get_name()
32
33 plt.rcParams.update({
34     "font.family": figure_font,
35     "font.size": 12,
36     "axes.unicode_minus": False,
37     "axes.spines.top": False,
38     "axes.spines.right": False,
39     "pdf.fonttype": 42,
40 })
41
42 # ---- fig: cost_components -----
43 fig, axes = plt.subplots(1, 2, figsize=(6.5, 2.6))
44 for ax, q in zip(axes, ["3", "4"]):
45     d = S["iid"][q]["vs56"]["decomposition_s"]
46     vals = list(d.values())
47     ax.bar(["走路", "测向", "换频", "失败清除"], vals,
48       color=["#187b7d" if v >= 0 else "#b64635" for v in vals])
49     ax.axhline(0, color="#222", lw=.7)
50     ax.set(ylabel="平均节省 (秒)", title="Q" + q)
51     ax.tick_params(axis="x", labelsz=12)
52     ax.tick_params(axis="y", labelsz=11)
53     ax.title.set_fontsize(13)
54     ax.yaxis.label.set_fontsize(12)
55     ax.grid(axis="y", alpha=.15)
56 fig.tight_layout(pad=0.6)
57 fig.savefig(R / "figures/cost_components.pdf")
58 fig.savefig(R / "figures/cost_components.png", dpi=200)
59 plt.close(fig)
60 print("cost_components regenerated")
61
62 # ---- fig: residual_geometry -----
63 a = json.loads((R / "artifacts/residual-witness.json").read_text(encoding="utf-8"))
64 sites = np.array(a["sites"])

```

```

65 boxes = a["boxes"]
66 fig, axes = plt.subplots(1, 2, figsize=(6.5, 2.9))
67 patches = [Rectangle((x0, y0), x1 - x0, y1 - y0) for x0, y0, x1, y1 in boxes]
68 axes[0].add_collection(PatchCollection(patches, facecolor="#dae7ec",
69                                     edgecolor="#adbdc4", linewidth=.15))
70 axes[0].add_patch(Circle((0, 0), 1800, fill=False, color="#333", lw=1))
71 axes[0].scatter(sites[:, 0], sites[:, 1], s=14, color="#187b7d")
72 axes[0].set(xlim=(-2100, 2100), ylim=(-2100, 2100), xlabel="x (m)", ylabel="y (m)",
73            title="连续覆盖认证分区")
74 axes[0].title.set_fontsize(13)
75 axes[0].xaxis.label.set_fontsize(12)
76 axes[0].yaxis.label.set_fontsize(12)
77 axes[0].tick_params(labelsize=11)
78 axes[0].set_aspect("equal")
79
80 old = np.array([995, 0])
81 q = sites[1]
82 axes[1].plot([old[0], q[0]], [old[1], q[1]], color="#187b7d")
83 axes[1].scatter([old[0]], [old[1]], label="静态点", marker="x", s=70, color="#b64635")
84 axes[1].scatter([q[0]], [q[1]], label="证书通过的替换点", s=55, color="#187b7d")
85 axes[1].set(xlim=(989, 999.5), ylim=(-3.4, 3.4), xlabel="x (m)", ylabel="y (m)",
86            title="可行替换位移3.595米")
87 axes[1].title.set_fontsize(13)
88 axes[1].xaxis.label.set_fontsize(12)
89 axes[1].yaxis.label.set_fontsize(12)
90 axes[1].tick_params(labelsize=11)
91 # legend inside the axes: an outside legend would be clipped by the figure edge
92 axes[1].legend(loc="lower right", fontsize=11, framealpha=.95, borderpad=.35,
93              handletextpad=.4)
94 axes[1].set_aspect("equal")
95 fig.tight_layout(pad=0.6)
96 fig.savefig(R / "figures/residual_geometry.pdf")
97 fig.savefig(R / "figures/residual_geometry.png", dpi=200)
98 plt.close(fig)
99 print("residual_geometry regenerated")

```

## src/build\_source\_appendix.py

```

1 """Regenerate paper/source_appendix.tex from the current source inventory.
2
3 Appendix B lists all shipped source files and writes the shared source count.
4 The build pipeline preserves reviewed numerical tables and their comparisons.
5 """
6 from pathlib import Path
7
8 R = Path(__file__).resolve().parents[1]
9 P = R / 'paper'
10
11 source_roots = [R / 'src', R / 'tests', R / 'review',
12                R / 'supplement/count_time/src', R / 'supplement/count_time_theory/src',
13                R / 'vendor/mocksim']
14 source_ext = {'.py', '.cpp', '.hpp', '.h', '.c', '.cc'}
15
16 files = []
17 for base in source_roots:
18     if base.exists():

```

```

19         files.extend(p for p in base.rglob('*') if p.is_file() and p.suffix.lower() in
        source_ext)
20 files.append(R / 'run_all.py')
21 files = sorted(set(files), key=lambda p: p.relative_to(R).as_posix())
22
23 tex = (r'\section{完整计算源程序}\small 此附录列出在线机器人、实验内核、消融模块、'
24        r'几何证书与附加分析脚本的完整源码；共' + str(len(files)) +
25        r'个文件，与支撑包内文件逐项对应。历史方法分支仅供复核，不表示默认启用。'
26        r'模块标题与支撑包同名。' + '\n')
27 for path in files:
28     rel = path.relative_to(R).as_posix()
29     title = rel.replace('_', r'\_')
30     if rel == 'run_all.py':
31         tex += r'\clearpage' + '\n'
32     tex += (r'\subsection*{\texttt{' + title + r'}}' + '\n' +
33            r'\lstinputlisting[basicstyle=\ttfamily\fontsize{8}{9.5}\selectfont,'
34            r'breaklines=true,breakatwhitespace=false,columns=fullflexible,'
35            r'keepspace=true,numbers=left,numberstyle=\tiny,numbersep=3pt]{../' + rel + '\n}')
36
37 (P / 'source_appendix.tex').write_text(tex, encoding='utf-8')
38 print('source_appendix.tex regenerated with', len(files), 'modules')
39
40 (P / "source_count.tex").write_text(r"\newcommand{\SourceFileCount}{" + str(len(files)) + "}"
    ", encoding="utf-8")

```

## src/certified\_route.hpp

```

1 #pragma once
2 #include <vector>
3 #include <algorithm>
4 #include <cmath>
5 #include <limits>
6 #include <numeric>
7 #include <stdexcept>
8 namespace cert_route {
9 struct Problem{std::vector<double>start;std::vector<std::vector<double>>edge;};
10 struct Solution{std::vector<int>path;double lower=0,upper=0;bool exact=false,local_optimal=
    false;};
11 inline double length(const Problem&p,const std::vector<int>&r){if(r.empty())return 0;double c=p
    .start[r[0]];for(size_t i=1;i<r.size();++i)c+=p.edge[r[i-1]][r[i]];return c;}
12 inline double mst_lower(const Problem&p){int n=p.start.size();std::vector<double>d(n+1,1e100);
    std::vector<bool>done(n+1);d[0]=0;double total=0;
13 for(int k=0;k<=n;++k){int u=-1;for(int i=0;i<=n;++i){if(!done[i]&&(u<0||d[i]<d[u]))u=i;done[u]=
    true;total+=d[u];
14 for(int j=0;j<=n;++j){if(!done[j]){double w=u==0?p.start[j-1]:j==0?p.start[u-1]:std::min(p
    .edge[u-1][j-1],p.edge[j-1][u-1]);d[j]=std::min(d[j],w);}}}
15 return total;
16 }
17 inline double assignment_lower(const Problem&p){int n=p.start.size()+1;double inf=1e12;std::
    vector<double>u(n+1),v(n+1);std::vector<int>match(n+1),way(n+1);
18 auto cost=[&](int i,int j){--i;--j;if(i==j)return inf;if(j==0)return 0;if(i==0)return p.start
    [j-1];return p.edge[i-1][j-1];};
19 for(int i=1;i<=n;++i){match[0]=i;int j0=0;std::vector<double>minv(n+1,inf);std::vector<bool>
    used(n+1);
20 do{used[j0]=true;int i0=match[j0],j1=0;double delta=inf;
21 for(int j=1;j<=n;++j){if(!used[j]){double cur=cost(i0,j)-u[i0]-v[j];if(cur<minv[j]){minv[j]=
    cur;way[j]=j0;if(minv[j]<delta){delta=minv[j];j1=j;}}}

```

```

22   for(int j=0;j<=n;++j)if(used[j]){u[match[j]]+=delta;v[j]-=delta;}else minv[j]-=delta;j0=j1;
23   }while(match[j0]);
24   do{int j1=way[j0];match[j0]=match[j1];j0=j1;}while(j0);
25 }
26 // Return a conservatively adjusted feasible assignment dual, not a cycle
27 // cover primal rounded upward. The adjustment exceeds arithmetic residuals.
28 double violation=0;for(int i=1;i<=n;++i)for(int j=1;j<=n;++j)violation=std::max(violation,u[i]
    ]+v[j]-cost(i,j));
29 double dual=0;for(int i=1;i<=n;++i)dual+=u[i]+v[i];return std::max(0.,dual-n*(violation+1e-8))
    ;
30 }
31 inline std::vector<int> nearest(const Problem&p,int first=-1){int n=p.start.size();std::vector<
    bool>used(n);std::vector<int>r;
32   if(first>=0){r.push_back(first);used[first]=true;}
33   while((int)r.size(<n){int j=-1;double best=1e100;for(int k=0;k<n;++k)if(!used[k]){double c=r.
        empty()?p.start[k]:p.edge[r.back()][k];if(c<best){best=c;j=k;}}r.push_back(j);used[j]=true
        ;}return r;
34 }
35 inline bool improve(const Problem&p,std::vector<int>&r,bool relocate){
36   for(int round=0;round<100;++round){double best=length(p,r)-1e-7;std::vector<int>choice;int n=r
        .size();
37   for(int i=0;i<n;++i)for(int j=i+1;j<n;++j){auto test=r;std::reverse(test.begin()+i,test.begin
        ()+j+1);double c=length(p,test);if(c<best){best=c;choice=test;}}
38   if(relocate)for(int i=0;i<n;++i)for(int j=0;j<n;++j)if(i!=j){auto test=r;int x=test[i];test.
        erase(test.begin()+i);test.insert(test.begin()+j,x);double c=length(p,test);if(c<best){
        best=c;choice=test;}}
39   if(choice.empty())return true;r=std::move(choice);
40 }return false;
41 }
42 inline Solution solve(const Problem&p){Solution s;int n=p.start.size();if(!n){s.exact=s.
    local_optimal=true;return s;}
43 if(n<=12){int states=1<<n;std::vector<double>dp(states*n,1e100);std::vector<signed char>parent
    (states*n,-1);
44   for(int j=0;j<n;++j)dp[(1<<j)*n+j]=p.start[j];
45   for(int mask=1;mask<states;++mask)for(int j=0;j<n;++j)if(mask&(1<<j)){double c=dp[mask*n+j];
        for(int k=0;k<n;++k)if(!(mask&(1<<k))){int index=(mask|(1<<k))*n+k;double v=c+p.edge[j][k
        ];if(v<dp[index]){dp[index]=v;parent[index]=j;}}}
46   int mask=states-1,last=0;for(int j=1;j<n;++j)if(dp[mask*n+j]<dp[mask*n+last])last=j;s.upper=
        dp[mask*n+last];
47   while(mask){s.path.push_back(last);int prev=parent[mask*n+last];mask^=1<<last;last=prev;}std
        ::reverse(s.path.begin(),s.path.end());s.lower=s.upper;s.exact=s.local_optimal=true;return
        s;
48 }
49 s.path=nearest(p);improve(p,s.path,false);s.upper=length(p,s.path);
50 for(int first=0;first<n;++first){auto r=nearest(p,first);improve(p,r,false);double c=length(p,
    r);if(c<s.upper-1e-7){s.upper=c;s.path=r;}}
51 s.local_optimal=improve(p,s.path,true);s.upper=length(p,s.path);
52 s.lower=std::max(assignment_lower(p),std::max(0.,mst_lower(p)-1e-6));
53 if(s.lower>s.upper+1e-5)throw std::runtime_error("route lower bound exceeds feasible route");
    return s;
54 }
55 }

```

## src/configurations.hpp

```

1 #pragma once
2 #include "policy.hpp"

```

```

3 inline Options configuration(int method){
4   Options o;o.fixedLayout=21;o.routing=2;o.opportunistic=true;
5   if(method==1){o.secondA=850;o.secondB=400;}
6   if(method==2){o.secondA=843.1178;o.secondB=545.4003;}
7   if(method>=3)o.stop_when_found16=true;
8   if(method==4)o.route_model=1;
9   if(method==5)o.route_model=2;
10  return o;
11 }

```

## src/continuous\_points.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 #define B_CONTINUOUS_POINTS 1
4 namespace continuous_points {
5   template<class Score,class Valid>
6   geo::Poly refine(const geo::Poly& seeds,Score score,Valid valid,int budget,double scale){
7     std::vector<std::pair<double,geo::Pt>> evaluated;
8     int calls=0;
9     auto evaluate=[&](geo::Pt q){if(calls>=budget||!valid(q))return;
10    for(auto x:evaluated)if(geo::dist(q,x.second)<1e-7)return;
11    ++calls;double value=score(q);if(!std::isfinite(value))return;evaluated.push_back({value,q});
12  };
13  for(auto q:seeds)evaluate(q);if(evaluated.empty())return {};
14  auto best=[&]()>{return std::min_element(evaluated.begin(),evaluated.end(),[](auto a,auto b){
15    return a.first<b.first;})->second;};
16  for(int level=0;level<6&&calls<budget;++level){
17    for(int repeat=0;repeat<2&&calls<budget;++repeat){auto center=best();
18      for(int k=0;k<8;++k)evaluate(center+geo::polar(scale,45.*k));
19      if(geo::dist(center,best())<1e-7)break;
20    }scale*=.5;
21  }
22  std::stable_sort(evaluated.begin(),evaluated.end(),[](auto a,auto b){return a.first<b.first;})
23  ;
24  geo::Poly out;for(auto x:evaluated){out.push_back(x.second);if(out.size()==6)break;}return out
25  ;
26 }
27 }

```

## src/discovery.hpp

```

1 #pragma once
2 #include "dynamic_selection.hpp"
3 namespace discovery {
4   class Belief {
5     std::vector<dynamic_selection::Hypothesis> particles;
6   public:
7     Belief()=default;
8     explicit Belief(int problem){for(int i=1;i<=2048;++i){double r=1800*sqrt(dynamic_selection::
9       radical(i,2)),a=360*dynamic_selection::radical(i,3);particles.push_back({geo::polar(r,a
10      ,1000+500*dynamic_selection::radical(i,5),problem==4&&dynamic_selection::radical(i,11)
11      <.5,360*dynamic_selection::radical(i,7),1});}}
12   size_t size()const{return particles.size();}
13   double probability(geo::Pt q)const{if(particles.empty())return 0;int yes=0;for(auto h:
14     particles)yes+=dynamic_selection::receives(h,q);return double(yes)/particles.size();}

```

```

11 void negative_scan(geo::Pt q){particles.erase(std::remove_if(particles.begin(),particles.end()
    ,[&](auto h){return dynamic_selection::receives(h,q);}),particles.end());}
12 };
13 }

```

## src/dynamic\_selection.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 #include "belief_dp.hpp"
4 #include "bearing_types.hpp"
5 #include "optical_terminal.hpp"
6 #include "continuous_points.hpp"
7 #include "robust_geometry.hpp"
8 #include <numeric>
9 namespace dynamic_selection {
10 #define B_REMAINING_DEPTH 1
11 #define B_BEARING_BINS 1
12 inline int bearing_outcome(double theta,int width=2){if(width<=0||360%width)throw std::
    invalid_argument("invalid bearing bin");double a=fmod(theta+720.,360.);if(a<0)a+=360.;
    return 10+int(floor(a/width));}
13 inline int remaining_depth(int requested,int allowed,int used){return std::max(0,std::min(
    requested,allowed-used));}
14 inline double finite_terminal_cost(const std::vector<Hypothesis>&hypotheses,uint64_t mask,geo::
    Pt now,geo::Pt first,geo::Pt u,geo::Pt v){
15
16     double xmin=1e99,xmax=-1e99,ymin=1e99,ymax=-1e99,total=0;
17     for(int i=0;i<(int)hypotheses.size();++i){if(mask>>i&1){auto d=hypotheses[i].z-first;double x=
        geo::dot(d,u),y=geo::dot(d,v);xmin=std::min(xmin,x);xmax=std::max(xmax,x);ymin=std::min(
        ymin,y);ymax=std::max(ymax,y);total+=hypotheses[i].weight;}
18     int nx=std::max(1,int(ceil((xmax-xmin)/28))),ny=std::max(1,int(ceil((ymax-ymin)/28)));geo::
        Poly sweep;
19     for(int i=0;i<nx;++i)for(int jj=0;jj<ny;++jj){int j=i%2?ny-1-jj:jj;sweep.push_back(first+u*(
        xmin+(i+.5)*(xmax-xmin)/nx)+v*(ymin+(j+.5)*(ymax-ymin)/ny));}
20     double best=1e100;
21     for(int reverse=0;reverse<2;++reverse){if(reverse)std::reverse(sweep.begin(),sweep.end());
        double cumulative=0,expected=0,uint64_t remaining=mask;auto position=now;
22     for(auto q:sweep){cumulative+=geo::dist(position,q)/5;position=q;for(int i=0;i<(int)
        hypotheses.size();++i){if((remaining>>i&1)&&geo::dist(q,hypotheses[i].z)<=20+1e-7){expected
        +=hypotheses[i].weight/total*(cumulative+5);remaining&=~(uint64_t(1)<<i);}if(!remaining)
        break;cumulative+=3;}
23     if(remaining)throw std::runtime_error("finite terminal sweep failed");best=std::min(best,
        expected);
24 }return best;
25
26 }
27 struct Alternative {geo::Pt point;bool measure;double value;};
28 struct Choice {bool available=false,measure=true,budget_hit=false,exact=false;geo::Pt point;
    double value=0;int depth=0,hypotheses=0,expanded=0;std::vector<Alternative>alternatives;};
29 inline bool receives(const Hypothesis&h,geo::Pt q){return geo::dist(h.z,q)<=h.radius+1e-9&&(!h.
    directional||geo::dot(q-h.z,geo::polar(1,h.heading))>=-1e-9);}
30 inline bool consistent(const Hypothesis&h,const std::vector<Event>&events){
31     for(auto e:events){double distance=geo::dist(h.z,e.q);
32     if(e.kind==3){if(distance<=20)return false;continue;}
33     if(e.kind==4){if(distance>20)return false;continue;}
34     bool heard=receives(h,e.q);
35     if(e.kind==0){if(heard)return false;}

```

```

36 else if(e.kind==1){if(!heard||distance>5)return false;}
37 else if(e.kind==2){double angle=atan2(h.z.y-e.q.y,h.z.x-e.q.x)*180/geo::pi;if(!heard||
    distance<=5||std::abs(std::remainder(angle-e.angle,360.))>1.0050001)return false;}
38 }return true;
39 }
40 inline double radical(int i,int base){double a=0,b=1.;while(i){b/=base;a+=b*(i%base);i/=base;}
    return a;}
41 inline std::vector<Hypothesis> make_hypotheses(const geo::Poly&p,const std::vector<Event>&
    history,int problem,int cap){
42 if(p.empty()||cap<1||cap>63)return {};
43 geo::Pt center;for(auto q:p)center=center+q;center=center/double(p.size());
44 std::vector<double>areas;double area=0;
45 for(size_t i=0;i<p.size();++i){area+=std::abs(geo::cross(p[i]-center,p[(i+1)%p.size()]-center)
    );areas.push_back(area);}
46 geo::Poly positions{center};
47 for(int i=1;i<cap;++i){
48 if(area<1e-10){positions.push_back(p.front()*(1-i/double(cap))+p.back()*(i/double(cap)));
    continue;}
49 size_t tri=std::lower_bound(areas.begin(),areas.end(),radical(i,2)*area)-areas.begin();double
    a=sqrt(radical(i,3)),b=radical(i,5);
50 positions.push_back(center*(1-a)+p[tri]*(a*(1-b))+p[(tri+1)%p.size()]*(a*b));
51 }
52 std::vector<Hypothesis>all;
53 for(auto z:positions){
54 double low=1000;for(auto e:history)if(e.kind==1||e.kind==2)low=std::max(low,geo::dist(z,e.q))
    ;if(low>1500+1e-7)continue;low=std::min(low,1500.);
55 std::vector<double>headings;for(int i=0;i<12;++i)headings.push_back(i*30.);
56 // Event boundary midpoints sample even very narrow feasible heading arcs.
57 std::vector<double>cuts{0};for(auto e:history)if(e.kind<=2){double a=atan2(e.q.y-z.y,e.q.x-z.
    x)*180/geo::pi;for(double d:{-90.,90.})cuts.push_back(fmod(a+d*720.,360.));}
58 std::sort(cuts.begin(),cuts.end());for(size_t i=0;i<cuts.size();++i){double b=i+1<cuts.size()
    ?cuts[i+1]:cuts[0]+360.;headings.push_back(fmod((cuts[i]+b)/2,360.));}
59 for(double r:{low,(low+1500)/2,1500.}){
60 Hypothesis h{z,r,false,0,problem==3?1.:.5};if(consistent(h,history))all.push_back(h);
61 if(problem==4)for(double a:headings){h={z,r,true,a,.5/headings.size()};if(consistent(h,
    history))all.push_back(h);}
62 }
63 }
64 if(all.size()<=size_t(cap))return all;
65 double total=0;for(auto h:all)total+=h.weight;
66 std::vector<Hypothesis>selected;double cumulative=all[0].weight;size_t j=0;
67 for(int i=0;i<cap;++i){double target=total*(i+.5)/cap;while(j+1<all.size()&&cumulative<target)
    cumulative+=all[++j].weight;auto h=all[j];h.weight=1.;selected.push_back(h);}
68 return selected;
69 }
70 inline geo::Poly retreat_points(geo::Pt center,const std::vector<Event>&history){
71 geo::Poly points;for(auto it=history.rbegin();it!=history.rend();++it)if(it->kind==2){double
    length=geo::dist(center,it->q);if(length<1)continue;auto ray=(it->q-center)/length;for(
    double d:{25.,50.,100.,200.})points.push_back(center+ray*std::min(d,.8*length));break;}
    return points;
72 }
73 inline geo::Poly safe_clear_points(const geo::Poly&p,geo::Pt now){
74 if(p.empty())return {};auto c=geo::mec(p);double r=20-c.r-1e-6;if(r<0)return {};
75 double d=geo::dist(now,c.c);geo::Poly out{d<=r?now:c.c+(now-c.c)*(r/d)};
76 for(int i=0;i<16;++i)out.push_back(c.c+geo::polar(r,i*22.5));return out;
77 }
78 inline Choice plan(const geo::Poly&p,geo::Pt now,geo::Pt first,double angle,const std::vector<
    Event>&history,int problem,int depth,bool tuned,int limit,bool retreat=false,int samples

```

```

        =32,bool clear_neighborhood=false,int terminal_mode=0,int refinement=0,int bearing_bin=2,
        int robust_mode=0){
79 Choice choice;choice.depth=depth;
80 auto hypotheses=make_hypotheses(p,history,problem,samples);choice.hypotheses=hypotheses.size()
    ;if(hypotheses.empty())return choice;
81 geo::Pt center=geo::mec(p).c,axis=geo::polar(1,angle);double longest=0;
82 for(auto a:p)for(auto b:p){double d=geo::dist(a,b);if(d>longest){longest=d,axis=(b-a)/d;}}
83 geo::Pt transverse{-axis.y,axis.x},u=geo::polar(1,angle),v{-u.y,u.x};
84 std::vector<geo::Pt>locations;std::vector<bool>measure;
85 auto add=[&](geo::Pt q,bool sensing){
86     if(!std::isfinite(q.x)||std::isfinite(q.y)||geo::norm(q)>2671.99)return;
87     for(auto e:history)if((sensing?e.kind<=2:e.kind>=3)&&geo::dist(e.q,q)<1e-6)return;
88     for(size_t i=0;i<locations.size();++i)if(measure[i]==sensing&&geo::dist(locations[i],q)<1e-6)
        return;
89     locations.push_back(q);measure.push_back(sensing);
90 };
91 add(now,true);add(center,true);
92 if(retreat)for(auto q:retreat_points(center,history))add(q,true);
93 for(double side:{-1.,1.}){add(first+u*750+v*(side*300),true);for(double d:{50.,150.,300.})add(
    center+transverse*(side*d),true);}
94 for(double along:{-.25,.25})for(double side:{-1.,1.})add(center+axis*(along*longest)+
    transverse*(side*80),true);
95 add(center,false);for(double along:{-.4,-.2,.2,.4})add(center+axis*(along*longest),false);
96 if(clear_neighborhood)for(auto q:safe_clear_points(p,now))add(q,false);
97 if(refinement){geo::Poly seeds;for(size_t i=0;i<locations.size();++i)if(measure[i])seeds.
    push_back(locations[i]);
98     double total=0;for(auto h:hypotheses)total+=h.weight;
99     auto valid=[&](geo::Pt q){if(!std::isfinite(q.x)||std::isfinite(q.y)||geo::norm(q)>2671.99)
        return false;for(auto e:history)if(e.kind<=2&&geo::dist(e.q,q)<1e-6)return false;return
        true;};
100     auto score=[&](geo::Pt q){double cost=geo::dist(now,q)/5+(tuned?0:1);std::map<int,uint64_t>
        groups;
101         for(size_t i=0;i<hypotheses.size();++i){auto h=hypotheses[i];double distance=geo::dist(q,h.z
            );int out=0;double seconds=5;
102             if(receives(h,q)){if(distance<=5){out=-1;seconds=10;}else{double theta=atan2(h.z.y-q.y,h.z.
                x-q.x)*180/geo::pi;out=bearing_outcome(theta,bearing_bin);}}
103             cost+=h.weight/total*seconds;groups[out]|=uint64_t(1)<<i;
104         }
105         for(auto [out,mask]:groups)if(out>=0){double mass=0;for(size_t i=0;i<hypotheses.size();++i)
            if(mask>>i&1)mass+=hypotheses[i].weight;cost+=mass/total*finite_terminal_cost(hypotheses,
                mask,q,first,u,v);}
106         return cost;
107     };
108     for(auto q:continuous_points::refine(seeds,score,valid,refinement==1?80:160,std::clamp(
        longest*.15,20.,180.)))add(q,true);
109 }
110 if(locations.empty())return choice;
111 belief_dp::Model model;for(auto h:hypotheses)model.weights.push_back(h.weight);
112 int n=locations.size();model.travel.assign(n+1,std::vector<double>(n+1));geo::Poly at=
    locations;at.push_back(now);
113 for(int i=0;i<=n;++i)for(int j=0;j<=n;++j)model.travel[i][j]=geo::dist(at[i],at[j])/5;
114 for(int j=0;j<=n;++j){belief_dp::Action a;a.measure=measure[j];
115     for(auto h:hypotheses){double distance=geo::dist(h.z,locations[j]);int out;double seconds;
116         if(!a.measure){bool hit=distance<=20;out=hit?-1:0;seconds=hit?5:3;}
117         else if(!receives(h,locations[j])){out=0;seconds=5;}
118         else if(distance<=5){out=-1;seconds=10;}
119         else{double theta=atan2(h.z.y-locations[j].y,h.z.x-locations[j].x)*180/geo::pi;out=
            bearing_outcome(theta,bearing_bin);seconds=5;}

```

```

120     a.outcome.push_back(out);a.seconds.push_back(seconds);
121 }model.actions.push_back(a);
122 }
123 model.terminal=[&](uint64_t mask,int from){return finite_terminal_cost(hypotheses,mask,at[from
    ],first,u,v);};
124 if(terminal_mode&&bearing_bin!=2)throw std::invalid_argument("context terminal requires 2-
    degree bins");
125 optical_terminal::Contexts contexts(p,locations,measure,hypotheses,first,angle,terminal_mode
    >=2);
126 if(terminal_mode){model.advance=[&](int context,int action,int outcome){return contexts.
    advance(context,action,outcome);};model.terminal_context=[&](uint64_t mask,int from,int
    context){return contexts.value(mask,at[from],context);};}
127 auto result=belief_dp::solve(model,depth,tuned,limit);choice.value=result.value;choice.
    expanded=result.expanded;choice.budget_hit=result.budget_hit;choice.exact=result.exact;
128 if(result.action>=0){choice.available=true;choice.measure=measure[result.action];choice.point=
    locations[result.action];}
129 if(choice.available){for(int j=0;j<n;++j)if(std::isfinite(result.action_values[j]))choice.
    alternatives.push_back({locations[j],measure[j],result.action_values[j]});std::stable_sort
    (choice.alternatives.begin(),choice.alternatives.end(),[](const auto&a,const auto&b){
    return a.value<b.value;});if(choice.alternatives.size()>6)choice.alternatives.resize(6);}
130 if(robust_mode&&choice.available){
131     auto old=geo::mec(p);
132     int positive=0;for(auto e:history)positive+=e.kind==2;
133     if(old.r>35&&(robust_mode!=2||positive==1)){
134         auto fresh=robust_geometry::candidate(p,now,first,angle,history,problem,robust_mode==2?2:1);
135         if(fresh.valid){
136             double oldrho=choice.measure&&robust_geometry::reception(p,choice.point,history,problem)?
                robust_geometry::worst_radius(p,choice.point,128).upper:old.r;
137             double extra=std::max(0.,geo::dist(now,fresh.q)-geo::dist(now,choice.point))/5+5;
138             // Time-scaled diameter reduction is a selection heuristic, not a theorem
139             // about total time. The held-out paired test is the promotion gate.
140             bool accept=robust_mode==2?(fresh.rho<=35&&geo::dist(now,fresh.q)+25<geo::dist(now,choice.
                point)):(.4*(oldrho-fresh.rho)>extra&&fresh.rho<old.r*.85);
141             if(accept){choice.point=fresh.q;choice.measure=true;choice.alternatives={{fresh.q,true,
                choice.value}};}}
142     }
143 }
144 }
145 return choice;
146 }
147 }

```

## src/figqa.py

```

1 #!/usr/bin/env python3
2 """figqa.py -- matplotlib bbox-collision QA gate for award-tier figures.
3
4 This is both an importable library and a CLI. It never renders to a contact
5 sheet and never reads source image files: it inspects a live ``matplotlib``
6 ``Figure`` after it has been drawn and reports where text/annotation/legend
7 artists collide with data artists (bars, lines, scatter points) or with each
8 other, plus any text-like artist whose box escapes the figure bounds.
9
10 Library API
11 -----
12     from figqa import collisions, assert_no_overlap
13     collisions(fig)          -> list[dict]    # empty means clean

```

```

14     assert_no_overlap(fig)                # raises AssertionError listing hits
15
16 Wire ``assert_no_overlap(fig)`` into the figure factory (and into ``run_all``)
17 so a rendering defect fails the run the same way a failed numeric check does.
18
19 Design notes (hard-won production pitfalls, do not "simplify" away)
20 -----
21 * Line2D: a diagonal line's bounding box covers a huge rectangle, so comparing
22   a text box against a *line bbox* false-positives almost everything. Instead we
23   sample points along the line's screen-space segments and only flag when a
24   sampled point actually lands inside a text box.
25 * Annotation: ``Annotation.get_window_extent`` returns only the TEXT box, never
26   the arrow. We rely on that and additionally exclude ``FancyArrowPatch`` from
27   the data-patch set so an annotation arrow can never be treated as a bar.
28 * Host patch: a value label that sits fully inside its own bar is legitimate.
29   Only a text box that *partially* overlaps a patch (crossing into a foreign
30   filled region) is a collision. Legends get no such exemption -- a legend over
31   bars is always a defect.
32 * Zero-size / invisible artists are ignored.
33
34 Only matplotlib (and its bundled numpy) are required.
35 """
36
37 from __future__ import annotations
38
39 import sys
40
41 try:
42     import numpy as np
43     from matplotlib.collections import Collection # noqa: F401 (documented surface)
44     from matplotlib.legend import Legend # noqa: F401
45     from matplotlib.lines import Line2D # noqa: F401
46     from matplotlib.patches import FancyArrowPatch
47     from matplotlib.text import Annotation, Text # noqa: F401
48     from matplotlib.transforms import Bbox, IdentityTransform
49 except ImportError as exc: # pragma: no cover - environment-dependent
50     print(
51         f"figqa: missing prerequisite: {exc}.\n"
52         "Install with: python3 -m pip install matplotlib\n"
53         "(or run under an environment that has matplotlib, e.g. "
54         "'uv run --with matplotlib python3 ...')",
55         file=sys.stderr,
56     )
57     raise SystemExit(2)
58
59 # Line2D linestyle values that draw no connecting stroke; only markers show.
60 _NO_LINESTYLE = ("None", "none", "", " ")
61
62 # Pixel tolerances. Kept small so a genuine 1px edge touch is not a "defect"
63 # while a real overlap (many px^2) always trips.
64 TOL = 1.0          # generic slack for "fully inside" / figure-bounds tests
65 MIN_AREA = 2.0     # minimum bbox intersection area (px^2) counted as overlap
66 INSET = 0.75       # a sampled point must penetrate a text box by this many px
67
68
69 # -----
70 # rendering helpers
71 # -----

```

```

72 def _renderer(fig):
73     """Draw the figure and return a usable renderer across backends."""
74     fig.canvas.draw()
75     getter = getattr(fig.canvas, "get_renderer", None)
76     if getter is not None:
77         try:
78             return getter()
79         except Exception:
80             pass
81     # Headless / non-Agg backend fallback: rasterise once via Agg. The
82     # FigureCanvasAgg constructor re-parents fig.canvas as a side effect, which
83     # would silently rebind a GUI-backend caller's figure -- restore it.
84     from matplotlib.backends.backend_agg import FigureCanvasAgg
85
86     orig_canvas = fig.canvas
87     canvas = FigureCanvasAgg(fig)
88     canvas.draw()
89     renderer = canvas.get_renderer()
90     if orig_canvas is not None and orig_canvas is not canvas:
91         fig.set_canvas(orig_canvas)
92     return renderer
93
94
95 def _win_extent(artist, renderer):
96     try:
97         return artist.get_window_extent(renderer)
98     except Exception:
99         try:
100             return artist.get_window_extent(renderer=renderer)
101         except Exception:
102             return None
103
104
105 def _usable(bb):
106     return bb is not None and bb.width > 0 and bb.height > 0
107
108
109 def _inter_area(a, b):
110     inter = Bbox.intersection(a, b)
111     if inter is None:
112         return 0.0
113     w, h = inter.width, inter.height
114     if w <= 0 or h <= 0:
115         return 0.0
116     return w * h
117
118
119 def _fully_inside(inner, outer, tol=TOL):
120     return (
121         inner.x0 >= outer.x0 - tol
122         and inner.x1 <= outer.x1 + tol
123         and inner.y0 >= outer.y0 - tol
124         and inner.y1 <= outer.y1 + tol
125     )
126
127
128 def _point_in(bb, x, y, inset=INSET):
129     return (

```

```

130         bb.x0 + inset <= x <= bb.x1 - inset
131         and bb.y0 + inset <= y <= bb.y1 - inset
132     )
133
134
135 def _label(text_artist):
136     try:
137         s = (text_artist.get_text() or "").strip()
138     except Exception:
139         s = ""
140     s = " ".join(s.split())
141     if len(s) > 40:
142         s = s[:37] + "..."
143     return f'"{s}"' if s else f"<{type(text_artist).__name__}>"
144
145
146 def _describe(artist, index):
147     return f'{type(artist).__name__}#{index}'
148
149
150 # -----
151 # sampling data artists into screen-space points (avoids diagonal-bbox trap)
152 # -----
153 def _line_segments(line):
154     """Screen-space polyline segments [(p0, p1), ...] with finite endpoints.
155
156     Used instead of point-sampling: a fixed sample count steps over a label on
157     a long segment (e.g. an axhline/axvline crossing a data label), so we test
158     each segment against the text box exactly, independent of screen length.
159     """
160     xy = line.get_xydata()
161     if xy is None or len(xy) == 0:
162         return []
163     disp = line.get_transform().transform(np.asarray(xy, dtype=float))
164     disp = disp[np.isfinite(disp).all(axis=1)]
165     if len(disp) < 2:
166         return []
167     return [(disp[i], disp[i + 1]) for i in range(len(disp) - 1)]
168
169
170 def _seg_box_hit(p0, p1, xmin, ymin, xmax, ymax):
171     """True if segment p0->p1 intersects the axis-aligned box (Liang-Barsky).
172
173     Density-independent: catches a line crossing a label on any segment length,
174     including endpoints that fall inside the box.
175     """
176     if xmax <= xmin or ymax <= ymin:
177         return False
178     x0, y0 = float(p0[0]), float(p0[1])
179     x1, y1 = float(p1[0]), float(p1[1])
180     dx, dy = x1 - x0, y1 - y0
181     t0, t1 = 0.0, 1.0
182     for p, q in ((-dx, x0 - xmin), (dx, xmax - x0), (-dy, y0 - ymin), (dy, ymax - y0)):
183         if p == 0.0:
184             if q < 0.0:
185                 return False # parallel to this edge and outside the slab
186         else:
187             r = q / p

```

```

188         if p < 0.0:
189             if r > t1:
190                 return False
191             if r > t0:
192                 t0 = r
193         else:
194             if r < t0:
195                 return False
196             if r < t1:
197                 t1 = r
198     return t0 <= t1
199
200
201 def _collection_points(coll):
202     try:
203         offs = coll.get_offsets()
204     except Exception:
205         offs = None
206     if offs is None or len(offs) == 0:
207         return np.empty((0, 2))
208     try:
209         trans = coll.get_offset_transform()
210     except Exception:
211         trans = coll.get_transform()
212     arr = np.asarray(offs, dtype=float)
213     # LineCollections built by vlins/errorbar report a phantom single offset
214     # at (0, 0) with an identity offset transform even though no point exists
215     # there; mapped to display pixel (0, 0) it would false-positive any text
216     # box at the figure's bottom-left corner.
217     if len(arr) == 1 and np.allclose(arr, 0.0) and isinstance(trans, IdentityTransform):
218         return np.empty((0, 2))
219     disp = trans.transform(arr)
220     return disp[np.isfinite(disp).all(axis=1)]
221
222
223 def _finite_points(pts):
224     if len(pts) == 0:
225         return pts
226     return pts[np.isfinite(pts).all(axis=1)]
227
228
229 # -----
230 # public API
231 # -----
232 def _mk(kind, ax_index, a, b, area=None):
233     d = {"kind": kind, "axes": ax_index, "a": a, "b": b}
234     if area is not None:
235         d["overlap_px2"] = round(float(area), 1)
236     return d
237
238
239 def collisions(fig, tol=TOL, min_area=MIN_AREA, inset=INSET):
240     """Return a list of collision dicts for ``fig`` (empty == clean).
241
242     Each dict has ``kind`` (one of ``text-over-patch``, ``legend-over-patch``,
243     ``line-through-text``, ``point-under-text``, ``text-over-text``,
244     ``clipped-out-of-figure``), the axes index (``-1`` for figure-level text
245     such as the suptitle), and the two artists involved. Titles, axis labels,

```

```

246 figure-level text, and the subtitle participate in the text checks; tick
247 labels participate only in the legend-overlap check.
248 """
249 renderer = _renderer(fig)
250 fig_bbox = fig.bbox
251 found = []
252
253 def _visible_text(t):
254     try:
255         return t is not None and t.get_visible() and (t.get_text() or "").strip()
256     except Exception:
257         return False
258
259 # --- figure-level text (fig.text(...) entries and the subtitle, which
260 # lives on fig._suptitle, NOT in fig.texts) ---
261 fig_text_items = []
262 for t in list(fig.texts) + [getattr(fig, "_suptitle", None)]:
263     if not _visible_text(t):
264         continue
265     bb = _win_extent(t, renderer)
266     if _usable(bb):
267         fig_text_items.append((_label(t), bb))
268
269 for ai, ax in enumerate(fig.axes):
270     if not ax.get_visible():
271         continue
272
273     # --- text-like artists: Text + Annotation live in ax.texts; the
274     # title and axis labels are separate artists and are exactly where
275     # dense contest figures collide (legend over title, suptitle over an
276     # axis label), so they join the text set too. ---
277     text_items = []
278     for t in ax.texts:
279         if not t.get_visible():
280             continue
281         bb = _win_extent(t, renderer) # Annotation -> text box only
282         if _usable(bb):
283             text_items.append((t, bb, _label(t)))
284     for t in (ax.title, getattr(ax, "_left_title", None),
285             getattr(ax, "_right_title", None),
286             ax.xaxis.label, ax.yaxis.label):
287         if not _visible_text(t):
288             continue
289         bb = _win_extent(t, renderer)
290         if _usable(bb):
291             text_items.append((t, bb, _label(t)))
292
293     legend = ax.get_legend()
294     legend_item = None
295     if legend is not None and legend.get_visible():
296         lb = _win_extent(legend, renderer)
297         if _usable(lb):
298             legend_item = (legend, lb)
299
300     # boxes that data artists must not cross
301     text_boxes = [(lbl, bb) for (_, bb, lbl) in text_items]
302     if legend_item is not None:
303         text_boxes.append(("legend", legend_item[1]))

```

```

304
305     # --- data artists ---
306     patches = [
307         p for p in ax.patches
308         if p.get_visible() and not isinstance(p, FancyArrowPatch)
309     ]
310     lines = [ln for ln in ax.lines if ln.get_visible()]
311     colls = [c for c in ax.collections if c.get_visible()]
312
313     # 1) text (labels) vs patches -- host patch exempted
314     for (_, tb, lbl) in text_items:
315         for pi, p in enumerate(patches):
316             pb = _win_extent(p, renderer)
317             if not _usable(pb):
318                 continue
319             area = _inter_area(tb, pb)
320             if area <= min_area:
321                 continue
322             if _fully_inside(tb, pb, tol):
323                 continue # value label inside its own bar -> allowed
324             found.append(_mk("text-over-patch", ai, lbl, _describe(p, pi), area))
325
326     # 2) legend vs patches -- no host exemption
327     if legend_item is not None:
328         lb = legend_item[1]
329         for pi, p in enumerate(patches):
330             pb = _win_extent(p, renderer)
331             if not _usable(pb):
332                 continue
333             area = _inter_area(lb, pb)
334             if area > min_area:
335                 found.append(_mk("legend-over-patch", ai, "legend",
336                                 _describe(p, pi), area))
337
338     # 3) lines vs text boxes -- exact segment-vs-box test (NOT the line bbox,
339     #     and NOT fixed-count point sampling which steps over long segments).
340     #     A marker-only line (linestyle 'None', e.g. plot(..., 'o')) draws no
341     #     connecting stroke, so its vertices are tested as points instead of
342     #     testing the phantom segments between them.
343     for li, ln in enumerate(lines):
344         if str(ln.get_linestyle()) in _NO_LINESTYLE:
345             if str(ln.get_marker()) in ("", "None"):
346                 continue # draws nothing at all
347             xy = ln.get_xydata()
348             if xy is None or len(xy) == 0:
349                 continue
350             pts = ln.get_transform().transform(np.asarray(xy, dtype=float))
351             pts = _finite_points(pts)
352             for (lbl, tb) in text_boxes:
353                 if any(_point_in(tb, x, y, inset) for x, y in pts):
354                     found.append(_mk("point-under-text", ai, lbl,
355                                     _describe(ln, li)))
356             continue
357         segs = _line_segments(ln)
358         if not segs:
359             continue
360         for (lbl, tb) in text_boxes:
361             xmin, ymin = tb.x0 + inset, tb.y0 + inset

```

```

362         xmax, ymax = tb.x1 - inset, tb.y1 - inset
363         if any(_seg_box_hit(p0, p1, xmin, ymin, xmax, ymax) for p0, p1 in segs):
364             found.append(_mk("line-through-text", ai, lbl,
365                             _describe(ln, li)))
366
367     # 4) collection points vs text boxes
368     for ci, c in enumerate(colls):
369         pts = _finite_points(_collection_points(c))
370         if len(pts) == 0:
371             continue
372         for (lbl, tb) in text_boxes:
373             if any(_point_in(tb, x, y, inset) for x, y in pts):
374                 found.append(_mk("point-under-text", ai, lbl,
375                                 _describe(c, ci)))
376
377     # 5) text vs text (legend included; figure-level text such as the
378     #     subtitle is checked against this axes' text set here, and against
379     #     itself once after the axes loop)
380     for i in range(len(text_boxes)):
381         for j in range(i + 1, len(text_boxes)):
382             area = _inter_area(text_boxes[i][1], text_boxes[j][1])
383             if area > min_area:
384                 found.append(_mk("text-over-text", ai, text_boxes[i][0],
385                                 text_boxes[j][0], area))
386     for (flbl, fb) in fig_text_items:
387         for (lbl, tb) in text_boxes:
388             area = _inter_area(fb, tb)
389             if area > min_area:
390                 found.append(_mk("text-over-text", ai, flbl, lbl, area))
391
392     # 5b) tick labels vs legend ONLY. Tick labels legitimately sit dense
393     #     and close to data, so they stay out of the general text set; a
394     #     legend parked on top of them is still always a defect.
395     if legend_item is not None:
396         lb = legend_item[1]
397         for t in ax.get_xticklabels() + ax.get_yticklabels():
398             if not _visible_text(t):
399                 continue
400             bb = _win_extent(t, renderer)
401             if not _usable(bb):
402                 continue
403             area = _inter_area(lb, bb)
404             if area > min_area:
405                 found.append(_mk("text-over-text", ai, "legend",
406                                 _label(t), area))
407
408     # 6) clipped: text-like artist STRADDLING the figure edge (genuinely
409     #     cut off). A box fully outside the canvas is an intentional
410     #     outside-axes placement that a bbox_inches='tight' save captures, so
411     #     it is not flagged; only a partial straddle is a real clip.
412     for (lbl, bb) in text_boxes:
413         if _fully_inside(bb, fig_bbox, tol):
414             continue
415         if _inter_area(bb, fig_bbox) <= min_area:
416             continue # fully off-canvas -> intentional outside placement
417         found.append(_mk("clipped-out-of-figure", ai, lbl, "figure-bounds"))
418
419     # --- figure-level text vs itself + figure bounds (outside the axes loop

```

```

420 # so multi-axes figures do not repeat the same pair per axes) ---
421 for i in range(len(fig_text_items)):
422     for j in range(i + 1, len(fig_text_items)):
423         area = _inter_area(fig_text_items[i][1], fig_text_items[j][1])
424         if area > min_area:
425             found.append(_mk("text-over-text", -1, fig_text_items[i][0],
426                             fig_text_items[j][0], area))
427 for (lbl, bb) in fig_text_items:
428     if _fully_inside(bb, fig_bbox, tol):
429         continue
430     if _inter_area(bb, fig_bbox) <= min_area:
431         continue
432     found.append(_mk("clipped-out-of-figure", -1, lbl, "figure-bounds"))
433
434 return found
435
436
437 def assert_no_overlap(fig, **kwargs):
438     """Raise ``AssertionError`` listing collisions; no-op when the figure is clean."""
439     hits = collisions(fig, **kwargs)
440     if hits:
441         lines = []
442         for d in hits:
443             extra = f" ({d['overlap_px2']}px^2)" if "overlap_px2" in d else ""
444             lines.append(f" - [{d['kind']}] axes#{d['axes']}: {d['a']} X {d['b']}{extra}")
445         raise AssertionError(
446             "figqa: %d figure collision(s) detected:\n%s" % (len(hits), "\n".join(lines))
447         )
448
449
450 # -----
451 # self-test
452 # -----
453 def _build_colliding_figure():
454     import matplotlib.pyplot as plt
455
456     fig, ax = plt.subplots(figsize=(4, 3))
457     ax.bar([0, 1, 2], [3, 5, 4], label="Series A")
458     ax.set_ylim(0, 6) # tight: no headroom
459     # legend forced onto the bars
460     ax.legend(loc="center")
461     # a wide annotation straddling a foreign bar (text box only measured)
462     ax.annotate(
463         "annotation label",
464         xy=(2, 4), xytext=(2, 2),
465         ha="center", va="center",
466         arrowprops=dict(arrowstyle="->"),
467     )
468     return fig
469
470
471 def _build_line_through_label_figure():
472     """A reference line crossing a data label on ONE long segment.
473
474     Regression for the fixed-sample-count false negative: a vertical guide line
475     running the full axes height is a single 2-vertex segment, so point sampling
476     stepped over the label; segment-vs-box catches it.
477     """

```

```

478 import matplotlib.pyplot as plt
479
480 fig, ax = plt.subplots(figsize=(4, 6))
481 ax.plot([0, 10], [0, 100])
482 ax.axvline(5.0) # one tall segment, full axes height
483 ax.text(5, 20, "crossed label", ha="center", va="center")
484 ax.set_xlim(0, 10)
485 ax.set_ylim(0, 100)
486 return fig
487
488
489 def _build_clean_figure():
490     import matplotlib.pyplot as plt
491
492     fig, ax = plt.subplots(figsize=(4, 3))
493     heights = [3, 5, 4]
494     ax.bar([0, 1, 2], heights, label="Series A")
495     ax.set_ylim(0, 11) # reserved headroom for the legend
496     for x, h in enumerate(heights):
497         ax.text(x, h / 2.0, str(h), ha="center", va="center") # inside its own bar
498     ax.legend(loc="upper center") # sits in the reserved headroom
499     return fig
500
501
502 def _build_marker_only_figure():
503     """Scatter-via-plot('o') with a mid-plot label the phantom segment crossed.
504
505     Regression: linestyle 'None' draws no stroke between vertices, but the
506     segment test used to connect them anyway, so any marker chart with a
507     centred annotation falsely failed the gate.
508     """
509     import matplotlib.pyplot as plt
510
511     fig, ax = plt.subplots(figsize=(4, 4))
512     ax.plot([0, 10], [0, 100], "o", linestyle="None") # two corner markers
513     ax.text(5, 50, "centered label", ha="center", va="center")
514     ax.set_xlim(0, 10)
515     ax.set_ylim(0, 100)
516     return fig
517
518
519 def _build_marker_under_label_figure():
520     """A marker sitting under a label must still be flagged (as a point)."""
521     import matplotlib.pyplot as plt
522
523     fig, ax = plt.subplots(figsize=(4, 4))
524     ax.plot([5], [50], "o", linestyle="None", markersize=10)
525     ax.text(5, 50, "label on marker", ha="center", va="center")
526     ax.set_xlim(0, 10)
527     ax.set_ylim(0, 100)
528     return fig
529
530
531 def _build_legend_over_title_figure():
532     """A legend parked on the axes title -- invisible to the old text set."""
533     import matplotlib.pyplot as plt
534
535     fig, ax = plt.subplots(figsize=(4, 3))

```

```

536     ax.plot([0, 1, 2], [1, 3, 2], label="Series A")
537     ax.set_title("Axes title under the legend")
538     ax.legend(loc="lower center", bbox_to_anchor=(0.5, 0.98)) # on the title
539     return fig
540
541
542 def _build_suptitle_over_label_figure():
543     """A figure suptitle dropped onto the axes title (suptitle is not in
544     fig.texts, so it needs its own collection path).
545
546     The suptitle takes figure coordinates while the axes title renders in axes
547     coordinates, so the title is measured after a draw and the suptitle is
548     centred exactly on it to guarantee the overlap the test needs.
549     """
550     import matplotlib.pyplot as plt
551
552     fig, ax = plt.subplots(figsize=(4, 3))
553     ax.plot([0, 1, 2], [1, 3, 2])
554     ax.set_title("Axes title")
555     fig.canvas.draw()
556     bb = ax.title.get_window_extent(fig.canvas.get_renderer())
557     cx = (bb.x0 + bb.x1) / 2.0 / fig.bbox.width
558     cy = (bb.y0 + bb.y1) / 2.0 / fig.bbox.height
559     fig.suptitle("Figure suptitle", x=cx, y=cy, va="center")
560     return fig
561
562
563 def _self_test():
564     import matplotlib
565     matplotlib.use("Agg")
566     import matplotlib.pyplot as plt
567
568     ok = True
569
570     colliding = _build_colliding_figure()
571     hits = collisions(colliding)
572     if hits:
573         print(f"PASS colliding figure -> {len(hits)} collision(s) detected")
574         for d in hits:
575             print(f"    {d['kind']}: {d['a']} X {d['b']}")
576     else:
577         ok = False
578         print("FAIL colliding figure -> no collision detected (expected >= 1)")
579     try:
580         assert_no_overlap(colliding)
581         ok = False
582         print("FAIL assert_no_overlap did not raise on the colliding figure")
583     except AssertionError:
584         print("PASS assert_no_overlap raised on the colliding figure")
585     plt.close(colliding)
586
587     crossing = _build_line_through_label_figure()
588     hits = collisions(crossing)
589     if any(d["kind"] == "line-through-text" for d in hits):
590         print("PASS reference-line-through-label -> line-through-text detected")
591     else:
592         ok = False
593         print("FAIL reference-line-through-label -> line crossing a label not detected")

```

```

594 plt.close(crossing)
595
596 marker_only = _build_marker_only_figure()
597 hits = collisions(marker_only)
598 if any(d["kind"] == "line-through-text" for d in hits):
599     ok = False
600     print("FAIL marker-only plot -> phantom segment falsely flagged line-through-text")
601 else:
602     print("PASS marker-only plot -> no phantom line-through-text")
603 plt.close(marker_only)
604
605 marker_under = _build_marker_under_label_figure()
606 hits = collisions(marker_under)
607 if any(d["kind"] == "point-under-text" for d in hits):
608     print("PASS marker under a label -> point-under-text detected")
609 else:
610     ok = False
611     print("FAIL marker under a label -> not detected")
612 plt.close(marker_under)
613
614 over_title = _build_legend_over_title_figure()
615 hits = collisions(over_title)
616 if any(d["kind"] == "text-over-text" for d in hits):
617     print("PASS legend over the axes title -> text-over-text detected")
618 else:
619     ok = False
620     print("FAIL legend over the axes title -> not detected")
621 plt.close(over_title)
622
623 sup = _build_suptitle_over_label_figure()
624 hits = collisions(sup)
625 if any(d["kind"] == "text-over-text" for d in hits):
626     print("PASS suptitle over the axes title -> text-over-text detected")
627 else:
628     ok = False
629     print("FAIL suptitle over the axes title -> not detected")
630 plt.close(sup)
631
632 clean = _build_clean_figure()
633 hits = collisions(clean)
634 if hits:
635     ok = False
636     print(f"FAIL clean figure -> {len(hits)} unexpected collision(s):")
637     for d in hits:
638         print(f"    {d['kind']}: {d['a']} X {d['b']}")
639 else:
640     print("PASS clean figure -> zero collisions")
641 try:
642     assert_no_overlap(clean)
643     print("PASS assert_no_overlap silent on the clean figure")
644 except AssertionError as exc:
645     ok = False
646     print(f"FAIL assert_no_overlap raised on the clean figure:\n{exc}")
647 plt.close(clean)
648
649 print("figqa self-test:", "OK" if ok else "FAILED")
650 return 0 if ok else 1
651

```

```

652
653 def _usage():
654     print(
655         "usage: figqa.py --self-test\n"
656         "\n"
657         "figqa is primarily an importable gate:\n"
658         "    from figqa import assert_no_overlap\n"
659         "    assert_no_overlap(fig)  # raises on any figure collision\n"
660     )
661
662
663 def main(argv=None):
664     argv = list(sys.argv[1:] if argv is None else argv)
665     if "--self-test" in argv:
666         return _self_test()
667     if "-h" in argv or "--help" in argv:
668         _usage()
669         return 0
670     if not argv:
671         # A bare invocation in an `&&-chained gate script must not read as a
672         # passed check, so it is an error, unlike an explicit --help.
673         _usage()
674         return 2
675     print(f"figqa: unknown argument(s): {' '.join(argv)}", file=sys.stderr)
676     _usage()
677     return 2
678
679
680 if __name__ == "__main__":
681     raise SystemExit(main())

```

## src/final\_check\_joint.py

```

1 import csv,json,hashlib,math
2 from pathlib import Path
3 from analyze_joint import ROOT,paired
4 for stage,start,method in [(1,1510001,38),(2,1710001,39),(3,1910001,56)]:
5     suffix='' if stage==1 else str(stage)
6     filename='summary.json' if stage==1 else f'summary_stage{stage}.json'
7     result=json.loads((ROOT/'results'/filename).read_text(encoding='utf-8'))
8     actual=paired(f'iid{suffix}_baseline.csv',f'iid{suffix}_final.csv',start,200,method)
9     assert json.loads(json.dumps(actual))==result['iid']
10    for k in range(1,6):
11        prefix=f'stress{k}' if stage==1 else f'stress{stage}_{k}'
12        check=paired(f'{prefix}_0.csv',f'{prefix}_{method}.csv',start+10000*k,40,method)
13        assert json.loads(json.dumps(check))==result['stress'][str(k)]
14    for file in (ROOT/'baseline/src').iterdir():
15        if file.is_file():assert file.read_bytes()==(ROOT.parent/'B题动态规划/src'/file.name).
            read_bytes(),file
16    for file in (ROOT/'baseline/review').iterdir():
17        if file.is_file():assert file.read_bytes()==(ROOT.parent/'B题动态规划/review'/file.name).
            read_bytes(),file
18    data=json.loads((ROOT/'results/summary_stage3.json').read_text(encoding='utf-8'))
19    assert data['iid']['4']['saving_percent']>10
20    assert 100*(1-data['iid']['4']['new_single_s']/data['iid']['4']['baseline_single_s'])>10
21    for name,count in [('joint',26),('baseline',10)]:
22        rows=json.loads((ROOT/f'results/original_mock_{name}_depth2.json').read_text())

```

```

23     assert len(rows)==count
24     for r in rows:
25         if r['layer']=='local_HTTP':assert r['total']==r['cleared']
26         else:assert r['jammer_total']==r['cleared_count']
27 doc=(ROOT/'最终结果.md').read_text(encoding='utf-8');readme=(ROOT/'README.md').read_text(
    encoding='utf-8')
28 for p in ['3','4']:
29     for key in ['baseline_single_s','new_single_s','saving_percent']:
30         value=f"{data['iid'][p][key]:.2f}";assert value in doc and value in readme
31 bound=(133*5344+16*107*77+16*1520)/5+516*6+1840*3+32
32 assert abs(bound-182027.2)<1e-6 and bound<360000
33 assert 354574.4<360000
34 files={}
35 for file in sorted(ROOT.rglob('*')):
36     if not file.is_file() or '__pycache__' in file.parts or file.name=='manifest.json':continue
37     files[file.relative_to(ROOT).as_posix()]=hashlib.sha256(file.read_bytes()).hexdigest()
38 summary={'three_stage_paired_runs':4800,'q4_total_improvement_pct':data['iid']['4']['
    saving_percent'],'q4_single_improvement_pct':100*(1-data['iid']['4']['new_single_s']/data
    ['iid']['4']['baseline_single_s']),
39     'q3_total_improvement_pct':data['iid']['3']['saving_percent'],'
    original_sources_unchanged':True,'default_mock_runs':26,'baseline_mock_runs':10,'
    report_matches_data':True}
40 (ROOT/'manifest.json').write_text(json.dumps({'verification':summary,'sha256':files},
    ensure_ascii=False,indent=2),encoding='utf-8')
41 print(json.dumps(summary,ensure_ascii=False,indent=2))

```

## src/finalize\_pdfs.py

```

1 """Strip incidental metadata without changing page content or source results."""
2 from pathlib import Path
3 from pypdf import PdfReader,PdfWriter
4 R=Path(__file__).resolve().parents[1]
5 for source,target,title in [(R/'paper/main.pdf',R/'B题几何策略优化_论文.pdf','有界测向误差下干扰
    源的连续几何规划与清除'),(R/'B题几何策略优化_题解.pdf',R/'B题几何策略优化_题解.pdf','B题几何策
    略优化题解与计算交接')]:
6     reader=PdfReader(source);writer=PdfWriter();writer.clone_document_from_reader(reader);writer.
        metadata=None
7     writer.add_metadata({'/Title':title,'/Author':'','/Creator':'LaTeX','/Producer':'XeLaTeX'})
8     tmp=target.with_suffix('.tmp.pdf')
9     with tmp.open('wb') as f:writer.write(f)
10    check=PdfReader(tmp);assert len(check.pages)==len(reader.pages)
11    assert all(a.get_contents().get_data()==b.get_contents().get_data() for a,b in zip(reader.
        pages,check.pages))
12    tmp.replace(target);print(target.name,len(check.pages),'pages; page streams unchanged')

```

## src/geometry.hpp

```

1 #pragma once
2 #include <algorithm>
3 #include <cmath>
4 #include <limits>
5 #include <string>
6 #include <vector>
7 namespace geo {
8 constexpr double pi=3.14159265358979323846;
9 struct Pt {double x=0,y=0; Pt operator+(Pt b)const{return{x+b.x,y+b.y};} Pt operator-(Pt b)
    const{return{x-b.x,y-b.y};} Pt operator*(double t)const{return{x*t,y*t};} Pt operator/(

```

```

        double t)const{return{x/t,y/t};}};
10 inline double dot(Pt a,Pt b){return a.x*b.x+a.y*b.y;}
11 inline double cross(Pt a,Pt b){return a.x*b.y-a.y*b.x;}
12 inline double norm(Pt a){return std::hypot(a.x,a.y);}
13 inline double dist(Pt a,Pt b){return norm(a-b);}
14 inline Pt polar(double r,double deg){double t=deg*pi/180;return{r*cos(t),r*sin(t)};}
15 using Poly=std::vector<Pt>;
16 struct HP{Pt a;double b;}; // a dot x <= b
17 inline Poly hull(Poly p){
18     std::sort(p.begin(),p.end(),[](Pt a,Pt b){return a.x!=b.x?a.x<b.x:a.y<b.y;});
19     p.erase(std::unique(p.begin(),p.end(),[](Pt a,Pt b){return dist(a,b)<1e-9;}),p.end());
20     if(p.size()<3)return p;
21     Poly h(2*p.size());int k=0;
22     for(auto x:p){while(k>=2&&cross(h[k-1]-h[k-2],x-h[k-1])<=1e-10)--k;h[k++]=x;}
23     int t=k+1;for(int i=(int)p.size()-2;i>=0;--i){auto x=p[i];while(k>=t&&cross(h[k-1]-h[k-2],x-h[
        k-1])<=1e-10)--k;h[k++]=x;}h.resize(k-1);return h;
24 }
25 inline Poly clip(const Poly&p,Pt a,double b){
26     Poly q;if(p.empty())return q;
27     for(size_t i=0;i<p.size();++i){Pt x=p[i],y=p[(i+1)%p.size()];double fx=dot(a,x)-b,fy=dot(a,y)-
        b;
28         bool ix=fx<=1e-9,iy=fy<=1e-9;if(ix)q.push_back(x);
29         if(ix!=iy){double t=fx/(fx-fy);q.push_back(x+(y-x)*t);}}
30     return q;
31 }
32 inline Poly outer_disk(Pt c,double r,int n=64){Poly p;double rr=r/cos(pi/n);for(int i=0;i<n;++i
        )p.push_back(c+polar(rr,(i+.5)*360/n));return p;}
33 inline Poly disk_clip(Poly p,Pt c,double r,int n=64){for(int i=0;i<n;++i){Pt a=polar(1,i*360./n
        );p=clip(p,a,r+dot(a,c));}return p;}
34 inline Poly observe(Poly p,Pt s,double theta,double eps=1.005){
35     Pt u=polar(1,theta),v={-u.y,u.x};double t=tan(eps*pi/180);
36     for(Pt a:{v-u*t,v*(-1)-u*t})p=clip(p,a,dot(a,s));
37     return p;
38 }
39 inline bool contains(const Poly&p,Pt z){if(p.empty())return false;if(p.size()==1)return dist(p
        [0],z)<1e-7;
40     if(p.size()==2)return std::abs(cross(p[1]-p[0],z-p[0]))<1e-6&&dot(z-p[0],z-p[1])<=1e-6;
41     for(size_t i=0;i<p.size();++i)if(cross(p[(i+1)%p.size()-p[i],z-p[i])< -1e-6)return false;
42     return true;}
43 inline double diameter(const Poly&p){double d=0;for(auto a:p)for(auto b:p)d=std::max(d,dist(a,b
        ));return d;}
44 struct Circle{Pt c;double r=-1;};
45 inline bool inside(Circle c,Pt p){return c.r>=0&&dist(c.c,p)<=c.r+1e-7;}
46 inline Circle paircircle(Pt a,Pt b){return{(a+b)/2,dist(a,b)/2};}
47 inline Circle triplecircle(Pt a,Pt b,Pt c){
48     Pt u=b-a,v=c-a;double d=2*cross(u,v);
49     if(std::abs(d)<1e-12){Circle best{{},1e100};for(auto q:{paircircle(a,b),paircircle(a,c),
        paircircle(b,c)})if(inside(q,a)&&inside(q,b)&&inside(q,c)&&q.r<best.r)best=q;return best;}
50     double uu=dot(u,u),vv=dot(v,v);Pt o=a+Pt{(uu*v.y-vv*u.y)/d,(u.x*vv-v.x*uu)/d};return{o,dist(o,
        a)};
51 }
52 // Incremental enclosing circle; final radius enlarged to actual max vertex distance.
53 inline Circle mec(const Poly&p){Circle c;
54     for(size_t i=0;i<p.size();++i)if(!inside(c,p[i])){c={p[i],0};for(size_t j=0;j<i;++j)if(!inside
        (c,p[j])){c=paircircle(p[i],p[j]);for(size_t k=0;k<j;++k)if(!inside(c,p[k]))c=triplecircle
        (p[i],p[j],p[k]);}}
55     for(auto x:p)c.r=std::max(c.r,dist(x,c.c));
56     return c;

```

```

57 }
58 struct HResult{std::string kind;Poly p;};
59 inline HResult halfplanes(const std::vector<HP>&h){
60 auto feasible=[&](Pt x){for(auto q:h)if(dot(q.a,x)>q.b+1e-8)return false;return true;};
61 Poly v;bool exists=feasible({0,0});
62 for(auto q:h)if(dot(q.a,q.a)>0&&feasible(q.a*(q.b/dot(q.a,q.a))))exists=true;
63 for(size_t i=0;i<h.size();++i)for(size_t j=0;j<i;++j){auto a=h[i],b=h[j];double d=cross(a.a,b.
a);if(std::abs(d)<1e-12)continue;
64 Pt x=((a.b*b.a.y-a.a.y*b.b)/d,(a.a.x*b.b-a.b*b.a.x)/d);if(feasible(x)){v.push_back(x);exists=
true;}}
65 if(!exists)return{"empty",{}};
66 Poly dirs={{1,0},{-1,0},{0,1},{0,-1}};for(auto q:h){dirs.push_back({-q.a.y,q.a.x});dirs.
push_back({q.a.y,-q.a.x});}
67 for(auto d:dirs)if(norm(d)>1e-12){d=d/norm(d);bool ok=true,uncertain=false;
68 for(auto q:h){double v=dot(q.a,d);if(v>1e-10)ok=false;if(v>0)uncertain=true;}
69 // A positive violation accepted only by tolerance is not an unboundedness
70 // certificate: report uncertainty instead of inventing a recession ray.
71 if(ok)return{uncertain?"numerically_uncertain":"unbounded",{}};
72 }
73 return{"bounded",hull(v)};
74 }
75 inline double segment_distance(Pt z,Pt a,Pt b){Pt d=b-a;double t=dot(d,d)>0?dot(z-a,d)/dot(d,d)
:0;t=std::clamp(t,0.,1.);return dist(z,a+d*t);}
76 inline bool meets_disk(Poly p,double r){if(contains(hull(p),{0,0}))return true;for(size_t i=0;i
<p.size();++i)if(segment_distance({0,0},p[i],p[(i+1)%p.size()])<=r+1e-7)return true;return
false;}
77 inline Poly coverage(int problem,double h=950,bool square=false){
78 if(problem==3){Poly p={{0,0}};for(int i=0;i<6;++i)p.push_back(polar(1200,i*60));return p;}
79 Poly p={{0,0}};int n=(int)ceil(4000/h)+2;
80 auto v=[&](int i,int j){return square?Pt{h*(i+.5*j),h*sqrt(3.)/2*j};};
81 for(int i=-n;i<n;++i)for(int j=-n;j<n;++j){Pt a=v(i,j),b=v(i+1,j),c=v(i,j+1),d=v(i+1,j+1);
82 if(square){Poly q={a,b,d,c};if(meets_disk(q,1800))p.insert(p.end(),q.begin(),q.end());}
83 else for(auto q:{Poly{a,b,c},Poly{d,c,b}})if(meets_disk(q,1800))p.insert(p.end(),q.begin(),q.
end());}
84 Poly unique;for(auto x:p){bool duplicate=false;for(auto y:unique)if(dist(x,y)<1e-6)duplicate=
true;if(!duplicate)unique.push_back(x);}return unique;
85 }
86 }

```

## src/joint\_config.hpp

```

1 #pragma once
2 #include "configurations.hpp"
3 #define B_SHORTCUT_CONFIG 1
4 inline Options joint_configuration(int method,int problem){
5 if(method==84){auto o=joint_configuration(problem==3?78:81,problem);o.shortcut=true;return o;}
6 if(method==86){auto o=joint_configuration(83,problem);if(problem==4)o.neighborhood_variable=
true;return o;}
7 if(method==85){auto o=joint_configuration(81,problem);if(problem==4)o.residual_opportunistic=
true;return o;}
8 if(method>=79&&method<=84){auto o=joint_configuration(56,problem);if(method==79)o.robust_mode
=1;if(method==80)o.robust_mode=2;if(method==81&&problem==4)o.residual_cover=true;if(method
==82)o.robust_mode=3;if(method==83)o.neighborhood_tour=true;return o;}
9 if(method==78){auto o=joint_configuration(56,problem);o.shortcut=true;return o;}
10 if(method>=74&&method<=77){auto o=joint_configuration(56,problem);if(problem==4)o.
optical_portfolio=method-73;return o;}

```

```

11 if(method>=70&&method<=73){auto o=joint_configuration(56,problem);o.bearing_bin=method==70?4:
    method==71?3:method==72?6:8;return o;}
12 if(method>=67&&method<=69){auto o=joint_configuration(56,problem);o.refinement=method==69?2:1;
    if(method==68&&problem==3)o.planning_samples=63;return o;}
13 if(method==66){auto o=joint_configuration(56,problem);o.budget_aware=true;return o;}
14 if(method>=61&&method<=65){auto o=joint_configuration(method==61||method==63?56:60,problem);if
    (problem==4)o.optical_search=method<=62?1:method<=64?2:3;return o;}
15 if(method==60){auto o=joint_configuration(56,problem);if(problem==4)o.variable_optical=true;
    return o;}
16 if(method==58){auto o=joint_configuration(56,problem);if(problem==4)o.terminal_mode=1;return o
    ;}
17 if(method==59){auto o=joint_configuration(58,problem);if(problem==4)o.terminal_mode=2;return o
    ;}
18 if(method>=55&&method<=57){auto o=joint_configuration(method==55?38:39,problem);if(problem==4)
    o.adaptive_optical=true;if(method==57)o.optical_weight=.5;return o;}
19 if(method==53||method==54){auto o=joint_configuration(39,problem);if(problem==4)o.
    dynamic_limit=method==53?10:14;return o;}
20 if(method>=50&&method<=52){auto o=joint_configuration(39,problem);o.discovery_weight=method
    ==50?.25:method==51?.5:1.;return o;}
21 if(method>=45&&method<=49){auto o=joint_configuration(method==48?40:method==49?42:39,problem);
    o.immediate_fee_weight=method==45?.25:method==47?1:.5;return o;}
22 if(method>=40&&method<=44){auto o=joint_configuration(method==41?38:39,problem);if(method
    ==40||method==41||method==44)o.use_lns=true;if(method==42||method==44)o.optical_weight=.5;
    if(method==43)o.optical_weight=.25;return o;}
23 if(method==39){auto o=joint_configuration(38,problem);o.weighted_optical=true;return o;}
24 if(method==38)return joint_configuration(problem==3?32:27,problem);
25 if(method>=34&&method<=37){auto o=joint_configuration(method==34||method==37?29:method
    ==35?21:27,problem);o.defer_single=true;if(method==36)o.auxiliary=1;if(method==37)o.
    q3_radius=1200;return o;}
26 if(method==32||method==33){auto o=joint_configuration(method==32?29:27,problem);o.
    clear_neighborhood=true;return o;}
27 auto o=configuration(5);o.dynamic_depth=2;o.active_mode=problem==3?1:0;
28 if(method==1)o.auxiliary=1;
29 if(method==2)o.joint_actions=true;
30 if(method==3){o.auxiliary=1;o.joint_actions=true;}
31 if(method==4)o.route_model=1;
32 if(method==5){o.auxiliary=2;o.joint_actions=true;}
33 if(method==6){o.auxiliary=2;}
34 if(method>=7){o.auxiliary=2;o.joint_actions=true;}
35 if(method==7)o.start_residual=.5;
36 if(method==8)o.start_residual=1;
37 if(method==9)o.start_residual=2;
38 if(method==10)o.retreat=true;
39 if(method==11){o.retreat=true;o.start_residual=1;}
40 if(method==12){o.auxiliary=0;o.direct_optical=true;}
41 if(method==13){o.direct_optical=true;o.optical_rotate=true;}
42 if(method==14){o.candidate_menu=true;}
43 if(method==15){o.candidate_menu=true;o.complete_source=true;}
44 if(method==16){o.candidate_menu=true;o.complete_source=true;o.direct_optical=true;o.
    optical_rotate=true;}
45 if(method==17){o.auxiliary=0;o.direct_optical=true;o.optical_rotate=true;}
46 if(method==18){o.auxiliary=0;o.direct_optical=true;o.retreat=true;}
47 if(method==19){o.auxiliary=3;o.joint_actions=false;}
48 if(method==20){o.auxiliary=3;}
49 if(method==21){o.auxiliary=4;o.direct_optical=true;}
50 if(method==22){o.auxiliary=4;o.joint_actions=false;}
51 if(method>=23){o.auxiliary=0;o.direct_optical=true;}
52 if(method==23||method==24||method==25||method==26||method==28)o.dynamic_depth=3;

```

```

53 if(method==24)o.optical_rotate=true;
54 if(method==25)o.candidate_menu=true;
55 if(method==26){o.candidate_menu=true;o.complete_source=true;}
56 if(method==27||method==28)o.planning_samples=63;
57 if(method==29){o.q3_radius=1124;o.candidate_menu=true;o.auxiliary=2;o.direct_optical=false;}
58 if(method==30){o.q3_radius=1124;o.dynamic_depth=3;o.optical_rotate=true;}
59 if(method==31){o.q3_radius=1150;o.candidate_menu=true;o.auxiliary=2;o.direct_optical=false;}
60 return o;
61 }

```

## src/lns.hpp

```

1 #pragma once
2 #include "certified_route.hpp"
3 #include <random>
4 namespace new_route {
5 // Deterministic, non-neural destroy/repair prototype. This is not NLNS or LKH.
6 inline std::vector<int> repair(const cert_route::Problem&p,std::vector<int> r,std::vector<int>
    missing){
7 while(!missing.empty()){
8 int chosen=-1,at=-1;double regret=-1,bestCost=1e100;
9 for(int a=0;a<(int)missing.size();++a){double best=1e100,second=1e100;int index=0;
10 for(int j=0;j<=(int)r.size();++j){auto q=r;q.insert(q.begin()+j,missing[a]);double c=
    cert_route::length(p,q);if(c<best){second=best;best=c;index=j;}else second=std::min(second
    ,c);}
11 double diff=second<1e99?second-best:0;
12 if(diff>regret+1e-8||(std::abs(diff-regret)<1e-8&&best<bestCost)){regret=diff;bestCost=best;
    chosen=a;at=index;}
13 }
14 r.insert(r.begin()+at,missing[chosen]);missing.erase(missing.begin()+chosen);
15 }return r;
16 }
17 inline cert_route::Solution solve(const cert_route::Problem&p){
18 auto s=cert_route::solve(p);int n=p.start.size();if(s.exact)return s;
19 std::mt19937 rng(120926+n);
20 for(int iteration=0;iteration<10;++iteration){
21 auto r=s.path;std::vector<int>missing;int k=3+iteration%4;
22 for(int t=0;t<k;++t){int j=(iteration%2==0)?int(rng()%r.size()):int((iteration+t)%r.size());
    missing.push_back(r[j]);r.erase(r.begin()+j);}
23 r=repair(p,r,missing);
24 if(cert_route::length(p,r)<s.upper-1e-7||iteration%3==0)cert_route::improve(p,r,true);
25 double cost=cert_route::length(p,r);if(cost<s.upper-1e-7){s.path=r;s.upper=cost;}
26 }
27 s.local_optimal=cert_route::improve(p,s.path,true);s.upper=cert_route::length(p,s.path);return
    s;
28 }
29 }

```

## src/neighborhood\_tour.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 #include "bearing_types.hpp"
4 namespace neighborhood_tour {
5 inline bool valid(geo::Pt q,const geo::Poly&cell){for(auto z:cell)if(geo::dist(q,z)>20-1e-6)
    return false;return true;}

```

```

6 inline geo::Pt project(geo::Pt q,const geo::Poly&cell){for(int pass=0;pass<24;++pass){double
    shift=0;for(auto z:cell){double d=geo::dist(q,z);if(d>20-1e-5){auto next=z+(q-z)*((20-1e
    -5)/d);shift=std::max(shift,geo::dist(q,next));q=next;}}if(shift<1e-8)break;}return q;}
7 inline double length(const geo::Poly&r,geo::Pt start){double out=0;for(auto q:r){out+=geo::dist
    (start,q);start=q;}return out;}
8 inline geo::Poly optimize(geo::Poly route,const std::vector<geo::Poly>&cells,geo::Pt start){
9     if(route.size()!=cells.size())throw std::runtime_error("neighborhood size mismatch");
10    for(int pass=0;pass<5;++pass)for(size_t ii=0;ii<route.size();++ii){size_t i=pass%2?route.size
        ()-1-ii:ii;if(cells[i].empty())continue;
11    auto prev=i?route[i-1]:start;auto next=i+1<route.size()?route[i+1]:prev;
12    auto cost=[&](geo::Pt q){return geo::dist(prev,q)+(i+1<route.size()?geo::dist(q,next):0);};
13    auto best=route[i];double value=cost(best);auto trial=[&](geo::Pt q){q=project(q,cells[i]);if
        (valid(q,cells[i])&&cost(q)<value-1e-8){best=q;value=cost(q);}};
14    auto d=next-prev;double fraction=geo::dot(d,d)>0?std::clamp(geo::dot(best-prev,d)/geo::dot(d,
        d),0.,1.):0;
15    trial(prev+d*fraction);trial((prev+next)/2);
16    for(int step=0;step<12;++step){auto a=best-prev,b=best-next;auto grad=a/std::max(1e-9,geo::
        norm(a));if(i+1<route.size())grad=grad+b/std::max(1e-9,geo::norm(b));trial(best-grad
        *(8./(1+step)));}
17    route[i]=best;
18 }
19 return route;
20 }
21 inline double expected(const geo::Poly&r,geo::Pt start,const std::vector<dynamic_selection::
    Hypothesis>&h){double mass=0,out=0;for(auto s:h){double t=0;auto q=start;bool found=false;
    for(auto p:r){t+=geo::dist(q,p)/5;q=p;if(geo::dist(p,s.z)<=20){t+=5;found=true;break;}t
    +=3;}if(!found)return INFINITY;mass+=s.weight;out+=s.weight*t;}return mass>0?out/mass:
    INFINITY;}
22 inline geo::Poly optical(const geo::Poly&base,const geo::Poly&centers,const std::vector<geo::
    Poly>&original_cells,geo::Pt start,const std::vector<dynamic_selection::Hypothesis>&h){
23     std::vector<geo::Poly>cells;std::vector<bool>used(centers.size());
24     for(auto q:base){bool mapped=false;for(size_t i=0;i<centers.size();++i)if(!used[i]&&geo::dist(
        q,centers[i])<1e-7){used[i]=true;cells.push_back(original_cells[i]);mapped=true;break;}if
        (!mapped)return base;}
25     auto best=base;double bestlen=length(base,start),bestexp=expected(base,start,h);
26     auto consider=[&](geo::Poly candidate,const std::vector<geo::Poly>&regions){candidate=optimize
        (candidate,regions,start);double l=length(candidate,start),e=expected(candidate,start,h);
        if(l<bestlen-1e-6&&e<bestexp-1e-6){best=candidate;bestlen=l;bestexp=e;}};
27     consider(base,cells);
28     // Jointly reorder the neighborhoods, then resolve their visit positions.
29     auto tour=base;auto regions=cells;
30     for(int pass=0;pass<2;++pass){double gain=0;size_t ai=0,aj=0;for(size_t i=0;i<tour.size();++i)
        for(size_t j=i+1;j<tour.size();++j){auto a=i?tour[i-1]:start;double old=geo::dist(a,tour[i
        ]),fresh=geo::dist(a,tour[j]);if(j+1<tour.size()){old+=geo::dist(tour[j],tour[j+1]);fresh
        +=geo::dist(tour[i],tour[j+1]);}if(old-fresh>gain){gain=old-fresh;ai=i;aj=j;}}if(gain<1e
        -7)break;std::reverse(tour.begin()+ai,tour.begin()+aj+1);std::reverse(regions.begin()+ai,
        regions.begin()+aj+1);tour=optimize(tour,regions,start);consider(tour,regions);}
31     return best;
32 }
33 }

```

## src/optical\_order.hpp

```

1 #pragma once
2 #include "bearing_types.hpp"
3 #include <numeric>
4 #include <cstdint>

```

```

5 #include <stdexcept>
6 #define B_SEARCH_ORDER 1
7 namespace optical_order {
8 inline geo::Poly choose(const geo::Poly&p,geo::Pt now,const std::vector<dynamic_selection::
    Hypothesis>&original,double mixture=1.){
9   if(p.empty())return p;
10  auto fallback=p;if(geo::dist(now,fallback.back())<geo::dist(now,fallback.front()))std::reverse
    (fallback.begin(),fallback.end());
11  if(original.empty())return fallback;
12  auto hypotheses=original;
13  if(mixture<1.){double mass=0;for(auto h:hypotheses)mass+=h.weight;for(auto &h:hypotheses)h.
    weight*=mixture/mass;for(auto q:p)hypotheses.push_back({q,1000,false,0,(1-mixture)/p.size
    ()});}
14  double total=0;for(auto h:hypotheses)total+=h.weight;double best=1e100;auto chosen=fallback;
15  int n=p.size();std::vector<int>offsets{0,n-1};for(int i=1;i<=12;++i)offsets.push_back((i*n)
    /13);
16  for(int reverse=0;reverse<2;++reverse)for(int offset:offsets){geo::Poly route;for(int k=0;k<n
    ;++k){int j=reverse?(offset-k+n)%n:(offset+k)%n;route.push_back(p[j]);}
17  std::vector<bool>done(hypotheses.size());int pending=hypotheses.size();double cost=0,expected
    =0;auto position=now;
18  for(auto q:route){cost+=geo::dist(position,q)/5;position=q;
19  for(int i=0;i<(int)hypotheses.size();++i)if(!done[i]&&geo::dist(q,hypotheses[i].z)<=20*1e-7)
    {done[i]=true;--pending;expected+=hypotheses[i].weight/total*(cost+5);}
20  if(!pending)break;cost+=3;
21  }
22  if(pending)continue;if(expected<best){best=expected;chosen=route;}
23  }return chosen;
24  }
25  // Minimize expected time to first successful clear, retaining the entire cover.
26  // The cell-center mixture is a ranking regularizer, never a geometric proof.
27  inline geo::Poly search(const geo::Poly&p,geo::Pt now,const std::vector<dynamic_selection::
    Hypothesis>&original,double mixture=.1){
28  if(p.empty()||original.empty())return choose(p,now,original);
29  if(!(mixture>0&&mixture<=1))throw std::invalid_argument("invalid search mixture");
30  auto h=original;double mass=0;for(auto x:h)mass+=x.weight;
31  for(auto &x:h)x.weight*=(1-mixture)/mass;
32  if(mixture>0)for(auto q:p)h.push_back({q,1000,false,0,mixture/p.size()});
33  const int n=p.size(),chunks=(h.size()+63)/64;
34  std::vector<std::vector<uint64_t>> hits(n,std::vector<uint64_t>(chunks));
35  std::vector<std::vector<double>> travel(n,std::vector<double>(n));std::vector<double>entry(n);
36  for(int i=0;i<n;++i){entry[i]=geo::dist(now,p[i])/5;for(int j=0;j<n;++j)travel[i][j]=geo::dist
    (p[i],p[j])/5;
37  for(size_t j=0;j<h.size();++j)if(geo::dist(p[i],h[j].z)<=20*1e-7)hits[i][j/64]|=uint64_t(1)
    <<(j%64);
38  }
39  std::vector<uint64_t>full(chunks,~uint64_t(0));if(h.size()%64)full.back()=(uint64_t(1)<<(h.
    size()%64))-1;
40  auto cost=[&](const std::vector<int>&order)->double{auto remaining=full;size_t pending=h.size
    ();double elapsed=0,expected=0;int previous=-1;
41  for(int j:order){elapsed+=previous<0?entry[j]:travel[previous][j];previous=j;
42  for(int k=0;k<chunks;++k){uint64_t found=remaining[k]&hits[j][k];remaining[k]&=~found;
43  while(found){int bit=__builtin_ctzll(found);expected+=h[k*64+bit].weight*(elapsed+5);--
    pending;found&=found-1;}
44  }if(!pending)return expected;elapsed+=3;
45  }return INFINITY;
46  };
47  auto cyclic=choose(p,now,h);std::vector<int>best_order;std::vector<bool>mapped(n);

```

```

48 for(auto q:cyclic)for(int i=0;i<n;++i)if(!mapped[i]&&geo::dist(q,p[i])<1e-8){mapped[i]=true;
    best_order.push_back(i);break;}
49 if(best_order.size()!=p.size())throw std::runtime_error("search route permutation mismatch");
50 double best=cost(best_order);
51 for(double power:{.5,1.,1.5}){std::vector<int>order;std::vector<bool>used(n);auto remaining=
    full;int previous=-1;
52 for(int step=0;step<n;++step){int chosen=-1;double score=-1;
53 for(int j=0;j<n;++j)if(!used[j]){double probability=0;
54 for(int k=0;k<chunks;++k){uint64_t found=remaining[k]&hits[j][k];while(found){int bit=
    __builtin_ctzll(found);probability+=h[k*64+bit].weight;found&=found-1;}}
55 double seconds=3+(previous<0?entry[j]:travel[previous][j]);double value=(probability+1e-12)
    /std::pow(seconds,power);
56 if(value>score){score=value;chosen=j;}
57 }used[chosen]=true;order.push_back(chosen);previous=chosen;for(int k=0;k<chunks;++k)
    remaining[k]&=~hits[chosen][k];
58 }double value=cost(order);if(value<best){best=value;best_order=std::move(order);}
59 }
60 for(int pass=0;pass<2;++pass){auto chosen=best_order;double improved=best;
61 for(int i=0;i<n-1;++i)for(int j=i+1;j<n;++j){auto candidate=best_order;std::reverse(candidate
    .begin()+i,candidate.begin()+j+1);double value=cost(candidate);if(value<improved-1e-10){
    improved=value;chosen=std::move(candidate);}}
62 if(improved>=best-1e-10)break;best=improved;best_order=std::move(chosen);
63 }
64 geo::Poly route;for(int i:best_order)route.push_back(p[i]);return route;
65 }
66 }

```

## src/optical\_portfolio.hpp

```

1 #pragma once
2 #include "optical_terminal.hpp"
3 #define B_OPTICAL_PORTFOLIO 1
4 namespace optical_portfolio {
5 inline geo::Poly choose(const geo::Poly& incumbent,const geo::Poly&candidate,geo::Pt now,const
    std::vector<dynamic_selection::Hypothesis>&h,double fraction,double seconds){
6 if(h.empty()||candidate.empty())return incumbent;
7 double before=optical_terminal::expected(incumbent,now,h),after=optical_terminal::expected(
    candidate,now,h);
8 return after<before*(1-fraction)&&before-after>seconds?candidate:incumbent;
9 }
10 }

```

## src/optical\_terminal.hpp

```

1 #pragma once
2 #include "adaptive_optical.hpp"
3 #include "optical_order.hpp"
4 #include <map>
5 #include <memory>
6 #define B_PREPARED_SWEEP 1
7 namespace optical_terminal {
8 // Geometric branch regions remain separate from finite ranking hypotheses.
9 // Prediction bins currently group noiseless bearings, not noisy report draws.
10 // Their angular envelope bounds geometry; branch probabilities remain approximate.
11 inline geo::Poly update(geo::Poly region,geo::Pt q,int outcome){
12 if(outcome<10)return region;

```

```

13 double angle=2.*(outcome-10)+1.;
14 region=geo::observe(region,q,angle,2.005);
15 region=geo::disk_clip(region,q,1500);
16 auto u=geo::polar(1,angle);
17 return geo::clip(region,u,geo::dot(u,q)+1500);
18 }
19 inline double expected(const geo::Poly& route,geo::Pt from,const std::vector<dynamic_selection
    ::Hypothesis>& hypotheses){
20 double total=0,cost=0,value=0;for(auto h:hypotheses)total+=h.weight;
21 if(hypotheses.empty())return 0;
22 std::vector<bool> done(hypotheses.size());size_t left=hypotheses.size();
23 for(auto q:route){cost+=geo::dist(from,q)/5;from=q;
24 for(size_t i=0;i<hypotheses.size();++i)if(!done[i]&&geo::dist(q,hypotheses[i].z)<=20+1e-7){
    done[i]=true;--left;value+=hypotheses[i].weight/total*(cost+5);}
25 if(!left)return value;cost+=3;
26 }
27 throw std::runtime_error("terminal route does not cover conditional hypotheses");
28 }
29 class PreparedSweep {
30 geo::Poly points;
31 std::vector<uint64_t> hits;
32 std::vector<std::vector<double>>travel;
33 std::vector<double>weights;
34 public:
35 PreparedSweep(const geo::Poly& p,const std::vector<dynamic_selection::Hypothesis>& h):points(p
    ){
36 if(p.empty()||h.empty()||h.size()>63)throw std::invalid_argument("invalid prepared sweep");
37 for(auto x:h)weights.push_back(x.weight);
38 hits.assign(p.size(),0);travel.assign(p.size(),std::vector<double>(p.size()));
39 for(size_t i=0;i<p.size();++i){for(size_t j=0;j<h.size();++j)if(geo::dist(p[i],h[j].z)<=20+1e
    -7)hits[i]|=uint64_t(1)<<j;
40 for(size_t j=0;j<p.size();++j)travel[i][j]=geo::dist(p[i],p[j])/5;
41 }
42 }
43 double value(uint64_t mask,geo::Pt now,bool match_entry=true)const{
44 if(!mask)return 0;
45 if(mask>>weights.size())throw std::invalid_argument("invalid sweep mask");
46 double total=0;for(size_t i=0;i<weights.size();++i)if(mask>>i&1)total+=weights[i];
47 int n=points.size();bool flip=match_entry&&geo::dist(now,points.back())<geo::dist(now,points.
    front());
48 std::vector<int>offsets{0,n-1};for(int i=1;i<=12;++i)offsets.push_back(i*n/13);
49 std::vector<double>entry;for(auto p:points)entry.push_back(geo::dist(now,p)/5);
50 double best=INFINITY;
51 for(int reverse=0;reverse<2;++reverse)for(int offset:offsets){
52 uint64_t remaining=mask;double cost=0,value=0;int previous=-1;
53 for(int k=0;k<n;++k){int logical=reverse?(offset-k+n)%n:(offset+k)%n;int j=flip?n-1-logical:
    logical;
54 cost+=previous<0?entry[j]:travel[previous][j];previous=j;
55 uint64_t found=remaining&hits[j];remaining&=~found;
56 while(found){int i=__builtin_ctzll(found);value+=weights[i]/total*(cost+5);found&=found-1;}
57 if(!remaining)break;cost+=3;
58 }
59 if(!remaining&&value<best)best=value;
60 }
61 if(!std::isfinite(best))throw std::runtime_error("terminal route does not cover conditional
    hypotheses");
62 return best;
63 }

```

```

64 };
65 class Contexts {
66     struct State {geo::Poly region;std::unique_ptr<PreparedSweep>sweep;std::map<std::pair<int,int>,int>children;};
67     std::vector<State> states;
68     const std::vector<geo::Pt>&locations;
69     const std::vector<bool>&sensing;
70     const std::vector<dynamic_selection::Hypothesis>&hypotheses;
71     geo::Pt first;double angle;bool match_entry;
72     public:
73     Contexts(const geo::Poly& p,const std::vector<geo::Pt>& q,const std::vector<bool>& m,const std::vector<dynamic_selection::Hypothesis>& h,geo::Pt f,double a,bool match=false):locations(q),sensing(m),hypotheses(h),first(f),angle(a),match_entry(match){states.push_back({p,{},{}});}
74     int advance(int context,int action,int outcome){
75         // Negative readings and failed clears do not yet have a continuous set update.
76         if(!sensing.at(action)||outcome<10)return context;
77         auto key=std::make_pair(action,outcome);auto it=states.at(context).children.find(key);
78         if(it!=states[context].children.end())return it->second;
79         auto region=update(states[context].region,locations.at(action),outcome);
80         if(region.empty())throw std::runtime_error("empty predicted continuous region");
81         int id=states.size();states[context].children[key]=id;states.push_back({std::move(region),{},{},{}});return id;
82     }
83     double value(uint64_t mask,geo::Pt now,int context){
84         auto& state=states.at(context);
85         if(!state.sweep)state.sweep=std::make_unique<PreparedSweep>(adaptive_optical::cover(state.region,first,angle),hypotheses);
86         return state.sweep->value(mask,now,match_entry);
87     }
88 };
89 }

```

## src/package\_delivery.py

```

1 """Create an anonymous, source-only support archive and verify relocation."""
2 import argparse,hashlib,json,shutil,subprocess,sys,tempfile,time,zipfile
3 from pathlib import Path
4 R=Path(__file__).resolve().parents[1]
5 p=argparse.ArgumentParser();p.add_argument('--check-relocation',action='store_true');a=p.parse_args()
6 final=R/'B题几何策略优化_代码与数据.zip'
7 sha=lambda p:hashlib.sha256(p.read_bytes()).hexdigest()
8 with tempfile.TemporaryDirectory(prefix='b-delivery-') as temp:
9     stage=Path(temp)/'B_solution';stage.mkdir()
10    for folder in ['src','tests','results','artifacts','paper','figures','inputs','evidence','vendor','review']:
11        def ignore(path, names):
12            return [n for n in names if n=='__pycache__' or n.endswith((''.pyc',''.aux',''.log',''.out',''.png')) or n in ['paper_pages','solution_pages','main.pdf','panel-aggregate.txt']]
13        shutil.copypath(R/folder,stage/folder,ignore=ignore)
14    for n in ['run_all.py','README.md','requirements.txt','modeling-manifest.json','validation_frozen.json','B题几何策略优化_题解.md','B题几何策略优化_题解.pdf','B题几何策略优化_论文.pdf']:
15        shutil.copy(R/n,stage/n)
16    # Prefix normalization only affects delivery copies, never frozen source or CSV.
17    # Original absolute commands remain in the user's research directory.

```

```

18 normalized=[]
19 for f in stage.rglob('*'):
20     if not f.is_file() or f.suffix not in {'.json','.md','.txt','.tex','.py','.cpp','.hpp','.yaml'
        '}:continue
21     data=f.read_text();new=data.replace(str(R),'PROJECT_ROOT').replace(str(R.parent),'
        WORKSPACE_ROOT')
22     # Tool invocation metadata may mention installed skill or runtime paths.
23     new=new.replace(str(Path.home()),'USER_HOME')
24     if new!=data:f.write_text(new);normalized.append(f.relative_to(stage).as_posix())
25 # Hashes point to the exported files after path normalization.
26 rep=json.loads((stage/'artifacts/reproducibility.json').read_text())
27 rep['output_hashes']={k:sha(stage/k) for k in rep['output_hashes']}
28 rep['delivery_path_normalization']='Host/user path prefixes replaced for anonymity only;
        original records preserved in the research root.'
29 (stage/'artifacts/reproducibility.json').write_text(json.dumps(rep,ensure_ascii=False,indent
        =2))
30 (stage/'PACKAGE.md').write_text('本包为可复现的本地研究交付，六份官方加密日志仍未提供。\\n\\n解压后
        进入本目录运行 python3 run_all.py; --full重跑固定独立/压力集。无需原工作目录，目录可改名。运行
        前安装requirements.txt所列依赖和C++17编译器。PDF重建另需XeLaTeX/ctex/Pandoc及字体。\\n\\n为匿名
        交付，命令记录中的绝对用户路径已替换为PROJECT_ROOT/WORKSPACE_ROOT/USER_HOME；数值CSV和冻结模
        型源码未改动。原始路径记录保留在研究目录。以PACKAGE_HASHES.json校验包内文件，校验表不包含自
        身。\\n')
31 if a.check_relocation:
32     check=Path(temp)/'renamed_portable_solver';shutil.copypath(stage,check)
33     begin=time.time();run=subprocess.run([sys.executable,'run_all.py'],cwd=check,capture_output=
        True,text=True)
34     (R/'artifacts/relocation.log').write_text(run.stdout+'\\n'+run.stderr)
35     assert run.returncode==0,run.stderr[-4000:]
36     old=json.loads((R/'artifacts/summary.json').read_text());new=json.loads((check/'artifacts/
        summary.json').read_text());assert old==new
37     cert=json.loads((check/'artifacts/regression-tests.json').read_text());replay=json.loads((
        check/'artifacts/residual-fraction-replay.json').read_text());assert cert['records'][0]['
        fresh_witness_sha256']==replay['witness_sha256']
38     report={'status':'PASS','command':[sys.executable,'run_all.py'],'location':'fresh temporary
        directory named renamed_portable_solver','exit_code':run.returncode,'runtime_s':time.time
        ()-begin,'summary_equal':True,'fresh_witness_replay':True,'scope':'Recompiled both
        binaries, all11 regression programs, Q2/Q3/Q4 certificates and CSV summary. Does not
        rerun2000 model cases or any official test.'}
39     (R/'artifacts/relocation.json').write_text(json.dumps(report,ensure_ascii=False,indent=2));
        shutil.copy(R/'artifacts/relocation.json',stage/'artifacts/relocation.json')
40     manifest={f.relative_to(stage).as_posix():sha(f) for f in stage.rglob('*') if f.is_file()}
41     (stage/'PACKAGE_HASHES.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2))
42     with zipfile.ZipFile(final,'w',zipfile.ZIP_DEFLATED,compresslevel=9) as z:
43         for f in sorted(stage.rglob('*')):
44             if f.is_file():z.write(f,Path(stage.name)/f.relative_to(stage))
45     assert final.stat().st_size<20*1024*1024
46     with zipfile.ZipFile(final) as z:
47         assert z.testzip() is None
48         for p,h in manifest.items():assert hashlib.sha256(z.read('B_solution/'+p)).hexdigest()==h
49     out={'status':'PASS','archive':final.name,'sha256':sha(final),'bytes':final.stat().st_size,'
        files':len(manifest)+1,'source_only':True,'path_normalized_files':normalized,'
        formal_logs_present':False}
50 (R/'artifacts/package-verification.json').write_text(json.dumps(out,ensure_ascii=False,indent
        =2));print(json.dumps({k:v for k,v in out.items() if k!='path_normalized_files'},
        ensure_ascii=False))

```

## src/pair\_analysis.py

```
1 """Paired, frozen-method evaluation. This script never selects a controller."""
2 import argparse
3 import csv
4 import json
5 import math
6 import statistics as stats
7 from collections import defaultdict
8 from pathlib import Path
9
10
11 def interval(values):
12     center=stats.mean(values)
13     half=1.96*stats.stdev(values)/math.sqrt(len(values)) if len(values)>1 else 0.
14     return [center-half,center+half]
15
16
17 def summarize(old,new):
18     a=[r['time_s'] for r in old]
19     b=[r['time_s'] for r in new]
20     differences=[x-y for x,y in zip(a,b)]
21     single_old=stats.mean(r['avg_s'] for r in old)
22     single_new=stats.mean(r['avg_s'] for r in new)
23     ratio=stats.mean(b)/stats.mean(a)
24     centered=[y-ratio*x for x,y in zip(a,b)]
25     se=stats.stdev(centered)/math.sqrt(len(a))/stats.mean(a) if len(a)>1 else 0.
26     cpu=[r['runtime_s'] for r in new]
27     return dict(cases=len(old),baseline_total_s=stats.mean(a),candidate_total_s=stats.mean(b),
28         total_gain_pct=100*(1-ratio),total_gain_ci95_pct=[100*(1-ratio-1.96*se),100*(1-ratio
29         +1.96*se)],
30         baseline_single_s=single_old,candidate_single_s=single_new,single_gain_pct=100*(1-
31         single_new/single_old),
32         saving_s_mean=stats.mean(differences),saving_s_ci95=interval(differences),
33         wins=sum(x>1e-4 for x in differences),ties=sum(abs(x)<=1e-4 for x in differences),
34         losses=sum(x < -1e-4 for x in differences),
35         worst_saving_s=min(differences),best_saving_s=max(differences),cpu_mean_s=stats.mean(
36         cpu),cpu_max_s=max(cpu))
37
38
39 def read_rows(files):
40     rows={}
41     for file in files:
42         with open(file,newline='',encoding='utf-8-sig') as stream:
43             for raw in csv.DictReader(stream):
44                 key=tuple(int(raw[x]) for x in ('problem','seed','stress'))
45                 if key in rows:raise ValueError(f'duplicate case {key}')
46                 row=dict(raw)
47                 for k in ('time_s','avg_s','runtime_s','move_m'):row[k]=float(row[k])
48                 if int(row['n'])!=int(row['cleared']):raise ValueError(f'incomplete clearance {
49                 key}')
50                 rows[key]=row
51     if not rows:raise ValueError('empty input')
52     return rows
53
54
55 def main():
56     parser=argparse.ArgumentParser()
```

```

52 parser.add_argument('--baseline',nargs='+',required=True)
53 parser.add_argument('--candidate',nargs='+',required=True)
54 parser.add_argument('--output',required=True)
55 parser.add_argument('--expected-per-problem',type=int,required=True)
56 parser.add_argument('--shortcut-check',action='store_true')
57 args=parser.parse_args()
58 old,new=read_rows(args.baseline),read_rows(args.candidate)
59 if old.keys()!=new.keys():raise ValueError('paired case keys differ')
60 if {int(r['method']) for r in old.values()}!={56}:raise ValueError('comparison baseline
    must be frozen method56')
61 methods={int(r['method']) for r in new.values()}
62 if len(methods)!=1:raise ValueError('candidate policies must not be pooled')
63 groups=defaultdict(list)
64 unchanged=('n','cleared','measures','switches','miss','fallbacks','dp_calls','dp_actions','
    dp_fallbacks','dp_expanded','dp_budget_hits','aux_calls','aux_positive','joint_steps','
    certificate')
65 for key in sorted(old):
66     if args.shortcut_check:
67         for field in unchanged:
68             if old[key][field]!=new[key][field]:raise ValueError(f'controller changed {key
    }: {field}')
69             delta=old[key]['time_s']-new[key]['time_s']
70             if delta < -1e-4:raise ValueError(f'shortcut regressed {key}: {delta}')
71             if abs(delta-float(new[key]['shortcut_saved_m'])/5)>1e-4:raise ValueError(f'
    shortcut accounting mismatch {key}')
72             groups[(key[0],key[2])].append(key)
73 result=dict(baseline_method=56,candidate_method=methods.pop(),paired_cases=len(old),groups
    ={})
74 for (problem,stress),keys in groups.items():
75     if len(keys)!=args.expected_per_problem:raise ValueError(f'unexpected group count {
    problem,stress,len(keys)}')
76     a,b=[old[k] for k in keys],[new[k] for k in keys]
77     row=summarize(a,b)
78     row['all_cleared']=True
79     row['worst_seed']=keys[min(range(len(keys)),key=lambda i:a[i]['time_s']-b[i]['time_s'])
    ][1]
80     row['budget_hits']=sum(int(r['dp_budget_hits']) for r in b)
81     row['shortcuts']=sum(int(r.get('shortcuts',0)) for r in b)
82     result['groups'][f'p{problem}_stress{stress}']=row
83 Path(args.output).write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf
    -8')
84 print(json.dumps(result,ensure_ascii=False,indent=2))
85
86
87 if __name__=='__main__':main()

```

## src/policy.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 #include "routing.hpp"
4 #include "certified_route.hpp"
5 #include "position_uncertainty.hpp"
6 #include "lns.hpp"
7 #include "active.hpp"
8 #include "dynamic_selection.hpp"
9 #include "optical_order.hpp"

```

```

10 #include "discovery.hpp"
11 #include "residual_cover.hpp"
12 #include "neighborhood_tour.hpp"
13 #include "adaptive_optical.hpp"
14 #include "optical_portfolio.hpp"
15 #include "../review/certified_layout21_hex.hpp"
16 #include <array>
17 #include <stdexcept>
18 using geo::Pt; using geo::Poly;
19 struct Reading {int kind=0; double angle=0;}; // no_signal=0, near=1, direction=2
20 struct Sensor {virtual Reading measure(Pt,int)=0; virtual bool clear(Pt,int)=0; virtual void
    audit(int,const Poly&){}; virtual void phase(int){}; virtual ~Sensor()=default;};
21 #include "rollout_sensor.hpp"
22 struct Track {Poly p; Pt first; double angle=0; int obs=0; bool cleared=false; std::vector<
    dynamic_selection::Event> history; int dynamic_used=0, aux_used=0;};
23 struct Options {double spacing=950, early=35; bool square=false, dynamic=true; int routing=0; double
    shiftA=0, shiftB=0; bool shifted=false, includeOrigin=true, opportunistic=false; int
    fixedLayout=0; double secondA=750, secondB=300, eta=0; bool stop_when_found16=false; int
    route_model=0; bool use_lns=false; int active_mode=0; int dynamic_depth=0, dp_node_limit=6000;
    bool joint_actions=false; int auxiliary=0, aux_limit=4; double start_residual=0; bool retreat=
    false, candidate_menu=false, complete_source=false, direct_optical=false, optical_rotate=false
    ; int planning_samples=32; double q3_radius=1200; bool clear_neighborhood=false, defer_single=
    false, weighted_optical=false; double optical_weight=1, immediate_fee_weight=0,
    discovery_weight=0; int dynamic_limit=6; bool adaptive_optical=false; int terminal_mode=0;
    bool variable_optical=false; int optical_search=0; bool budget_aware=false; int refinement=0;
    int bearing_bin=2; int optical_portfolio=0; bool shortcut=false; int robust_mode=0; bool
    residual_cover=false, neighborhood_tour=false, residual_opportunistic=false,
    neighborhood_variable=false;};
24 class Policy {
25     Sensor& io; int problem; Options opt; std::array<Track,21> t; bool auxiliary_busy=false; discovery
        ::Belief unseen; residual_cover::State remaining_cover;
26     public: Pt pos; int channel=1, cleared=0, fallbacks=0, plans=0, exact_plans=0, local_plans=0,
        nonlocal_plans=0, covered_skips=0; double max_route_gap=0, total_route_gap=0,
        total_route_upper=0, max_relative_gap=0; std::string certificate="incomplete";
27     int dynamic_plans=0, dynamic_actions=0, dynamic_fallbacks=0, dp_expanded=0, dp_budget_hits=0;
28     int auxiliary_calls=0, auxiliary_positive=0, joint_steps=0, residual_calls=0, residual_replaced=0,
        residual_retired=0, rollout_calls=0, rollout_replaced=0;
29     Policy(Sensor& s, int prob, Options o={}) : io(s), problem(prob), opt(o) {if(opt.discovery_weight>0)
        unseen=discovery::Belief(prob);}
30     Reading measure(Pt p, int k) {auto r=io.measure(p,k); pos=p; channel=k;
31         if(opt.dynamic_depth>0) t[k].history.push_back({p,r.kind,r.angle});
32         if(r.kind==1) {if(!clear(p,k)) throw std::runtime_error("near followed by failed clear");}
33         else if(r.kind==2) {auto &a=t[k]; if(a.obs==0) {a.p=geo::outer_disk({0,0},1800); a.first=p; a.
            angle=r.angle;}
34         if(opt.eta>0) a.p=position_error::update(a.p,p,r.angle,opt.eta);
35         else {a.p=geo::observe(a.p,p,r.angle); a.p=geo::disk_clip(a.p,p,1500);}
36         // Valid axial bound additionally prevents circumscribed-disk overshoot along bearing.
37         Pt u=geo::polar(1,r.angle); a.p=geo::clip(a.p,u,geo::dot(u,p)+1500+opt.eta);
38         ++a.obs; if(a.p.empty()) throw std::runtime_error("empty feasible region"); io.audit(k,a.p);}
39     if(opt.auxiliary>=3&&!auxiliary_busy) supplement(p,k);
40     return r;
41 }
42 bool clear(Pt p, int k) {bool ok=io.clear(p,k); pos=p; if(opt.dynamic_depth>0) t[k].history.
    push_back({p,ok?4:3,0}); if(ok&&!t[k].cleared) {t[k].cleared=true; ++cleared;} if(opt.
    auxiliary>=3&&!auxiliary_busy) supplement(p,k); return ok;}
43 void optical(int k) {io.phase(2); ++fallbacks; auto &a=t[k]; Pt u=geo::polar(1,a.angle), v={-u.y,u.
    x};
44     double xmin=1e99, xmax=-1e99, ymin=1e99, ymax=-1e99;

```

```

45  for(auto p:a.p){Pt d=p-a.first;double x=geo::dot(d,u),y=geo::dot(d,v);xmin=std::min(xmin,x);
    xmax=std::max(xmax,x);ymin=std::min(ymin,y);ymax=std::max(ymax,y);}
46  double step=opt.eta>0?position_error::grid_side(opt.eta):28;
47  int nx=std::max(1,(int)ceil((xmax-xmin)/step)),ny=std::max(1,(int)ceil((ymax-ymin)/step));
48  Poly route;for(int i=0;i<nx;++i)for(int jj=0;jj<ny;++jj){int j=i%2?ny-1-jj:jj;route.push_back
    (a.first+u*(xmin+(i+.5)*(xmax-xmin)/nx)+v*(ymin+(j+.5)*(ymax-ymin)/ny));}
49  if(opt.adaptive_optical&&opt.eta==0){auto candidate=adaptive_optical::cover(a.p,a.first,a.
    angle);if(!candidate.empty())route=candidate;}
50  if(opt.variable_optical&&opt.eta==0)route=adaptive_optical::variable_cover(a.p,a.first,a.
    angle);
51  if(geo::dist(pos,route.back())<geo::dist(pos,route.front()))std::reverse(route.begin(),route.
    end());
52  if(opt.optical_rotate){size_t best=0;for(size_t i=1;i<route.size();++i)if(geo::dist(pos,route
    [i])<geo::dist(pos,route[best]))best=i;std::rotate(route.begin(),route.begin()+best,route.
    end());}
53  if(opt.weighted_optical&&opt.eta==0)route=optical_order::choose(route,pos,dynamic_selection::
    make_hypotheses(a.p,a.history,problem,63),opt.optical_weight);
54  if(opt.optical_search&&opt.eta==0)route=optical_order::search(route,pos,dynamic_selection::
    make_hypotheses(a.p,a.history,problem,63),opt.optical_search==1?0:opt.optical_search
    ==2?.1:.3);
55  if(opt.optical_portfolio&&opt.eta==0){
56  auto h=dynamic_selection::make_hypotheses(a.p,a.history,problem,63);
57  auto candidate=adaptive_optical::variable_cover(a.p,a.first,a.angle);
58  if(geo::dist(pos,candidate.back())<geo::dist(pos,candidate.front()))std::reverse(candidate.
    begin(),candidate.end());
59  candidate=optical_order::choose(candidate,pos,h);
60  if(opt.optical_portfolio==2)candidate=optical_order::search(candidate,pos,h,.1);
61  double fraction=opt.optical_portfolio==1?.05:opt.optical_portfolio==2?.1:opt.
    optical_portfolio==3?.2:0;
62  route=optical_portfolio::choose(route,candidate,pos,h,fraction,100*fraction);
63  }
64  if(opt.neighborhood_tour&&opt.eta==0){std::vector<Poly>cells;auto centers=adaptive_optical::
    cover(a.p,a.first,a.angle,&cells);route=neighborhood_tour::optical(route,centers,cells,pos
    ,dynamic_selection::make_hypotheses(a.p,a.history,problem,63));}
65  if(opt.neighborhood_variable&&opt.eta==0){std::vector<Poly>cells;auto centers=
    adaptive_optical::variable_cover(a.p,a.first,a.angle,&cells);auto h=dynamic_selection::
    make_hypotheses(a.p,a.history,problem,63);auto order=optical_order::choose(centers,pos,h);
    auto candidate=neighborhood_tour::optical(order,centers,cells,pos,h);
66  double oldcost=neighborhood_tour::expected(route,pos,h),newcost=neighborhood_tour::expected(
    candidate,pos,h);
67  if(newcost+5<oldcost&&newcost<.95*oldcost&&neighborhood_tour::length(candidate,pos)<
    neighborhood_tour::length(route,pos))route=candidate;
68  }
69  for(auto p:route)if(clear(p,k))return;
70  throw std::runtime_error("continuous optical coverage exhausted without success");
71  }
72  dynamic_selection::Choice rollout_choice(int k,dynamic_selection::Choice base){
73  auto &a=t[k];if(!base.available||geo::mec(a.p).r<=35)return base;
74  std::vector<dynamic_selection::Alternative>candidate{{base.point,base.measure,base.value}};
75  auto fresh=robust_geometry::candidate(a.p,pos,a.first,a.angle,a.history,problem,1);
76  if(fresh.valid){double oldrho=base.measure&&robust_geometry::reception(a.p,base.point,a.
    history,problem)?robust_geometry::worst_radius(a.p,base.point,128).upper:geo::mec(a.p).r;
77  double extra=std::max(0.,geo::dist(pos,fresh.q)-geo::dist(pos,base.point))/5+5;
78  if(.4*(oldrho-fresh.rho)>extra&&fresh.rho<geo::mec(a.p).r*.85)candidate.push_back({fresh.q,
    true,0});
79  }
80  if(candidate.size()==1)return base;
81  auto h=dynamic_selection::make_hypotheses(a.p,a.history,problem,4);

```

```

82 // Include polygon boundary extrema, not just interior particles, for Q3.
83 if(problem==3){auto far=a.p.front();for(auto z:a.p)if(geo::dist(z,a.first)>geo::dist(far,a.
    first))far=z;
84 for(auto z:{a.p.front(),far}){dynamic_selection::Hypothesis x{z,1500,false,0,1};if(
    dynamic_selection::consistent(x,a.history))h.push_back(x);}
85 if(h.empty())return base;
86 std::vector<std::vector<double>>costs(candidate.size());
87 for(size_t j=0;j<candidate.size();++j)for(auto source:h)for(double error:{-1.,1.}){
88 RolloutSensor sensor(source,pos,channel,error,a.history);Options continuation=opt;
    continuation.robust_mode=0;continuation.residual_cover=false;continuation.
    neighborhood_tour=false;continuation.auxiliary=0;
89 Policy shadow(sensor,problem,continuation);shadow.pos=pos;shadow.channel=channel;shadow.t[k
    ]=a;
90 auto d=candidate[j];++shadow.t[k].dynamic_used;
91 try{if(d.measure)shadow.measure(d.point,k);else shadow.clear(d.point,k);if(!shadow.t[k].
    cleared)shadow.locate(k);if(!sensor.cleared)throw std::runtime_error("rollout incomplete")
    ;costs[j].push_back(sensor.time);}
92 catch(const std::exception&){costs[j].push_back(INFINITY);}++rollout_calls;
93 }
94 auto mean=[](const std::vector<double>&v){double s=0;for(auto x:v)s+=x;return s/v.size();};
95 double best=mean(costs[0]);int selected=0;
96 for(size_t j=1;j<candidate.size();++j){double value=mean(costs[j]);double worst_regret=-
    INFINITY;for(size_t i=0;i<costs[j].size();++i)worst_regret=std::max(worst_regret,costs[j][
    i]-costs[0][i]);
97 if(value+5<best&&worst_regret<=5){best=value;selected=j;}}
98 if(selected){++rollout_replaced;base.point=candidate[selected].point;base.measure=candidate[
    selected].measure;base.value=best;base.alternatives={{base.point,base.measure,best}};}
99 return base;
100 }
101 void locate(int k){io.phase(1);auto &a=t[k];if(a.cleared)return;
102 if(opt.dynamic_depth>0&&opt.eta==0){
103 for(int step=a.dynamic_used;step<opt.dynamic_limit&&!a.cleared;++step){
104 int depth=opt.budget_aware?dynamic_selection::remaining_depth(opt.dynamic_depth,opt.
    dynamic_limit,a.dynamic_used):opt.dynamic_depth;
105 auto decision=dynamic_selection::plan(a.p,pos,a.first,a.angle,a.history,problem,depth,
    channel==k,opt.dp_node_limit,opt.retrear,opt.planning_samples,opt.clear_neighborhood,opt.
    terminal_mode,opt.refinement,opt.bearing_bin,opt.robust_mode==3?0:opt.robust_mode);
106 if(opt.robust_mode==3)decision=rollout_choice(k,decision);
107 ++dynamic_plans;dp_expanded+=decision.expanded;dp_budget_hits+=decision.budget_hit;
108 if(!decision.available)break;
109 ++dynamic_actions;++a.dynamic_used;if(decision.measure)measure(decision.point,k);else clear
    (decision.point,k);
110 }
111 if(a.cleared)return;++dynamic_fallbacks;
112 }
113 if(opt.direct_optical)optical(k);else locate_baseline(k);
114 }
115 void locate_baseline(int k){auto &a=t[k];if(a.cleared)return;
116 if(opt.opportunistic&&a.obs==1){Pt u=geo::polar(1,a.angle);if(std::abs(geo::cross(u,pos-a.
    first))>=50){measure(pos,k);if(a.cleared)return;}}
117 if(a.obs==1){Pt u=geo::polar(1,a.angle),v={-u.y,u.x};Pt q1=a.first+u*opt.secondA+v*opt.
    secondB,q2=a.first+u*opt.secondA-v*opt.secondB;
118 if(geo::dist(pos,q2)<geo::dist(pos,q1))std::swap(q1,q2);
119 if(opt.active_mode)q1=active::choose(a.p,pos,a.first,a.angle,opt.secondA,opt.secondB,opt.
    active_mode);
120 auto r=measure(q1,k);if(a.cleared)return;
121 if(r.kind==0){measure(q2,k);if(a.cleared)return;}
122 }

```

```

123 for(int attempt=0;attempt<3&&!a.cleared;++attempt){auto c=geo::mec(a.p);
124   if(c.r<=opt.early || c.r<=20-opt.eta-1e-6){if(clear(c.c,k))return;}
125   if(a.obs<2)break;
126   Pt q=c.c;
127   if(geo::dist(pos,q)<1){Pt u=geo::polar(1,a.angle);q=q+Pt{-u.y,u.x}*(attempt%2?-50:50);}
128   auto r=measure(q,k);if(a.cleared)return;
129   if(r.kind==0){Pt u=geo::polar(1,a.angle),v={-u.y,u.x};measure(c.c+v*50,k);if(a.cleared)
     return;}
130 }
131 if(!a.cleared)optical(k);
132 }
133 Poly covering_points(){
134   if(problem==3&&opt.q3_radius!=1200){Poly p{{0,0}};for(int i=0;i<6;++i)p.push_back(geo::polar(
     opt.q3_radius,60*i));return p;}
135   if(problem==4&&opt.fixedLayout==21){Poly p;for(auto x:certified_layout21::points)p.push_back
     ({x.x,x.y});return p;}
136   if(problem==3||!opt.shifted)return geo::coverage(problem,opt.square?600:opt.spacing,opt.
     square);
137   Poly points;if(opt.includeOrigin)points.push_back({0,0});double h=opt.spacing;auto v=[&](int
     i,int j){return Pt{h*(i+opt.shiftA+.5*(j+opt.shiftB)),h*sqrt(3.)/2*(j+opt.shiftB)};};
138   for(int i=-6;i<6;++i)for(int j=-6;j<6;++j){Pt a=v(i,j),b=v(i+1,j),c=v(i,j+1),d=v(i+1,j+1);
139     for(auto triangle:{Poly{a,b,c},Poly{d,c,b}})if(geo::meets_disk(triangle,1800))for(auto q:
     triangle){bool duplicate=false;for(auto p:points)if(geo::dist(p,q)<1e-6)duplicate=true;if
     (!duplicate)points.push_back(q);}}
140   return points;
141 }
142 void supplement(Pt q,int exclude=0){
143   if(opt.auxiliary==0||opt.eta!=0||auxiliary_busy)return;
144   auxiliary_busy=true;
145   int start=channel;
146   for(int j=0;j<20;++j){int k=1+(start-1+j)%20;auto &a=t[k];
147     if(k==exclude||a.cleared||a.obs==0||a.aux_used>opt.aux_limit)continue;
148     auto c=geo::mec(a.p);if(c.r<=20||geo::dist(q,c.c)>1000)continue;
149     auto u=geo::polar(1,a.angle);if(std::abs(geo::cross(u,q-a.first))<50)continue;
150     bool repeated=false;for(auto e:a.history)if(e.kind<=2&&geo::dist(e.q,q)<1e-6)repeated=true;
     if(repeated)continue;
151     if(opt.auxiliary>=4){auto sample=dynamic_selection::make_hypotheses(a.p,a.history,problem
     ,32);double total=0,heard=0;for(auto h:sample){total+=h.weight;if(dynamic_selection::
     receives(h,q)heard+=h.weight;}if(total==0||heard<.6*total)continue;}
152     ++a.aux_used;++auxiliary_calls;auto r=measure(q,k);if(r.kind>0)++auxiliary_positive;
153   }
154   auxiliary_busy=false;
155 }
156 void scan(Pt q){io.phase(0);int start=channel;for(int a=0;a<20;++a){int k=1+(start-1+a)%20;if
     (!t[k].cleared&&t[k].obs==0)measure(q,k);}supplement(q);if(opt.discovery_weight>0)unseen.
     negative_scan(q);}
157 int known()const{int n=0;for(int k=1;k<=20;++k)n+=(t[k].cleared||t[k].obs>0);return n;}
158 double scan_fee()const{int n=0;for(int k=1;k<=20;++k)if(!t[k].cleared&&t[k].obs==0)++n;return
     n?6*n-(t[channel].cleared&&t[channel].obs==0?1:0):0;}
159 double service_distance(Pt from,Pt center,int kind){
160   if(opt.route_model!=2||kind<0||t[kind].obs!=1)return geo::dist(from,center);
161   auto &tr=t[kind];auto u=geo::polar(1,tr.angle);Pt v={-u.y,u.x};auto a=tr.first+u*opt.secondA+
     v*opt.secondB,b=tr.first+u*opt.secondA-v*opt.secondB;
162   // Entry side follows the production nearest-side rule. The exit remains
163   // a belief-center approximation, not a secretly observed source position.
164   Pt entry=geo::dist(from,a)<=geo::dist(from,b)?a:b;
165   return geo::dist(from,entry)+geo::dist(entry,center);
166 }

```

```

167 void joint_run(const Poly&initial_points){Poly points=initial_points;std::vector<bool>visited(
    points.size());
168 while(cleared<16){
169     if(opt.residual_opportunistic&&opt.eta==0){bool scanned_before=false;for(auto q:
        remaining_cover.past)if(geo::dist(q,pos)<1)scanned_before=true;
170     if(!scanned_before)for(int i=0;i<(int)points.size();++i)if(!visited[i]&&geo::dist(pos,
        points[i])>1){
171         if(remaining_cover.verify(remaining_cover.sites(points,visited,i,pos,true),true)){visited[
            i]=true;scan(pos);remaining_cover.scanned(pos);++remaining_cover.retired;remaining_cover.
            retire(points,visited);residual_retired=remaining_cover.retired;break;}
172     }
173 }
174 Poly nodes;std::vector<int> kind;std::vector<dynamic_selection::Choice>choices;
175 bool complete_discovery=opt.stop_when_found16&&known()==16;
176 for(int i=0;i<(int)points.size();++i)if(!visited[i]){if(complete_discovery){visited[i]=true
    ;++covered_skips;}else{nodes.push_back(points[i]);kind.push_back(-1-i);choices.push_back
    ({});}}
177 bool coverage_left=!nodes.empty();
178 for(int k=1;k<=20;++k)if(t[k].obs&&t[k].cleared){
179     if(opt.defer_single&&coverage_left&&t[k].obs==1)continue;
180     dynamic_selection::Choice decision;auto &a=t[k];
181     if(opt.joint_actions&&opt.dynamic_depth>0&&opt.eta==0&&a.dynamic_used<opt.dynamic_limit){
182         int depth=opt.budget_aware?dynamic_selection::remaining_depth(opt.dynamic_depth,opt.
            dynamic_limit,a.dynamic_used):opt.dynamic_depth;
183         decision=dynamic_selection::plan(a.p,pos,a.first,a.angle,a.history,problem,depth,channel==
            k,opt.dp_node_limit,opt.retreat,opt.planning_samples,opt.clear_neighborhood,opt.
            terminal_mode,opt.refinement,opt.bearing_bin,opt.robust_mode==3?0:opt.robust_mode);
184         if(opt.robust_mode==3)decision=rollout_choice(k,decision);
185         ++dynamic_plans;dp_expanded+=decision.expanded;dp_budget_hits+=decision.budget_hit;
186     }
187     nodes.push_back(decision.available&&!opt.candidate_menu?decision.point:geo::mec(a.p).c);
        kind.push_back(k);choices.push_back(decision);
188 }
189 if(nodes.empty())break;std::vector<int>order;
190 if(opt.route_model){cert_route::Problem task;int n=nodes.size();task.start.resize(n);task.
    edge.assign(n,std::vector<double>(n));
191     for(int j=0;j<n;++j){task.start[j]=opt.joint_actions?geo::dist(pos,nodes[j]):
        service_distance(pos,nodes[j],kind[j]);for(int i=0;i<n;++i)if(i!=j)task.edge[i][j]=opt.
        joint_actions?geo::dist(nodes[i],nodes[j]):service_distance(nodes[i],nodes[j],kind[j]);}
192     if(opt.candidate_menu){auto cost=[&(Pt from,int j){if(!choices[j].available)return geo::
        dist(from,nodes[j]);double best=1e100;for(auto d:choices[j].alternatives)best=std::min(
        best,geo::dist(from,d.point)+std::max(0.,5*d.value-geo::dist(pos,d.point)));return best;};
        for(int j=0;j<n;++j){task.start[j]=cost(pos,j);for(int i=0;i<n;++i)if(i!=j)task.edge[i][
            j]=cost(nodes[i],j);}}
193     if(opt.joint_actions&&opt.start_residual>0)for(int j=0;j<n;++j)if(kind[j]>0&&choices[j].
        available)task.start[j]+=opt.start_residual*std::max(0.,choices[j].value*5-geo::dist(pos,
        nodes[j]));
194     if(opt.immediate_fee_weight>0)for(int j=0;j<n;++j){double fee=kind[j]<0?scan_fee():(choices
        [j].available&&choices[j].measure?5+(channel!=kind[j]):4);task.start[j]+=5*opt.
        immediate_fee_weight*fee;}
195     if(opt.discovery_weight>0){int left=0;for(int k:kind)left+=k<0;double missing=std::max(0,
        std::min(16-known(),std::max(1,13-known())));for(int j=0;j<n;++j)if(kind[j]<0){double
        reward=opt.discovery_weight*missing*unseen.probability(nodes[j])*2.5*left;task.start[j]=
        std::max(0.,task.start[j]-5*reward);}}
196     auto solution=opt.use_lns?new_route::solve(task):cert_route::solve(task);order=solution.
        path;+plans;if(solution.exact)+exact_plans;if(solution.local_optimal)+local_plans;else
        ++nonlocal_plans;

```

```

197     double gap=std::max(0.,solution.upper-solution.lower);total_route_gap+=gap;max_route_gap=
        std::max(max_route_gap,gap);total_route_upper+=solution.upper;max_relative_gap=std::max(
            max_relative_gap,solution.upper>0?gap/solution.upper:0.);
198 }else order=route::optimize(pos,nodes);
199 if(opt.neighborhood_tour&&opt.eta==0){Poly path;std::vector<Poly>cells;bool any=false;
200     for(int j:order){path.push_back(nodes[j]);if(kind[j]>0&&geo::mec(t[kind[j]].p).r<20-1e-4){
        cells.push_back(t[kind[j]].p);path.back()=geo::mec(cells.back()).c;any=true;}else cells.
        push_back({});}
201     if(any){auto moved=neighborhood_tour::optimize(path,cells,pos);for(size_t jj=0;jj<order.
        size();++jj)if(!cells[jj].empty()){int j=order[jj];nodes[j]=moved[jj];choices[j].available
        =true;choices[j].measure=false;choices[j].point=moved[jj];}}
202 }
203 int chosen=order[0],act=kind[chosen];if(opt.joint_actions)++joint_steps;
204 if(act<0){int i=-1-act;
205     if(opt.residual_cover&&opt.eta==0){points[i]=remaining_cover.replace(points,visited,i,pos)
        ;}
206     visited[i]=true;scan(points[i]);
207     if(opt.residual_cover&&opt.eta==0){remaining_cover.scanned(points[i]);remaining_cover.
        retire(points,visited);residual_calls=remaining_cover.calls;residual_replaced=
        remaining_cover.accepted;residual_retired=remaining_cover.retired;}
208 }
209 else{
210     if(opt.joint_actions){io.phase(1);auto d=choices[chosen];if(d.available&&t[act].
        dynamic_used<opt.dynamic_limit){++t[act].dynamic_used;++dynamic_actions;if(d.measure)
        measure(d.point,act);else clear(d.point,act);if(opt.complete_source&&!t[act].cleared)
        locate(act);}else{++dynamic_fallbacks;if(opt.direct_optical)optical(act);else
        locate_baseline(act);}}
211     else locate(act);
212     if(opt.auxiliary>=2)supplement(pos,act);
213 }
214 }
215 }
216 void run(){Poly points=covering_points();std::vector<bool> visited(points.size());
217 if(opt.routing==2){joint_run(points);if(cleared==16){certificate="upper_bound_16";return;}
        certificate=problem==3?"seven_point_per_channel":"convex_mesh_per_channel";return;}
218 for(size_t step=0;step<points.size();++step){size_t j=points.size();double best=1e99;
219     if(opt.routing==1){Poly nodes;std::vector<int>ids;for(int i=0;i<(int)points.size();++i)if(!
        visited[i]){nodes.push_back(points[i]);ids.push_back(i);}j=ids[route::optimize(pos,nodes)
        [0]];}
220     else for(size_t i=0;i<points.size();++i)if(!visited[i]){double score=opt.dynamic?geo::dist(
        pos,points[i]):double(i);if(score<best){best=score;j=i;}}
221     visited[j]=true;scan(points[j]);
222     while(cleared<16){int kbest=0;best=1e99;Poly nodes;std::vector<int>ids;for(int k=1;k<=20;++k
        )if(t[k].obs>0&&!t[k].cleared){auto center=geo::mec(t[k].p).c;nodes.push_back(center);ids.
        push_back(k);double d=opt.dynamic?geo::dist(pos,center):k;if(d<best){best=d;kbest=k;}}if(!
        kbest)break;if(opt.routing==1)kbest=ids[route::optimize(pos,nodes)[0]];locate(kbest);}
223     if(cleared==16){certificate="upper_bound_16";return;}
224 }
225 for(int k=1;k<=20;++k)if(t[k].obs&&!t[k].cleared)throw std::runtime_error("pending source at
        exit");
226 certificate=problem==3?"seven_point_per_channel":"convex_mesh_per_channel";
227 }
228 };

```

## src/position\_uncertainty.hpp

```
1 #pragma once
```

```

2 #include "geometry.hpp"
3 namespace position_error {
4 inline geo::Poly update(geo::Poly p,geo::Pt s,double angle,double eta){
5   double eps=1.005*geo::pi/180,t=tan(eps);auto u=geo::polar(1,angle);geo::Pt v={-u.y,u.x};
6   for(auto n:{v-u*t,v*(-1)-u*t})p=geo::clip(p,n,geo::dot(n,s)+eta*geo::norm(n));
7   p=geo::disk_clip(p,s,1500+eta);p=geo::clip(p,u,geo::dot(u,s)+1500+eta);p=geo::clip(p,u*(-1),-
   geo::dot(u,s)+eta);
8   for(auto n:{v,v*(-1)})p=geo::clip(p,n,geo::dot(n,s)+1500*sin(eps)+eta);
9   return p;
10 }
11 inline double grid_side(double eta){return std::min(28.,sqrt(2.)*(20-eta)-1e-6);}
12 }

```

### src/profile.cpp

```

1 #include "simulator.hpp"
2 #include "configurations.hpp"
3 #include <iostream>
4 struct Profile:Sim {int stage=0;double times[3]{};moves[3]{};int counts[3]{};
5   using Sim::Sim;void phase(int p)override{stage=p;}
6   Reading measure(Pt p,int k)override{double before=time,d=length;auto r=Sim::measure(p,k);times
   [stage]+=time-before;moves[stage]+=length-d;++counts[stage];return r;}
7   bool clear(Pt p,int k)override{double before=time,d=length;auto r=Sim::clear(p,k);times[stage
   ]+=time-before;moves[stage]+=length-d;++counts[stage];return r;}
8 };
9 int main(){std::ofstream out("results/profile.csv");out<<"problem,seed,total,scan,local,optical
   ,scan_move,local_move,optical_move,scan_actions,local_actions,optical_actions\n";
10  for(int p:{3,4})for(int seed=1;seed<=40;++seed){auto o=configuration(5);o.dynamic_depth=2;o.
   active_mode=p==3?1:0;Profile s(make_case(p,seed),seed,0);Policy b(s,p,o);b.run();if(s.
   success!=s.total())return 1;
11   out<<std::setprecision(13)<<p<<"<<seed<<"<<s.time;for(auto v:s.times)out<<"<<v;for(auto
   v:s.moves)out<<"<<v;for(auto v:s.counts)out<<"<<v;out<<"\n";
12 }std::cout<<"profiled 80 baseline cases\n";
13 }

```

### src/q2\_full\_domain\_certificate.py

```

1 """Exact-rational minimax audit on the FULL robust reception domain.
2
3 Trigonometric enclosures: Machin pi + Taylor alternating-series remainders.
4 Floating-point centers are proposals only; every leaf is accepted by exact
5 rational squared-distance checks. No library floating trig is trusted.
6 """
7 from fractions import Fraction as F
8 from functools import lru_cache
9 from math import hypot, isqrt
10 import json
11 import sys
12
13 S = 10**20
14
15 def rounded_interval(lo, hi):
16     return F((lo*S).__floor__(), S), F((hi*S).__ceil__(), S)
17
18 def atan_interval(inv, n):
19     x = F(1, inv)

```

```

20     v = sum((-1)**k*x**(2*k+1)/F(2*k+1) for k in range(n)), F(0))
21     tail = (-1)**n*x**(2*n+1)/F(2*n+1)
22     return min(v,v+tail),max(v,v+tail)
23
24 a,b=atan_interval(5,45)
25 c,d=atan_interval(239,14)
26 PI_L0,PI_HI=rounded_interval(16*a-4*d,16*b-4*c)
27
28 @lru_cache(None)
29 def trig(degrees, cosine=False):
30     degrees=F(degrees)
31     lo,hi=sorted((degrees*PI_L0/180,degrees*PI_HI/180))
32     x=F(((lo+hi)*S/2).__floor__(),S)
33     err=max(abs(x-lo),abs(x-hi))
34     term=F(1) if cosine else x
35     v=term
36     for k in range(1,32):
37         term *= -x*x/F((2*k-1)*(2*k) if cosine else (2*k)*(2*k+1))
38         v += term
39     k=32
40     tail=term*(-x*x)/F((2*k-1)*(2*k) if cosine else (2*k)*(2*k+1))
41     # |x|<4 here, so the omitted alternating tail decreases in magnitude.
42     assert abs(x)<4
43     return rounded_interval(min(v,v+tail)-err,max(v,v+tail)+err)
44
45 class I:
46     def __init__(self,a,b=None):self.a=F(a);self.b=F(a if b is None else b)
47     def __add__(x,y):
48         y=as_i(y);return I(x.a+y.a,x.b+y.b)
49     __radd__=__add__
50     def __neg__(x):return I(-x.b,-x.a)
51     def __sub__(x,y):return x+-as_i(y)
52     def __rsub__(x,y):return as_i(y)+-x
53     def __mul__(x,y):
54         y=as_i(y);v=[x.a*y.a,x.a*y.b,x.b*y.a,x.b*y.b];return I(min(v),max(v))
55     __rmul__=__mul__
56     def __truediv__(x,y):
57         y=as_i(y);assert y.a*y.b>0;return x*I(1/y.b,1/y.a)
58     def sq(x):
59         assert x.a>=0;return I(x.a*x.a,x.b*x.b)
60     def pair(x):return [float(x.a),float(x.b)]
61
62 def as_i(x):return x if isinstance(x,I) else I(x)
63 def sin(x):return I(*trig(x))
64 def cos(x):return I(*trig(x,True))
65 def sqrt_bounds(x):
66     lo=isqrt((x.a*S*S).__floor__())
67     hi=isqrt((x.b*S*S).__ceil__()+1)
68     return I(F(lo,S),F(hi,S))
69
70 def global_lower(e1,e2,t_upper,claimed_floor):
71     R=1500; eps=F('1.005'); T=F(t_upper)
72     k=sin(e1+2*e2);h=cos(e1+2*e2);s=sin(2*e2);l=sin(2*e1+2*e2)
73     # Every q in G has 0<=a<=1005, |b|<=1005, norm(q)<=1005.
74     # b<=30 is handled by the fixed normal-readout witness pair (1340,0),(1500,0).
75     # Their angle difference has positive dot product and tan(delta)<=4800/165825.
76     assert F(4800,165825)<(sin(2*e2)/cos(2*e2)).a
77     # b>=30: denominator positive, B on the forward rotated ray, r_B>5,

```

```

78 # and |B-q|>5 so neither witness triggers the near response.
79 denlo=(R*1-1005*k).a
80 denhi=(R*1+1005*h).b
81 tlo=(R*h*30-s*1005**2).a/denhi
82 lambda_lo=(30*cos(e1)-1005*sin(e1)).a/denhi
83 assert denlo>0 and tlo>5
84 assert (R*h-1005).a>0 # x-component of rotated A-q is positive
85 assert lambda_lo*(R-1005)>5
86 assert T<(R*cos(2*e1)).a
87 # N-T*D = -s*||q||^2 + w.q - T*R*1. G is contained in
88 # disk(center=(5*cos(eps),-5*sin(eps)), radius=1000).
89 cx,cy=5*cos(eps),-5*sin(eps)
90 wx,wy=k*(R+T),h*(R-T)
91 gx,gy=wx-2*s*cx,wy-2*s*cy
92 assert gx.a>0 and gy.a>0
93 gnorm=sqrt_bounds(gx.sq()+gy.sq())
94 assert gnorm.a>(2000*s).b
95 # Norm(center)^2=25 exactly. The quadratic maximum occurs at radius 1000.
96 quad_max=-s*(1000000+25)+wx*cx+wy*cy-T*R*1+1000*gnorm
97 assert quad_max.b<=0, ('increase t_upper',quad_max.pair())
98 rad=sqrt_bounds(I(R*R)+I(T*T)-2*R*T*cos(2*e1))/2
99 assert rad.a>F(claimed_floor)
100 assert F(claimed_floor)<80
101 return {'certified_floor_m':claimed_floor,'global_lower_m':rad.pair(),
102         't_upper_m':t_upper,'quadratic_max_upper':float(quad_max.b),
103         'positive_denominator_lower':float(denlo),'witness_r_lower_m':float(tlo),
104         'witness_second_distance_lower_m':float(lambda_lo*(R-1005)),
105         'small_b_witness_radius_m':80}
106
107 def reception(q):
108     a,b=map(F,q); eps=F('1.005')
109     d=a*cos(eps)-abs(b)*sin(eps); normsq=a*a+b*b
110     c1=2000*d-normsq;c2=999975+10*d-normsq
111     assert c1.a>=0 and c2.a>=0
112     return {'q':[str(a),str(b)],'r1000_constraint_margin_m2':c1.pair(),
113           'r5_constraint_margin_m2':c2.pair()}
114
115 def clip(poly, normal, rhs):
116     out=[]
117     for i,x in enumerate(poly):
118         y=poly[(i+1)%len(poly)]
119         f=sum(normal[j]*x[j] for j in range(2))-rhs
120         g=sum(normal[j]*y[j] for j in range(2))-rhs
121         if f<=0:out.append(x)
122         if (f<=0)!=(g<=0):
123             t=f/(f-g);out.append(tuple(x[j]+t*(y[j]-x[j]) for j in range(2)))
124     return out
125
126 def center_proposal(poly):
127     p=[tuple(map(float,z)) for z in poly]
128     if not p:return (F(0),F(0))
129     # Any center is permitted. Select best among pair midpoints and circumcenters.
130     candidates=list(p)
131     for i,a in enumerate(p):
132         for j,b in enumerate(p[i:]):
133             candidates.append(((a[0]+b[0])/2,(a[1]+b[1])/2))
134         for c in p[i+1:]:
135             ux,uy=b[0]-a[0],b[1]-a[1];vx,vy=c[0]-a[0],c[1]-a[1]

```

```

136         d=2*(ux*vy-uy*vx)
137         if abs(d)>1e-15:
138             uu=ux*ux+uy*uy;vv=vx*vx+vy*vy
139             candidates.append((a[0]+(uu*vy-vv*uy)/d,a[1]+(ux*vv-vx*uu)/d))
140     center=min(candidates,key=lambda c:max(hypot(c[0]-z[0],c[1]-z[1]) for z in p))
141     return tuple(F(f"{v:.12f}") for v in center)
142
143 def certify_fixed_q(bound=F('55.739')):
144     q=(F('843.1178'),F('545.4003'));eps=F('1.005')
145     # Outer triangle plus the exact disk tangents at +/-eps (with rational
146     # coefficient error allowances). This contains every point of the arc.
147     tangent=sin(eps).b/cos(eps).a
148     first=[(F(0),F(0)),(F(1500),-1500*tangent),(F(1500),1500*tangent)]
149     for angle in [-eps,eps]:
150         nx,ny=cos(angle),sin(angle)
151         normal=((nx.a+nx.b)/2,(ny.a+ny.b)/2)
152         pad=3000*((nx.b-nx.a)+(ny.b-ny.a))/2
153         first=clip(first,normal,F(1500)+pad)
154     stack=[(F(-182+23*k),F(-182+23*(k+1))) for k in range(8)]
155     leaves=[];nodes=0
156     while stack:
157         lo,hi=stack.pop();mid=(lo+hi)/2;half=(hi-lo)/2
158         poly=first
159         for angle,sign in [(mid+eps+half,1),(mid-eps-half,-1)]:
160             si,co=sin(angle),cos(angle)
161             nx,ny=-si*sign,co*sign
162             normal=((nx.a+nx.b)/2,(ny.a+ny.b)/2)
163             # Every source has |z_x-q_x|, |z_y-q_y|<3000.
164             pad=3000*((nx.b-nx.a)+(ny.b-ny.a))/2
165             rhs=sum(normal[j]*q[j] for j in range(2))+pad
166             poly=clip(poly,normal,rhs)
167         nodes+=1
168         center=center_proposal(poly)
169         sq=max((sum((z[j]-center[j])**2 for j in range(2)) for z in poly),default=F(0))
170         if sq<=bound*bound:
171             leaves.append({"lo":str(lo),"hi":str(hi),"center":[str(v) for v in center],"
vertices":len(poly)})
172         else:
173             assert hi-lo>F(1,10**12),"Requested radius is too small for this outer triangle"
174             stack.extend([(lo,mid),(mid,hi)])
175     leaves.sort(key=lambda a:F(a['lo']))
176     assert F(leaves[0]['lo'])==F(-182) and F(leaves[-1]['hi'])==F(2)
177     assert all(x['hi']==y['lo'] for x,y in zip(leaves,leaves[1:]))
178     return {"q":[str(v) for v in q],"upper_m":str(bound),"verified_by":"exact Fraction squared-
distance checks", "nodes":nodes,"leaves":leaves}
179
180 if __name__=='__main__':
181     # Tiny inward margins avoid first-reading rounding ties and ensure the
182     # physical quantized witnesses are feasible under an unknown tie rule.
183     e1=F('1.004999999')
184     result={"reception":reception(['843.1178','545.4003']),
185            "continuous_model_lower":global_lower(e1,F('1.004999999'),'1400.796371','55.73823'),
186            "physical_quantized_lower":global_lower(e1,F('0.999999999'),'1400.99','55.6528'),
187            "continuous_and_quantized_upper":certify_fixed_q()}
188     with open(sys.argv[1] if len(sys.argv)>1 else 'q2_rational_certificate.json','w',encoding='
utf-8') as f:
189         json.dump(result,f,ensure_ascii=False,indent=2)

```

```

190     print(json.dumps({k:({kk:vv for kk,vv in v.items() if kk!='leaves'}) for k,v in result.
        items()}),ensure_ascii=False,indent=2))

```

### src/q4\_area\_arc\_certificate.py

```

1  """Exact-rational certificate: fixed universal Q4 designs need at least 15 sites.
2
3  Uses upper Riemann sums for two-disk overlap area and rational Taylor bounds
4  for the boundary arc. No floating point is used in any acceptance condition.
5  """
6  from fractions import Fraction as F
7  from math import isqrt
8  import json
9  import sys
10
11  R,L=1800,1000
12  K=F(138,100)
13  SCALE=10**12
14
15  def atan_bounds(inv,n):
16      x=F(1,inv)
17      v=sum((-1)**j*x**(2*j+1)/F(2*j+1) for j in range(n)),F(0))
18      tail=(-1)**n*x**(2*n+1)/F(2*n+1)
19      return min(v,v+tail),max(v,v+tail)
20
21  a,b=atan_bounds(5,25)
22  c,d=atan_bounds(239,8)
23  PI_L0=16*a-4*d
24  PI_HI=16*b-4*c
25  assert PI_L0>F('3.14159')
26
27  def cos_bounds(x):
28      x=F(x);assert 0<=x<=F('0.508')
29      term=F(1);total=term
30      for j in range(1,24):
31          term*=-x*x/F((2*j-1)*(2*j))
32          total+=term
33      tail=term*(-x*x)/F(47*48)
34      return min(total,total+tail),max(total,total+tail)
35
36  def sqrt_upper(n):
37      assert n>=0
38      v=n*SCALE*SCALE
39      root=isqrt(v)
40      if root*root<v:root+=1
41      return F(root,SCALE)
42
43  def overlap_upper(r):
44      # f(y)=[sqrt(R^2-y^2)+sqrt(L^2-y^2)-r]_+ is nonincreasing on [0,L].
45      # Thus the unit-spaced LEFT rectangle sum is an UPPER integral bound.
46      total=F(0)
47      for y in range(L):
48          total+=max(F(0),sqrt_upper(R*R-y*y)+sqrt_upper(L*L-y*y)-r)
49      return 2*total
50
51  def main():
52      # On r<=sqrt(R^2+L^2), delta(r)=acos(R/r). For all r, delta(r)<=atan(L/R).

```

```

53 edges=[1800,1900,1950,2000,2025,2050,2800]
54 angle_upper=[F('0.326'),F('0.395'),F('0.452'),F('0.476'),F('0.4991'),F('0.508')]
55 records=[]
56 for lo,hi,u in zip(edges,edges[1:],angle_upper):
57     clo,chi=cos_bounds(u)
58     assert clo>0 and u<PI_L0/2
59     if hi<=2050:
60         assert hi*hi<R*R+L*L
61         assert chi<=F(R,hi)
62         angle_method='cos(upper)<=R/right_endpoint'
63     else:
64         assert chi*chi<=F(R*R,R*R+L*L)
65         angle_method='global delta<=atan(L/R)<upper'
66     area=overlap_upper(lo)
67     deficit=1-area/(PI_L0*L*L)
68     slack=deficit-K*u
69     assert slack>0
70     records.append({'r_interval_m':[lo,hi],'overlap_area_upper_m2':float(area),
71                    'deficit_lower':float(deficit),'delta_upper_rad':str(u),
72                    'verified_slack_lower':float(slack),'angle_method':angle_method,
73                    'exact_area_upper':str(area)})
74     n_lower=F(3*R*R,L*L)+K*PI_L0
75     assert n_lower>14
76     result={'statement':'N >= 15 for every finite fixed universal directional-reception site
77             set',
78             'source_disk_radius_m':R,'minimum_reception_radius_m':L,'k':str(K),
79             'pi_lower':str(PI_L0),'riemann_step_m':1,'intervals':records,
80             'N_real_lower':float(n_lower),'N_integer_lower':15,
81             'scope':'All source positions in the radius-1800 disk and all closed 180-degree
82             emission orientations; fixed finite sites; no optical fallback or adaptation counted as
83             sites.'}
84     with open(sys.argv[1] if len(sys.argv)>1 else 'q4_area_arc_certificate.json','w',encoding='
85             utf-8') as f:
86         json.dump(result,f,ensure_ascii=False,indent=2)
87     print(json.dumps({k:v for k,v in result.items() if k!='pi_lower'},ensure_ascii=False,indent
88                     =2))
89
90 if __name__=='__main__':main()

```

## src/residual\_cover.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 #include <array>
4 namespace residual_cover {
5 // Outward-rounded long-double intervals certify every accepted mutation.
6 // Input binary64 site coordinates are represented exactly in long double.
7 struct I{long double lo,hi;I(long double a=0):lo(a),hi(a){}I(long double a,long double b):lo(a),hi(b){}};
8 inline long double down(long double x){return std::nextafter(x,-INFINITY);}
9 inline long double up(long double x){return std::nextafter(x,INFINITY);}
10 inline I operator+(I a,I b){return {down(a.lo+b.lo),up(a.hi+b.hi)};}
11 inline I operator-(I a,I b){return {down(a.lo-b.hi),up(a.hi-b.lo)};}
12 inline I operator*(I a,I b){std::array<long double,4> v{a.lo*b.lo,a.lo*b.hi,a.hi*b.lo,a.hi*b.hi};return {down(*std::min_element(v.begin(),v.end()),up(*std::max_element(v.begin(),v.end()))};}

```

```

13 inline I det(geo::Pt a,geo::Pt b,geo::Pt c){return (I(b.x)-I(a.x))*(I(c.y)-I(a.y))-(I(b.y)-I(a.
    y))*(I(c.x)-I(a.x));}
14 inline I distance2(geo::Pt a,geo::Pt b){I x=I(a.x)-I(b.x),y=I(a.y)-I(b.y);return x*x+y*y;}
15 struct Box{double x0,y0,x1,y1;int depth=0;geo::Poly corners()const{return {{x0,y0},{x1,y0},{x1,
    y1},{x0,y1}};}};
16 inline bool outside(Box b){double x=b.x0>0?b.x0:b.x1<0?b.x1:0,y=b.y0>0?b.y0:b.y1<0?b.y1:0;
    return distance2({x,y},{0,0}).lo>1800.L*1800.L;}
17 inline bool safe_box(Box b,const geo::Poly&sites){
18     auto vertices=b.corners();geo::Poly safe;
19     for(auto q:sites){bool in=true;for(auto z:vertices)if(distance2(q,z).hi>999.999L*999.999L){in=
        false;break;}if(in)safe.push_back(q);}
20     auto h=geo::hull(safe);if(h.size()<3)return false;
21     for(size_t i=0;i<h.size();++i){if(det(h[i],h[(i+1)%h.size()],h[(i+2)%h.size()]).lo<=0)return
        false;
22         for(auto z:vertices)if(det(h[i],h[(i+1)%h.size()],z).lo<0)return false;}
23     return true;
24 }
25 struct Certificate{bool pass=false;int checks=0;std::vector<Box>leaves;};
26 inline Certificate certify(const geo::Poly&sites,const std::vector<Box>&prior={},int budget
    =100000){
27     Certificate out;std::vector<Box>todo=prior.empty()?std::vector<Box>
        >{{-1800,-1800,1800,1800,0}}:prior;
28     while(!todo.empty()){
29         auto b=todo.back();todo.pop_back();if(outside(b))continue;
30         if(++out.checks>budget)return out;
31         if(!safe_box(b,sites)){out.leaves.push_back(b);continue;}
32         if(b.depth>=30)return out;
33         // An uncovered center is a quick rejection, never an acceptance test.
34         geo::Pt c{(b.x0+b.x1)/2,(b.y0+b.y1)/2};
35         if(geo::norm(c)<1800){geo::Poly near;for(auto p:sites)if(geo::dist(c,p)<=1000)near.push_back(
            p);if(!geo::contains(geo::hull(near),c))return out;}
36         if(b.x1-b.x0>=b.y1-b.y0){double m=(b.x0+b.x1)/2;todo.push_back({b.x0,b.y0,m,b.y1,b.depth+1});
            todo.push_back({m,b.y0,b.x1,b.y1,b.depth+1});}
37         else{double m=(b.y0+b.y1)/2;todo.push_back({b.x0,b.y0,b.x1,m,b.depth+1});todo.push_back({b.x0
            ,m,b.x1,b.y1,b.depth+1});}
38     }
39     out.pass=true;return out;
40 }
41 class State {
42 public:geo::Poly past;std::vector<Box>leaves;int calls=0,accepted=0,retired=0;double saved=0;
43     geo::Poly sites(const geo::Poly&points,const std::vector<bool>&visited,int omit=-1,geo::Pt
        replacement={},bool add=false)const{
44         geo::Poly out=past;for(int i=0;i<(int)points.size();++i)if(!visited[i]&&i!=omit)out.push_back
            (points[i]);if(add)out.push_back(replacement);return out;
45     }
46     bool verify(const geo::Poly&s,bool commit){++calls;auto c=certify(s,leaves);if(c.pass&&commit)
        leaves=std::move(c.leaves);return c.pass;}
47     void scanned(geo::Pt q){past.push_back(q);}
48     void retire(geo::Poly&points,std::vector<bool>&visited){
49         // With no extra/replaced site, deleting an essential static point cannot
50         // become sound merely because other static points gave negative readings.
51         for(int i=0;i<(int)points.size();++i)if(!visited[i]&&verify(sites(points,visited,i),true)){
            visited[i]=true;++retired;}
52     }
53     geo::Pt replace(const geo::Poly&points,const std::vector<bool>&visited,int i,geo::Pt now){
54         auto old=points[i];if(geo::dist(old,now)<5)return old;
55         auto valid=[&](geo::Pt q,bool commit){return verify(sites(points,visited,i,q,true),commit);};
56         if(valid(now,true)){++accepted;saved+=geo::dist(old,now);return now;}

```

```

57 // Closest certified point on this segment; feasibility is checked at every
58 // accepted point. Bisection is a heuristic search, not a monotonicity claim.
59 double low=0,high=1;geo::Pt best=old;
60 for(int it=0;it<7;++it){double m=(low+high)/2;auto q=old+(now-old)*m;if(valid(q,false)){low=m
    ;best=q;}else high=m;}
61 if(geo::dist(best,old)>=2&&valid(best,true)){++accepted;saved+=geo::dist(best,old);return
    best;}return old;
62 }
63 };
64 }

```

## src/robot.cpp

```

1 #include "policy.hpp"
2 #include "configurations.hpp"
3 #include "joint_config.hpp"
4 #include "shortcut_sensor.hpp"
5 #include <iostream>
6 #include <iomanip>
7 // Solver receives observations only; no simulator headers or case data are linked.
8 struct PipeSensor:Sensor {
9     Reading measure(Pt p,int k)override{std::cout<<std::setprecision(17)<<"measure "<<k<<" "<<p.x
        <<" "<<p.y<<std::endl;Reading r;if(!(std::cin>>r.kind>>r.angle))throw std::runtime_error("
        bridge ended");return r;}
10 bool clear(Pt p,int k)override{std::cout<<std::setprecision(17)<<"clear "<<k<<" "<<p.x<<" "<<p
    .y<<std::endl;int ok;if(!(std::cin>>ok))throw std::runtime_error("bridge ended");return ok
    ==1;}
11 };
12 int main(int argc,char**argv){try{
13     int prob=argc>1?atoi(argv[1]):3;if(prob!=3&&prob!=4)throw std::runtime_error("problem must be
        3 or 4");
14     std::string strategy=argc>5?argv[5]:"joint";if(strategy!="geometric"&&strategy!="joint"&&
        strategy!="previous"&&strategy!="baseline"&&strategy!="stage1"&&strategy!="stage2")throw
        std::runtime_error("strategy must be geometric, joint, previous, stage1, stage2 or
        baseline");
15     Options o=joint_configuration(strategy=="geometric"?84:strategy=="joint"?78:strategy=="
        previous"?56:strategy=="stage2"?39:strategy=="stage1"?38:0,prob);if(argc>2)o.early=atof(
        argv[2]);if(argc>3)o.eta=atof(argv[3]);
16     int depth=argc>4?std::stoi(argv[4]):2;if(depth<0||depth>3)throw std::runtime_error("planner
        depth must be in [0,3]");
17     if(!(o.eta>=0&&o.eta<=.4))throw std::runtime_error("position error must be in [0,0.4]");
18     if(o.eta==0){if(depth==0){double early=o.early;o=joint_configuration(0,prob);o.early=early;o.
        dynamic_depth=depth;}
19     else{double early=o.early,eta=o.eta;o=configuration(5);o.early=early;o.eta=eta;}
20     PipeSensor pipe;ShortcutSensor s(pipe,o.shortcut&&o.eta==0);Policy bot(s,prob,o);bot.run();
21     std::cerr<<"strategy="<<strategy<<" planner_depth="<<o.dynamic_depth<<" dp_calls="<<bot.
        dynamic_plans<<" dp_actions="<<bot.dynamic_actions<<" dp_fallbacks="<<bot.
        dynamic_fallbacks<<" budget_hits="<<bot.dp_budget_hits<<" shortcuts="<<s.shortcuts<<"
        saved_m="<<s.saved_distance()<<"\n";
22     std::cout<<"exit "<<bot.cleared<<" "<<bot.certificate<<std::endl;
23 }catch(const std::exception&e){std::cerr<<e.what()<<"\n";return 1;}}

```

## src/robust\_geometry.hpp

```

1 #pragma once
2 #include "geometry.hpp"

```

```

3 #include "bearing_types.hpp"
4 #include <array>
5 #include <queue>
6 namespace robust_geometry {
7 constexpr double eps=1.005; // ±1 degree plus the public 0.01-degree readout rounding.
8 struct Bound {double lower=0,upper=0;int cells=0;};
9 inline bool reception(const geo::Poly&p,geo::Pt q,const std::vector<dynamic_selection::Event>&h
    ,int problem,double range=950.7){
10 if(p.empty())return false;
11 for(auto z:p)if(geo::dist(q,z)>range-1e-5)return false;
12 if(problem==3)return true;
13 // For every z, the positive observations constrain heading by homogeneous
14 // halfplanes. q in the convex hull of heard sites is a sufficient dual cone
15 // certificate simultaneously for all z. No heading particles certify this.
16 geo::Poly heard;for(auto e:h)if(e.kind==1||e.kind==2)heard.push_back(e.q);
17 return geo::contains(geo::hull(heard),q);
18 }
19 inline std::pair<double,double> angles(const geo::Poly&p,geo::Pt q){
20 if(geo::contains(p,q))return {0,360};
21 std::vector<double>a;for(auto z:p){double b=atan2(z.y-q.y,z.x-q.x)*180/geo::pi;if(b<0)b+=360;a
    .push_back(b);}
22 std::sort(a.begin(),a.end());double gap=-1,start=0;
23 for(size_t i=0;i<a.size();++i){double b=i+1<a.size()?a[i+1]:a[0]+360;if(b-a[i]>gap){gap=b-a[i]
    ];start=b;}}
24 return {start-eps,start+360-gap+eps};
25 }
26 inline Bound worst_radius(const geo::Poly&p,geo::Pt q,int budget=96,double tolerance=.10){
27 if(p.empty())return {};auto old=geo::mec(p);
28 auto range=angles(p,q);
29 struct Cell{double a,b,upper,lower;bool operator<(const Cell&o)const{return upper<o.upper;}};
30 auto cell=[&](double a,double b){double mid=(a+b)/2;
31 auto outer=geo::observe(p,q,mid,eps+(b-a)/2+1e-9);
32 auto exact=geo::observe(p,q,mid,eps);
33 double up=outer.empty()?0:std::min(old.r,geo::mec(outer).r+1e-6);
34 double lo=exact.empty()?0:std::max(0.,geo::mec(exact).r);
35 return Cell{a,b,up,lo};
36 };
37 std::priority_queue<Cell>todo;int initial=std::max(1,int(ceil((range.second-range.first)/12.))
    );double lower=0;int used=0;
38 for(int i=0;i<initial;++i){auto x=cell(range.first+(range.second-range.first)*i/initial,range.
    first+(range.second-range.first)*(i+1)/initial);lower=std::max(lower,x.lower);todo.push(x)
    ;++used;}
39 while(used+2<=budget&&!todo.empty()&&todo.top().upper>lower+tolerance){auto x=todo.top();todo.
    pop();double mid=(x.a+x.b)/2;for(auto y:{cell(x.a,mid),cell(mid,x.b)}){lower=std::max(
    lower,y.lower);todo.push(y);++used;}}
40 return {lower,todo.empty()?0:std::max(lower,todo.top().upper)+1e-5,used};
41 }
42 inline geo::Pt project_reception(geo::Pt q,const geo::Poly&p,double range=950.69){
43 for(int pass=0;pass<24;++pass){double maxmove=0;for(auto z:p){double d=geo::dist(q,z);if(d>
    range){auto n=z+(q-z)*(range/d);maxmove=std::max(maxmove,geo::dist(q,n));q=n;}}if(maxmove
    <1e-6)break;}return q;
44 }
45 // Deterministic simplex optimization of the geometric objective. It neither
46 // quantizes bearings nor performs eight-direction particle pattern search.
47 template<class Score> geo::Pt simplex(geo::Pt seed,double scale,Score score,int steps=26){
48 struct V{geo::Pt q;double s;};auto at=[&](geo::Pt q){return V{q,score(q)};};
49 std::array<V,3>x{at(seed),at(seed+geo::Pt{scale,0}),at(seed+geo::Pt{0,scale})};

```

```

50 for(int it=0;it<steps;++it){std::sort(x.begin(),x.end(),[](auto a,auto b){return a.s<b.s;});
    auto c=(x[0].q+x[1].q)/2;auto r=at(c*2-x[2].q);
51 if(r.s<x[0].s){auto e=at(c+(r.q-c)*2);x[2]=e.s<r.s?e:r;}
52 else if(r.s<x[1].s)x[2]=r;
53 else{auto s=at(c+(r.s<x[2].s?r.q-c:x[2].q-c)*.5);if(s.s<std::min(r.s,x[2].s))x[2]=s;else for(
    int j=1;j<3;++j)x[j]=at((x[0].q+x[j].q)/2);}
54 }
55 return std::min_element(x.begin(),x.end(),[](auto a,auto b){return a.s<b.s;})->q;
56 }
57 struct Candidate{bool valid=false;geo::Pt q;double rho=INFINITY,cost=INFINITY;};
58 inline Candidate candidate(const geo::Poly&p,geo::Pt now,geo::Pt first,double angle,const std::
    vector<dynamic_selection::Event>&history,int problem,int mode){
59 Candidate best;if(p.empty()||problem==4)return best; // one positive bearing has no new
    universal heading guarantee.
60 auto c=geo::mec(p);if(c.r<=20)return best;
61 auto u=geo::polar(1,angle);geo::Pt v{-u.y,u.x};
62 geo::Poly seeds{project_reception(now,p),project_reception(c.c,p)};
63 for(double side:{-1.,1.})seeds.push_back(project_reception(first+u*750+v*(side*300),p));
64 auto valid=[&](geo::Pt q){if(geo::norm(q)>2671.99||!reception(p,q,history,problem))return
    false;for(auto e:history)if(e.kind<=2&&geo::dist(e.q,q)<.5)return false;return true;};
65 auto score=[&](geo::Pt q){if(!valid(q))return 1e8+geo::dist(q,project_reception(q,p))*1000;
    double r=worst_radius(p,q,64,.25).upper;
66 return mode==2?geo::dist(now,q)+500*std::max(0.,r-34.8):r;};
67 std::sort(seeds.begin(),seeds.end(),[&](auto a,auto b){return score(a)<score(b);});
68 for(int i=0;i<2;++i){auto q=simplex(seeds[i],75,score,22);if(!valid(q))continue;double r=
    worst_radius(p,q,256,.03).upper;
69 if(mode==2&&r>35)continue;double cost=mode==2?geo::dist(now,q):r;
70 if(cost<best.cost)best={true,q,r,cost};}
71 return best;
72 }
73 }

```

## src/rollout\_sensor.hpp

```

1 #pragma once
2 // Included after Sensor/Reading declarations in policy.hpp.
3 struct RolloutSensor:Sensor {
4 dynamic_selection::Hypothesis source;Pt pos;int channel;double error,time=0;bool cleared=false
    ;
5 std::vector<dynamic_selection::Event>events;
6 RolloutSensor(dynamic_selection::Hypothesis h,Pt p,int k,double e,const std::vector<
    dynamic_selection::Event>&history):source(h),pos(p),channel(k),error(e),events(history){}
7 void move(Pt p){time+=geo::dist(pos,p)/5;pos=p;}
8 Reading measure(Pt p,int k)override{move(p);time+=5+(channel!=k);channel=k;if(cleared)return
    {};}
9 for(auto e:events)if(e.kind<=2&&geo::dist(p,e.q)<1e-7)return {e.kind,e.angle};
10 Reading r;if(dynamic_selection::receives(source,p)){if(geo::dist(source.z,p)<=5)r.kind=1;else
    {r.kind=2;r.angle=fmod(atan2(source.z.y-p.y,source.z.x-p.x)*180/geo::pi+error+720,360);}}
11 events.push_back({p,r.kind,r.angle});return r;
12 }
13 bool clear(Pt p,int)override{move(p);bool ok=!cleared&&geo::dist(p,source.z)<=20;time+=ok?5:3;
    if(ok)cleared=true;return ok;}
14 };

```

## src/routing.hpp

```

1 #pragma once
2 #include "geometry.hpp"
3 #include <numeric>
4 namespace route {
5 inline double length(geo::Pt start, const geo::Poly&p, const std::vector<int>&r){double d=0;for(
    int i:r){d+=geo::dist(start,p[i]);start=p[i];}return d;}
6 inline std::vector<int> nearest(geo::Pt start, const geo::Poly&p, int first=-1){
7     std::vector<int> r; std::vector<bool> used(p.size());
8     if(first>=0){r.push_back(first);used[first]=true;start=p[first];}
9     while(r.size()<p.size()){int best=-1;double d=1e100;for(int i=0;i<(int)p.size();++i)if(!used[i]
        &&geo::dist(start,p[i])<d){d=geo::dist(start,p[i]);best=i;}r.push_back(best);used[best]=
        true;start=p[best];}return r;
10 }
11 inline void two_opt(geo::Pt start, const geo::Poly&p, std::vector<int>&r){
12     for(int iter=0;iter<100;++iter){double best=-1e-7;int bi=-1,bj=-1;
13         for(int i=0;i<(int)r.size()-1;++i)for(int j=i+1;j<(int)r.size();++j){auto a=i?p[r[i-1]]:start
            ;double delta=geo::dist(a,p[r[j]])-geo::dist(a,p[r[i]]);
14             if(j+1<(int)r.size())delta+=geo::dist(p[r[i]],p[r[j+1]])-geo::dist(p[r[j]],p[r[j+1]]);
15             if(delta<best){best=delta;bi=i;bj=j;}}
16         if(bi<0)break;std::reverse(r.begin()+bi,r.begin()+bj+1);
17     }
18 }
19 inline std::vector<int> optimize(geo::Pt start, const geo::Poly&p){auto best=nearest(start,p);
    two_opt(start,p,best);double cost=length(start,p,best);
20 // Deterministic starts: all possible first nodes, capped only by small problem size.
21 for(int first=0;first<(int)p.size();++first){auto r=nearest(start,p,first);two_opt(start,p,r);
    double c=length(start,p,r);if(c<cost-1e-7){cost=c;best=r;}}
22 return best;
23 }
24 }

```

## src/run\_experiments.py

```

1 """Frozen paired experiments; raw seeds are never selected from final results."""
2 import argparse, concurrent.futures, hashlib, json, platform, subprocess, time
3 from pathlib import Path
4 R=Path(__file__).resolve().parents[1]
5 def sha(p):return hashlib.sha256(p.read_bytes()).hexdigest()
6 def run(job):
7     name, start, n, stress, method=job; cmd=[str(R/'bin/benchmark_final'), str(R/'results'/f'{name}.csv'
        ), str(start), str(n), str(stress), str(method), str(method)]
8     begin=time.time(); out=subprocess.run(cmd, capture_output=True, text=True); record={'command':cmd
        , 'exit_code':out.returncode, 'started_unix':begin, 'runtime_s':time.time()-begin, 'stdout':
        out.stdout, 'stderr':out.stderr, 'output':f'results/{name}.csv'}
9     if out.returncode==0:record['output_sha256']=sha(R/record['output'])
10    (R/'artifacts'/f'run-{name}.json').write_text(json.dumps(record, ensure_ascii=False, indent=2));
    print(name, out.returncode, round(record['runtime_s'], 2), flush=True)
11    if out.returncode:raise RuntimeError(record)
12    return record
13 if __name__=='__main__':
14    p=argparse.ArgumentParser(); p.add_argument('--suite', choices=['iid', 'stress', 'dev'], required=
        True); p.add_argument('--workers', type=int, default=3); a=p.parse_args()
15    if a.suite=='iid':jobs=[(f'iid{m}', 2610001, 200, 0, m) for m in [56, 78, 84]]
16    elif a.suite=='stress':jobs=[(f'stress{s}_{m}', 2620001+(s-1)*10000, 40, s, m) for s in range(1, 6)
        for m in [56, 84]]
17    else:jobs=[(f'dev{m}_replay', 2110001, 40, 0, m) for m in [56, 78, 84]]

```

```

18 with concurrent.futures.ThreadPoolExecutor(max_workers=a.workers) as pool:records=list(pool.
    map(run,jobs))
19 (R/'artifacts'/f'execution-{a.suite}.json').write_text(json.dumps({'suite':a.suite,'
    environment':platform.platform(),'binary_sha256':sha(R/'bin/benchmark_final'),'records':
    records},ensure_ascii=False,indent=2))

```

## src/shortcut\_sensor.hpp

```

1 #pragma once
2 #include "policy.hpp"
3 #define B_SHORTCUT_SENSOR 1
4 // Execution adapter: the controller retains its original route anchors, while
5 // physical clear commands may stop earlier on the incoming segment. Measurements
6 // are never moved. With nominal conservative support, clear outcomes are unchanged.
7 // Construct only for a fresh session starting at (0,0), never attach mid-session.
8 class ShortcutSensor:public Sensor {
9     Sensor& io;bool enabled;std::array<geo::Poly,21> support;
10     geo::Pt physical,anchor;double physical_length=0,anchor_length=0;
11     bool fits(const geo::Poly& p,geo::Pt q)const{
12         if(p.empty())return false;for(auto z:p)if(geo::dist(z,q)>20-1e-4)return false;return true;
13     }
14     void record(geo::Pt requested,geo::Pt executed){
15         anchor_length+=geo::dist(anchor,requested);physical_length+=geo::dist(physical,executed);
16         anchor=requested;physical=executed;
17         if(physical_length+geo::dist(physical,anchor)>anchor_length+1e-5)throw std::runtime_error("
            shortcut violated anchored path dominance");
18     }
19     public:int shortcuts=0;
20     explicit ShortcutSensor(Sensor& sensor,bool active=true):io(sensor),enabled(active){}
21     Reading measure(geo::Pt q,int k)override{auto r=io.measure(q,k);record(q,q);return r;}
22     bool clear(geo::Pt q,int k)override{
23         geo::Pt target=q;
24         if(enabled&&k>=1&&k<=20&&fits(support[k],q)){
25             if(fits(support[k],physical))target=physical;
26             else{double low=0,high=1;for(int i=0;i<60;++i){double mid=(low+high)/2;auto trial=physical+(
                q-physical)*mid;if(fits(support[k],trial))high=mid;else low=mid;}target=physical+(q-
                physical)*high;}
27             // A fresh endpoint check handles arithmetic roundoff without assuming success.
28             if(!fits(support[k],target))target=q;
29         }
30         bool result=io.clear(target,k);record(q,target);
31         if(geo::dist(q,target)>1e-7){++shortcuts;if(!result)throw std::runtime_error("guaranteed
            shortcut clear failed");}
32         if(result&&k>=1&&k<=20)support[k].clear();return result;
33     }
34     void audit(int k,const geo::Poly&p)override{if(k>=1&&k<=20)support[k]=p;io.audit(k,p);}
35     void phase(int stage)override{io.phase(stage);}
36     double saved_distance()const{return anchor_length-physical_length;}
37 };

```

## src/simulator.hpp

```

1 #pragma once
2 #include "policy.hpp"
3 #include <cstdint>
4 #include <fstream>

```

```

5 #include <iomanip>
6 #include <random>
7 struct Jammer {int ch;Pt p;double radius;bool directional;double heading;bool removed=false;};
8 inline uint64_t mix(uint64_t z){z+=0x9e3779b97f4a7c15ULL;z=(z^(z>>30))*0xbf58476d1ce4e5b9ULL;z
   =(z^(z>>27))*0x94d049bb133111ebULL;return z^(z>>31);}
9 inline std::vector<Jammer> make_case(int problem,int seed,int stress=0){
10  std::mt19937_64 rng(seed);auto U=[&]() {return std::generate_canonical<double,53>(rng);};
11  int n=10+rng()%7;std::vector<int>ch;for(int k=1;k<=20;++k)ch.push_back(k);std::shuffle(ch.
   begin(),ch.end(),rng);
12  int nd=1+rng()%(n-1);std::vector<Jammer> j;
13  for(int k=0;k<n;++k){Pt p;do{p=geo::polar(1800*sqrt(U()),360*U());}while(std::any_of(j.begin()
   ,j.end(), [&](Jammer a){return geo::dist(a.p,p)<40;});});
14  double r=1000+500*U(),heading=360*U();bool dir=problem==4&&k<nd;
15  if(stress==1){r=1000;p=geo::polar(1800,(k+.137)*360/n);heading=atan2(p.y,p.x)*180/geo::pi;dir
   =problem==4&&k<n-1;}
16  if(stress==2){r=1000;p={-1700.+3400.*k/(n-1),double(k%2)*.01};heading=k%2?90:270;dir=problem
   ==4&&k<n-1;}
17  if(stress==3){r=1000;p={double(k%5)*5,0};heading=k%2?0:180;dir=problem==4&&k<n-1;}
18  j.push_back({ch[k],p,r,dir,heading});return j;
19 }
20 class Sim:public Sensor {
21  std::vector<Jammer> truth;int seed,errmode;std::ofstream trace;
22  public:Pt pos;int channel=1,measures=0,miss=0,success=0,switches=0,audits=0;double time=0,
   length=0,max_violation=0;
23  Sim(std::vector<Jammer> j,int s,int e,const std::string&file=""):truth(j),seed(s),errmode(e){
   if(!file.empty()){trace.open(file);trace<<"kind,channel,x,y,result,angle,time\n";}}
24  void move(Pt q){double d=geo::dist(q,pos);length+=d;time=round((time+d/5)*1e6)/1e6;pos=q;}
25  Jammer* pending(int k){for(auto &j:truth)if(j.ch==k&&!j.removed)return &j;return nullptr;}
26  void check(){if(time>=360000)throw std::runtime_error("virtual deadline");}
27  Reading measure(Pt p,int k)override {check();move(p);++measures;if(k!=channel){++switches;time
   +=1;}channel=k;time+=5;Reading r;
28  auto j=pending(k);if(j&&geo::dist(p,j->p)<=j->radius&&(!j->directional||geo::dot(p-j->p,geo::
   polar(1,j->heading))>=-1e-9)){
29  if(geo::dist(p,j->p)<=5)r.kind=1;else{r.kind=2;double a=atan2(j->p.y-p.y,j->p.x-p.x)*180/geo
   ::pi;
30  uint64_t v=mix((uint64_t)llround(p.x*100))^mix((uint64_t)llround(p.y*100)+1234567)^mix(seed
   );
31  double e=2*(mix(v)>>11)*0x1.0p-53-1;
32  if(errmode==1)e=1;if(errmode==2)e=-1;if(errmode==3)e=sin(p.x/350)*cos(p.y/350);if(errmode
   ==4)e=(p.x>0?1:-1);
33  r.angle=fmod(round(fmod(a+e+720,360)*100)/100,360);}}
34  if(trace)trace<<std::setprecision(12)<<"measure,"<<k<<","<<p.x<<","<<p.y<<","<<r.kind<<","<<r
   .angle<<","<<time<<"\n";return r;
35 }
36 bool clear(Pt p,int k)override{check();move(p);auto j=pending(k);bool ok=j&&geo::dist(p,j->p)
   <=20;time+=ok?5:3;if(ok){j->removed=true;++success;}else ++miss;
37  if(trace)trace<<std::setprecision(12)<<"clear,"<<k<<","<<p.x<<","<<p.y<<","<<ok<<","<<time
   <<"\n";return ok;}
38  void audit(int k,const Poly&p)override{++audits;auto j=pending(k);if(j&&!geo::contains(p,j->p)
   )throw std::runtime_error("truth excluded from conservative polygon");}
39  int total()const{return(int)truth.size();}
40 };

```

## src/validate\_mock.py

```

1 """Independent nonofficial public-protocol Python simulator kernel + real HTTP tests of C++
   solver."""

```

```

2 import argparse
3 import json
4 import sys
5 import threading
6 import time
7 from pathlib import Path
8 from bridge import ROOT, drive, DeadlineClient
9 from mocksim.arena import Session, make_case
10 from mocksim.client import RobotClient
11 from mocksim.server import MockSimulator, serve
12
13 class KernelClient:
14     def __init__(self, problem, seed):
15         self.session = Session(make_case(problem=problem, seed=seed))
16     def enter(self):
17         self.session.enter()
18         return dict(accepted=True, remaining_real_duration_s=1200)
19     def measure(self, x, y, k):
20         o=self.session.measure(x, y, k)
21         return dict(accepted=True, virtual_time_s=o.virtual_time_s, **o.payload)
22     def clear(self, x, y, k):
23         o=self.session.clear(x, y, k)
24         return dict(accepted=True, virtual_time_s=o.virtual_time_s, **o.payload)
25     def exit(self):
26         o=self.session.exit()
27         return dict(accepted=True, virtual_time_s=o.virtual_time_s)
28
29 def main():
30     p=argparse.ArgumentParser();p.add_argument('--count',type=int,default=30);p.add_argument
31     ('--early',type=float,default=35);p.add_argument('--planner-depth',type=int,choices
32     =[0,1,2,3],default=2);p.add_argument('--strategy',choices=['geometric','joint','previous
33     ','stage1','stage2','baseline'],default='joint')
34     a=p.parse_args();rows=[]
35     for problem in [3,4]:
36         for seed in range(1610001,1610001+a.count):
37             client=KernelClient(problem,seed)
38             out=drive(problem,client,a.early,planner_depth=a.planner_depth,strategy=a.strategy)
39             assert out['cleared']==client.session.case.total
40             rows.append(dict(layer='original_kernel',problem=problem,seed=seed,strategy=out['
41             strategy'],planner_depth=out['planner_depth'],**client.session.statistics(out['runtime_s
42             ']),certificate=out['certificate']))
43         for seed in [1620001,1620002,1620003]:
44             sim=MockSimulator(problem=problem,mode='practice',seed=seed,team_id='LOCAL_RESEARCH
45             ',port=0)
46             server=serve(sim);threading.Thread(target=server.serve_forever,daemon=True).start()
47             ;sim.start()
48             file=ROOT/'results'/f'LOCAL_HTTP_{a.strategy}_D{a.planner_depth}_P{problem}_{seed}.
49             jsonl'
50             try:
51                 with file.open('w',encoding='utf-8') as f:
52                     client=DeadlineClient(base_url=f'http://127.0.0.1:{server.server_address
53                     [1]}',robot_id='LOCAL_RESEARCH',log_sink=lambda r:f.write(json.dumps(r,ensure_ascii=False)
54                     +'\n'))
55                     out=drive(problem,client,a.early,planner_depth=a.planner_depth,strategy=a.
56                     strategy)
57                     assert out['cleared']==sim.case.total
58                     rows.append(dict(layer='local_HTTP',problem=problem,seed=seed,total=sim.case.
59                     total,**out))

```

```

48         finally:
49             server.shutdown();server.server_close()
50             (ROOT/'results'/f'original_mock_{a.strategy}_depth{a.planner_depth}.json').write_text(json.
                dumps(rows,ensure_ascii=False,indent=2),encoding='utf-8')
51             print(f'validated {len(rows)} nonofficial simulator cases (including 6 HTTP runs)')
52
53 if __name__=='__main__':main()

```

## src/write\_current\_report.py

```

1  """Verify independent results, original simulator traces and write the handoff."""
2  import hashlib
3  import json
4  from pathlib import Path
5  from pair_analysis import read_rows, summarize
6
7  ROOT=Path(__file__).resolve().parents[1]
8  RESULTS=ROOT/'results'
9
10
11 def main():
12     iid=json.loads((RESULTS/'independent_summary.json').read_text(encoding='utf-8'))
13     stress=json.loads((RESULTS/'stress_summary.json').read_text(encoding='utf-8'))
14     baseline=ROOT.parent/'B题联合优化'
15     frozen=json.loads((baseline/'manifest.json').read_text(encoding='utf-8'))['sha256']
16     for name,digest in frozen.items():
17         if name.startswith('src/'):
18             assert hashlib.sha256((baseline/name).read_bytes()).hexdigest()==digest, name
19     old=json.loads((RESULTS/'original_mock_previous_depth2.json').read_text(encoding='utf-8'))
20     new=json.loads((RESULTS/'original_mock_joint_depth2.json').read_text(encoding='utf-8'))
21     assert len(old)==len(new)==26
22     for a,b in zip(old,new):
23         assert (a['layer'],a['problem'],a['seed'])==(b['layer'],b['problem'],b['seed'])
24         if a['layer']=='original_kernel':
25             for field in ('jammer_total','cleared_count','measure_count','clear_count','
                clear_miss_count'):
26                 assert a[field]==b[field]
27                 assert b['cleared_count']==b['jammer_total']
28                 assert b['total_locate_clear_time_s']<=a['total_locate_clear_time_s']+1e-4
29         else:
30             assert a['total']==a['cleared']==b['total']==b['cleared']
31             assert b['virtual_time_s']<=a['virtual_time_s']+1e-4
32     http_actions=0
33     for problem in (3,4):
34         for seed in (1620001,1620002,1620003):
35             def trace(strategy):
36                 file=RESULTS/f'LOCAL_HTTP_{strategy}_D2_P{problem}_{seed}.jsonl'
37                 return [json.loads(line) for line in file.read_text(encoding='utf-8').
                    splitlines()]
38             a,b=trace('previous'),trace('joint')
39             assert len(a)==len(b)
40             for x,y in zip(a,b):
41                 assert x['path']==y['path']
42                 if x['path'] not in ('/measure','/clear'):continue
43                 http_actions+=1
44                 assert x['request']['channel']==y['request']['channel']

```

```

45         if x['path']=='/measure':assert x['request']['position']==y['request']['
position']
46         for field in ('measure_result','svd_deg','clear_result'):
47             assert x['response'].get(field)==y['response'].get(field)
48     development={}
49     dev_base=read_rows([RESULTS/'dev56.csv'])
50     files=[*sorted(RESULTS.glob('dev58_[abcd].csv')),RESULTS/'dev60.csv',*sorted(RESULTS.glob('
dev_search_*.csv')),RESULTS/'dev66.csv',*sorted(RESULTS.glob('dev_continuous_*.csv')),*
sorted(RESULTS.glob('dev_bins_*.csv')),*sorted(RESULTS.glob('dev_portfolio_*.csv')),
RESULTS/'dev78.csv']
51     import csv
52     variants={}
53     for file in files:
54         with file.open(newline='',encoding='utf-8') as stream:
55             for row in csv.DictReader(stream):
56                 method=int(row['method']);key=tuple(int(row[k]) for k in ('problem','seed','
stress'))
57                 for k in ('time_s','avg_s','runtime_s'):row[k]=float(row[k])
58                 assert key not in variants.setdefault(method,{})
59                 variants[method][key]=row
60     for method,rows in sorted(variants.items()):
61         assert rows.keys()==dev_base.keys(),method
62         development[method]={}
63         for problem in (3,4):
64             keys=sorted(k for k in rows if k[0]==problem)
65             development[method][problem]=summarize([dev_base[k] for k in keys],[rows[k] for k
in keys])
66     (RESULTS/'development_summary.json').write_text(json.dumps(development,ensure_ascii=False,
indent=2),encoding='utf-8')
67     lines=['# B题本轮改进结果','',
68           '结论：只获得很小的正收益，未达到新增10%。默认方法78仅在原方法56外增加“保证清除的沿线提前停
车”，其它试验方法均未采用。','',
69           '## 独立验证：每问200例','',
70           '同一批新案例，种子2310001—2310200。比较基线是上一轮最佳method56，不是更早的两步DP方法0。
以下为本地模拟，不是官方正式成绩。','',
71           '|问题|原总均时|s|新总均时|s|节省|s|总均时下降|原单源均时|s|新单源均时|s|全清除|',
72           '|---|---:|---:|---:|---:|---:|---:|---:|',
73     for p in (3,4):
74         r=iid['groups'][f'p{p}_stress0']
75         lines.append(f"|{p}|{r['baseline_total_s']:.6f}|{r['candidate_total_s']:.6f}|{r['
saving_s_mean']:.6f}|{r['total_gain_pct']:.6f}%|{r['baseline_single_s']:.6f}|{r['
candidate_single_s']:.6f}|200/200|")
76     lines+=['','配对节省时间的95%正态近似区间：','']
77     for p in (3,4):
78         r=iid['groups'][f'p{p}_stress0'];lo,hi=r['saving_s_ci95']
79         lines.append(f"- Q{p}: [{lo:.6f}, {hi:.6f}]秒；{r['wins']}局更快，{r['ties']}局持平，{r
['losses']}局变慢（0.0001秒容差）；最大CPU耗时{r['cpu_max_s']:.3f}秒。")
80     lines+=['','浮点累计舍入可产生1—3微秒的差异；无变慢指超过上述容差的变慢。几何论证针对未逐步取
整的路程，不能声称逐微秒严格改善。',
81           '','## 为什么可以保持原控制器行为','',
82           '执行适配器保存真实物理位置与原计划路标。所有测向命令原样发送。只有原清除位置能够以至少0
.0001米余量覆盖整个保守多边形时，才在“当前物理位置→原清除位置”线段上二分找到最早可保证清除的
位置。实际发送清除命令、等待真实返回；不伪造成功。其它清除命令原样发送。',
83           '',
84           '控制器继续使用原计划路标，因此未来测向坐标不因这项改动改变。对同一测量噪声实现且清除几何确
定的名义位置模型，清除结果与后续控制器分支不变。',
85           '']

```

```

86         '记录计实际路程为L, 原计划路程为L, 实际位置为x, 计划路标为a。每个动作后验证  $\|L + \|x-a\| - L$ 
      ~ (浮点容差 $1e-5$ 米)。提前停车点在线段上, 下一次移向原路标时由三角不等式保持该不变量。它是固定路
      标控制器的执行路程上界, 不是全任务最优性证明。',
87     '',
88     '适配器仅支持从原点启动的新会话, 不能中途接管。非零位置误差模式自动禁用快捷清除; 需要重新计
      入位置误差后才能扩展保证。',
89     '', '## 压力测试', '',
90     '五组分别为边界外向、共线端点、重合/近重合、相关角误差、分区端点误差; 每组每问40例, 种子与开
      发/IID不重叠。',
91     '', '|组别|Q3总均时下降|Q4总均时下降|清除结果|', '|---|---|---|---|']
92 for k in range(1,6):
93     a=stress['groups'][f'p3_stress{k}'];b=stress['groups'][f'p4_stress{k}']
94     lines.append(f'|{k}|{a['total_gain_pct']:.6f}%|{b['total_gain_pct']:.6f}%|两问均40
      /40|')
95 lines+=['','## 未采用的开发试验','',
96     '每项每问40例, 使用同一开发集。不能将这些反复使用的开发数据称为独立验证。59是修正入口方向的
      终端实现, 未进行完整开发对照; 58保留原入口顺序作为历史负结果。',
97     '', '|方法|变化|Q3总均时变化 (正为改善)|Q4总均时变化|', '|---|---|---|---|']
98 descriptions={58:'连续域DP叶端费用校准 (历史入口顺序)',60:'可变宽度覆盖, 典型78+62点',61:'原
      覆盖+期望搜索时间排序',62:'可变覆盖+搜索排序',63:'原覆盖+10%覆盖点先验混合',64:'可变覆盖+10%混
      合',65:'可变覆盖+30%混合',66:'DP剩余动作预算校准',67:'连续候选点80次预算',68:'连续候选点+Q3取
      63假设',69:'连续候选点160次预算',70:'4°预测分箱',71:'3°预测分箱',72:'6°预测分箱',73:'8°预测分
      箱',74:'可变覆盖, 预测收益5%/5秒门槛',75:'覆盖+排序, 10%/10秒门槛',76:'覆盖20%/20秒门槛',77:'
      覆盖仅要求预测费用更低',78:'冻结方法56+保证清除提前停车'}
99 for method,groups in development.items():
100     lines.append(f'|{method}|{descriptions[method]}|{groups[3]['total_gain_pct']:.6f}%|{
      groups[4]['total_gain_pct']:.6f}%|')
101 lines+=['','连续候选搜索是确定性局部模式搜索, 不是完整ADVT或POMCPow实现。2°/3°等分箱仅是有限观
      测模型敏感性试验, 不等于有界噪声的概率模型。连续域叶端保留几何域, 但条件粒子权重仍是近似; 未采用
      其增大的CPU开销来换取不存在的收益。',
102     '', '## 验证记录','',
103     '- IID与压力共800个配对案例、1600次策略运行, 全部清除; 动作计数与DP计数一致; 时间差与执行路
      程差/5一致。',
104     '- 12个C++完整逐动作对照案例, 涵盖两问及五类压力; 测向位置、角度、清除响应保持一致。',
105     f'- 原模拟器两策略各26次 (20次内核+6次本地HTTP), 全部清除; 六对HTTP日志共核对{
      http_actions}个动作。',
106     '- 旧成果src哈希逐项与其冻结manifest一致; 最终编译的method56与旧CSV十行复现一致, method78
      十行开发回放一致 (不比较CPU时间)。',
107     '- 独立源码审查确认当前method78及默认接入无关键或重要问题, 并核对了路程不变量的适用条件。该
      次审查未独立重跑最终基准或复核统计总数, 统计核验由本轮主执行完成。',
108     '- 传输测试在并行工作期间曾出现一次“静默子进程退出时间”超出1秒; 未改测试阈值或传输逻辑。隔
      离重跑约0.241秒通过, 随后完整9项通过。不能将该单次波动归因于已确认的具体原因。',
109     '', '## 文件与复现','',
110     '- `src/shortcut_sensor.hpp`: 本轮默认唯一启用的新优化。',
111     '- `validation_frozen.json`: 验证前冻结的策略与样本。',
112     '- `results/independent_summary.json`、`stress_summary.json`: 配对统计; 原始CSV和HTTP日
      志同目录。',
113     '- `results/development_summary.json`: 含负结果的完整开发对照。',
114     '- `run_all.ps1`: 重建与验证; `Full`重新运行冻结的1600次策略测试。',
115     '', '## 文献关联','',
116     '把清除点视为可访问区域, 而非必须到达的固定中心, 受[Peng, Wei, Isler, ICRA 2023,
      Stochastic Traveling Salesperson Problem with Neighborhoods for Object Detection](https://
      arxiv.org/html/2407.06366v1)的建模方式启发。本轮沿线提前停车和路程不变量为此项目的具体推导, 未
      移植该文近似比。',
117     '', '继续冲击两位数提升需要新的结构性改进; 本轮结果不支持声称已经找到或接近全局最优方案。
      ', '']
118 (ROOT/'最终结果.md').write_text('\n'.join(lines),encoding='utf-8')

```

```

119     manifest=dict(baseline_method=56,default_method=78,verification=dict(iid_paired_cases=400,
    stress_paired_cases=400,original_mock_runs=52,http_action_pairs=http_actions,
    baseline_sources_unchanged=True),sha256={})
120     for folder in ('src','tests','review'):
121         for file in sorted((ROOT/folder).rglob('*')):
122             if file.is_file() and file.suffix in ('.hpp','.cpp','.py','.md'):
123                 manifest['sha256'][file.relative_to(ROOT).as_posix()]=hashlib.sha256(file.
    read_bytes()).hexdigest()
124     for file in sorted(RESULTS.iterdir()):
125         if file.is_file() and file.suffix in ('.csv','.json','.jsonl'):
126             manifest['sha256'][file.relative_to(ROOT).as_posix()]=hashlib.sha256(file.
    read_bytes()).hexdigest()
127     for name in ('robot.exe','benchmark.exe','README.md','run_all.ps1','最终结果.md','
    validation_frozen.json'):
128         file=ROOT/name
129         manifest['sha256'][name]=hashlib.sha256(file.read_bytes()).hexdigest()
130     (ROOT/'manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding
    ='utf-8')
131     print(f'PASS report: 800 paired cases,52 original runs,{http_actions} matched HTTP actions;
    baseline unchanged')
132
133
134 if __name__=='__main__':main()

```

## src/write\_report.py

```

1 import json, csv
2 from pathlib import Path
3 R=Path(__file__).resolve().parents[1]
4 stage1=json.loads((R/'results/summary.json').read_text(encoding='utf-8'))
5 stage2=json.loads((R/'results/summary_stage2.json').read_text(encoding='utf-8'))
6 p3,p4=stage2['iid']['3'],stage2['iid']['4']
7 table=[]
8 for p in ['3','4']:
9     a=stage2['iid'][p]
10    table.append(f"|问题{p}|{a['baseline_single_s']:.2f}|{a['new_single_s']:.2f}|{a['
    saving_percent']:.2f}|{a['wins']}/{a['losses']}|200/200|")
11 stress=[]
12 for k,label in enumerate(['边界向外','近共线端点','重合近距离','平滑相关','分区端点'],1):
13     a,b=stage2['stress'][str(k)]['3'],stage2['stress'][str(k)]['4']
14     stress.append(f"|{label}|{a['saving_percent']:.2f}|{b['saving_percent']:.2f}|40/40, 40
    /40|")
15 def ci(v):return f"[{v[0]:.2f}, {v[1]:.2f}]"
16 outcome='达到' if p4['saving_percent']>=10 else '尚未达到'
17 doc=f'''# 联合优化结果与方法
18
19 日期: 2026-09-12.主基线固定为上一轮默认两步DP (B题动态规划), 不是更早的31点方案。第一阶段未达标结
    果保留; 第二阶段仅新增按观测权重选择完整光学扫描顺序。
20
21 ## 当前结果
22
23 问题4在第二阶段新IID中的总均时下降 **{p4['saving_percent']:.2f}%**, {outcome}样本均值10%目标。问题
    3下降 **{p3['saving_percent']:.2f}%**。是否达到目标按题号分别判断, 不能将一个小问的结果代替另
    一个。
24
25 |新同案例IID|基线单源秒|新单源秒|总均时下降|更快/更慢|全清除|
26 |---|---|---|---|---|---|

```

```

27 {chr(10).join(table)}
28
29 本轮seed1710001—1710200，每问200个基础环境，两种策略都在同一环境运行。单源均值按 $\text{mean}(T/N)$ ，节省率
    按 $\text{mean}(T_{\text{旧}}-T_{\text{新}})/\text{mean}(T_{\text{旧}})$ 。它们不是同一个百分比口径，不能用不同轮次的单源均值相除算收益。
30
31 - Q3总均时： $\{p3['\text{baseline\_total\_s}']:.2f\}+\{p3['\text{new\_total\_s}']:.2f\}$ 秒，配对每局节省 $\{p3['\text{saving\_mean\_s}']:.2f\}$ 秒，95%t区间 $\{ci(p3['\text{paired\_ci95\_s}'])\}$ 。
32 - Q4总均时： $\{p4['\text{baseline\_total\_s}']:.2f\}+\{p4['\text{new\_total\_s}']:.2f\}$ 秒，配对每局节省 $\{p4['\text{saving\_mean\_s}']:.2f\}$ 秒，95%t区间 $\{ci(p4['\text{paired\_ci95\_s}'])\}$ 。
33 - Q4超过10%门槛的额外节省量定义为 $0.9T_{\text{旧}}-T_{\text{新}}$ ，均值 $\{p4['\text{excess\_over\_10pct\_s}']:.2f\}$ 秒，95%区间 $\{ci(p4['\text{excess\_over\_10pct\_ci95\_s}'])\}$ 。区间若跨零，仅能说本组样本均值超过10%，不能将其升级为已经统计
    证明总体改善大于10%。
34
35 ## 优化机制
36
37 1. **动作级联合调度。** 已发现目标不再只以固定第二测点作为全局路线代理。规划器每执行一次真实检测或试
    清就回到全局排序，更新下一目标与下一测点。每频道累计动态动作最多6次，之后进入确定完成流程，避免
    无限重规划。
38 2. **直接连续光学兜底。** 问题4的DP结束后直接覆盖当前保守区域，避免重新走旧固定测点。所有网格点仍覆
    盖完整连续区域。
39 3. **条件权重光学顺序。** 从真实观测历史构造有限位置权重，比较原蛇形正反方向及若干循环起点，选取较低
    预测清除成本的完整序列。没有删除任何网格点；即使权重错误，完整扫描仍保留。权重只是规划近似，不是
    官方真实分布。
40 4. **问题3的联动与安全区域。** 采用入口候选菜单、机会补测和1124米半径的六点环加原点；定位足够精确时
    允许在安全清除邻域内选择位置。机会补测每频道最多4次。
41
42 最终Q4仍使用21点固定覆盖、两步前瞻、最多63个假设、每次规划6000状态展开上限。Q3使用32假设、两步前瞻。
    更深前瞻、候选延迟处理、无条件机会补测、多个扫描顺序等方案均在开发记录中保留，没有仅保留获胜版
    本。
43
44 ## 开发和独立测试的区分
45
46 开发组为1—40及1001—1040，每问80环境。前10例曾用于较重方法预筛，不算新的独立样本。方法参数只依据开
    发数据选择。
47
48 第一阶段冻结38后，用1510001—1510200测试：Q3下降 $\{\text{stage1}['\text{iid}']['3']['\text{saving\_percent}']:.2f\}\%$ ，Q4下
    降 $\{\text{stage1}['\text{iid}']['4']['\text{saving\_percent}']:.2f\}\%$ ，未达到10%。记录位于`iid_baseline.csv`、`
    iid_final.csv`、`summary.json`，没有覆盖。
49
50 第二阶段新增光学顺序机制，仍只在开发组测试；开发Q4降幅11.8379%后冻结39，再使用事先指定的新1710001
    组。记录为`iid2_baseline.csv`、`iid2_final.csv`、`summary_stage2.json`。两阶段的数据不能拼成
    同一个固定策略样本。
51
52 ## 压力组和不情况
53
54 五类压力组采用1720001—1760001各起40种子、每问40环境。相对同案例旧两步DP：
55
56 |压力类别|Q3总均时下降|Q4总均时下降|全清除|
57 |---|---:|---:|---|
58 {chr(10).join(stress)}
59
60 压力场景不与IID混合计算达标比例。负百分比表示变慢，不能被全清除掩盖。IID中Q3最差种子 $\{p3['\text{worst\_seed}']\}$ 额外耗时 $\{p3['\text{max\_slowdown\_s}']:.2f\}$ 秒；Q4最差种子 $\{p4['\text{worst\_seed}']\}$ 额外耗时 $\{p4['\text{max\_slowdown\_s}']:.2f\}$ 秒。
61
62 ## 几何与完成依据
63

```

64 问题4沿用已认证21点。问题3新环的连续覆盖按径向凸性证明：半径小于999.6米由原点覆盖；剩余圆域任一点与最近环点角差不超过30°，距离平方对源半径为凸函数，只需检查999.6和1800两个端点。用有理数外包 $\sqrt{3}$ 计算得两端均不超过999.6²。实际C++坐标与理想环坐标的误差也已逐坐标核验，小于1e-9米，远小于0.4米接收余量。证据为`results/q3\_coverage\_bound.json`及`q3\_production\_points.json`。

65

66 安全清除点属于B(c,20-r- )，其中B(c,r)包围全部当前多边形，由三角不等式保证距离任意可行源不超过20米。概率模型仅用于行动排序，不替代这个连续区域。

67

68 循环移位的光学路径仍含全部网格点，至多新增一个首尾长连接。保守地沿用上一轮349246.4秒界，增加最多64次辅助检测及16次near清除，再增加16个不超过1520米的光学连接，得到354574.4秒<100小时。这里继承原几何和合法响应条件，不能当作现实网络20分钟的保证，也不是全局最优耗时界。

69

70 ## 运行开销与验证

71

72 IID中新版Q3整局C++算法平均{p3['new\_cpu\_mean\_s']:.3f}秒，最大{p3['new\_cpu\_max\_s']:.3f}秒；Q4平均{p4['new\_cpu\_mean\_s']:.3f}秒，最大{p4['new\_cpu\_max\_s']:.3f}秒。Q3/Q4的状态预算命中数分别为{p3['budget\_hits']}/{p4['budget\_hits']}。该开销是算法墙钟，不是表中的虚拟任务时间。

73

74 原始数据逐行核验实际清除数、费用分解、源数、种子和配对。新CLI默认joint，提供stage1和baseline对照。已有Bellman穷举对照、发射方向/历史约束、动作次数、网格保留、安全清除区域和通信截止回归；本地原模拟器核与HTTP演练另有记录。没有将自检称为外部独立审稿，也没有执行官方正式测试。

75 '''

76 (R/'结果与方法.md').write\_text(doc,encoding='utf-8')

77 print('generated results report from both stages')

## src/write\_report\_stage3.py

```
1 import json
2 from pathlib import Path
3 R=Path(__file__).resolve().parents[1]
4 s1=json.loads((R/'results/summary.json').read_text(encoding='utf-8'))
5 s2=json.loads((R/'results/summary_stage2.json').read_text(encoding='utf-8'))
6 s3=json.loads((R/'results/summary_stage3.json').read_text(encoding='utf-8'))
7 def ci(v):return f'[{v[0]:.2f}, {v[1]:.2f}]'
8 table=[]
9 for p in ['3','4']:
10     x=s3['iid'][p];single=100*(1-x['new_single_s']/x['baseline_single_s'])
11     table.append(f"|问题{p}|{x['baseline_single_s']:.2f}|{x['new_single_s']:.2f}|{single:.2f}%|{x['saving_percent']:.2f}%|200/200|")
12 stress=[]
13 for k,label in enumerate(['边界向外','近共线端点','重合近距离','平滑相关','分区端点'],1):
14     a,b=s3['stress'][str(k)]['3'],s3['stress'][str(k)]['4'];stress.append(f"|{label}|{a['saving_percent']:.2f}%|{b['saving_percent']:.2f}%|40/40, 40/40|")
15 p3,p4=s3['iid']['3'],s3['iid']['4']
16 doc=f'''# 联合优化最终结果（第三阶段）
17
18 主基线为刚交付的B题动态规划默认两步DP。所有百分比基于同案例比较，没有切换到较弱历史基线。
19
20 ## 新独立测试
21
22 seed1910001—1910200，每问200个基础环境，两方法各运行一次。方法56已在打开此组之前冻结。
23
24 |小问|基线单源秒|新单源秒|单源均时下降|总均时下降|全清除|
25 |---|---|---|---|---|---|
26 {chr(10).join(table)}
27
28 单源均时=mean(T/N)，总均时下降=mean(T旧-T新)/mean(T旧)。各局N不同，这两个百分比不相同。
```

29

30 - Q3总均时: {p3['baseline\_total\_s']:.2f}+{p3['new\_total\_s']:.2f}秒; 节省95%配对t区间{ci(p3['paired\_ci95\_s'])}秒; 更快/更慢={p3['wins']}/{p3['losses']}。

31 - Q4总均时: {p4['baseline\_total\_s']:.2f}+{p4['new\_total\_s']:.2f}秒; 节省95%配对t区间{ci(p4['paired\_ci95\_s'])}秒; 更快/更慢={p4['wins']}/{p4['losses']}。

32 - Q4超过10%门槛的量为0.9T旧-T新, 均值{p4['excess\_over\_10pct\_s']:.2f}秒, 95%区间{ci(p4['excess\_over\_10pct\_ci95\_s'])}秒。若此区间跨零, 样本均值达标与总体改善已证超过10%必须区分。

33

34 ## 三阶段记录

35

36 |冻结阶段|新测试种子|Q3总均时下降|Q4总均时下降|

37 |---|---|---:|---:|

38 |38: 联合调度/直接光学|1510001—1510200|{s1['iid']['3']['saving\_percent']:.2f}%|{s1['iid']['4']['saving\_percent']:.2f}%|

39 |39: 增加条件权重顺序|1710001—1710200|{s2['iid']['3']['saving\_percent']:.2f}%|{s2['iid']['4']['saving\_percent']:.2f}%|

40 |56: 增加自适应几何覆盖|1910001—1910200|{p3['saving\_percent']:.2f}%|{p4['saving\_percent']:.2f}%|

41

42 前两阶段没有达到10%, 记录及失败/变慢案例全部保留。每次新机制都先在1—40、1001—1040开发组比较, 冻结后才打开新测试组。三阶段不是同一固定策略的重复样本, 不能合并为一个性能区间。开发探索及多轮验证本身也意味着不能把最后一次结果当作未经任何模型探索的普适保证。

43

44 ## 最终实际采用的机制

45

46 \*\*Q4: \*\* 两步DP、最多63个位置/接收半径/朝向假设, 每个实际动作之后重新参与全局路线排序; 每源最多6次动态动作; 之后直接进入连续光学覆盖。固定发现布局仍为21点, 未声称减少到20点。

47

48 光学覆盖不再直接填满外接矩形。沿首次示向轴将保守多边形按不超过28米宽的条带切分: 某条带的最小包围圆若小于20-1e-5米, 用一个清除点覆盖; 否则继续纵向分割并逐块检查包围圆。条带覆盖整个多边形, 子块均被清除圆覆盖。典型1500米长扇形由108个清除点降为78个。这是局部清除点数的降低, 不是21个全局发现点的降低。

49

50 随后按已收到观测的有限条件权重比较完整扫描顺序。排序不删除任何新覆盖子块。权重近似出错会影响效率, 但不会使已构造的覆盖子块消失。实际几何使用浮点计算和余量, 未冒充形式化实数证书。

51

52 \*\*Q3: \*\* 入口候选菜单、动作级联动、每频道最多4次机会补测、1124米六点环加原点、安全清除邻域。问题3在这一轮没有同样达到Q4的改善幅度, 须单独报告。

53

54 ## 覆盖与预算

55

56 Q3新环的连续覆盖通过径向凸性和√3的有理数界核验, 理想覆盖半径999.6米, 实际C++坐标逐点误差小于1e-9米。见q3\_coverage\_bound.json和q3\_production\_points.json。

57

58 Q4适配光学最多54条带、每条至多两个清除圆, 仍以108点作保守上界。相邻保留中心的距离可取77米上界; 若循环移位, 另取1520米连接上界。以最多133个长移动段、每段5344米, 再加局部扫描, 可得到保守虚拟费用界182027.2秒, 低于100小时。Q3沿用更宽松的354574.4秒界。都以合法响应和正确维护保守区域为条件; 网络现实截止仍由通信桥控制。

59

60 ## 压力和不利情况

61

62 第三阶段压力组使用1920001—1960001各起40种子, 每问每类40环境。相对同案例旧两步DP:

63

64 |压力组|Q3总均时下降|Q4总均时下降|全清除|

65 |---|---:|---:|---:|

66 {chr(10).join(stress)}

67

```

68 负数表示变慢。压力组不参与IID达标计算。Q3 IID最差种子{p3['worst_seed']}慢{p3['max_slowdown_s']:.2f}秒；Q4最差种子{p4['worst_seed']}慢{p4['max_slowdown_s']:.2f}秒。不会仅保留获胜案例。
69
70 ## 开销、测试与使用
71
72 新版Q3整局C++平均{p3['new_cpu_mean_s']:.3f}秒，最大{p3['new_cpu_max_s']:.3f}秒；Q4平均{p4['new_cpu_mean_s']:.3f}秒，最大{p4['new_cpu_max_s']:.3f}秒。状态预算命中数Q3/Q4={p3['budget_hits']/p4['budget_hits']}。
73
74 测试包括Bellman穷举对照、观测一致性、每源预算、全局有限进度、光学完整序列、100个不同定位区域的覆盖交叉检查、实际坐标、CLI和通信回归。默认控制器通过原模拟器核与本地HTTP演练，原始记录在original_mock_joint_depth2.json。未执行官方正式测试，未安排外部独立审稿。
75
76 默认`--strategy joint --planner-depth 2`。`--strategy baseline`回到原两步DP；stage1、stage2保留此前版本。源代码、开发记录、三阶段独立数据、压力数据和复现脚本均保存在本目录。
77 '''
78 (R/'最终结果.md').write_text(doc,encoding='utf-8')
79 print('final stage3 report generated')

```

## src/write\_solution.py

```

1 from pathlib import Path
2 import json
3 R=Path(__file__).resolve().parents[1];S=json.loads((R/'artifacts/summary.json').read_text());a=S['iid']['3']['vs56'];b=S['iid']['4']['vs56'];c=S['iid']['4']['vs78']
4 f=lambda x:f'{x:.2f}'
5 s=r'''# B题几何策略优化题解与计算交接
6
7 本轮完成最坏半径选点、最近保证第二测、剩余覆盖认证、真实单源控制器续跑和光学邻域巡游的实现与对照。独立验证尚不能确认新的稳定提速，推荐默认保留方法78；方法84作为经过全清除检验的实验候选单独提供。论文涵盖问题1至4的推导与实验，并附在线代码和几何证书。
8
9 ## 1 先看最终结果
10
11 时间单位均为秒；每问独立200局。
12
13 |问题|56均时|84均时|节省|95%区间|慢局|
14 |---|---:|---:|---:|---|---:|
15 '''
16 for q,d in [('Q3',a),('Q4',b)]:s+=f"|{q}|{f(d['baseline_mean_s'])}|{f(d['candidate_mean_s'])}|{f(d['saving_s'])}|{f(d['ci95_s'][0])},{f(d['ci95_s'][1])}|{d['slower']}|{d['n']}"
17 s+=f'''
18 推荐默认78的独立总均时为Q3 {f(S['iid']['3']['vs78']['baseline_mean_s'])}秒、Q4 {f(S['iid']['4']['vs78']['baseline_mean_s'])}秒。Q3与默认78逐局相同；其{f(a['saving_s'])}秒收益属于旧提前停车，不能算本轮新增。Q4候选相对78节省{f(c['saving_s'])}秒，但95%区间为[{f(c['ci95_s'][0])},{f(c['ci95_s'][1])}]秒，仍跨零。按冻结规则，候选没有进入默认程序。两问IID和五组压力测试全部清除；未达到新增10%。
19
20 以上表格是本地模拟结果，与官方正式成绩分开解释。Q3、Q4各三次官方正式测试已经完成，案例编码与成绩见论文及`paper/formal-result-claims.json`，六份官方加密日志按原文件名放入提交支撑材料。正式案例不公开干扰源总数，不能据主动退出虚构清除比例。
21 '''
22 s+=r'''
23 ## 2 问题1的完整算法
24
25 示向点  $s$ 、读数  $\theta$  和半角  $\varepsilon$  对应两个前向半平面。令  $u=(\cos\theta,\sin\theta)$ 、 $v=(-\sin\theta,\cos\theta)$ ，约束为

```

26

27  $\$ \$ |v^T(z-s)| \leq \tan \epsilon, u^T(z-s). \$ \$$

28

29 逐条裁剪初始区域，得到凸多边形  $\$P\$$ 。若没有场地先验，先判断半平面交为空、无界、有界或退化；空域报不一致，无界域的直径为无穷。顶点数为  $m$  时，枚举顶点对以  $O(m^2)$  求直径，或采用旋转卡壳。

30

31 不能以直径不超过40米作为保证清除：边长40米的等边三角形需要半径  $40/\sqrt{3}=23.094$  米的圆。正确条件是最小包围圆半径  $\rho(P) \leq 20$  米。其支持点至多三个，计算圆心后检查全部顶点距离。

32

33 本地读数按0.01度量化，实际实现用1.005度半角；物理误差仍为题设的正负1度。

34

35 ## 3 问题2及保证区域的空集边界

36

37 首测归一化为原点及零度。源  $z=r\phi$  满足  $r \leq 1500, |\phi| \leq \epsilon$ 。首次收到信号隐含  $R_s \geq \max(1000, r)$ ，完整保证第二次接收域为

38

39  $\$G=\bigcap_{r \in \{5, 1000\}, \sigma \in \{-1, 1\}} B(r(\cos \epsilon, \sigma \sin \epsilon), 1000). \$ \$$

40

41 分别用径向凸性和大于1000米后约束的单调性，便可把所有位置约束化到四个端点圆盘。测点  $(843.1178, \pm 545.4003)$  的最坏后验半径上界为55.739米，物理误差加量化模型的全域下界为55.6528米；精确有理数证书本轮已重跑。

42

43 因此，对未被场地裁剪的完整首测扇区，要求“一次补测后无论读数如何都保证半径不超过20米或35米”没有可行点。这不是局部优化器没找到。边界裁剪后的  $\$P\$$  另当别论，仍可以搜索最近保证区域；本轮没有证明所有裁剪实例均为空。

44

45 在线新局部候选额外使用  $\max_{z \in P} |q-z| \leq 950.7$  米的充分接收门槛，保持经验点的余量。这比完整保证接收域更小；应避免将它称为全域最优搜索。

46

47 ## 4 算法1与算法4的实际实现

48

49 仅对多边形顶点的无噪声方位切角锥，不能认证最坏剩余半径。实现首先构造所有可能示向区间，并包含正负误差，再分割示向区间。如果一段中心为  $\theta_j$ 、半宽为  $\delta_j$ ，则

50

51  $\$ \$ \bigcup_{|\theta-\theta_j| \leq \delta_j} W_\epsilon(q, \theta) \subseteq W_\epsilon(q, \theta_j). \$ \$$

52

53 对每个加宽角锥与  $\$P\$$  的交求最小包围圆，最大半径是连续读数目标的外包。对最大上界区间递归二分。候选位置使用单纯形优化，不再按粒子分箱和八方向加密。

54

55 只保留一个新点，要求其半径改善及行走门槛通过。门槛把直径减少量除以5换成秒数，只是启发式，不能承诺后续实际时间减少。开发40局中，该方法Q3反而平均多用165.79秒，7胜33负。该结果说明半径几何仍需与完成时间衔接。

56

57 方法82进一步用原单源控制器从每个候选动作继续执行，保留测量更新、近距清除、剩余动作预算和真正的光学覆盖。最多取4个内点假设及2个可行边界点，并分别取两端误差。平均时间至少减少5秒且有限场景最差退步不超过5秒才替换。实际执行34624次续跑、采纳22次动作，Q3开发均时仅减少1.10秒。

58

59 “真实续跑”指调用同一控制器至清除成功。未来观测使用每个场景固定的正负1度误差，不执行0.01度量化，未复用评测的位置相关误差场；只在同位置重放真实历史读数。这是单源控制器的有限场景续跑，未模拟所有其它源的未联合调度；有限场景的最坏值不等于连续源状态的最坏值。代码没有接入完整POMCPOW，也没有把新候选回灌旧的八方向粒子分。

60

61 ## 5 算法3的剩余覆盖证明

62

63 对Q4，状态必须含位置、半径、朝向及类型。未发现频道的状态集按  $\Omega_k \leftarrow \Omega_k \setminus \min C(q)$  更新，不能只裁位置圆盘。若只有一次正常示向，要求换一个新点后对所有合法朝向仍必收，通常只

能得到原点本身：对每个可能源位置，新的接收向量必须是旧向量的非负倍数；两条不同源视线同时成立立即退化。

64

65 实现使用比显式四维状态更容易复核的覆盖不变量。令  $H$  为已对所有未发现频道完整扫描的点， $F$  为未来点；对全部  $z \in \Omega$  要求

66

67 
$$z \in \operatorname{conv}\{q \in H \cup F : \|q - z\| \leq 1000\}.$$

68

69 任何朝向均至少有一个近邻点在源的正向闭半平面。如果该点全在历史  $H$  中，源此前就会被发现，故未发现状态必须由未来  $F$  覆盖。这同时处理所有位置及朝向，不靠粒子接收概率。

70

71 空间认证从场地外包正方形开始二分，若一个矩形的所有角点都在公共近邻点的凸包内，且这些点到所有角点均不超过999.999米，就通过。距离平方和叉积以向外舍入区间认证；任一不确定判断、递归上限或预算耗尽都拒绝本次修改。

72

73 初始21点使用4552个叶单元通过认证。一个实际接受的点替换用4564个叶单元通过，并由独立Python Fraction核验全部单元和空间分区无缺口。

74

75 每个待走发现点尝试向当前点连续移动，只有全域证书仍成立才接受。另实现顺路已停车位扫描后删点的版本；开发集没有成功删掉整点。负响应只把一个点从  $F$  移进  $H$ ，两者并集不变，所以它不会自动把其它必需静态点变冗余。需要新增几何信息或额外测点支撑，不能凭“已经扫过附近”削减全清除保证。

76

77 固定点数下界的推导为：除有限条站点连线外，全朝向接收需要至少三个距离1000米内的测点，故  $\sum_i A_i \geq \pi 1800^2$ 。对边界径向向外发射的源，测点可接收边界弧半角  $\delta_i$  满足  $\sum_i \delta_i \geq \pi$ 。外部测点的面积损失  $d_i = 1 - A_i / (\pi 1000^2)$  经有理数区间逐段认证满足  $d_i \geq 1.38 \delta_i$ ，内部测点的边界弧为零。因此  $N \geq (1800/1000)^2 + 1.38 \pi > 14$ ，所以  $N \geq 15$ 。详细弧公式、积分上界和端点处理已列入论文。

78

79 本轮固定通用布点下界15已经重新核验，上界21保留；这个静态下界不是局内必能删除6点的承诺。

80

81 ## 6 算法5的清除邻域与巡游

82

83 覆盖子区域  $P_i$  的合法清除位置是

84

85 
$$\mathcal{C}(P_i) = \bigcap_{v \in V(P_i)} B(v, 20).$$

86

87 若子区域包围圆为  $(c_i, r_i)$ ，则  $B(c_i, 20 - r_i)$  是一个安全内域。不能将原20米覆盖圆直接当成“只要触碰就能清除”的TSPN邻域，否则两端源与机器人可能相距40米。

88

89 实现对各子区域保持覆盖证书，交替优化清除点和2-opt顺序，同时尝试可变条带覆盖。只有完整扫描长度和有限假设的平均首次命中时间都下降才接受。多个已保证近清源也可联合调整访问坐标。它没有求得全部顺序的CETSP全局最优解。

90

91 开发集Q4中，原覆盖邻域巡游多用5.21秒，可变覆盖版本多用4.09秒。因此这些候选保留代码和消融，未启用到默认策略。

92

93 ## 7 开发消融与压力结果

94

95 本地普通场景每局有10至16个源。位置在圆内按面积均匀提议，但拒绝源距小于40米的样本；Q4定向源数在1至总数减1之间生成。误差为坐标和种子决定的固定场，随后量化读数。局间独立，局内位置有排斥约束，不能把此均值外推为任意分布的官方均值。

96

97 历史CSV在当前平台同种子标签下不逐行相同。本轮重新编译旧的未修改源码，80行开发基线与新版本的方法56完全对齐；所有比较使用本轮同环境重跑的基线，历史数据不混入。

98

99

100 |方法|Q3平均节省/秒|Q4平均节省/秒|用途|

101 |---|---:|---:|---|

```

102 '''
103 for m in [79,80,81,82,83,85,86,84]:
104     d=S['development'][str(m)];s+=f"|{m}|{f(d['3']['saving_s'])}|{f(d['4']['saving_s'])}|{'冻结独
        立验证候选' if m==84 else '开发消融'}|\n"
105 s+='\n|压力组, 每问40局|Q3平均节省/秒|Q4平均节省/秒|清除|\n|---|---:|---:|---|\n'
106 for i,name in enumerate(['边界外向','共线端点','近重合','相关误差','分区端点误差'],1):
107     d=S['stress'][str(i)];s+=f"|{name}|{f(d['3']['saving_s'])}|{f(d['4']['saving_s'])}|两问均40
        /40|\n"
108 s+=r'''
109 ## 8 运行与复现
110
111 默认控制器先将未知频道扫描点与已知源下一动作联合排路, 只执行首个任务后重算。有限动作树深度2、节点预
        算6000, 每源最多6次局部动作, Q3/Q4假设上限32/63。Q3最多4次机会补测, 局部阶段后最多3轮中心试探
        再进入28米格光学; Q4直接进入自适应条带光学。路由节点不超过12时动态规划精确求解, 否则多起点、2-
        opt和重定位。发现16个源即可停止余下发现, 未达16则须完成剩余证书扫描。每个光学子区域都需实际发出
        清除指令。方法78只把保证成功的清除沿原路径提前, 方法84额外允许Q4发现点通过连续证书后移动。
112
113 全部操作在解压后的项目根目录执行, 目录可以改名。默认方法78, 实验候选84使用 `geometric`, 对照56使用
        `previous`。新会话从原点开始, 不能用提前停车适配器中途接管已经移动的机器人。
114
115 ```bash
116 python3 run_all.py
117 python3 run_all.py --full
118 python3 src/bridge.py --problem 4 --team-id 实际队号 --strategy joint --planner-depth 2
119 python3 src/bridge.py --problem 4 --team-id 实际队号 --strategy geometric --planner-depth 2
120 ```
121
122 ```--full```重跑固定独立与压力样本, 不重新选参; 正式测试须先在官方模拟器界面由队员启动并等待接口就绪。脚
        本不会自动启动正式测试。`vendor/mocksim` 是所给压缩包内的非官方公开协议实现, 不能将其本地HTTP
        测试称为官方验证。
123
124 主要路径:
125
126 - `src/robust_geometry.hpp`: 连续读数半径外包和局部测点候选。
127 - `src/residual_cover.hpp`: 向外舍入空间认证和发现点替换。
128 - `src/rollout_sensor.hpp` 与 `policy.hpp`: 真实单源控制器续跑。
129 - `src/neighborhood_tour.hpp`: 安全清除邻域与光学序列优化。
130 - `artifacts/summary.json`: 逐局配对与分组统计。
131 - `validation_frozen.json`: 独立集运行前的冻结配置与哈希。
132 - `artifacts/promotion-decision.json`: 置信区间跨零、不推广候选的决定。
133 - `paper/main.tex`: 完整论文, 含程序附录。
134
135 ## 9 文献核对与不能直接移植的结论
136
137 谨慎贪心原文采用高斯状态与EKF; Bayram的定位标准是GDOP, 并带双准则近似损失。这些论文支持时间意识和先
        布点后巡游的结构, 不提供本题20米角锥半径保证。Sunberg的退化结论针对朴素渐进加宽, POMCPOW恰是其
        修正算法; 不能把它写成POMCPOW普遍退化。割草路径上自动覆盖与本题停车光学指令也有区别。
138
139 核验的一手来源:
140
141 '''
142 for x in json.loads((R/'evidence/literature.json').read_text()):s+=f"- {x['authors']} ({x['year']
        }) [{x['title']}]({x['url']})。\\n"
143 (R/'B题几何策略优化_题解.md').write_text(s)
144 print('solution markdown created')

```

## supplement/count\_time/src/analyze\_count\_time.py

```

1 """Fit only original IID data; verify the frozen model on new seed data."""
2 import csv,hashlib,json,math,sys,time
3 from pathlib import Path
4 import numpy as np
5 from scipy.stats import t,pearsonr,spearmanr
6 R=Path(__file__).resolve().parents[1];P=R.parents[1]
7 def read(p):return list(csv.DictReader(p.open()))
8 def col(a,k):return np.array([float(z[k]) for z in a])
9 def design(n,name):
10     if name=='proportional':return n[:,None]
11     if name=='affine':return np.column_stack([np.ones(len(n)),n])
12     if name=='quadratic':return np.column_stack([np.ones(len(n)),n,n*n])
13     return np.column_stack([np.ones(len(n)),n,n=16])
14 def fit(a,name):
15     n=col(a,'n');y=col(a,'time_s');x=design(n,name);b=np.linalg.lstsq(x,y,rcond=None)[0];res=y-x@b
16     ;inv=np.linalg.inv(x.T@x);hat=np.sum((x@inv)*x,axis=1);w=res/(1-hat)
17     cov=inv@((x*w[:,None]).T@((x*w[:,None]))@inv;crit=t.ppf(.975,len(n)-x.shape[1]);ci=np.stack([b-
18     crit*np.sqrt(np.diag(cov)),b+crit*np.sqrt(np.diag(cov))],axis=1)
19     return {'model':name,'n_cases':len(n),'coefficients':b.tolist(),'HC3_covariance':cov.tolist()
20     , 'HC3_t95_intervals':ci.tolist(),'total_R2':float(1-np.sum(res**2)/np.sum((y-y.mean())**2)
21     ), 'avg_R2':float(1-np.sum((res/n)**2)/np.sum((y/n-(y/n).mean())**2)), 'LOOCV_avg_RMSE_s':
22     float(np.sqrt(np.mean((res/(1-hat)/n)**2))), 'LOOCV_total_RMSE_s':float(np.sqrt(np.mean((
23     res/(1-hat)**2))))}
24 def assess(f,a):
25     n=col(a,'n');y=col(a,'time_s');x=design(n,f['model']);pred=x@np.array(f['coefficients']);e=y-
26     pred
27     return {'n_cases':len(n),'mean_avg_error_s':float(np.mean(e/n)), 'avg_RMSE_s':float(np.sqrt(np.
28     mean((e/n)**2))), 'total_RMSE_s':float(np.sqrt(np.mean(e**2))), 'avg_R2':float(1-np.sum((e/n
29     )**2)/np.sum((y/n-(y/n).mean())**2)), 'total_R2':float(1-np.sum(e**2)/np.sum((y-y.mean()
30     )**2))}
31 def groups(a):
32     out=[]
33     for n in range(10,17):
34         g=[z for z in a if int(z['n'])==n];y=col(g,'avg_s');crit=t.ppf(.975,len(g)-1);margin=crit*y.
35         std(ddof=1)/np.sqrt(len(g))
36         out.append({'N':n,'cases':len(g),'total_mean_s':float(col(g,'time_s').mean()), 'avg_mean_s':
37         float(y.mean()), 'avg_mean_t95_s':[float(y.mean()-margin),float(y.mean()+margin)], 'avg_sd_s
38         ':float(y.std(ddof=1)), 'discovery_sites_mean':float(col(g,'scans').mean()), '
39         discovery_measure_mean':float(col(g,'discovery_measures').mean()), '
40         directional_sources_mean':float(col(g,'directional_n').mean())})
41     return out
42 begin=time.time();old=read(P/'results/iid78.csv');out={'method':78,'unit':'seconds; per-source
43     average is total/N','training_role':'Post-hoc explanatory fit on original IID; not a new
44     independent confirmation.','validation_role':'Fresh seeds frozen before run, never
45     refitted.','Q':{}}
46 max_identity=0;matching=0
47 for q in [3,4]:
48     a=read(R/'results'/f'training_replay_q{q}.csv');b=read(R/'results'/f'validation_q{q}.csv');
49     reference={int(z['seed']):z for z in old if int(z['problem'])==q}
50     assert len(a)==len(b)==200 and not {z['seed'] for z in a}&{z['seed'] for z in b}
51     for z in a:
52         for k in ['n','time_s','avg_s','move_m','measures','switches','miss']:assert abs(float(z[k])-
53         float(reference[int(z['seed'])][k]))<.0001
54     matching+=1
55     for z in a+b:
56         N=int(z['n']);m=int(z['scans']);assert int(z['discovery_measures'])==m*(20-N)+int(z['Jsum'])

```

```

37 exact=float(z['move_m'])/5+6*(m*(20-N)+int(z['Jsum']))-int(z['discovery_same_channel_savings
    '])+5*int(z['local_measures'])+int(z['local_switches'])+3*int(z['miss'])+5*N
38 max_identity=max(max_identity,abs(exact-float(z['time_s'])));assert max_identity<.001
39 assert (m==(7 if q==3 else 21)) if N<16 else m<= (7 if q==3 else 21)
40 models={name:fit(a,name) for name in ['proportional','affine','quadratic','affine_step16']}
41 for name,f in models.items():f['fresh_validation']=assess(f,b)
42 c=models['affine_step16'];predictions=[]
43 x=design(col(a,'n'),'affine_step16')
44 components={walking':col(a,'move_m')/5,'discovery_scan':5*col(a,'discovery_measures')+col(a,'
    discovery_switches'),'known_sensing':5*col(a,'local_measures')+col(a,'local_switches'),'
    failed_clear':3*col(a,'miss'),'successful_clear':5*col(a,'n')}]
45 decomposition={k:np.linalg.lstsq(x,y,rcond=None)[0].tolist() for k,y in components.items()}
46 assert np.max(np.abs(np.sum(list(decomposition.values()),axis=0)-np.array(c['coefficients'])))
    <.001
47 # Independently sum per-row HC3 sandwich contributions.
48 e=col(a,'time_s')-x@np.array(c['coefficients']);inv=np.linalg.inv(x.T@x);direct=np.zeros((3,3)
    )
49 for row,residual in zip(x,e):
50     v=inv@row;h=float(row@v);direct+=np.outer(v,v)*(residual/(1-h))*2
51 assert np.allclose(direct,c['HC3_covariance'],rtol=1e-10,atol=1e-10)
52 c['accounting_coefficient_decomposition']=decomposition
53 for n in range(10,17):
54     g=np.array([1/n,1,(1/n if n==16 else 0)]);mu=float(g@np.array(c['coefficients']));se=math.
        sqrt(float(g@np.array(c['HC3_covariance'])@g));margin=t.ppf(.975,197)*se
55     predictions.append({'N':n,'predicted_avg_s':mu,'conditional_mean_HC3_t95_s':[mu-margin,mu+
        margin]})
56 corr={}
57 for split,rows in [('training',a),('validation',b)]:
58     n=col(rows,'n');y=col(rows,'time_s');corr[split]={ 'pearson_N_total':float(pearsonr(n,y).
        statistic),'pearson_N_avg':float(pearsonr(n,y/n).statistic),'spearman_N_avg':float(
        spearmanr(n,y/n).statistic)}
59 out['Q'][str(q)]={'models':models,'selected':'affine_step16','training_groups':groups(a),'
    validation_groups':groups(b),'predictions':predictions,'correlations':corr}
60 out['checks']={'original_rows_matched':matching,'identity_checked_cases':800,'
    max_identity_residual_s':max_identity,'all_cleared_and_counter_checks':'C++ producer
    rejects any violation','validation_seed_disjoint':True};out['analysis_runtime_s']=time.
    time()-begin
61 (R/'artifacts/count_time_results.json').write_text(json.dumps(out,ensure_ascii=False,indent=2))
62 print(json.dumps({q:{'coefficients':d['models']['affine_step16']['coefficients'],'CI':d['models
    ']['affine_step16']['HC3_t95_intervals'],'validation':d['models']['affine_step16']['
    fresh_validation'],'group_scans':[(z['N'],z['discovery_sites_mean']) for z in d['
    validation_groups']] for q,d in out['Q'].items()},ensure_ascii=False,indent=2))

```

## supplement/count\_time/src/build\_count\_report.py

```

1 import json,sys,hashlib
2 from pathlib import Path
3 import numpy as np
4 import matplotlib
5 matplotlib.use('Agg')
6 import matplotlib.pyplot as plt
7 from scipy.stats import t
8 R=Path(__file__).resolve().parents[1];P=R.parents[1]
9 sys.path.insert(0,str(P/'src'))
10 from figqa import assert_no_overlap
11 D=json.loads((R/'artifacts/count_time_results.json').read_text());Q=D['Q']

```

```

12 plt.rcParams.update({'font.family': 'Songti SC', 'font.size': 11, 'axes.unicode_minus': False, 'axes.
    spines.top': False, 'axes.spines.right': False, 'pdf.fonttype': 42})
13 fig, axes = plt.subplots(1, 2, figsize=(10, 4.4)); handles = []
14 for ax, q in zip(axes, ['3', '4']):
15     d = Q[q]; f = d['models']['affine_step16']; beta = np.array(f['coefficients']); cov = np.array(f['
        HC3_covariance'])
16     x = np.linspace(10, 15.99, 150); g = np.column_stack([1/x, np.ones(len(x)), np.zeros(len(x))]); y = g@beta
        ; margin = t.ppf(.975, 197) * np.sqrt(np.einsum('ij, jk, ik -> i', g, cov, g))
17     line, = ax.plot(x, y, color='163e5d', lw=1.6, label='冻结公式'); ax.fill_between(x, y-margin, y+margin
        , color='163e5d', alpha=.13)
18     point = d['predictions'][-1]; ax.plot([16], [point['predicted_avg_s']], marker='s', color='163e5d',
        ms=5)
19     a = d['training_groups']; old = ax.scatter([z['N']-.05 for z in a], [z['avg_mean_s'] for z in a],
        color='8c979e', marker='x', s=30, label='原样本均值')
20     b = d['validation_groups']; new = ax.errorbar([z['N']+.05 for z in b], [z['avg_mean_s'] for z in b],
        yerr=[z['avg_mean_s']-z['avg_mean_t95_s'][0] for z in b], [z['avg_mean_t95_s'][1]-z['
        avg_mean_s'] for z in b], fmt='o', ms=4, capsize=3, color='c06927', label='新样本均值及95%区间
        ')
21     ax.set(xlabel='信号源数量 N', ylabel='单源平均清除时长 (秒)', title='Q3: 全向源' if q=='3' else '
        Q4: 全向与定向混合', xticks=range(10, 17), xlim=(9.7, 16.3)); ax.grid(alpha=.16)
22     handles = [line, old, new]
23     fig.legend(handles=handles, loc='lower center', ncol=3, bbox_to_anchor=(.5, .015), frameon=False,
        fontsize=10)
24     fig.tight_layout(rect=(0, .09, 1, 1), pad=1.5); assert_no_overlap(fig)
25     fig.savefig(R/'figures/count_time_relation.pdf'); fig.savefig(R/'figures/count_time_relation.png
        ', dpi=200); plt.close(fig)
26     (R/'artifacts/figure_qa.json').write_text(json.dumps({'status': 'PASS', 'producer': 'src/
        build_count_report.py', 'gate': 'assert_no_overlap', 'figure': 'figures/count_time_relation.
        pdf'}, indent=2))
27     f3 = Q['3']['models']['affine_step16']; f4 = Q['4']['models']['affine_step16']; a3, b3, c3 = f3['
        coefficients']; a4, b4, c4 = f4['coefficients']
28     s = r'''# 信号源数量与平均清除时长的关系
29
30 本节以默认方法78为对象，定义总源数为 $N$、全部清除的总虚拟时长为 $T$、单源平均时长为 $\tau=T/N$。结
        论限于本地场景分布及题设的 $10 \le N \le 16$。研究的是条件均值，源的位置、接收半径、方向及观测误差
        仍影响每一局的实际时间。
31
32 ## 1 严格的计时关系
33
34 令 $L$ 为总行走距离（米），$S$ 为换频次数，$M$ 为测向次数，$F$ 为失败清除次数。全部清除时，成功清除
        次数等于 $N$，由题设费用得到
35
36 $T = \frac{L + S + 5M + 3F + 5N}{5 + \frac{L + S + 5M + 3F}{N}}$
37
38 因此，源数量本身不唯一确定时长；需要考察上述各项如何随 $N$ 变化。
39
40 对默认控制器，记实际完成的发现点数为 $m$，源 $k$ 首次被发现时的发现点序号为 $J_k$。未占用的 $20-N$
        个频道必须在每个发现点检测；有源频道仅检测至首次发现。因此严格有
41
42 $M - \{\mathrm{disc}\} = m(20-N) + \sum_{k=1}^N J_k$
43
44 每个发现点按频道顺序扫描未知频道，只有首次检测可能免换频。令 $d = M - \{\mathrm{disc}\} - S - \{\mathrm{disc}\}$，则 $0 \le d \le m$。将其它测向和换频数记为 $M_{\mathrm{loc}}, S_{\mathrm{loc}}$，得到
45
46 $\boxed{\tau = \frac{L}{5N} + 6m \left( \frac{20}{N} - 1 \right) + 6 \overline{J} + \frac{5M_{\mathrm{loc}}}{N} + S_{\mathrm{loc}} + 3F - d} \frac{1}{N} + 5, \quad \overline{J} = \frac{1}{N} \sum_{k=1}^N J_k$
47

```

48 这表明源增多会减少空频道的重复扫描，并使发现路程与共同开销分摊到更多源上。对  $N < 16$ ，Q3的  $m=7$ 、Q4的  $m=21$ ；到  $N=16$  时，已知源数上界允许停止余下发现， $m$  可以下降。

49

50 ## 2 可用于论文的均值模型

51

52 在固定策略与同一场景分布下，将总时长在有限数量范围内作仿射近似，并对16源的停止规则单设指示项：

53

54 
$$\mathbb{E}[T \mid N] \approx A + BN + C \mathbf{1}_{\{N=16\}}.$$

55

56 由于给定  $N$  后除数固定，严格有  $\mathbb{E}[T/N \mid N] = \mathbb{E}[T \mid N]/N$ ，故得到

57

58 
$$\boxed{\mu(N)} := \mathbb{E}[\tau \mid N] \approx B + \frac{A}{N} + \frac{C}{N} \mathbf{1}_{\{N=16\}}.$$

59

60 这一低维模型对应共同成本分摊、净增量成本及上界停止修正。 $A, B, C$  为经验系数；它不是每个源具有固定独立定位成本的定理， $B$  也不是不可为负的单次服务费用。

61

62 先用原独立集每问200局作事后关系建模，随后冻结公式，并在新种子2710001至2710200（每问另200局）验证，不用新样本重估参数。

63

64 '''

65 s+= '按总时长作最小二乘估计，单位为秒，得到\n\n'

66 for q,a,b,c in [( '3',a3,b3,c3), ( '4',a4,b4,c4)]:

67 s+= r'\$\boxed{\widehat{\mu}}\_{{}^q} + (N) = {}^f\{b:.2f\} + r + \frac{{}^f\{a:.2f\} + r}{N} - \frac{{}^f\{-c:.2f\} + r}{N} \mathbf{1}\_{\{N=16\}}\$'+ '\n\n'

68 s+= r'''当  $N=10, \ldots, 14$ ，模型中相邻数量的差为

69

70 
$$\widehat{\mu}(N+1) - \widehat{\mu}(N) = -\frac{\widehat{A}}{N(N+1)}.$$

71

72 两问估计的  $A$  均显著为正，因此该区间内预测的每源均时随源数增加而下降；15到16的变化另包含  $\widehat{C}/16$ 。这不是对所有固定空间布局的逐局单调性保证。

73

74 ## 3 数值可信度与独立检验

75

76 系数区间使用HC3异方差稳健协方差与Student-t近似；均值带表示条件均值的不确定性，不是单局预测区间。HC3采用  $\widehat{V} = (X^{\top} X)^{-1} \sum_i [x_{ix_i}^{\top} e_i^2 / (1 - h_{ii})^2] (X^{\top} X)^{-1}$ ，见 [statsmodels官方实现说明]([https://www.statsmodels.org/dev/generated/statsmodels.regression.linear\\_model.OLSResults.html](https://www.statsmodels.org/dev/generated/statsmodels.regression.linear_model.OLSResults.html))。

77

78 |问题|A及95%区间|B及95%区间|C及95%区间|

79 |---|---|---|---|

80 '''

81 for q,f in [( 'Q3',f3), ( 'Q4',f4)]: s+= '|'+q+'|'+ '|'.join(f'{v:.2f} [{ci[0]:.2f}, {ci[1]:.2f}]' for v,ci in zip(f['coefficients'],f['HC3\_t95\_intervals'])))+'|\n'

82 s+= '\n|冻结新样本验证|Q3|Q4|\n|---|---:|---:|\n'

83 for label,key in [( '单源时长RMSE/秒', 'avg\_RMSE\_s'), ( '单源时长R²', 'avg\_R2'), ( '总时长RMSE/秒', 'total\_RMSE\_s'), ( '总时长R²', 'total\_R2')]: s+= f'|{label}|{f3[ 'fresh\_validation' ][key]:.3f}|{f4[ 'fresh\_validation' ][key]:.3f}|\n'

84 s+= '\n总时长的解释力低于每源均时，说明空间布局、方向和观测误差仍有显著作用，不能由源数精确预测单局总耗时。 \n\n|源数|Q3预测每源/秒|Q3新样本均值/秒|Q4预测每源/秒|Q4新样本均值/秒|\n|---|---:|---:|---:|---:|\n'

85 for i in range(7): s+= f'|{i+10}|{Q[ '3' ][ 'predictions' ][i][ 'predicted\_avg\_s' ]:.2f}|{Q[ '3' ][ 'validation\_groups' ][i][ 'avg\_mean\_s' ]:.2f}|{Q[ '4' ][ 'predictions' ][i][ 'predicted\_avg\_s' ]:.2f}|{Q[ '4' ][ 'validation\_groups' ][i][ 'avg\_mean\_s' ]:.2f}|\n'

86 s+= '\n|源数量与每源平均时长|(figures/count\_time\_relation.png)\n\n阴影为拟合均值的HC3近似95%区间，误差棒为新样本分组均值的t区间；16处方点为包含停止修正项的预测值。 \n\n'

87 s+= f"原400行默认策略记录逐行复现一致；800个剖析案例均通过计时和发现次数恒等式，最大计时残差{D[ 'checks' ][ 'max\_identity\_residual\_s' ]:.8f}秒。新样本中16源的平均发现点数为Q3 {Q[ '3' ][ 'validation\_groups' ][-1][ 'discovery\_sites\_mean' ]:.3f}、Q4 {Q[ '4' ][ 'validation\_groups' ]

```

    '][-1]['discovery_sites_mean']:.3f}; 其余数量分别固定为7和21。\\n\\n"
88 s+=r'## 4 直接写入论文的文字
89
90 “在本题固定20频道及既定发现覆盖策略下，未占用频道需要重复检测，有源频道在首次发现后退出发现扫描。由
    此可得发现测向次数恒等式  $M_{\mathrm{disc}}=m(20-N)+\sum_{k=1}^N J_k$ 。结合总时间计费规则，建
    立条件均值模型  $E[T/N\mid N]\approx B+A/N+C\mathbf{f}_1\backslash\{N=16\}/N$ 。两问的系数估计均
    表现为  $A>0$ 、 $C<0$ ，对应共同开销分摊与达到已知源数上界后的提前停止。冻结新样本检验支持该有限范
    围内的平均下降趋势。该关系描述本地场景分布下的条件均值，不意味着所有布局逐局单调，也不应外推到题
    设数量范围以外。”
91
92 ## 5 复现与适用范围
93
94 - 方法78，题设源数10至16；Q3全向，Q4全向与定向混合。
95 - 本地普通场景使用40米位置排斥采样、均匀提议半径和朝向、固定位置误差场。结论不是官方成绩规律。
96 - 原200局用于事后建模，新200局用于冻结验证；没有把新验证结果并回系数估计。
97 - 代码和原始逐局记录见本目录 `src/`、`results/`，统计字段见 `artifacts/count_time_results.json`。
98 - 先在本目录编译 `clang++ -O2 -std=c++17 src/count_profile.cpp -o count_profile`，再运行 `
    python3 src/run_profiles.py`、`python3 src/analyze_count_time.py`、`python3 src/
    build_count_report.py`。重跑会覆盖本目录生成结果，冻结原始交付不变。
99 '''
100 extra='## 4a 净增量系数的计费分解\\n\\n下表将同一线性拟合的B系数按计时项目分解；各项相加等于总B，这是
    计费恒等分解，不是因果效应估计。\\n\\n项目|Q3的B分量/秒|Q4的B分量/秒|\\n|---|---|---|\\n'
101 for k,name in [('walking','行走'),('discovery_scan','发现扫描'),('known_sensing','已知源补测')
    ,('failed_clear','失败清除'),('successful_clear','成功清除')]:
102     extra+=f'|{name}|{f3[\"accounting_coefficient_decomposition\"][k][1]:.2f]}{f4[\"
        accounting_coefficient_decomposition\"][k][1]:.2f]}\\n'
103 extra+='\\nQ4的负B主要对应发现扫描减少抵消新增行走与定位费用。不能把负B解释成负的单源操作费用。16源的
    验证均值仍与公式有明显差异，模型用于平均趋势与粗略估计，不替代逐局仿真。\\n\\n'
104 s=s.replace('## 4 直接写入论文的文字',extra+'## 4 直接写入论文的文字')
105 (R/'信号源数量与平均时长_论文补充.md').write_text(s)
106 print('Scientific figure and paper-ready note generated.')

```

## supplement/count\_time/src/count\_profile.cpp

```

1 #include "../././src/simulator.hpp"
2 #include "../././src/joint_config.hpp"
3 #include "../././src/shortcut_sensor.hpp"
4 #include <array>
5 #include <chrono>
6 #include <iostream>
7 class CountSensor:public Sensor{
8     Sim& sim;std::array<bool,21>seen{};
9     public:int scans=0,discovery=0,switch_discovery=0,Jsum=0;
10     explicit CountSensor(Sim&s):sim(s){}
11     void phase(int p)override{if(p==0)++scans;}
12     Reading measure(Pt q,int k)override{
13         bool unknown=!seen[k];if(unknown){++discovery;switch_discovery+=(sim.channel!=k);}
14         auto out=sim.measure(q,k);
15         if(unknown&&out.kind>0){seen[k]=true;Jsum+=scans;}
16         return out;
17     }
18     bool clear(Pt q,int k)override{return sim.clear(q,k);}
19     void audit(int k,const Poly&p)override{sim.audit(k,p);}
20 };
21 int main(int argc,char**argv){
22     if(argc!=5)return 2;int problem=atoi(argv[2]),start=atoi(argv[3]),count=atoi(argv[4]);
23     if((problem!=3&&problem!=4)||start<1||count<1)return 2;

```

```

24 std::ofstream f(argv[1]);if(!f)return 3;
25 f<<"problem,seed,method,n,directional_n,time_s,avg_s,move_m,measures,switches,miss,scans,
    discovery_measures,discovery_switches,Jsum,local_measures,local_switches,
    discovery_same_channel_savings,cpu_s\n";
26 for(int seed=start;seed<start+count;++seed){
27     auto truth=make_case(problem,seed);int n=truth.size(),nd=0;for(auto j:truth)nd+=j.directional
    ;
28     Sim sim(truth,seed,0);CountSensor counted(sim);auto opt=joint_configuration(78,problem);
    ShortcutSensor sensor(counted,true);Policy policy(sensor,problem,opt);
29     auto begin=std::chrono::steady_clock::now();policy.run();double cpu=std::chrono::duration<
    double>(std::chrono::steady_clock::now()-begin).count();
30     int nominal=problem==3?7:21,m=counted.scans,d=counted.discovery-counted.switch_discovery;
31     if(sim.success!=n||counted.discovery!=m*(20-n)+counted.Jsum||d<0||d>m||m>nominal||(n<16&&m!=
        nominal))return 4;
32     int ml=sim.measures-counted.discovery,sl=sim.switches-counted.switch_discovery;
33     double exact=sim.length/5+6*(m*(20-n)+counted.Jsum)-d+5*ml+sl+3*sim.miss+5*n;
34     if(std::abs(exact-sim.time)>.001)return 5;
35     f<<std::setprecision(15)<<problem<<','<<seed<<","78,"<<n<<','<<nd<<','<<sim.time<<','<<sim.
        time/n<<','<<sim.length<<','<<sim.measures<<','<<sim.switches<<','<<sim.miss<<','<<m
        <<','<<counted.discovery<<','<<counted.switch_discovery<<','<<counted.Jsum<<','<<ml<<','<<
        sl<<','<<d<<','<<cpu<<'\n';
36     if((seed-start+1)%25==0){f.flush();std::cout<<problem<< ' '<<(seed-start+1)<<'\n';}
37 }
38 }

```

## supplement/count\_time/src/run\_profiles.py

```

1 import concurrent.futures,hashlib,json,platform,subprocess,time
2 from pathlib import Path
3 R=Path(__file__).resolve().parents[1];P=R.parents[1]
4 sha=lambda p:hashlib.sha256(p.read_bytes()).hexdigest()
5 freeze={'created_unix':time.time(),'method':78,'training_csv':'../results/iid78.csv','
    training_sha256':sha(P/'results/iid78.csv'),'training_seeds':[2610001,2610200],
    new_validation_seeds':[2710001,2710200],'cases_per_question_per_split':200,'model':'Fit
    total seconds  $T=a+b*N+c*I(N=16)$  on original 200 each question only; derive mean per source
    as  $b+a/N+c*I(N=16)/N$ ','alternatives':[' $T=b*N$ ',' $T=a+b*N$ ',' $T=a+b*N+c*N^2$ '],'uncertainty':
    'HC3 covariance and Student-t reference; prediction assessed on new seeds without refitting
    .','no_causal_claim':'Counts vary with scenes; no claim adding a source in every fixed
    scene helps.','measurement_identity':' $M_{disc}=m*(20-N)+sum(J_k)$ ,  $S_{disc}=M_{disc}-d$  with  $0 \leq d \leq m$ ','binary_sha256':sha(R/'count_profile'),'profile_source_sha256':sha(R/'src/
    count_profile.cpp')}
6 (R/'artifacts/design_frozen.json').write_text(json.dumps(freeze,ensure_ascii=False,indent=2))
7 def job(q,split,start):
8     out=R/'results'/f'{split}_q{q}.csv';cmd=[str(R/'count_profile'),str(out),str(q),str(start)
        ,200];begin=time.time();p=subprocess.run(cmd,capture_output=True,text=True)
9     record={'command':cmd,'started_unix':begin,'runtime_s':time.time()-begin,'exit_code':p.
        returncode,'stdout':p.stdout,'stderr':p.stderr,'output':str(out.relative_to(R))}
10    if p.returncode==0:record['sha256']=sha(out)
11    (R/'artifacts'/f'run_{split}_q{q}.json').write_text(json.dumps(record,ensure_ascii=False,
        indent=2));assert p.returncode==0,record
12    print(q,split,'complete',flush=True);return record
13    with concurrent.futures.ThreadPoolExecutor(max_workers=2) as pool:
14        futures=[pool.submit(job,q,s,start) for s,start in [('training_replay',2610001),('validation
            ',2710001)] for q in [3,4]]
15        records=[f.result() for f in futures]
16    (R/'artifacts/execution.json').write_text(json.dumps({'environment':platform.platform(),'
        records':records},ensure_ascii=False,indent=2))

```

## supplement/count\_time/src/write\_count\_section.py

```

1 import json
2 from pathlib import Path
3 R=Path(__file__).resolve().parents[1];D=json.loads((R/'artifacts/count_time_results.json').
    read_text());q3=D['Q']['3']['models']['affine_step16'];q4=D['Q']['4']['models']['
    affine_step16']
4 s='''% 可从现有 paper/main.tex 用 input 引入；图像路径以该文件所在目录为基准。
5 @subsection{信号源数量与单源平均清除时长}
6 设信号源数量为$N$，全部清除总时长为$T$，单源平均时长为$\tau=T/N$。以下研究默认方法78在$10\le
    N\le 16$内的条件均值，不将源数量视为决定单局时长的唯一因素。
7
8 总行走距离、换频数、测向数和失败清除数分别记为$L,S,M,F$，计时规则严格给出
9 @begin{equation}T=L/5+S+5M+3F+5N.\end{equation}
10 记实际发现点数为$m$，第$k$个源首次被发现的发现点序号为$J_k$。空频道在每一点检测，有源频道检测至首次
    发现，故
11 @begin{equation}M_{\mathrm{disc}}=m(20-N)+\sum_{k=1}^N J_k.\end{equation}
12 每一发现点的首次测量至多节省一次换频。记$d=M_{\mathrm{disc}}-S_{\mathrm{disc}}$，则$0\le d\le m$
    ；其它检测和换频数记为$M_{\mathrm{loc}},S_{\mathrm{loc}}$。由此得到
13 @begin{equation}\begin{aligned}
14 \tau=& \frac{L}{5N}+6m\left(\frac{20}{N}-1\right)+6\overline{J+5}\backslash
15 & +\frac{5M_{\mathrm{loc}}}{N}+S_{\mathrm{loc}}+3F-d\backslash N,\quad \overline{J}=\frac{1}{N}\sum_k J_k.
16 \end{aligned}\end{equation}
17 源数增加会减少空频道重复扫描，并分摊共同开销。$N<16$时，Q3与Q4分别需完成7个和21个发现点；$N=16$时，
    已知数量上界允许提前结束发现。
18
19 据此在有限数量范围内建立均值近似
20 @begin{equation}
21 \mathbb{E}[T_{\mathrm{mid}} N]\approx A+BN+C\mathbf{1}_{\{N=16\}},\quad
22 \mu(N):=\mathbb{E}[\tau_{\mathrm{mid}} N]\approx B+\frac{A}{N}+\frac{C}{N}\mathbf{1}_{\{N=16\}}.
23 \end{equation}
24 原独立集每问200局用于事后关系建模；冻结公式后，新增种子2710001至2710200、每问200局作验证，不用新样
    本重估参数。总时长最小二乘拟合得到（单位：秒）
25 @begin{align}
26 \widehat{\mu}_3(N)&=\mathrm{NUM\_B3}+\frac{\mathrm{NUM\_A3}}{N}-\frac{\mathrm{NUM\_C3}}{N}\mathbf{1}_{\{N=16\}},\backslash
27 \widehat{\mu}_4(N)=\mathrm{NUM\_B4}+\frac{\mathrm{NUM\_A4}}{N}-\frac{\mathrm{NUM\_C4}}{N}\mathbf{1}_{\{N=16\}}.
28 \end{align}
29 HC3异方差稳健标准误给出两问$A$的95%近似区间分别为$[2014.89,2466.19]$、$[5889.35,6524.36]$，均为
    正；$C$的相应区间均为负。因而对$N=10,\ldots,14$，模型预测的相邻变化为
30 @begin{equation}\widehat{\mu}(N+1)-\widehat{\mu}(N)=-\frac{\widehat{A}}{N(N+1)}<0.\end{equation}
31 从15到16还包含$\widehat{C}/16$的修正。$B$是减少空频道检测后的净增量系数，不是独立单源的操作费用，因
    此可以为负。
32
33 @begin{table}[H]\centering\small\caption{冻结的新200局验证，逐局计算误差}
34 @begin{tabular}{lrr}\toprule 指标&Q3&Q4\midrule
35 单源均时RMSE/秒&\mathrm{NUM\_RMSE3}&\mathrm{NUM\_RMSE4}\backslash
36 单源均时$R^2$&\mathrm{NUM\_R23}&\mathrm{NUM\_R24}\backslash
37 总时长$R^2$&\mathrm{NUM\_TR23}&\mathrm{NUM\_TR24}\bottomrule
38 \end{tabular}\end{table}
39 原400行结果逐行重现，800个剖析案例均通过计时及发现次数恒等式。公式主要解释每源平均时长的下降趋势；总
    时长解释力较低，空间位置、接收半径、朝向和误差仍造成较大变动。尤其16源分组存在可见预测误差，不
    能把公式当作单局精确预测或所有布局的单调性定理。结论限于含40米位置排斥规则的本地场景分布，不外
    推官方成绩或题设数量范围之外。
40
41 @begin{figure}[H]\centering
42 @includegraphics[width=\linewidth]{../补充分析/信号源数量与平均时长/figures/count_time_relation.
    pdf}

```

```

43 @caption{源数量与每源均时。阴影为拟合均值HC3近似95%区间，误差棒为新样本分组均值区间；16处方点包含
    停止规则修正。}
44 @end{figure}
45 '''
46 for num,f in [(3,q3),(4,q4)]:
47     for k,v in zip(['A','B','C'],f['coefficients']):s=s.replace('NUM_'+k+num,f'{abs(v) if k=="C"
        else v:.2f}')
48     s=s.replace('NUM_RMSE'+num,f'{f['fresh_validation']['avg_RMSE_s']:.2f}').replace('NUM_R2'+num,
        f'{f['fresh_validation']['avg_R2']:.3f}').replace('NUM_TR2'+num,f'{f['fresh_validation']['
        total_R2']:.3f}')
49 s=s.replace('@',chr(92))
50 # In this template Python backslash escapes leave one slash at row endings;
51 # restore the LaTeX row break tokens before a newline or table rule.
52 s=s.replace(chr(92)+'\n',chr(92)*2+'\n').replace(chr(92)+chr(92)+'midrule',chr(92)*2+chr(92)+'
    midrule').replace(chr(92)+chr(92)+'bottomrule',chr(92)*2+chr(92)+'bottomrule')
53 (R/'论文插入段落.tex').write_text(s)

```

## supplement/count\_time\_theory/src/check\_algebra.py

```

1 """Exact arithmetic for the illustrative oracle example; not a policy experiment."""
2 from fractions import Fraction as Q
3 from pathlib import Path
4 import hashlib, json, time
5 R = Path(__file__).resolve().parents[1]
6 start = time.perf_counter()
7 n0, n1, r, v, d = 10, 11, 20, 5, 1800
8 t0 = Q(5 * n0)
9 t1 = Q(5 * n1) + Q(d - r, v)
10 mu0, mu1 = t0 / n0, t1 / n1
11 assert mu1 > mu0
12 assert mu1 - mu0 == (n0 * (t1 - t0) - t0) / (n0 * n1)
13 # The first ten distinct centers may be (0,0),..., (9,0), all within the start disk.
14 # All points are collinear and nonnegative, so the farthest left disk endpoint
15 # proves the exact open path optimum after adding (1800,0).
16 assert max(0, max(range(10)) - r) == 0
17 assert max(0, d - r) == 1780
18
19
20 def statement_sha():
21     """Hash the problem statement if a copy is shipped; otherwise record the absence."""
22     for cand in (R.parents[1] / 'inputs' / 'B题.pdf',
23                 R.parents[2] / 'inputs' / 'B题.pdf'):
24         if cand.is_file():
25             return hashlib.sha256(cand.read_bytes()).hexdigest()
26     return 'not shipped in this package (see the official problem statement)'
27
28
29 out = {
30     'status': 'PASS', 'kind': 'exact illustrative calculation, not simulation',
31     'constants': {'speed_m_s': v, 'clear_radius_m': r, 'successful_clear_s': 5,
32                  'switch_s': 1, 'bearing_s': 5, 'failed_optical_s': 3,
33                  'sources_min': 10, 'sources_max': 16},
34     'example': {'source_counts': [n0, n1], 'distance_m': d, 'added_route_m': d - r,
35                'total_time_s': [float(t0), float(t1)],
36                'mean_time_s': [float(mu0), float(mu1)],
37                'mean_time_exact': [str(mu0), str(mu1)],
38                'marginal_identity_exact': True},

```

```

39     'input_statement_sha256': statement_sha(),
40     'source_sha256': hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
41     'runtime_s': time.perf_counter() - start,
42     'limitations': ['No new cross-policy empirical validation.',
43                    'No independent mathematical review.']}
44 (R / 'artifacts/algebra-check.json').write_text(
45     json.dumps(out, ensure_ascii=False, indent=2) + '\n', encoding='utf-8')
46 print(json.dumps(out, ensure_ascii=False))

```

## tests/probe\_residual.cpp

```

1 #include "../src/residual_cover.hpp"
2 #include "../review/certified_layout21_hex.hpp"
3 #include <iostream>
4 int main(){geo::Poly p;for(auto q:certified_layout21::points)p.push_back({q.x,q.y});auto c=
    residual_cover::certify(p);std::cout<<c.pass<<" "<<c.checks<<" "<<c.leaves.size()<<"\n";}

```

## tests/probe\_robust.cpp

```

1 #include "../src/policy.hpp"
2 #include <iostream>
3 int main(){auto p=geo::observe(geo::outer_disk({0,0},1800),{0,0},0);p=geo::disk_clip(p
    ,{0,0},1500);p=geo::clip(p,{1,0},1500);std::vector<dynamic_selection::Event>h
    {{0,0},2,0}};
4 for(int mode:{1,2}){auto q=robust_geometry::candidate(p,{0,0},{0,0},0,h,3,mode);std::cout<<
    mode<<" valid "<<q.valid<<" q "<<q.x<<" "<<q.y<<" rho "<<q.rho<<"\n";}
5 for(auto q:geo::Poly{{750,300},{750,0},{680,150},{700,150},{650,250}}){auto b=robust_geometry
    ::worst_radius(p,q,512);std::cout<<q.x<<" "<<q.y<<" "<<b.lower<<" "<<b.upper<<"\n";}}

```

## tests/q3\_coverage\_bound.py

```

1 from fractions import Fraction as F
2 import json
3 from pathlib import Path
4 root=Path(__file__).resolve().parents[1]
5 sqrt3lo=F('1.7320508075688772935');sqrt3hi=F('1.7320508075688772936')
6 assert sqrt3lo**2<3<sqrt3hi**2
7 R=F(1800);reception=F('999.6');a=F(1124)
8 # Radius r <= reception is covered by the origin. In every angular sector
9 # |delta| <=30 degrees, squared distance to its ring point is convex in r.
10 # Its maximum on [reception,1800] is at an endpoint.
11 boundary=R*R+a*a-R*a*sqrt3lo
12 inner=reception*reception+a*a-reception*a*sqrt3lo
13 assert max(boundary,inner)<reception**2
14 points=json.loads((root/'results/q3_production_points.json').read_text())
15 assert len(points)==7
16 ideal=[(F(0),F(0)),(a,F(0)),(a/2,a*sqrt3lo/2),(-a/2,a*sqrt3lo/2),(-a,F(0)),(-a/2,-a*sqrt3lo/2),
    (a/2,-a*sqrt3lo/2)]
17 for actual,reference in zip(points,ideal):
18     error=sum((F.from_float(float(x))-y)**2 for x,y in zip(actual,reference))
19     assert error<F('0.0000000001')**2
20 assert F('0.0000000001')+a*(sqrt3hi-sqrt3lo)/2<F('0.0000000001')
21 result={'ring_radius':1124,'certified_ideal_cover_radius':'999.6','boundary_squared_upper':str(
    boundary),'inner_squared_upper':str(inner),'sqrt3_interval':[str(sqrt3lo),str(sqrt3hi)],'
    coordinate_perturbation_allowance_m':'0.4','scope':'ideal sixfold ring + origin;
    implementation floating coordinates must be within 0.4m'}

```

```

22 result['production_binary64_error_bound_m']='0.000000001'
23 (root/'results/q3_coverage_bound.json').write_text(json.dumps(result,indent=2))
24 print('PASS rational radial/angular proof: 1124m seven-point ring covers the 1800m disk within
    999.6m')

```

## tests/replay\_residual\_fraction.py

```

1 """Independent exact-rational replay of accepted spatial witnesses.
2
3 No production interval predicate imported. JSON numbers are converted through
4 binary64, matching the robot's actual coordinates exactly, then to Fraction.
5 """
6 import json,sys,time,hashlib
7 from fractions import Fraction as F
8 from pathlib import Path
9 start=time.monotonic();root=Path(__file__).resolve().parents[1]
10 witness=Path(sys.argv[1]) if len(sys.argv)>1 else root/'artifacts/residual-witness.json'
11 a=json.loads(witness.read_text())
12 P=[tuple(F(float(v)) for v in z) for z in a['sites']]
13 def cross(a,b,c):return (b[0]-a[0])*(c[1]-a[1])-(b[1]-a[1])*(c[0]-a[0])
14 def hull(p):
15     p=sorted(set(p));lo=[];hi=[]
16     for chain,points in [(lo,p),(hi,p[::-1])]:
17         for q in points:
18             while len(chain)>1 and cross(chain[-2],chain[-1],q)<=0:chain.pop()
19             chain.append(q)
20     return lo[::-1]+hi[::-1]
21 for raw in a['boxes']:
22     x0,y0,x1,y1=map(lambda x:F(float(x)),raw);v=[(x0,y0),(x1,y0),(x1,y1),(x0,y1)]
23     safe=[p for p in P if all(sum((p[i]-z[i])**2 for i in [0,1])<=F('999.999'))**2 for z in v)]
24     h=hull(safe);assert len(h)>=3
25     assert all(cross(h[i],h[(i+1)%len(h)],z)>=0 for i in range(len(h)) for z in v)
26 # Rebuild the dyadic partition to prove every disk-intersecting box is present,
27 # instead of trusting a list of successful leaves that might have gaps.
28 leaves={tuple(F(float(x)) for x in b) for b in a['boxes']};seen=0
29 stack=[(F(-1800),F(-1800),F(1800),F(1800))]
30 while stack:
31     b=stack.pop();x0,y0,x1,y1=b
32     if b in leaves:seen+=1;continue
33     x=x0 if x0>0 else x1 if x1<0 else 0;y=y0 if y0>0 else y1 if y1<0 else 0
34     if x*x+y*y>1800**2:continue
35     assert max(x1-x0,y1-y0)>F(1,10000),'missing leaf'
36     if x1-x0>=y1-y0:
37         m=(x0+x1)/2;stack.extend([(x0,y0,m,y1),(m,y0,x1,y1)])
38     else:
39         m=(y0+y1)/2;stack.extend([(x0,y0,x1,m),(x0,m,x1,y1)])
40     assert seen==len(leaves)
41 out={'status':'PASS','arithmetic':'Python Fraction, independent of C++ interval code','sites':
    len(P),'verified_leaf_boxes':seen,'full_disk_partition':True,'witness_sha256':hashlib.
    sha256(witness.read_bytes()).hexdigest(),'runtime_s':time.monotonic()-start}
42 (root/'artifacts/residual-fraction-replay.json').write_text(json.dumps(out,indent=2));print(
    json.dumps(out))

```

## tests/run\_checks.py

```

1 """Compile and run checks; the rational replay must consume this run's witness."""

```

```

2 import hashlib,json,os,shutil,subprocess,sys,tempfile,time
3 from pathlib import Path
4 R=Path(__file__).resolve().parents[1];records=[]
5 cc=os.environ.get('CXX') or shutil.which('clang++') or shutil.which('g++')
6 assert cc,'C++17 compiler required'
7 (R/'bin').mkdir(exist_ok=True);(R/'artifacts').mkdir(exist_ok=True)
8 digest=lambda p:hashlib.sha256(Path(p).read_bytes()).hexdigest()
9 witness=R/'artifacts/residual-witness.json'
10 for name in ['test_geometry_revision','test_joint','test_controller','test_adaptive_optical','
    test_shortcut_controller','test_action_budget','test_scan_fee','test_discovery','
    test_variable_cover']:
11     source=R/'tests'/f'{name}.cpp';binary=R/'bin'/name
12     cmd=[cc,'-O2','-std=c++17',str(source),'-o',str(binary)];begin=time.time();out=subprocess.run(
        cmd,capture_output=True,text=True)
13     if out.returncode:raise RuntimeError(out.stderr)
14     run_cmd=[str(binary)];extra={}
15     if name=='test_geometry_revision':
16         witness.unlink(missing_ok=True)
17         with tempfile.TemporaryDirectory(prefix='geometry-export-') as tmp:
18             bad=subprocess.run([str(binary),str(Path(tmp)/'absent'/witness.json)],cwd=tmp,
                capture_output=True,text=True)
19             assert bad.returncode!=0 and 'PASS' not in bad.stdout
20             run_cmd.append(str(witness));out=subprocess.run(run_cmd,cwd=tmp,capture_output=True,text=
                True)
21             assert out.returncode==0 and witness.is_file() and witness.stat().st_mtime>=begin
22             extra={'source_sha256':digest(source),'binary_sha256':digest(binary),'fresh_witness_sha256':
                digest(witness),'invalid_output_rejected':True,'execution_cwd':'fresh temporary directory
                '}
23     else:out=subprocess.run(run_cmd,cwd=R,capture_output=True,text=True)
24     records.append({'name':name,'compile_command':cmd,'command':run_cmd,'exit_code':out.returncode
        ,'runtime_s':time.time()-begin,'stdout':out.stdout,'stderr':out.stderr,**extra})
25     if out.returncode:raise RuntimeError(records[-1])
26     print(name,'PASS',flush=True)
27 for name in ['replay_residual_fraction.py','test_bridge_deadline.py']:
28     cmd=[sys.executable,str(R/'tests'/name)]
29     if name.startswith('replay'):cmd.append(str(witness))
30     begin=time.time();out=subprocess.run(cmd,cwd=R,capture_output=True,text=True)
31     records.append({'name':name,'command':cmd,'exit_code':out.returncode,'runtime_s':time.time()-
        begin,'stdout':out.stdout,'stderr':out.stderr})
32     if out.returncode:raise RuntimeError(records[-1])
33     print(name,'PASS',flush=True)
34 assert json.loads((R/'artifacts/residual-fraction-replay.json').read_text())['witness_sha256
    ']==records[0]['fresh_witness_sha256']
35 (R/'artifacts/regression-tests.json').write_text(json.dumps({'status':'PASS','records':records
    },ensure_ascii=False,indent=2))

```

## tests/test\_action\_budget.cpp

```

1 #include "../src/simulator.hpp"
2 #include "../src/joint_config.hpp"
3 #include <cassert>
4 #include <iostream>
5 int main(){for(int p:{3,4}){auto o=joint_configuration(39,p);o.dynamic_limit=1;Sim s(make_case(
    p,73),73,0);Policy b(s,p,o);b.run();assert(s.success==s.total());assert(b.dynamic_actions
    <=s.total());}std::cout<<"PASS per-source dynamic budget persists across global replans\n
    "};}

```

## tests/test\_adaptive\_optical.cpp

```

1 #include "../src/adaptive_optical.hpp"
2 #include <cassert>
3 #include <iostream>
4 #include <random>
5 int main(){geo::Poly triangle{{0,0},{1500,-26},{1500,26}};auto q=adaptive_optical::cover(
    triangle,{0,0},0);assert(!q.empty()&&q.size()<108);
6  for(int i=0;i<=150;+i)for(int j=-10;j<=10;+j){geo::Pt z{10.*i,(i/150.)*26*j/10.};double d=1
    e99;for(auto p:q)d=std::min(d,geo::dist(p,z));assert(d<=20);}
7  geo::Pt offset{100,200};geo::Poly rotated;auto u=geo::polar(1,71.3);geo::Pt v{-u.y,u.x};for(
    auto z:triangle)rotated.push_back(offset+u*z.x+v*z.y);auto r=adaptive_optical::cover(
    rotated,offset,71.3);assert(r.size()==q.size());for(size_t i=0;i<q.size();+i)assert(geo::
    dist(r[i],offset+u*q[i].x+v*q[i].y)<1e-6);
8  auto point=adaptive_optical::cover({{42,10}},{0,0},0);assert(point.size()==1&&geo::dist(point
    [0],{42,10})<1e-6);
9  std::mt19937 rng(913);auto U=[&]() {return double(rng())/rng.max();};
10 for(int trial=0;trial<100;+trial){auto source=geo::polar(1800*sqrt(U()),360*U());auto first=
    source+geo::polar(300+900*U(),360*U());double direction=atan2(source.y-first.y,source.x-
    first.x)*180/geo::pi+2*U()-1;
11  auto polygon=geo::observe(geo::outer_disk({0,0},1800),first,direction);polygon=geo::disk_clip
    (polygon,first,1500);auto axis=geo::polar(1,direction);polygon=geo::clip(polygon,axis,geo
    ::dot(axis,first)+1500);
12  for(int k=0;k<trial%4;+k){auto station=source+geo::polar(100+900*U(),360*U());double bearing
    =atan2(source.y-station.y,source.x-station.x)*180/geo::pi+2*U()-1;polygon=geo::observe(
    polygon,station,bearing);}
13  assert(geo::contains(polygon,source));auto centers=adaptive_optical::cover(polygon,first,
    direction);assert(!centers.empty()&&centers.size()<=108);
14  geo::Pt mean;for(auto z:polygon)mean=mean+z;mean=mean/double(polygon.size());
15  for(auto vertex:polygon)for(int k=0;k<=20;+k){auto z=mean*(1-k/20.)+vertex*(k/20.);double
    best=1e99;for(auto c:centers)best=std::min(best,geo::dist(z,c));assert(best<20);}
16 }
17 std::cout<<"PASS adaptive optical: "<<q.size()<<" instead of 108 centers; triangle coverage,
    rotation and singleton\n";
18 }

```

## tests/test\_bearing\_bins.cpp

```

1 #include "../src/dynamic_selection.hpp"
2 #include <cassert>
3 #include <iostream>
4 int main(){
5 #ifndef B_BEARING_BINS
6  std::cerr<<"FAIL bearing prediction bin sensitivity is not configurable\n";return 1;
7 #else
8  using dynamic_selection::bearing_outcome;
9  assert(bearing_outcome(41.9,4)==bearing_outcome(42.1,4));
10 assert(bearing_outcome(41.9,2)!=bearing_outcome(42.1,2));
11 for(int width:{1,2,3,4,6,8,12})for(double a:{-361.,-1.,0.,41.99,180.,359.99,360.,721.}){
12  int b=bearing_outcome(a,width);assert(b>=10&&b<10+360/width);assert(b==bearing_outcome(a+720,
    width));
13 }
14 std::cout<<"PASS bearing bins: angular wrap and configurable finite observation resolution\n";
15 #endif
16 }

```

## tests/test\_belief\_dp.cpp

```

1 #include "../src/belief_dp.hpp"
2 #include <cassert>
3 #include <iostream>
4 #include <random>
5 #include <map>
6 using belief_dp::Model; using belief_dp::Action;
7 double brute(const Model&m, uint64_t mask, int at, int h, bool tuned){
8     double best=m.terminal(mask,at); if(h==0) return best;
9     double mass=0; for(int i=0; i<(int)m.weights.size(); ++i) if(mask>>i&1) mass+=m.weights[i];
10    for(int a=0; a<(int)m.actions.size(); ++a){ auto &x=m.actions[a]; double cost=m.travel[at][a]+(x.
        measure&&!tuned?1:0); std::map<int, uint64_t> groups;
11        for(int i=0; i<(int)m.weights.size(); ++i) if(mask>>i&1){ cost+=m.weights[i]/mass*x.seconds[i];
            groups[x.outcome[i]] |= uint64_t(1)<<i;}
12        if(groups.size()==1&&groups.begin()->first>=0) continue;
13        for(auto [out, sub]: groups) if(out>=0){ double w=0; for(int i=0; i<(int)m.weights.size(); ++i) if(
            sub>>i&1) w+=m.weights[i]; cost+=w/mass*brute(m, sub, a, h-1, tuned||x.measure);}
14        best=std::min(best, cost);
15    } return best;
16 }
17 int main(){
18     Model m; m.weights={.5, .5}; m.actions={{true, {10,11}, {5,5}}, {false, {-1,0}, {5,3}}, {false
        , {0,-1}, {3,5}}}; m.travel.assign(4, std::vector<double>(4));
19     m.travel[3][1]=m.travel[3][2]=10.; m.terminal=[](uint64_t mask, int){ return mask==3?100.:.5; };
20     auto result=belief_dp::solve(m, 2, false, 10000);
21     assert(result.action==0&&std::abs(result.value-11)<1e-9); // sensing + one switch + adaptive
        clear
22     assert(result.exact&&result.expanded>0);
23     assert(result.action_values.size()==m.actions.size()); assert(std::abs(result.action_values
        [0]-11)<1e-9); assert(result.action_values[1]>result.action_values[0]);
24     auto zero=belief_dp::solve(m, 0, false, 10000); assert(zero.action==1&&zero.value==100);
25     auto limited=belief_dp::solve(m, 2, false, 0); assert(limited.action==1&&!limited.exact&&limited.
        budget_hit);
26     // A near response includes immediate optical success, with no continuation.
27     Model near=m; near.actions={{true, {-1,-1}, {10,10}}}; near.travel.assign(2, std::vector<double>(2)
        ); assert(belief_dp::solve(near, 2, false, 100).value==11);
28     Model clear=m; clear.weights={.99, .01}; clear.actions.erase(clear.actions.begin()); clear.travel.
        assign(3, std::vector<double>(3)); assert(belief_dp::solve(clear, 1, true, 100).action==0);
29     clear.weights={.01, .99}; assert(belief_dp::solve(clear, 1, true, 100).action==1);
30     // No-signal (outcome0) retains a genuine branch, never terminal success.
31     Model dropout=m; dropout.actions[0].outcome={0,10}; assert(belief_dp::solve(dropout, 2, false, 100)
        .value==11);
32     std::mt19937 rng(12);
33     for(int trial=0; trial<100; ++trial){ Model t; t.weights={1., 2., 3.}; int n=4; t.travel.assign(n+1,
        std::vector<double>(n+1));
34         for(auto &row:t.travel) for(auto &x:row) x=rng()%20;
35         for(int a=0; a<n; ++a){ Action x; x.measure=(a%2==0); for(int i=0; i<3; ++i){ x.outcome.push_back(int
            (rng()%4)-1); x.seconds.push_back(x.measure?5:3); } t.actions.push_back(x); }
36         t.terminal=[](uint64_t mask, int at){ return 20.+3*_builtin_popcountll(mask)+at; };
37         auto r=belief_dp::solve(t, 3, false, 10000); assert(r.exact); assert(std::abs(r.value-brute(t, 7, n
            , 3, false))<1e-9);
38     }
39     bool invalid=false; try{ Model bad=m; bad.weights[0]=-1; belief_dp::solve(bad, 2, true, 100); } catch(
        const std::invalid_argument&){ invalid=true; } assert(invalid);
40     std::cout<<"PASS Bellman: information, priors, near/no-signal, tuning, depth/budget; 100
        exhaustive comparisons\n";
41 }

```

## tests/test\_bridge\_deadline.py

```
1 """Transport failure tests: local HTTP only, no formal simulator session."""
2 import json
3 import math
4 import sys
5 import threading
6 import time
7 import unittest
8 import subprocess
9 from pathlib import Path
10 from unittest.mock import patch
11 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
12
13 sys.path.insert(0, str(Path(__file__).resolve().parents[1] / 'src'))
14 import bridge
15 from mocksim.client import ProtocolError
16
17 class FakeClient:
18     def __init__(self, remaining=1200, rejected=False):
19         self.remaining=remaining; self.rejected=rejected; self.actions=[]
20     def enter(self): return {'accepted':True,'remaining_real_duration_s':self.remaining}
21     def measure(self,*args):
22         self.actions.append('measure')
23         return {'accepted':not self.rejected,'virtual_time_s':0,'measure_result':'no_signal'}
24     def exit(self):
25         self.actions.append('exit')
26         return {'accepted':True,'virtual_time_s':0}
27
28 class DeadlineTests(unittest.TestCase):
29     def test_non_200_success_status_is_rejected(self):
30         class Handler(BaseHTTPRequestHandler):
31             def do_POST(self):
32                 self.rfile.read(int(self.headers['Content-Length']))
33                 data=json.dumps({'accepted':True,'virtual_time_s':0}).encode()
34                 self.send_response(201);self.send_header('Content-Length',str(len(data)));self.
35                 end_headers();self.wfile.write(data)
36             def log_message(self,*args):pass
37         server=ThreadingHTTPServer(('127.0.0.1',0),Handler)
38         threading.Thread(target=server.serve_forever,daemon=True).start()
39         try:
40             c=bridge.DeadlineClient(base_url=f'http://127.0.0.1:{server.server_port}',robot_id
41             ='REVIEW')
42             with self.assertRaises(ProtocolError):c.enter()
43             finally:server.shutdown();server.server_close()
44         def test_remaining_zero_starts_no_action(self):
45             c=FakeClient(remaining=0)
46             with self.assertRaises(TimeoutError): bridge.drive(3,c)
47             self.assertEqual(c.actions,[])
48
49         def test_rejected_action_never_counts_as_observation(self):
50             c=FakeClient(rejected=True)
51             with self.assertRaises(bridge.Rejected): bridge.drive(3,c)
52             self.assertEqual(c.actions,['measure'])
53
54     def test_solver_without_stdout_honors_short_deadline(self):
55         c=FakeClient(remaining=.25)
56         original=subprocess.Popen
```

```

55     def silent(*args,**kwargs):
56         return original([sys.executable, '-c', 'import time; time.sleep(2)'],**kwargs)
57     start=time.perf_counter()
58     with patch.object(bridge.subprocess, 'Popen', side_effect=silent):
59         with self.assertRaises(TimeoutError): bridge.drive(3,c)
60     self.assertLess(time.perf_counter()-start, 1.0)
61     self.assertNotIn('measure', c.actions)
62
63 def test_retry_reuses_identical_id_and_body(self):
64     records=[]
65     class Handler(BaseHTTPRequestHandler):
66         def do_POST(self):
67             raw=self.rfile.read(int(self.headers['Content-Length']));records.append(raw)
68             if len(records)==1:
69                 self.close_connection=True; self.connection.close();return
70             data=json.dumps({'accepted':True, 'virtual_time_s':6, 'measure_result':'no_signal
71             '}).encode()
72             self.send_response(200);self.send_header('Content-Length',str(len(data)));self.
73             end_headers();self.wfile.write(data)
74         def log_message(self,*args): pass
75     server=ThreadingHTTPServer(('127.0.0.1',0),Handler)
76     threading.Thread(target=server.serve_forever,daemon=True).start()
77     try:
78         cls=getattr(bridge, 'DeadlineClient', bridge.RobotClient)
79         c=cls(base_url=f'http://127.0.0.1:{server.server_port}', robot_id='REVIEW',
80         retry_backoff_s=.001)
81         c.deadline_at=time.perf_counter()+1
82         self.assertTrue(c.measure(0,0,1)['accepted'])
83         self.assertEqual(len(records), 2)
84         self.assertEqual(records[0], records[1])
85         self.assertEqual(c.counter, 1)
86     finally: server.shutdown();server.server_close()
87
88 def test_expired_transport_deadline_sends_nothing(self):
89     cls=getattr(bridge, 'DeadlineClient', bridge.RobotClient)
90     c=cls(base_url='http://127.0.0.1:1', robot_id='REVIEW', timeout_s=.01, retries=0)
91     c.deadline_at=time.perf_counter()-1
92     with self.assertRaises(TimeoutError): c.measure(0,0,1)
93     self.assertEqual(c.stats.requests, 0)
94
95 def test_position_is_not_rounded_before_geometry_observation(self):
96     c=bridge.DeadlineClient(robot_id='REVIEW')
97     captures=[]
98     c.post=lambda path, payload: captures.append((path, payload)) or {'accepted':True}
99     x,y=0., -.00000049
100     c.measure(x,y,1);c.clear(x,y,1)
101     for _, payload in captures:
102         self.assertEqual(payload['position'], {'x':x, 'y':y})
103     eps=math.radians(1.005)
104     z=(1000*math.cos(eps-1e-12), 1000*math.sin(eps-1e-12))
105     # If the old serializer moved the sensor to (0,0), a legal reading of
106     # 0.00 degrees excludes this true source from the solver's retained cone.
107     excess=-math.tan(eps)*z[0]+z[1]-y
108     self.assertGreater(excess, 1e-9)
109     # Full float transport keeps physical and solver coordinates identical.
110     self.assertEqual(captures[0][1]['position']['y'], y)
111
112 def test_slow_http_body_cannot_extend_total_deadline(self):

```

```

110     records=[]
111     class Handler(BaseHTTPRequestHandler):
112         def do_POST(self):
113             records.append(self.rfile.read(int(self.headers['Content-Length'])))
114             data=json.dumps({'accepted':True,'virtual_time_s':6,'measure_result':'no_signal
115             '}).encode()
116             self.send_response(200);self.send_header('Content-Length',str(len(data)));self.
117             end_headers()
118             try:
119                 for value in data:
120                     self.wfile.write(bytes([value]));self.wfile.flush();time.sleep(.015)
121             except OSError:pass
122             def log_message(self,*args):pass
123             server=ThreadingHTTPServer(('127.0.0.1',0),Handler)
124             threading.Thread(target=server.serve_forever,daemon=True).start()
125             try:
126                 c=bridge.DeadlineClient(base_url=f'http://127.0.0.1:{server.server_port}',robot_id
127                 ='REVIEW',retries=5)
128                 start=time.perf_counter();c.deadline_at=start+.15
129                 with self.assertRaises(TimeoutError):c.measure(0,0,1)
130                 self.assertLess(time.perf_counter()-start,.5)
131                 with self.assertRaises(TimeoutError):c.exit()
132                 self.assertEqual(len(records),1)
133             finally:server.shutdown();server.server_close()
134
135     def test_retry_backoff_is_limited_by_remaining_duration(self):
136         records=[]
137         class Handler(BaseHTTPRequestHandler):
138             def do_POST(self):
139                 records.append(self.rfile.read(int(self.headers['Content-Length'])))
140                 self.close_connection=True;self.connection.close()
141             def log_message(self,*args):pass
142             server=ThreadingHTTPServer(('127.0.0.1',0),Handler)
143             threading.Thread(target=server.serve_forever,daemon=True).start()
144             try:
145                 c=bridge.DeadlineClient(base_url=f'http://127.0.0.1:{server.server_port}',robot_id
146                 ='REVIEW',retry_backoff_s=1)
147                 start=time.perf_counter();c.deadline_at=start+.1
148                 with self.assertRaises(TimeoutError):c.measure(0,0,1)
149                 self.assertLess(time.perf_counter()-start,.5)
150                 self.assertEqual(len(records),1)
151             finally:server.shutdown();server.server_close()
152
153 if __name__=='__main__':unittest.main(verbosity=2)

```

## tests/test\_calibrated\_controller.cpp

```

1 #include "../src/joint_config.hpp"
2 #include <iostream>
3 int main(){auto improved=joint_configuration(58,4);
4 if(!improved.adaptive_optical||!improved.weighted_optical){std::cerr<<"FAIL calibrated
5 configuration is not based on current method56\n";return 1;}
6 if(joint_configuration(56,4).planning_samples!=63)return 1;
7 std::cout<<"PASS calibrated controller keeps adaptive coverage and posterior ordering\n";
8 }

```

## tests/test\_clear\_neighborhood.cpp

```
1 #include "../src/dynamic_selection.hpp"
2 #include <cassert>
3 #include <iostream>
4 int main(){geo::Poly p{{-5,-5},{5,-5},{5,5},{-5,5}};auto points=dynamic_selection::
    safe_clear_points(p,{100,0});assert(!points.empty());double nearest=1e99;
5 for(auto q:points){for(auto z:p)assert(geo::dist(q,z)<=20);nearest=std::min(nearest,geo::dist(
    q,{100,0}));}assert(nearest<90);
6 auto here=dynamic_selection::safe_clear_points(p,{1,1});assert(geo::dist(here.front(),{1,1})<1
    e-8);
7 assert(dynamic_selection::safe_clear_points({{-30,0},{30,0}},{100,0}).empty());std::cout<<"
    PASS safe clear neighborhoods cover all polygon vertices and avoid unnecessary center
    travel\n";
8 }
```

## tests/test\_cli.py

```
1 import subprocess,sys
2 from pathlib import Path
3 root=Path(__file__).resolve().parents[1]
4 p=subprocess.run([sys.executable,str(root/'src/bridge.py'),'--help'],capture_output=True,text=
    True)
5 assert p.returncode==0,p.stderr
6 assert '--planner-depth' in p.stdout,'planner-depth switch missing from bridge'
7 assert '--strategy' in p.stdout,'baseline/joint strategy switch missing'
8 for depth in [-1,4]:
9     p=subprocess.run([str(root/'robot.exe'),'3','35','0',str(depth)],input='',capture_output=
        True,text=True)
10     assert p.returncode!=0 and 'planner depth' in p.stderr,p.stderr
11 print('PASS CLI: planner-depth discoverable, invalid depths rejected before sensor I/O')
```

## tests/test\_continuous\_points.cpp

```
1 #include "../src/geometry.hpp"
2 #include <cassert>
3 #include <iostream>
4 #if __has_include("../src/continuous_points.hpp")
5 #include "../src/continuous_points.hpp"
6 #endif
7 int main(){
8 #ifndef B_CONTINUOUS_POINTS
9     std::cerr<<"FAIL continuous candidate refinement is unavailable\n";return 1;
10 #else
11     int calls=0;auto score=[&](geo::Pt q){++calls;return (q.x-37)*(q.x-37)+(q.y+23)*(q.y+23);};
12     auto valid=[](geo::Pt q){return geo::norm(q)<100;};
13     auto points=continuous_points::refine({{0,0},{90,0}},score,valid,80,40);
14     assert(calls<=80&&!points.empty());double best=INFINITY;for(auto q:points){assert(valid(q));
        best=std::min(best,(q.x-37)*(q.x-37)+(q.y+23)*(q.y+23));}
15     assert(best<25);auto second=continuous_points::refine({{0,0},{90,0}},score,valid,80,40);assert
        (second.size()==points.size());for(size_t i=0;i<points.size();++i)assert(geo::dist(second[
        i],points[i])==0);
16     int rejected_calls=0;geo::Poly many;for(int i=0;i<100;++i)many.push_back({double(i),0});
17     auto rejected=continuous_points::refine(many,[&](geo::Pt){++rejected_calls;return INFINITY
        ;},[](geo::Pt){return true;},5,20);
18     if(rejected_calls>5){std::cerr<<"FAIL nonfinite scores exceeded evaluation budget\n";return
        1;}assert(rejected.empty());

```

```

19 std::cout<<"PASS bounded deterministic continuous candidate refinement\n";
20 #endif
21 }

```

## tests/test\_controller.cpp

```

1 #include "../src/simulator.hpp"
2 #include "../src/configurations.hpp"
3 #include <cassert>
4 #include <iostream>
5 int main(){
6     int actions=0,plans=0;
7     for(int p:{3,4})for(int stress:{0,1,2,3}){
8         auto o=configuration(5);o.dynamic_depth=2;o.active_mode=p==3?1:0;
9         Sim s(make_case(p,73,stress),73,stress==2?2:0);Policy b(s,p,o);b.run();
10        assert(b.dynamic_plans>0);assert(s.success==s.total());assert(b.dynamic_actions<=6*s.total());
11        ;assert(b.certificate!="incomplete");
12        assert(std::abs(s.time-(s.length/5+s.switches+5*s.measures+3*s.miss+5*s.success))<.001);
13        actions+=b.dynamic_actions;plans+=b.dynamic_plans;
14    }
15    auto o=configuration(5);o.dynamic_depth=2;o.dp_node_limit=0;Sim s(make_case(4,89),89,0);Policy
16        b(s,4,o);b.run();assert(b.dynamic_actions==0&&b.dynamic_fallbacks>0&&s.success==s.total()
17        );
18    assert(actions>0);std::cout<<"PASS controller: eight nominal/boundary/collinear/dense cases,
19        "<<plans<<" replans and "<<actions<<" actions; zero-budget fallback\n";
20 }

```

## tests/test\_discovery.cpp

```

1 #include "../src/discovery.hpp"
2 #include "../review/certified_layout21_hex.hpp"
3 #include <cassert>
4 #include <iostream>
5 int main(){discovery::Belief b(4);assert(b.size()==2048);assert(b.probability({0,0})>0&&b.
6        probability({0,0})<1;b.negative_scan({0,0});assert(b.probability({0,0})==0&&b.size()
7        <2048);
8    for(auto p:certified_layout21::points)b.negative_scan({p.x,p.y});assert(b.size()==0&&b.
9        probability({500,500})==0);
10    std::cout<<"PASS discovery model conditions only on completed common scans; empty finite
11        support reports zero probability, not completion\n";
12 }

```

## tests/test\_dp\_context.cpp

```

1 #include "../src/belief_dp.hpp"
2 #include <cassert>
3 #include <iostream>
4 #include <random>
5 double exhaustive(const belief_dp::Model& m,uint64_t mask,int at,int depth,bool tuned,int
6        context){
7     #ifdef B_CONTEXT_TERMINAL
8     double best=m.terminal_context(mask,at,context);
9     if(!depth)return best;double total=0;for(int i=0;i<(int)m.weights.size();++i)if(mask>>i&1)
10        total+=m.weights[i];
11    for(int a=0;a<(int)m.actions.size();++a){auto x=m.actions[a];double cost=m.travel[at][a]+(x.
12        measure&&!tuned?1:0);std::map<int,uint64_t> groups;

```

```

10  for(int i=0;i<(int)m.weights.size();++i)if(mask>>i&1){cost+=m.weights[i]/total*x.seconds[i];
    groups[x.outcome[i]]|=uint64_t(1)<<i;}
11  for(auto [out,sub]:groups)if(out>=0){double w=0;for(int i=0;i<(int)m.weights.size();++i)if(
    sub>>i&1)w+=m.weights[i];cost+=w/total*exhaustive(m,sub,a,depth-1,tuned||x.measure,m.
    advance(context,a,out));}
12  best=std::min(best,cost);
13  }return best;
14  #else
15  return 0;
16  #endif
17  }
18  int main(){
19  #ifndef B_CONTEXT_TERMINAL
20  std::cerr<<"FAIL DP cannot distinguish continuous regions sharing a particle mask\n";return 1;
21  #else
22  std::mt19937 random(613);
23  for(int trial=0;trial<100;++trial){belief_dp::Model m;m.weights={1,2,3};m.travel.assign(4,std
    ::vector<double>(4,0));
24  for(int a=0;a<3;++a){belief_dp::Action x;x.measure=a!=2;for(int i=0;i<3;++i){x.outcome.
    push_back(int(random()%4)-1);x.seconds.push_back(3+a);}m.actions.push_back(x);}
25  m.terminal=[](uint64_t,int){return 100.};};
26  m.advance=[](int c,int a,int out){return c*17+(a+1)*4+out+1};};
27  m.terminal_context=[](uint64_t mask,int at,int context){return context?double(5+((context*13+
    at+mask)%29)):100.};};
28  auto r=belief_dp::solve(m,3,false,10000);
29  assert(r.exact&&std::abs(r.value-exhaustive(m,7,3,3,false,0))<1e-9);
30  }
31  std::cout<<"PASS context-aware DP: 100 exhaustive history-dependent comparisons\n";
32  #endif
33  }

```

## tests/test\_geometry\_revision.cpp

```

1  #include "../src/policy.hpp"
2  #include <cassert>
3  #include <fstream>
4  #include <iomanip>
5  #include <iostream>
6  int main(int argc,char**argv){
7  if(argc!=2){std::cerr<<"usage: test_geometry_revision OUTPUT_WITNESS.json\n";return 2;}
8  auto p=geo::observe(geo::outer_disk({0,0},1800),{0,0},0);p=geo::disk_clip(p,{0,0},1500);p=geo
    ::clip(p,{1,0},1500);
9  int checks=0;double gap=0;
10 for(auto q:geo::Poly{{750,300},{810,-480},{300,0},{0,600},{1400,10}}){auto b=robust_geometry::
    worst_radius(p,q,512,.01);auto range=robust_geometry::angles(p,q);double sampled=0;
11 for(int i=0;i<=5000;++i){double angle=range.first+(range.second-range.first)*i/5000;auto
    region=geo::observe(p,q,angle);if(!region.empty())sampled=std::max(sampled,geo::mec(region
    ).r);++checks;}
12 assert(sampled<=b.upper+1e-6);assert(b.lower<=b.upper);gap=std::max(gap,b.upper-sampled);
13 }
14 std::vector<dynamic_selection::Event>history{{{0,0},2,0}};
15 assert(robust_geometry::reception(p,{750,300},history,3));
16 assert(!robust_geometry::reception(p,{750,300},history,4));
17 assert(!robust_geometry::candidate(p,{0,0},{0,0},0,history,3,2).valid);
18 std::vector<geo::Poly>cells;auto centers=adaptive_optical::cover(p,{0,0},0,&cells);assert(
    cells.size()==centers.size());

```

```

19 auto moved=neighborhood_tour::optimize(centers,cells,{0,200});assert(neighborhood_tour::length
    (moved,{0,200})<=neighborhood_tour::length(centers,{0,200})*1e-7);
20 for(size_t i=0;i<cells.size();++i){assert(neighborhood_tour::valid(moved[i],cells[i]));for(
    auto z:cells[i])assert(geo::dist(moved[i],z)<20);}
21 geo::Poly sites;for(auto q:certified_layout21::points)sites.push_back({q.x,q.y});auto
    certificate=residual_cover::certify(sites);assert(certificate.pass);
22 auto missing=sites;missing.erase(missing.begin()+9);assert(!residual_cover::certify(missing,
    certificate.leaves).pass);
23 residual_cover::State state;std::vector<bool>visited(sites.size());state.leaves=certificate.
    leaves;
24 geo::Pt shifted=state.replace(sites,visited,1,{850,50});assert(geo::dist(shifted,sites[1])>1);
    sites[1]=shifted;auto recert=residual_cover::certify(sites);assert(recert.pass);
25 // Export actual accepted coordinates and interval-certified leaf boxes for
26 // independent Fraction replay; doubles are round-trip decimal encoded.
27 std::ofstream f(argv[1]);if(!f){std::cerr<<"cannot open witness output\n";return 3;}
28 f<<std::setprecision(17)<<{"sites\":[\";for(size_t i=0;i<sites.size();++i){if(i)f<<",";f
    <<\"[\"<<sites[i].x<<\",\"<<sites[i].y<<\"]\";f<<\"],\"boxes\":[\";for(size_t i=0;i<recert.leaves
    .size();++i){auto b=recert.leaves[i];if(i)f<<\",\";f<<\"[\"<<b.x0<<\",\"<<b.y0<<\",\"<<b.x1<<\",\"<<
    b.y1<<\"]\";f<<\"}]\n\";
29 f.close();if(!f){std::cerr<<"cannot finish witness output\n";return 4;}
30 std::cout<<{"status\":\"PASS\",\"angle_checks\":"<<checks<<\",\"max_upper_gap_m\":\"<<gap
    <<\",\"optical_cells\":"<<cells.size()<<\",\"static_leaves\":"<<certificate.leaves.size()
    <<\",\"replacement_leaves\":"<<recert.leaves.size()<<\",\"replacement_shift_m\":\"<<geo::dist
    (shifted,geo::Pt{995,0})<<\"}\n\";
31 }

```

## tests/test\_joint.cpp

```

1 #include "../src/simulator.hpp"
2 #include "../src/joint_config.hpp"
3 #include <cassert>
4 #include <iostream>
5 int main(){
6     for(int p:{3,4}){auto o=configuration(5);o.dynamic_depth=2;o.active_mode=p==3?1:0;o.auxiliary
        =1;
7         Sim s(make_case(p,73),73,0);Policy b(s,p,o);b.run();assert(b.auxiliary_calls>0);assert(s.
            success==s.total());assert(b.auxiliary_calls<=4*s.total());
8     }
9     for(int m:{12,13,14,15,16,17,18,19,20,21,22,27,32,38,39,56})for(int p:{3,4}){auto o=
        joint_configuration(m,p);Sim s(make_case(p,89,3),89,0);Policy b(s,p,o);b.run();assert(s.
            success==s.total());assert(b.dynamic_actions<=6*s.total());assert(b.auxiliary_calls<=4*s.
            total());}
10    for(int p:{3,4})for(int stress:{0,1,2,3}){auto o=configuration(5);o.dynamic_depth=2;o.
        active_mode=p==3?1:0;o.auxiliary=2;o.joint_actions=true;
11        Sim s(make_case(p,89,stress),89,stress==2?2:0);Policy b(s,p,o);b.run();
12        assert(b.joint_steps>0&&s.success==s.total());assert(b.dynamic_actions<=6*s.total());assert(b
            .auxiliary_calls<=4*s.total());
13        assert(b.joint_steps<=21+7*s.total());assert(std::abs(s.time-(s.length/5+s.switches+5*s.
            measures+3*s.miss+5*s.success))<.001);
14    }
15    std::cout<<"PASS auxiliary/joint progress and action limits across ten complete scenarios\n";
16 }

```

## tests/test\_optical\_order.cpp

```

1 #include "../src/optical_order.hpp"

```

```

2 #include <cassert>
3 #include <iostream>
4 int main(){geo::Poly cells;for(int i=0;i<20;++i)cells.push_back({i*25.,0});
5 std::vector<dynamic_selection::Hypothesis> h{{250,0},1000,false,0,.99},{475,0},1000,false
    ,0,.01}};
6 auto r=optical_order::choose(cells,{240,0},h);assert(r.size()==cells.size());assert(geo::dist(
    r.front(),{250,0})<30);
7 auto sorted=r;auto less=[](auto a,auto b){return a.x!=b.x?a.x<b.x:a.y<b.y;};std::sort(sorted.
    begin(),sorted.end(),less);assert(sorted.size()==cells.size());for(size_t i=0;i<cells.size
    ());++i)assert(geo::dist(cells[i],sorted[i])==0);
8 auto empty=optical_order::choose(cells,{0,0},{});assert(empty.front().x==0);
9 std::cout<<"PASS posterior sweep prioritizes likely location and retains every covering cell\n
    ";
10 }

```

### tests/test\_optical\_portfolio.cpp

```

1 #include "../src/optical_terminal.hpp"
2 #if __has_include("../src/optical_portfolio.hpp")
3 #include "../src/optical_portfolio.hpp"
4 #endif
5 #include <iostream>
6 #include <cassert>
7 int main(){
8 #ifndef B_OPTICAL_PORTFOLIO
9 std::cerr<<"FAIL optical replacement lacks a minimum modeled saving guard\n";return 1;
10 #else
11 geo::Poly old{{100,0},{1000,0},{0,0}},candidate{{0,0},{100,0},{1000,0}};geo::Pt now{0,0};
12 std::vector<dynamic_selection::Hypothesis> h{{0,0},1000,false,0,1}};
13 auto chosen=optical_portfolio::choose(old,candidate,now,h,.1,10);assert(chosen.size()==
    candidate.size())&&geo::dist(chosen.front(),{0,0})==0);
14 auto retained=optical_portfolio::choose(candidate,old,now,h,.1,10);assert(geo::dist(retained.
    front(),{0,0})==0);
15 auto guarded=optical_portfolio::choose(old,candidate,now,h,.99,10000);assert(geo::dist(guarded
    .front(),old.front())==0);
16 assert(optical_portfolio::choose(old,candidate,now,{},0,0).size()==old.size());
17 std::cout<<"PASS optical portfolio keeps incumbent unless modeled improvement exceeds guard\n
    ";
18 #endif
19 }

```

### tests/test\_pair\_analysis.py

```

1 import sys, unittest
2 from pathlib import Path
3 sys.path.insert(0,str(Path(__file__).resolve().parents[1]/'src'))
4 try:
5     from pair_analysis import summarize
6 except ImportError:
7     print('FAIL independent paired analysis is unavailable')
8     raise SystemExit(1)
9
10 class PairedTests(unittest.TestCase):
11     def test_known_savings(self):
12         old=[dict(time_s=100.,avg_s=10.,runtime_s=.1),dict(time_s=200.,avg_s=20.,runtime_s=.2)]
13         new=[dict(time_s=80.,avg_s=8.,runtime_s=.1),dict(time_s=160.,avg_s=16.,runtime_s=.2)]

```

```

14         result=summarize(old,new)
15         self.assertAlmostEqual(result['total_gain_pct'],20)
16         self.assertAlmostEqual(result['saving_s_mean'],30)
17         self.assertEqual(result['wins'],2)
18     def test_equal(self):
19         rows=[dict(time_s=100.,avg_s=10.,runtime_s=.1)]*3
20         result=summarize(rows,rows)
21         self.assertEqual(result['total_gain_pct'],0)
22         self.assertEqual(result['ties'],3)
23         self.assertEqual(result['saving_s_ci95'],[0,0])
24
25 if __name__=='__main__':unittest.main()

```

## tests/test\_prepared\_sweep.cpp

```

1 #include "../src/dynamic_selection.hpp"
2 #include <cassert>
3 #include <iostream>
4 int main(){
5 #ifndef B_PREPARED_SWEEP
6     std::cerr<<"FAIL no reusable coverage/travel cache for repeated terminal estimates\n";return
7     1;
8 #else
9     for(double angle:{0.,37.,179.5,301.}){
10         auto p=geo::observe(geo::outer_disk({0,0},1500),{0,0},angle);
11         p=geo::clip(p,geo::polar(1,angle),1500);
12         auto h=dynamic_selection::make_hypotheses(p,{{0,0},2,angle}},4,63);
13         auto cover=adaptive_optical::cover(p,{0,0},angle);
14         optical_terminal::PreparedSweep sweep(cover,h);
15         uint64_t all=(uint64_t(1)<<h.size())-1;
16         for(uint64_t mask:{all,uint64_t(1),all&0x5555555555555555ULL})for(auto now:{geo::Pt{0,0},geo
17             ::polar(1700,angle),geo::polar(700,angle+45)}){
18             std::vector<dynamic_selection::Hypothesis> sub;for(size_t i=0;i<h.size();++i)if(mask>>i&1)
19                 sub.push_back(h[i]);
20             auto route=cover;if(geo::dist(now,route.back())<geo::dist(now,route.front()))std::reverse(
21                 route.begin(),route.end());
22             route=optical_order::choose(route,now,sub);
23             double reference=optical_terminal::expected(route,now,sub);
24             assert(std::abs(sweep.value(mask,now)-reference)<1e-8);
25         }
26     }
27     assert(optical_terminal::update({{0,0},{1,0},{0,1}},{0,0},0).size()==3);
28     for(int out=10;out<190;++out){geo::Pt q{200,-100};auto p=geo::outer_disk(q,1500);auto after=
29         optical_terminal::update(p,q,out);
30         for(double delta:{-2.004,0.,2.004})assert(geo::contains(after,q+geo::polar(1200,2*(out-10)+1+
31             delta)));
32     }
33     std::cout<<"PASS cached sweep equals execution policy for rotations, masks and both entry ends
34         ; bin envelope conservative\n";
35 #endif
36 }

```

## tests/test\_q3\_radius.cpp

```

1 #include "../src/simulator.hpp"
2 #include "../src/configurations.hpp"

```

```

3 #include <cassert>
4 #include <iostream>
5 int main(){auto o=configuration(5);o.q3_radius=1124;Sim s({},1,0);Policy b(s,3,o);auto pts=b.
    covering_points();assert(pts.size()==7);assert(std::abs(gео::norm(pts[1])-1124)<1e-8);
6 for(int k=0;k<=60;++k)for(int j=0;j<720;++j){auto z=geo::polar(30*k,j*.5);double best=1e100;
    for(auto p:pts)best=std::min(best,geo::dist(z,p));assert(best<999.6);}
7 std::ofstream f("results/q3_production_points.json");f<<std::setprecision(17)<<"[";for(size_t
    i=0;i<pts.size();++i)f<<("{","":"")<<["<<pts[i].x<<","<<pts[i].y<<"]<<"]<<"\n";
8 std::cout<<"PASS: seven shortened scan points; dense positive coverage check (analytic bound
    separate)\n";}

```

## tests/test\_remaining\_depth.cpp

```

1 #include "../src/dynamic_selection.hpp"
2 #include <iostream>
3 #include <cassert>
4 int main(){
5 #ifdef B_REMAINING_DEPTH
6 int depth=dynamic_selection::remaining_depth(2,6,5);
7 #else
8 int depth=2;
9 #endif
10 belief_dp::Model m;m.weights={1,1};m.actions={{true,{10,11},{5,5}},{false,{-1,0},{5,3}},{false
    ,{0,-1},{3,5}}};m.travel.assign(4,std::vector<double>(4));
11 m.travel[3][1]=m.travel[3][2]=15;
12 m.terminal=[](uint64_t mask,int){return mask==3?100.:50.;};
13 auto r=belief_dp::solve(m,depth,false,1000);
14 if(r.action==0){std::cerr<<"FAIL final allowed action relies on a second dynamic action\n";
    return 1;}
15 #ifdef B_REMAINING_DEPTH
16 assert(depth==1);assert(dynamic_selection::remaining_depth(3,6,6)==0);assert(dynamic_selection
    ::remaining_depth(2,6,0)==2);
17 assert(belief_dp::solve(m,2,false,1000).action==0);
18 #endif
19 std::cout<<"PASS Bellman horizon respects remaining dynamic action allowance\n";
20 }

```

## tests/test\_retreat.cpp

```

1 #include "../src/dynamic_selection.hpp"
2 #include <cassert>
3 #include <iostream>
4 int main(){using namespace dynamic_selection;geo::Pt center{500,0};std::vector<Event> h
    {{0,0},2,0};auto q=retreat_points(center,h);assert(q.size()==4);
5 for(auto p:q){assert(p.x>0&&p.x<500&&p.y==0);for(int angle=0;angle<360;++angle){Hypothesis
    source{center,1000,true,double(angle),1};if(receives(source,{0,0}))assert(receives(source,
    p));}}
6 assert(retreat_points(center,{}).empty());std::cout<<"PASS retreat rays remain on received
    half-plane for fixed target across 360 headings\n";
7 }

```

## tests/test\_scan\_fee.cpp

```

1 #include "../src/simulator.hpp"
2 #include "../src/configurations.hpp"

```

```

3 #include <cassert>
4 #include <iostream>
5 int main(){Sim s({},1,0);Policy b(s,4,configuration(5));double expected=b.scan_fee();assert(
    expected==119);b.scan({0,0});assert(s.time==expected);expected=b.scan_fee();double before=
    s.time;b.scan({0,0});assert(s.time-before==expected);std::cout<<"PASS scanning charge
    equals simulator measurements plus actual switches\n";}

```

## tests/test\_search\_order.cpp

```

1 #include "../src/optical_order.hpp"
2 #include "../src/optical_terminal.hpp"
3 #include <cassert>
4 #include <iostream>
5 int main(){geo::Poly p{{100,0},{1000,0},{-100,0},{-1000,0},{0,0}};geo::Pt now{0,50};
6 std::vector<dynamic_selection::Hypothesis> h;for(auto q:p)h.push_back({q,1000,false,0,q.x
    ==0?10.:1.});
7 auto old=optical_order::choose(p,now,h);
8 #ifdef B_SEARCH_ORDER
9 auto improved=optical_order::search(p,now,h,0);
10 #else
11 auto improved=old;
12 #endif
13 double before=optical_terminal::expected(old,now,h),after=optical_terminal::expected(improved,
    now,h);
14 if(after>=before-1){std::cerr<<"FAIL search order expected cost "<<after<<" did not beat
    cyclic "<<before<<"\n";return 1;}
15 #ifdef B_SEARCH_ORDER
16 for(double mix:{0.,.1,.3}){auto q=optical_order::search(p,now,h,mix);assert(q.size()==p.size()
    );for(auto v:p){int count=0;for(auto z:q)count+=geo::dist(z,v)<1e-8;assert(count==1);}}
17 assert(optical_order::search({},now,h,0).empty());
18 assert(optical_order::search(p,now,{},0).size()==p.size());
19 #endif
20 std::cout<<"PASS probability-per-time search improves cyclic route and preserves every cover
    center\n";
21 }

```

## tests/test\_selection.cpp

```

1 #include "../src/dynamic_selection.hpp"
2 #include <cassert>
3 #include <iostream>
4 int main(){
5 using namespace dynamic_selection;
6 Hypothesis omni{{500,0},1000,false,0,1},left{{500,0},1000,true,180,1};
7 std::vector<Event>history{{0,0},2,0},{1000,0},0,0}};
8 assert(consistent(left,history));assert(!consistent(omni,history));
9 history.back().q={2000,0};assert(consistent(omni,history));
10 history.push_back({490,0},3,0);assert(!consistent(omni,history));
11 assert(consistent(omni,{{496,0},1,0}));assert(!consistent(omni,{{496,0},2,0}));
12 assert(consistent(omni,{{1000,0},2,180}));assert(!consistent(omni,{{1000,0},2,0}));
13 auto p=geo::observe(geo::outer_disk({0,0},1500),{0,0},0);p=geo::clip(p,{1,0},1500);auto
    original=p;
14 history={{0,0},2,0},{1000,0},0,0}};
15 auto hypotheses=make_hypotheses(p,history,4,32);assert(!hypotheses.empty()&&hypotheses.size()
    <=32);
16 for(auto h:hypotheses){assert(geo::contains(p,h.z));assert(consistent(h,history));}

```

```

17 auto choice=plan(p,{0,0},{0,0},0,history,4,1,true,2000);
18 assert(choice.available&&choice.depth==1&&choice.hypotheses>0);
19 assert(std::isfinite(choice.value)&&geo::norm(choice.point)<2672);
20 if(choice.measure)for(auto e:history)if(e.kind<=2)assert(geo::dist(e.q,choice.point)>1e-6);
21 assert(p.size()==original.size());for(size_t i=0;i<p.size();++i)assert(geo::dist(p[i],original
    [i])==0);
22 auto exhausted=plan(p,{0,0},{0,0},0,history,4,2,true,0);assert(!exhausted.available&&exhausted
    .budget_hit);
23 assert(make_hypotheses({},history,3,32).empty());
24 std::cout<<"PASS selection: reception, direction, no-signal/near, failed clear, support, no
    repeats, polygon isolation and budget\n";
25 }

```

### tests/test\_shortcut.cpp

```

1 #include "../src/policy.hpp"
2 #if __has_include("../src/shortcut_sensor.hpp")
3 #include "../src/shortcut_sensor.hpp"
4 #endif
5 #include <iostream>
6 #include <cassert>
7 struct Probe:Sensor{geo::Pt position;double length=0;Reading measure(geo::Pt q,int)override{
    length+=geo::dist(position,q);position=q;return {0,0};}bool clear(geo::Pt q,int)override{
    length+=geo::dist(position,q);position=q;return true;}};
8 int main(){
9 #ifndef B_SHORTCUT_SENSOR
10 std::cerr<<"FAIL guaranteed clear still travels to nominal center\n";return 1;
11 #else
12 Probe original;ShortcutSensor sensor(original);geo::Poly region
    {{99,-1},{101,-1},{101,1},{99,1}};
13 sensor.audit(1,region);assert(sensor.clear({100,0},1));
14 assert(original.position.x>80&&original.position.x<83);for(auto v:region)assert(geo::dist(
    original.position,v)<20);
15 sensor.measure({-100,100},2);assert(geo::dist(original.position,{-100,100})==0);
16 assert(original.length<100+geo::dist({100,0},{-100,100}));assert(sensor.saved_distance()>0);
17 sensor.audit(2,{0,0},{100,0},{0,100});sensor.clear({0,0},2);assert(geo::dist(original.
    position,{0,0})==0);
18 sensor.audit(3,{20,0});sensor.clear({0,0},3);assert(geo::dist(original.position,{0,0})==0);
19 Probe disabled;ShortcutSensor off(disabled,false);off.audit(1,region);off.clear({100,0},1);
    assert(disabled.position.x==100);
20 std::cout<<"PASS clear shortcut: guaranteed support, unchanged measurements, insufficient
    support unchanged and path dominance\n";
21 #endif
22 }

```

### tests/test\_shortcut\_controller.cpp

```

1 #include "../src/simulator.hpp"
2 #include "../src/joint_config.hpp"
3 #include "../src/shortcut_sensor.hpp"
4 #include <cassert>
5 #include <iostream>
6 struct EventProbe:Sim{
7 struct Record{int kind,k,result;geo::Pt q;double angle;};std::vector<Record>events;using Sim::
    Sim;
8 Reading measure(geo::Pt q,int k)override{auto r=Sim::measure(q,k);events.push_back({0,k,r.kind
    ,q,r.angle});return r;}

```

```

9   bool clear(geo::Pt q,int k)override{auto r=Sim::clear(q,k);events.push_back({1,k,int(r),q,0});
    return r;}
10 };
11 int main(){
12 #ifndef B_SHORTCUT_CONFIG
13   std::cerr<<"FAIL shortcut strategy not integrated with the frozen controller\n";return 1;
14 #else
15   int stopped=0;double savings=0;
16   for(int problem:{3,4})for(int stress=0;stress<=5;++stress){int seed=2140001+stress;
17     auto environment=make_case(problem,seed,stress);int error=stress==2?2:stress==4?3:stress
        ==5?4:0;
18     EventProbe baseline(environment,seed,error),candidate(environment,seed,error);
19     Policy old(baseline,problem,joint_configuration(56,problem));old.run();
20     auto options=joint_configuration(78,problem);assert(options.shortcut);ShortcutSensor io(
        candidate);Policy improved(io,problem,options);improved.run();
21     assert(baseline.total()==baseline.success&&candidate.total()==candidate.success);
22     assert(baseline.events.size()==candidate.events.size());
23     for(size_t i=0;i<baseline.events.size();++i){auto a=baseline.events[i],b=candidate.events[i];
        assert(a.kind==b.kind&&a.k==b.k&&a.result==b.result);
24     if(a.kind==0){assert(a.q.x==b.q.x&&a.q.y==b.q.y);assert(a.angle==b.angle);}
25   }
26   assert(candidate.time<=baseline.time+1e-4);assert(std::abs(baseline.length-candidate.length-
        io.saved_distance())<1e-5);
27   assert(old.dynamic_plans==improved.dynamic_plans&&old.dynamic_actions==improved.
        dynamic_actions);
28   stopped+=io.shortcuts;savings+=baseline.time-candidate.time;
29 }
30 assert(stopped>0&&savings>0);std::cout<<"PASS shortcut controller: 12 matched nominal/stress
    traces; "<<stopped<<" shortcuts, "<<savings<<" seconds saved\n";
31 #endif
32 }

```

## tests/test\_terminal.cpp

```

1 #include "../src/dynamic_selection.hpp"
2 #include "../src/adaptive_optical.hpp"
3 #include "../src/optical_order.hpp"
4 #include <iostream>
5 #include <cassert>
6 double expected_cost(const geo::Poly& route,geo::Pt now,const std::vector<dynamic_selection::
    Hypothesis>& hypotheses){
7   double total=0,value=0;for(auto h:hypotheses)total+=h.weight;
8   for(auto h:hypotheses){double cost=0;auto at=now;bool found=false;
9     for(auto q:route){cost+=geo::dist(at,q)/5;at=q;if(geo::dist(q,h.z)<=20+1e-7){cost+=5;found=
        true;break;}cost+=3;}
10    assert(found);value+=h.weight/total*cost;
11  }return value;
12 }
13 int main(){
14   using namespace dynamic_selection;
15   auto p=geo::observe(geo::outer_disk({0,0},1500),{0,0},0);
16   p=geo::clip(p,{1,0},1500);geo::Pt now{400,120};
17   std::vector<Event> history{{0,0},2,0};
18   auto h=make_hypotheses(p,history,4,63);
19   auto route=optical_order::choose(adaptive_optical::cover(p,{0,0},0),now,h);
20 #ifdef B_CONTEXT_TERMINAL
21   auto selected=plan(p,now,{0,0},0,history,4,0,true,1000,false,63,false,1);

```

```

22 #else
23 auto selected=plan(p,now,{0,0},0,history,4,0,true,1000,false,63,false);
24 #endif
25 double expected=expected_cost(route,now,h);
26 if(std::abs(selected.value-expected)>1e-8){std::cerr<<"FAIL actual adaptive sweep cost="<<
    expected<<" planner terminal="<<selected.value<<"\n";return 1;}
27 assert(!selected.available);
28 std::cout<<"PASS terminal matches complete conservative adaptive sweep\n";
29 }

```

## tests/test\_variable\_cover.cpp

```

1 #include "../src/adaptive_optical.hpp"
2 #include <iostream>
3 #include <cassert>
4 #include <random>
5 int main(){
6 auto p=geo::observe(geo::outer_disk({0,0},1500),{0,0},0);p=geo::clip(p,{1,0},1500);
7 #ifdef B_VARIABLE_COVER
8 auto route=adaptive_optical::variable_cover(p,{0,0},0);
9 #else
10 auto route=adaptive_optical::cover(p,{0,0},0);
11 #endif
12 if(route.size()>70){std::cerr<<"FAIL variable strip packing should use <=70 centers, got "<<
    route.size()<<"\n";return 1;}
13 #ifdef B_VARIABLE_COVER
14 for(double angle:{0.,37.,147.,299.})for(double length:{1.,20.,200.,750.,1500.}){
15 auto z=geo::observe(geo::outer_disk({0,0},length),{0,0},angle);z=geo::clip(z,geo::polar(1,
    angle),length);
16 std::vector<geo::Poly> cells;auto q=adaptive_optical::variable_cover(z,{0,0},angle,&cells);
17 assert(!q.empty()&&q.size()==cells.size());
18 for(size_t i=0;i<q.size();++i){assert(!cells[i].empty());for(auto v:cells[i])assert(geo::dist
    (v,q[i])<20);}
19 for(double x=0;x<=length;x+=std::max(.2,length/500))for(double t
    :{-0.01754,-.01,0.,.01,.01754}){
20 auto v=geo::polar(x,angle)+geo::polar(t*x,angle+90);if(!geo::contains(z,v))continue;
21 bool hit=false;for(auto c:q)hit|=geo::dist(c,v)<=20;assert(hit);
22 }
23 }
24 auto single=adaptive_optical::variable_cover({{400,50}},{0,0},0);assert(single.size()==1&&geo
    ::dist(single[0],{400,50})<1e-9);
25 std::mt19937 random(842);auto U=[&](){}return double(random())/random.max();};
26 for(int trial=0;trial<100;++trial){auto source=geo::polar(1800*sqrt(U()),360*U());auto first=
    source+geo::polar(100+1200*U(),360*U());double angle=atan2(source.y-first.y,source.x-first
    .x)*180/geo::pi+2*U()-1;
27 auto region=geo::observe(geo::outer_disk({0,0},1800),first,angle);region=geo::disk_clip(
    region,first,1500);auto axis=geo::polar(1,angle);region=geo::clip(region,axis,geo::dot(
    axis,first)+1500);
28 for(int k=0;k<trial%4;++k){auto q=source+geo::polar(50+800*U(),360*U());double a=atan2(source
    .y-q.y,source.x-q.x)*180/geo::pi+2*U()-1;region=geo::observe(region,q,a);}
29 std::vector<geo::Poly> cells;auto centers=adaptive_optical::variable_cover(region,first,angle
    ,&cells);assert(centers.size()==cells.size()&&!centers.empty());
30 for(size_t i=0;i<centers.size();++i)for(auto z:cells[i])assert(geo::dist(centers[i],z)<20);
31 geo::Pt mean;for(auto z:region)mean=mean+z;mean=mean/double(region.size());
32 for(auto vertex:region)for(int k=0;k<=20;++k){auto z=mean*(1-k/20.)+vertex*(k/20.);bool hit=
    false;for(auto c:centers)hit|=geo::dist(c,z)<20;assert(hit);}
33 }

```

```

34 #endif
35 std::cout<<"PASS variable-width covering: "<<route.size()<<" centers, certified convex cells
    and rotated coverage\n";
36 }

```

## vendor/mocksim/\_\_init\_\_.py

```

1 # -*- coding: utf-8 -*-
2 """mocksim -- CUMCM 2026 B 题 本地测试环境（按公开协议规范实现，非官方模拟器）.
3
4 模块
5 ----
6 arena    : 物理规则 / 虚拟计时 / 案例生成（附录1、附录2 的常数）
7 server   : HTTP+JSON 接口 /enter /measure /clear /exit（附件2）
8 client   : 参考机器狗客户端（urllib, 逐次等待响应, request_id 幂等重试）
9 harness  : 演练批量运行、统计、表1 结果导出、明文行为日志
10 conformance : 协议一致性自测套件（校验任意客户端是否符合附件2）
11 """
12
13 from . import arena
14
15 __all__ = ["arena"]
16 __version__ = "1.0.0"

```

## vendor/mocksim/arena.py

```

1 # -*- coding: utf-8 -*-
2 """mock jammer arena -- core physics, virtual clock and case generation.
3
4 This module is a from-scratch implementation of the *documented* behaviour in
5 2026 CUMCM B 题 附录1/附录2 与 附件1《模拟器使用说明》/ 附件2《模拟器通信接口说明及编程指南》.
6
7 It performs no reverse engineering of the official simulator: every constant
8 below is quoted from the published specification (see CONSTANTS / docstrings).
9 Purpose: unlimited offline rehearsal of a robot-dog client program.
10
11 Unit conventions
12 -----
13 * length : metre
14 * angle  : degree, 0 = +x (east), counter-clockwise positive, [0, 360)
15 * time   : second (virtual, internal resolution 1e-6 s)
16 """
17
18 from __future__ import annotations
19
20 import hashlib
21 import math
22 import random
23 import struct
24 from dataclasses import dataclass, field
25 from typing import Any
26
27 # -----
28 # Constants quoted from the specification
29 # -----
30

```

```

31 ARENA_RADIUS_M = 1800.0          # 目标区域半径 1800 m (题目正文)
32 CHANNEL_MIN, CHANNEL_MAX = 1, 20 # 频道 1..20
33 NEAR_RANGE_M = 5.0                # 近距离阈值 5 m (附件2 §2.4, 附录2(9))
34 CLEAR_RANGE_M = 20.0              # 清除半径 20 m (附件2 §2.4, 附录2(8))
35 SPEED_MPS = 5.0                   # 移动速度 5 m/s (附录2(6))
36 MEASURE_COST_S = 5.0              # 检测动作 5 s (附录2(5))
37 SWITCH_COST_S = 1.0               # 任意两频道切换 1 s (附录2(4))
38 CLEAR_FOUND_COST_S = 5.0          # 光学定位 3 s + 激光清除 2 s (附录2(8))
39 CLEAR_MISS_COST_S = 3.0           # 只启用光学探测仪 3 s
40 SVD_ERROR_LIMIT_DEG = 1.0         # 示向度误差范围 [-1°, 1°] (附录2(1))
41 DIRECTIONAL_HALF_ANGLE_DEG = 90.0 # 定向源覆盖 = 定向方向两侧各 90° (附录1(3))
42 EFF_RADIUS_MIN_M, EFF_RADIUS_MAX_M = 1000.0, 1500.0 # 有效接收半径 1000~1500 m
43 JAMMER_COUNT_MIN, JAMMER_COUNT_MAX = 10, 16          # 干扰源总数 10~16 个
44
45 MAX_VIRTUAL_DURATION_S = 360000.0 # 虚拟世界限时 (附件2 §4.5)
46 MAX_REAL_DURATION_S = 1200.0      # /enter 后程序运行时间上限 20 min
47 WINDOW_DURATION_S = 1500.0        # 25 分钟测试窗口
48 ENTRY_GRACE_S = 300.0              # 窗口开启后 5 分钟内 /enter 可用足 20 分钟
49
50 COORD_LIMIT_M = 2000000.0          # |坐标分量| <= 2000000 (附件2 §1.1)
51 ID_BYTE_MIN, ID_BYTE_MAX = 1, 64   # robot_id 1..64 字节
52 REQ_ID_BYTE_MAX = 128               # request_id 1..128 字节
53 BODY_LIMIT_BYTES = 65536            # 请求体上限
54 MAX_JSON_DEPTH = 16                 # JSON 嵌套不超过 16 层
55
56 MICRO = 1_000_000 # 模拟器内部按微秒累计 (附件2 §4.1)
57
58
59 def quantise_time(seconds: float) -> float:
60     """模拟器内部按微秒累计, 响应最多保留 6 位小数并删除无意义末尾零."""
61     return round(seconds * MICRO) / MICRO
62
63
64 # -----
65 # Case (ground truth) generation
66 # -----
67
68
69 @dataclass
70 class Jammer:
71     """一个干扰源。位置/频道/有效接收半径/类型对机器狗不可见 (附录3)."""
72
73     channel: int
74     x: float
75     y: float
76     eff_radius: float
77     directional: bool
78     bearing_deg: float | None = None # 定向方向, 仅定向源有; 全向源为空
79
80     @property
81     def pos(self) -> tuple[float, float]:
82         return (self.x, self.y)
83
84     def covered(self, px: float, py: float) -> bool:
85         """检测点是否位于该干扰源信号有效覆盖角度范围内."""
86         if not self.directional:
87             return True
88         assert self.bearing_deg is not None

```

```

89         return angle_in_sector(px, py, self.x, self.y, self.bearing_deg)
90
91
92 @dataclass
93 class Case:
94     """一局测试的案例真值。problem=3 -> 全向；problem=4 -> 全向+定向混合。"""
95
96     case_code: str
97     problem: int
98     seed: int
99     jammers: list[Jammer]
100     created_ms: int = 0
101
102     @property
103     def total(self) -> int:
104         return len(self.jammers)
105
106     @property
107     def omni_count(self) -> int:
108         return sum(1 for j in self.jammers if not j.directional)
109
110     @property
111     def directional_count(self) -> int:
112         return sum(1 for j in self.jammers if j.directional)
113
114     def by_channel(self, channel: int) -> Jammer | None:
115         for j in self.jammers:
116             if j.channel == channel:
117                 return j
118         return None
119
120     def truth_table(self) -> list[dict[str, Any]]:
121         return [
122             {
123                 "channel": j.channel,
124                 "x": round(j.x, 6),
125                 "y": round(j.y, 6),
126                 "effective_radius_m": round(j.eff_radius, 6),
127                 "type": "directional" if j.directional else "omni",
128                 "bearing_deg": None if j.bearing_deg is None else round(j.bearing_deg, 6),
129             }
130             for j in sorted(self.jammers, key=lambda j: j.channel)
131         ]
132
133
134 def dist(ax: float, ay: float, bx: float, by: float) -> float:
135     return math.hypot(ax - bx, ay - by)
136
137
138 def bearing_deg(px: float, py: float, tx: float, ty: float) -> float:
139     """从 (px,py) 指向 (tx,ty) 的真实方位角，正东为 0°，逆时针为正，[0,360)。"""
140     return math.degrees(math.atan2(ty - py, tx - px)) % 360.0
141
142
143 def angle_in_sector(px: float, py: float, jx: float, jy: float, bearing: float) -> bool:
144     """检测点是否落在以 bearing 为中心、两侧各 90°(含边界) 的覆盖扇区内。
145
146     附件1(3): "定向干扰源具有特定的发射方向(简称定向方向)，有效覆盖角度范围

```

```

147 为定向方向两侧各 90°(含)". 注意这个角度是从**干扰源**出发量的,
148 等价于 (检测点 - 干扰源) 与 bearing 方向的夹角 <= 90°.
149 """
150 vx, vy = px - jx, py - jy      # 从干扰源指向检测点
151 if vx == 0.0 and vy == 0.0:
152     return True                # 与干扰源重合, 视为在覆盖范围内
153 rad = math.radians(bearing)
154 bx, by = math.cos(rad), math.sin(rad)
155 # 夹角 |Δ| <= 90° <=> v · b >= 0
156 return vx * bx + vy * by >= -1e-9
157
158
159 def svd_error_deg(px: float, py: float, salt: str = "") -> float:
160     """给定地点的固定示向度误差, 落在 [-1°, 1°] (附录2(1)).
161
162     "在一段时间内, 同一地点的电磁环境干扰是固定的, 所以重复检测不会改变
163     检测误差。只有在不同地点、不同电磁环境下, 误差才会呈现统计规律。
164     从目标区域全局来看, 所有这些误差在 [-1°, 1°] 范围内"
165
166     实现: 以 1 cm 网格量化位置, blake2b 摘要 -> uint64 -> U(-1, 1)。
167     因此同点恒定、异点近似独立均匀分布于 [-1°, 1°], 且不依赖调用顺序。
168     """
169     key = f"{salt}|{round(px * 100)}|{round(py * 100)}".encode("utf-8")
170     digest = hashlib.blake2b(key, digest_size=8).digest()
171     u = struct.unpack("<Q", digest)[0] / float(1 << 64)    # [0,1)
172     return (u * 2.0 - 1.0) * SVD_ERROR_LIMIT_DEG
173
174
175 def norm360(deg: float) -> float:
176     return deg % 360.0
177
178
179 def round2(deg: float) -> float:
180     """svd_deg 保留两位小数 (附件2 §2.3)."""
181     return round(deg, 2)
182
183
184 def make_case(
185     *,
186     problem: int,
187     seed: int | None = None,
188     count: int | None = None,
189     case_code: str | None = None,
190     min_speedup_margin: float = 0.0,
191 ) -> Case:
192     """随机生成一局案例。
193
194     problem=3: 全部为全向干扰源。
195     problem=4: 全向与定向混合, 定向源个数未知 (这里取 1..count-1 的随机值)。
196     count 缺省时在 10..16 之间随机。
197     """
198     if problem not in (3, 4):
199         raise ValueError("problem must be 3 or 4")
200     if seed is None:
201         seed = random.SystemRandom().randrange(1 << 63)
202     rng = random.Random(seed)
203
204     if count is None:

```

```

205         count = rng.randint(JAMMER_COUNT_MIN, JAMMER_COUNT_MAX)
206     if not (JAMMER_COUNT_MIN <= count <= JAMMER_COUNT_MAX):
207         raise ValueError(f"count must be in {JAMMER_COUNT_MIN}..{JAMMER_COUNT_MAX}")
208
209     channels = rng.sample(range(CHANNEL_MIN, CHANNEL_MAX + 1), count)
210
211     # 位置采样: 圆域内均匀分布, 但避免与其他干扰源重合(否则频道无法区分),
212     # 也避免贴边导致定位区域退化。
213     jammers: list[Jammer] = []
214     for ch in sorted(channels):
215         while True:
216             r = ARENA_RADIUS_M * math.sqrt(rng.random())
217             th = rng.uniform(0.0, 2.0 * math.pi)
218             x, y = r * math.cos(th), r * math.sin(th)
219             if all(dist(x, y, j.x, j.y) > 40.0 for j in jammers):
220                 break
221             eff = rng.uniform(EFF_RADIUS_MIN_M, EFF_RADIUS_MAX_M)
222             jammers.append(Jammer(channel=ch, x=x, y=y, eff_radius=eff,
223                                   directional=False, bearing_deg=None))
224
225     if problem == 4:
226         # 定向源个数未知: 至少 1 个、至多 count-1 个, 保证两类都存在
227         n_dir = rng.randint(1, count - 1) if count > 1 else 0
228         for j in rng.sample(jammers, n_dir):
229             j.directional = True
230             j.bearing_deg = rng.uniform(0.0, 360.0)
231
232     if case_code is None:
233         case_code = f"P{problem}-{seed:016X}"
234
235     return Case(case_code=case_code, problem=problem, seed=seed, jammers=jammers)
236
237
238 # -----
239 # Session: virtual clock + robot state machine
240 # -----
241
242
243 @dataclass
244 class ActionOutcome:
245     """一次合法动作的结果。delta_* 为该动作消耗的虚拟时间 (秒)。"""
246
247     virtual_time_s: float
248     move_s: float = 0.0
249     switch_s: float = 0.0
250     action_s: float = 0.0
251     payload: dict[str, Any] = field(default_factory=dict)
252
253     @property
254     def total_s(self) -> float:
255         return self.move_s + self.switch_s + self.action_s
256
257
258 class Session:
259     """一局测试的可变状态: 虚拟时钟、机器狗位置、测向机频道、已清除集合。"""
260
261     def __init__(self, case: Case, *, svd_salt: str = "") -> None:
262         self.case = case

```

```

263         self.svd_salt = svd_salt
264         self.case_salt = f"{case.case_code}:{case.seed}"
265
266         self.virtual_time_s = 0.0
267         self.entered = False
268         self.exited = False
269         self.pos: tuple[float, float] = (0.0, 0.0)
270         self.current_channel = 1          # 测向机初始频道 1
271         self.cleared: set[int] = set()    # 已清除的频道
272         self.measure_count = 0
273         self.clear_count = 0
274         self.clear_success = 0
275         self.actions = 0
276
277     # -- helpers -----
278     @property
279     def alive(self) -> bool:
280         return self.entered and not self.exited
281
282     @property
283     def remaining_count(self) -> int:
284         return self.case.total - len(self.cleared)
285
286     def pending(self, channel: int) -> Jammer | None:
287         j = self.case.by_channel(channel)
288         if j is None or j.channel in self.cleared:
289             return None
290         return j
291
292     # -- /enter /exit -----
293     def enter(self) -> ActionOutcome:
294         self.entered = True
295         self.pos = (0.0, 0.0)
296         self.current_channel = 1
297         return ActionOutcome(virtual_time_s=self.virtual_time_s)
298
299     def exit(self) -> ActionOutcome:
300         self.exited = True
301         return ActionOutcome(virtual_time_s=self.virtual_time_s)
302
303     # -- /measure -----
304     def measure(self, x: float, y: float, channel: int) -> ActionOutcome:
305         move_s = dist(*self.pos, x, y) / SPEED_MPS
306         switch_s = 0.0 if channel == self.current_channel else SWITCH_COST_S
307         self._advance(move_s + switch_s + MEASURE_COST_S)
308         self.pos = (x, y)
309         self.current_channel = channel
310         self.measure_count += 1
311
312         payload: dict[str, Any] = {"measure_result": "no_signal"}
313         jammer = self.pending(channel)
314         if jammer is not None:
315             d = dist(x, y, jammer.x, jammer.y)
316             in_range = d <= jammer.eff_radius
317             in_sector = jammer.covered(x, y)
318             if in_range and in_sector:
319                 if d <= NEAR_RANGE_M:
320                     payload = {"measure_result": "near"}

```

```

321         else:
322             true_brg = bearing_deg(x, y, jammer.x, jammer.y)
323             err = svd_error_deg(x, y, salt=self.case_salt + "|" + self.svd_salt)
324             payload = {
325                 "measure_result": "direction",
326                 "svd_deg": round2(norm360(true_brg + err)),
327             }
328         return ActionOutcome(
329             virtual_time_s=self.virtual_time_s,
330             move_s=quantise_time(move_s),
331             switch_s=switch_s,
332             action_s=MEASURE_COST_S,
333             payload=payload,
334         )
335
336     # -- /clear -----
337     def clear(self, x: float, y: float, channel: int) -> ActionOutcome:
338         move_s = dist(*self.pos, x, y) / SPEED_MPS
339         self._advance(move_s + CLEAR_MISS_COST_S) # 先按"未发现"推进
340         self.pos = (x, y)
341         self.clear_count += 1
342
343         jammer = self.pending(channel)
344         hit = jammer is not None and dist(x, y, jammer.x, jammer.y) <= CLEAR_RANGE_M
345         if hit:
346             # 已按 3 s 推进, 命中时补足到 5 s
347             self._advance(CLEAR_FOUND_COST_S - CLEAR_MISS_COST_S)
348             self.cleared.add(channel)
349             self.clear_success += 1
350             return ActionOutcome(
351                 virtual_time_s=self.virtual_time_s,
352                 move_s=quantise_time(move_s),
353                 action_s=CLEAR_FOUND_COST_S,
354                 payload={"clear_result": "success"},
355             )
356         return ActionOutcome(
357             virtual_time_s=self.virtual_time_s,
358             move_s=quantise_time(move_s),
359             action_s=CLEAR_MISS_COST_S,
360             payload={"clear_result": "no_target_in_range"},
361         )
362
363     # -- clock -----
364     def _advance(self, seconds: float) -> None:
365         self.virtual_time_s = quantise_time(self.virtual_time_s + seconds)
366         self.actions += 1
367
368     @property
369     def virtual_expired(self) -> bool:
370         return self.virtual_time_s >= MAX_VIRTUAL_DURATION_S
371
372     # -- 统计量 (题目问题3/问题4 要求) -----
373     def statistics(self, program_runtime_s: float) -> dict[str, Any]:
374         cleared = len(self.cleared)
375         ratio = (cleared / self.case.total) if self.case.total else 0.0
376         avg = (self.virtual_time_s / cleared) if cleared else None
377         return {
378             "case_code": self.case.case_code,

```

```

379         "jammer_total": self.case.total,
380         "cleared_count": cleared,
381         "cleared_ratio": ratio,
382         "total_locate_clear_time_s": self.virtual_time_s,
383         "avg_locate_clear_time_s": avg,
384         "program_runtime_s": program_runtime_s,
385         "measure_count": self.measure_count,
386         "clear_count": self.clear_count,
387         "clear_miss_count": self.clear_count - self.clear_success,
388     }

```

## vendor/mocksim/cli.py

```

1 # -*- coding: utf-8 -*-
2 """统一命令行入口。
3
4     python -m mocksim.cli doctor                # 环境自检
5     python -m mocksim.cli strategies            # 列出可用策略
6     python -m mocksim.cli run --problem 3 --runs 6
7     python -m mocksim.cli serve --problem 3
8     python -m mocksim.cli compare --left BaselineSweep --right MyStrategy
9     python -m mocksim.cli conformance           # 协议一致性断言
10 """
11
12 from __future__ import annotations
13
14 import argparse
15 import json
16 import sys
17 from pathlib import Path
18
19 from . import config as cfg
20 from . import discovery
21 from .harness import Harness, compare
22 from .version import BANNER, FORMAL_TEST_ADVICE, FORMAL_TEST_DEADLINE, __version__
23
24
25 def _add_common(p: argparse.ArgumentParser) -> None:
26     p.add_argument("--problem", type=int, choices=(3, 4),
27                    help="题目编号: 3=全向源, 4=全向+定向混合")
28     p.add_argument("--team", dest="team_id", help="robot_id (默认取 config/team.json)")
29     p.add_argument("--real-limit", type=float, dest="real_limit_s",
30                    help="/enter 后的现实时间上限秒数 (默认 1200)")
31     p.add_argument("--window", type=float, dest="window_s",
32                    help="测试窗口秒数 (默认 1500=25 分钟)")
33
34
35 def _parse_kwargs(text: str | None) -> dict:
36     if not text:
37         return {}
38     try:
39         data = json.loads(text)
40     except json.JSONDecodeError as exc:
41         raise SystemExit(f"--strategy-kwags 必须是合法 JSON: {exc}") from exc
42     if not isinstance(data, dict):
43         raise SystemExit("--strategy-kwags 必须是 JSON 对象, 例如 '{\"coarse_points\":18}'")
44     return data

```

```

45
46
47 def _fmt_strategy_table(found: dict) -> str:
48     if not found:
49         return " (strategies/ 目录里没有找到策略, 参考 strategies/_template.py) "
50     width = max(len(k) for k in found)
51     lines = []
52     for key, info in sorted(found.items()):
53         lines.append(f" {key:<{width}} {info.name:<18} {info.description}")
54         lines.append(f" {'':<{width}} 文件: {info.path.name}")
55     return "\n".join(lines)
56
57
58 def main(argv: list[str] | None = None) -> int:
59     ap = argparse.ArgumentParser(
60         prog="python -m mocksim.cli",
61         description="26B 本地测试环境 (mock 模拟器, 不消耗官方正式测试机会)",
62         formatter_class=argparse.RawDescriptionHelpFormatter,
63         epilog=f"提示: 官方模拟器 {FORMAL_TEST_DEADLINE} (北京时间) 后无法新开测试, "
64               f"建议 {FORMAL_TEST_ADVICE} 前完成正式测试。",
65     )
66     ap.add_argument("--version", action="version", version=f"mocksim {_version_}")
67     sub = ap.add_subparsers(dest="cmd", required=True)
68
69     # doctor
70     d = sub.add_parser("doctor", help="环境自检 (先跑这个)")
71     d.add_argument("--full", action="store_true", help="包含端到端自测")
72     d.add_argument("--conformance", action="store_true", help="包含协议一致性断言")
73     d.add_argument("--port", type=int)
74
75     # strategies
76     sub.add_parser("strategies", help="列出 strategies/ 里的策略")
77
78     # conformance
79     sub.add_parser("conformance", help="协议一致性断言 (对标附件2)")
80
81     # run
82     r = sub.add_parser("run", help="批量演练并出统计/表1/日志")
83     _add_common(r)
84     r.add_argument("--strategy", default=None, help="策略名 (默认取 config/defaults.json)")
85     r.add_argument("--runs", type=int, default=6, help="跑几局 (默认 6)")
86     r.add_argument("--mode", default="practice", choices=("practice", "formal"),
87                   help="practice=演练(显示真值), formal=模拟正式测试(不显示真值)")
88     r.add_argument("--seeds", type=int, nargs="*", help="指定案例 seed, 可复现")
89     r.add_argument("--counts", type=int, nargs="*", help="指定干扰源个数 10..16")
90     r.add_argument("--strategy-kwarg", default=None,
91                   help="策略构造参数 JSON, 例如 '{\"coarse_points\":20}'")
92     r.add_argument("--out", default=None, help="输出目录 (默认 runs/<问题>)")
93     r.add_argument("--quiet", action="store_true")
94     r.add_argument("--json", dest="json_out", action="store_true",
95                   help="最后打印机器可读的汇总 JSON")
96
97     # compare
98     c = sub.add_parser("compare", help="同一批 seed 下对比两个策略")
99     _add_common(c)
100    c.add_argument("--left", default="BaselineSweep")
101    c.add_argument("--right", required=True)
102    c.add_argument("--seeds", type=int, nargs="+", required=True)

```

```

103 c.add_argument("--left-kwargs", default=None)
104 c.add_argument("--right-kwargs", default=None)
105 c.add_argument("--out", default=None)
106
107 # serve
108 s = sub.add_parser("serve", help="起本地模拟器, 接你们自己的机器狗程序")
109 _add_common(s)
110 s.add_argument("--port", type=int, help="监听端口 (默认 2026)")
111 s.add_argument("--seed", type=int, help="固定案例 seed")
112 s.add_argument("--count", type=int, help="干扰源个数 10..16")
113 s.add_argument("--mode", default="practice", choices=("practice", "formal"))
114 s.add_argument("--expose-debug", action="store_true",
115                 help="开放 GET /debug/truth|status|log (仅供人工查看)")
116 s.add_argument("--verbose", action="store_true", help="打印每个 HTTP 请求")
117
118 args = ap.parse_args(argv)
119
120 if args.cmd == "doctor":
121     from .doctor import run_doctor
122     return run_doctor(skip_self_test=not args.full, conformance=args.conformance,
123                       port=args.port)
124
125 if args.cmd == "conformance":
126     from .conformance import main as conf_main
127     return conf_main()
128
129 if args.cmd == "strategies":
130     print(BANNER)
131     print("\n可用策略:\n")
132     print(_fmt_strategy_table(discovery.discover()))
133     print(f"\n策略目录: {discovery.STRATEGY_DIR}")
134     print("新增策略: 复制 strategies/_template.py 改名并在 run() 里实现")
135     return 0
136
137 overrides = {
138     "default_problem": args.problem,
139     "team_id": args.team_id,
140     "real_limit_s": args.real_limit_s,
141     "window_s": args.window_s,
142 }
143 conf = cfg.load(overrides=overrides)
144
145 if args.cmd == "run":
146     out_dir = Path(args.out) if args.out else (conf.resolved_out_dir()
147                                                / f"p{conf.default_problem}")
148     strategy = args.strategy or conf.strategy
149     kwargs = _parse_kwargs(args.strategy_kwargs)
150     print(BANNER)
151     print(f"\nteam_id={conf.team_id!r} 问题={conf.default_problem} "
152           f"模式={args.mode} 策略={strategy}")
153     if args.mode == "formal":
154         print("注意: formal 模式不显示干扰源真值, 统计口径与官方正式测试一致")
155     h = Harness(problem=conf.default_problem, team_id=conf.team_id, out_dir=out_dir,
156                 real_limit_s=conf.real_limit_s, window_s=conf.window_s,
157                 verbose=not args.quiet)
158     try:
159         h.run_batch(args.runs, strategy=strategy, mode=args.mode,
160                     seeds=args.seeds, counts=args.counts, strategy_kwargs=kwargs)

```

```

161     except SystemExit:
162         raise
163     table = h.export_results_table(out_dir / "table1-results.md", mode=args.mode)
164     h.export_json(out_dir / "summary.json")
165
166     agg = h.aggregate()
167     print("\n--- 汇总 ---")
168     if args.mode == "formal":
169         # 正式口径: 真值不可见, 只报接口能观测到的量
170         for r in h.records:
171             avg = f"{r.avg_locate_clear_time_s:.2f}" if r.avg_locate_clear_time_s else "-"
172             print(f" 测试{r.index}: 案例 {r.case_code} | 清除 {r.cleared_count} 个 | "
173                   f"平均定位清除时间 {avg}s | 程序运行时间 {r.program_runtime_s:.2f}s")
174             avgs = [r.avg_locate_clear_time_s for r in h.records if r.avg_locate_clear_time_s]
175             if avgs:
176                 print(f" 平均: 清除 {agg['mean_clear_count']:.1f} 个 | "
177                       f"平均定位清除时间 {sum(avgs) / len(avgs):.2f}s | "
178                       f"平均程序运行时间 {agg['mean_program_runtime_s']:.2f}s")
179             print(" (formal 模式按官方口径不显示干扰源总数, 因此无法给出清除比例)")
180         elif agg.get("mean_cleared_ratio") is not None:
181             print(f" 局数 {agg['runs']} | 平均清除比例 {agg['mean_cleared_ratio']:.3f} "
182                   f"| 最差 {agg['worst_cleared_ratio']:.3f} "
183                   f"| 满分 {agg['full_clear_runs']}/{agg['known_runs']}")
184             print(f" 平均定位清除时间 {agg['mean_avg_locate_clear_time_s']:.1f}s "
185                   f"| 平均检测 {agg['mean_measure_count']:.0f} 次 "
186                   f"| 平均清除 {agg['mean_clear_count']:.1f} 次 "
187                   f"| (落空 {agg['mean_clear_miss_count']:.1f})")
188         print(f" 结束原因分布: {agg.get('end_reasons')}")
189         if agg.get("runs_with_error"):
190             print(f" [WARN] {agg['runs_with_error']} 局出现异常, 见下方诊断")
191
192         print("\n--- 诊断 ---")
193         print(h.diagnosis() or "(无诊断信息)")
194         print(f"\n结果表: {table}")
195         print(f"汇总: {h.export_json(out_dir / 'summary.json')}")
196         print(f"日志: {out_dir}\\*.jsonl")
197         if args.json_out:
198             print("\n--- JSON ---")
199             print(json.dumps(agg, ensure_ascii=False, indent=2))
200         return 1 if agg.get("runs_with_error") else 0
201
202     if args.cmd == "compare":
203         report = compare(args.seeds, problem=conf.default_problem, team_id=conf.team_id,
204                          left=args.left, right=args.right,
205                          left_kwargs=_parse_kwargs(args.left_kwargs),
206                          right_kwargs=_parse_kwargs(args.right_kwargs),
207                          out_dir=Path(args.out) if args.out else conf.resolved_out_dir() / "
208                          compare")
209         print(BANNER)
210         print()
211         print(report)
212         return 0
213
214     if args.cmd == "serve":
215         port = args.port if args.port is not None else conf.port
216         from .server import MockSimulator, serve as serve_http
217         sim = MockSimulator(problem=conf.default_problem, mode=args.mode, seed=args.seed,
218                              count=args.count, team_id=conf.team_id, port=port,

```

```

218         real_limit_s=conf.real_limit_s, window_s=conf.window_s)
219     httpd = serve_http(sim, expose_debug=args.expose_debug, verbose=args.verbose)
220     sim.start()
221     print(BANNER)
222     print(f"\nmock 模拟器已启动: http://127.0.0.1:{port}")
223     print(f"   team_id (robot_id) = {conf.team_id!r}")
224     print(f"   问题={conf.default_problem}   模式={args.mode}   案例={sim.case.case_code}")
225     print(f"   接口已开放, 等机器狗调用 /enter")
226     if args.expose_debug:
227         print(f"   调试: http://127.0.0.1:{port}/debug/truth")
228     print("   停止: Ctrl+C")
229     try:
230         httpd.serve_forever()
231     except KeyboardInterrupt:
232         print("\n收到 Ctrl+C, 正在退出...")
233     finally:
234         sim.tick()
235         httpd.shutdown()
236         httpd.server_close()
237         print("\n--- 本局结果 ---")
238         print(json.dumps(sim.result_summary(), ensure_ascii=False, indent=2))
239     return 0
240
241     ap.error("未知命令")
242     return 2
243
244
245 if __name__ == "__main__":
246     sys.exit(main())

```

## vendor/mocksim/client.py

```

1 # -*- coding: utf-8 -*-
2 """机器狗客户端与几何工具 (不含具体策略)。
3
4 协议规则按 附件2 §5 / §12 实现:
5
6 * 每个新动作使用新的 ``request_id``
7 * 只在"超时 / 连接中断后重发同一动作"时复用原 payload 与原 id
8 * 严格串行: 同一时刻只有一个在途动作, 必须等完整响应
9 * 同时检查 HTTP 状态与 ``accepted``
10 * 连接被直接关闭 (无 JSON 体) 视为可重试, 不当作错误响应
11 * 使用 /enter 返回的 ``remaining_real_duration_s``, 不假定每次都有 1200 s
12 * 只在 ``measure_result == "direction"`` 时读取 ``svd_deg``
13 * ``no_signal`` 不代表附近没有干扰源
14
15 具体搜索/清除策略请放在 ``strategies/`` 目录里, 复制 ``strategies/_template.py`` 开始写;
16 运行入口统一是 ``python -m mocksim.cli``。
17 """
18
19 from __future__ import annotations
20
21 import http.client
22 import json
23 import math
24 import time
25 import urllib.error

```

```

26 import urllib.request
27 from dataclasses import dataclass
28 from typing import Any, Callable
29
30 ARENA_RADIUS_M = 1800.0
31
32 #: 回环地址的请求必须直连: 系统代理 (HTTP_PROXY/HTTPS_PROXY) 会把 127.0.0.1
33 #: 的请求转发出去, 表现为 502 Bad Gateway 或 407, 让人误以为是模拟器故障。
34 NO_PROXY = ("127.0.0.1", "localhost", "::1", "[::1]")
35
36
37 def make_opener() -> urllib.request.OpenerDirector:
38     """返回一个明确绕过系统代理的 opener。"""
39     return urllib.request.build_opener(urllib.request.ProxyHandler({}))
40
41
42 # -----
43 # 异常
44 # -----
45
46
47 class ProtocolError(RuntimeError):
48     """模拟器返回 HTTP 400/409/413/415/429/500 等硬错误。"""
49
50     def __init__(self, status: int, body: Any) -> None:
51         super().__init__(f"HTTP {status}: {body}")
52         self.status = status
53         self.body = body
54
55
56 class TransportFailure(RuntimeError):
57     """连不上, 或连接被直接关闭 (接口未开放 / 测试已结束)。"""
58
59
60 class Rejected(RuntimeError):
61     """HTTP 200 但 accepted=false: 动作没有生效, virtual_time_s 不可信。"""
62
63     def __init__(self, response: dict[str, Any]) -> None:
64         super().__init__(f"accepted=false: {response}")
65         self.response = response
66
67
68 @dataclass
69 class ClientStats:
70     requests: int = 0
71     retries: int = 0
72     http_errors: int = 0
73     rejected: int = 0
74     transport_failures: int = 0
75
76     def as_dict(self) -> dict[str, int]:
77         return {
78             "requests": self.requests,
79             "retries": self.retries,
80             "http_errors": self.http_errors,
81             "rejected": self.rejected,
82             "transport_failures": self.transport_failures,
83         }

```

```

84
85
86 # -----
87 # 客户端
88 # -----
89
90
91 class RobotClient:
92     """逐次等待响应的 HTTP+JSON 客户端。"""
93
94     def __init__(self, *, base_url: str = "http://127.0.0.1:2026",
95                  robot_id: str = "TEAM0000", timeout_s: float = 5.0,
96                  retries: int = 5, retry_backoff_s: float = 0.05,
97                  log_sink: Callable[[dict[str, Any]], None] | None = None,
98                  use_system_proxy: bool = False) -> None:
99         self.base_url = base_url.rstrip("/")
100         self.robot_id = robot_id
101         self.timeout_s = timeout_s
102         self.retries = retries
103         self.retry_backoff_s = retry_backoff_s
104         self.stats = ClientStats()
105         self.counter = 0
106         self.log_sink = log_sink
107         self.log: list[dict[str, Any]] = []
108         self.opener = (urllib.request.build_opener() if use_system_proxy
109                        else make_opener())
110
111     # -- transport -----
112     def _next_id(self, kind: str) -> str:
113         self.counter += 1
114         return f"{kind}--{self.counter}"
115
116     def post(self, path: str, payload: dict[str, Any]) -> dict[str, Any]:
117         """发送一个动作并等待完整响应；相同内容与 id 会按需重试。"""
118         raw = json.dumps(payload, ensure_ascii=False).encode("utf-8")
119         request = urllib.request.Request(
120             self.base_url + path, data=raw, method="POST",
121             headers={"Content-Type": "application/json"},
122         )
123         attempt = 0
124         while True:
125             attempt += 1
126             self.stats.requests += 1
127             try:
128                 with self.opener.open(request, timeout=self.timeout_s) as resp:
129                     body = json.loads(resp.read().decode("utf-8"))
130                     status = resp.status
131             except urllib.error.HTTPError as exc:
132                 detail = exc.read().decode("utf-8", "replace")
133                 try:
134                     body = json.loads(detail)
135                 except json.JSONDecodeError:
136                     body = {"detail": detail}
137                 self.stats.http_errors += 1
138                 self._log(path, payload, exc.code, body)
139                 raise ProtocolError(exc.code, body) from None
140             except (urllib.error.URLError, ConnectionError, TimeoutError, OSError,
141                    http.client.HTTPException, ValueError) as exc:

```

```

142         # 接口未开放 / 测试已结束 / 网络抖动: 连接被直接关闭, 没有 JSON 体。
143         # 复用完全相同的 payload 与 request_id 重试 (附件2 §5.3)。
144         self.stats.transport_failures += 1
145         if attempt > self.retries:
146             raise TransportFailure(f"{type(exc).__name__}: {exc}") from exc
147         self.stats.retries += 1
148         time.sleep(self.retry_backoff_s * attempt)
149         continue
150     self._log(path, payload, status, body)
151     if body.get("accepted") is not True:
152         self.stats.rejected += 1
153     return body
154
155 def _log(self, path: str, payload: dict[str, Any], status: int,
156         body: dict[str, Any]) -> None:
157     entry = {
158         "wall_ms": int(time.time() * 1000),
159         "path": path,
160         "http_status": status,
161         "request": payload,
162         "response": body,
163     }
164     self.log.append(entry)
165     if self.log_sink is not None:
166         self.log_sink(entry)
167
168 # -- actions -----
169 def _base(self, kind: str) -> dict[str, Any]:
170     return {"arena_id": "default", "robot_id": self.robot_id,
171           "request_id": self._next_id(kind)}
172
173 def enter(self) -> dict[str, Any]:
174     return self.post("/enter", self._base("enter"))
175
176 def exit(self) -> dict[str, Any]:
177     return self.post("/exit", self._base("exit"))
178
179 def measure(self, x: float, y: float, channel: int) -> dict[str, Any]:
180     p = self._base("measure")
181     p["position"] = {"x": _num(x), "y": _num(y)}
182     p["channel"] = int(channel)
183     return self.post("/measure", p)
184
185 def clear(self, x: float, y: float, channel: int) -> dict[str, Any]:
186     p = self._base("clear")
187     p["position"] = {"x": _num(x), "y": _num(y)}
188     p["channel"] = int(channel)
189     return self.post("/clear", p)
190
191 # -- 断言 accepted 的包装 -----
192 def measure_checked(self, x: float, y: float, channel: int) -> dict[str, Any]:
193     r = self.measure(x, y, channel)
194     if r.get("accepted") is not True:
195         raise Rejected(r)
196     return r
197
198 def clear_checked(self, x: float, y: float, channel: int) -> dict[str, Any]:
199     r = self.clear(x, y, channel)

```

```

200         if r.get("accepted") is not True:
201             raise Rejected(r)
202         return r
203
204     # -- 便捷读取 -----
205     @staticmethod
206     def virtual_time(resp: dict[str, Any], fallback: float = 0.0) -> float:
207         """只在 accepted=true 时可信; 否则返回 fallback。"""
208         if resp.get("accepted") is True:
209             return float(resp.get("virtual_time_s", fallback))
210         return fallback
211
212     @staticmethod
213     def svd(resp: dict[str, Any]) -> float | None:
214         """只有 measure_result == "direction" 时才有示向度。"""
215         if resp.get("measure_result") == "direction":
216             return float(resp["svd_deg"])
217         return None
218
219
220     def _num(v: float) -> float | int:
221         """JSON 里整数坐标更好读, 但必须是有限数值。"""
222         if float(v).is_integer():
223             return int(v)
224         return round(float(v), 6)
225
226
227 # -----
228 # 方位几何工具
229 # -----
230
231
232     def norm360(d: float) -> float:
233         return d % 360.0
234
235
236     def angular_diff(a: float, b: float) -> float:
237         """两个方位角的最小夹角, 单位度, 范围 [0,180]。"""
238         d = abs(norm360(a) - norm360(b)) % 360.0
239         return d if d <= 180.0 else 360.0 - d
240
241
242     @dataclass
243     class Bearing:
244         x: float
245         y: float
246         deg: float
247         virtual_time_s: float = 0.0
248
249         def as_line(self) -> tuple[float, float, float]:
250             """返回 (n_x, n_y, c) 使  $n \cdot p = c$  表示这条方位线 (n 为单位法向)。"""
251             rad = math.radians(self.deg)
252             nx, ny = -math.sin(rad), math.cos(rad)
253             return nx, ny, nx * self.x + ny * self.y
254
255
256     class Estimator:
257         """由若干条示向度直线做最小二乘交会, 得到干扰源位置估计。

```

```

258
259 ±1° 的示向度误差下，交会精度取决于基线与交角：基线越长、交角越接近 90°，
260 估计越准。``estimate`` 默认只用最后两条（帮助策略快速逼近），
261 ``estimate_all`` 用全部观测（帮助收敛到更精确的位置）。
262 """
263
264 def __init__(self) -> None:
265     self.bearings: dict[int, list[Bearing]] = {}
266
267 def add(self, channel: int, x: float, y: float, deg: float,
268         virtual_time_s: float = 0.0) -> None:
269     self.bearings.setdefault(channel, []).append(Bearing(x, y, deg, virtual_time_s))
270
271 def latest(self, channel: int) -> Bearing | None:
272     lst = self.bearings.get(channel)
273     return lst[-1] if lst else None
274
275 def count(self, channel: int) -> int:
276     return len(self.bearings.get(channel) or [])
277
278 def channels(self) -> list[int]:
279     return sorted(self.bearings)
280
281 @staticmethod
282 def _solve(pts: list[Bearing]) -> tuple[float, float, float] | None:
283     a11 = a12 = a22 = b1 = b2 = 0.0
284     for p in pts:
285         nx, ny, c = p.as_line()
286         a11 += nx * nx
287         a12 += nx * ny
288         a22 += ny * ny
289         b1 += nx * c
290         b2 += ny * c
291     det = a11 * a22 - a12 * a12
292     if abs(det) < 1e-12:
293         return None
294     x = (b1 * a22 - b2 * a12) / det
295     y = (a11 * b2 - a12 * b1) / det
296     acc = 0.0
297     for p in pts:
298         nx, ny, c = p.as_line()
299         acc += (nx * x + ny * y - c) ** 2
300     return x, y, math.sqrt(acc / len(pts))
301
302 def estimate(self, channel: int) -> tuple[float, float, float] | None:
303     """用最后两条方位线交会；不足两条返回 None。"""
304     pts = self.bearings.get(channel) or []
305     return self._solve(pts[-2:]) if len(pts) >= 2 else None
306
307 def estimate_all(self, channel: int) -> tuple[float, float, float] | None:
308     """用该频道全部方位线交会。"""
309     pts = self.bearings.get(channel) or []
310     return self._solve(pts) if len(pts) >= 2 else None
311
312 def baseline(self, channel: int) -> float:
313     """最长观测基线长度（米）。"""
314     pts = self.bearings.get(channel) or []
315     best = 0.0

```

```

316         for i in range(len(pts)):
317             for j in range(i + 1, len(pts)):
318                 best = max(best, math.hypot(pts[i].x - pts[j].x, pts[i].y - pts[j].y))
319         return best
320
321
322 # -----
323 # 策略运行结果
324 # -----
325
326
327 @dataclass
328 class StrategyResult:
329     """策略自报的结果（框架只用 note 做诊断，真实统计以模拟器为准）。"""
330
331     cleared: int = 0
332     attempts: int = 0
333     channels_seen: list[int] | None = None
334     note: str = ""
335
336     def __post_init__(self) -> None:
337         if self.channels_seen is None:
338             self.channels_seen = []

```

## vendor/mocksim/config.py

```

1 # -*- coding: utf-8 -*-
2 """配置加载：分层合并 CLI 参数 > 环境变量 > 用户配置 > 默认配置。"""
3
4 from __future__ import annotations
5
6 import json
7 import os
8 from dataclasses import dataclass, field
9 from pathlib import Path
10 from typing import Any
11
12 def _package_root() -> Path:
13     """包根目录（26B-team/）。
14
15     允许用环境变量 ``MOCKSIM_PKG_ROOT`` 覆盖，这样打包脚本可以在"暂存目录还没被
16     安装/复制成最终位置"时，直接对这个目录生成配置模板。
17     """
18     override = os.environ.get("MOCKSIM_PKG_ROOT")
19     if override:
20         return Path(override).resolve()
21     return Path(__file__).resolve().parent.parent
22
23
24 #: 包根目录（26B-team/）
25 ROOT = _package_root()
26 CONFIG_DIR = ROOT / "config"
27 TEAM_CONFIG = CONFIG_DIR / "team.json"
28 DEFAULTS_CONFIG = CONFIG_DIR / "defaults.json"
29
30 TEAM_CONFIG_TEMPLATE: dict[str, Any] = {
31     "_说明": "每位队员把 team_id 改成自己在竞赛系统里的参赛队号；该文件不要提交到公共仓库",

```

```

32     "team_id": "TEAM0000",
33     "team_name": "",
34     "member": "",
35     "team_check_enabled": True,
36 }
37
38 DEFAULTS_CONFIG_TEMPLATE: dict[str, Any] = {
39     "_说明": "通用默认值; 命令行参数优先级更高",
40     "default_problem": 3,
41     "port": 2026,
42     "real_limit_s": 1200,
43     "window_s": 1500,
44     "out_dir": "runs",
45     "seed": None,
46     "count": None,
47     "strategy": "BaselineSweep",
48     "verbose": True,
49 }
50
51
52 def _read_text(path: Path) -> str:
53     """读取配置文本; 用 utf-8-sig 兼容带不带 BOM 的文件。
54
55     Windows 上用 PowerShell 的 Set-Content/Out-File 写 JSON 很容易带上 BOM,
56     而 JSON 解析器不接受 BOM, 所以这里统一按 utf-8-sig 解码。
57     """
58     return path.read_text(encoding="utf-8-sig")
59
60
61 def _read_json(path: Path, template: dict[str, Any], *, create: bool) -> dict[str, Any]:
62     if not path.exists():
63         if create:
64             path.parent.mkdir(parents=True, exist_ok=True)
65             path.write_text(json.dumps(template, ensure_ascii=False, indent=2),
66                             encoding="utf-8")
67             return dict(template)
68         try:
69             data = json.loads(_read_text(path))
70         except (json.JSONDecodeError, UnicodeDecodeError) as exc:
71             raise SystemExit(
72                 f"配置文件格式错误: {path}\n {exc}\n"
73                 f" 提示: JSON 文件必须是不带 BOM 的 UTF-8; 删掉这个文件重新运行 doctor "
74                 f"会自动生成模板。"
75             ) from exc
76     if not isinstance(data, dict):
77         raise SystemExit(f"配置文件必须是 JSON 对象: {path}")
78     return {k: v for k, v in data.items() if not k.startswith("_")}
79
80
81 @dataclass
82 class Config:
83     team_id: str = "TEAM0000"
84     team_name: str = ""
85     member: str = ""
86     team_check_enabled: bool = True
87     default_problem: int = 3
88     port: int = 2026
89     real_limit_s: float = 1200.0

```

```

90     window_s: float = 1500.0
91     out_dir: str = "runs"
92     seed: int | None = None
93     count: int | None = None
94     strategy: str = "BaselineSweep"
95     verbose: bool = True
96     sources: list[str] = field(default_factory=list)
97
98     def resolved_out_dir(self) -> Path:
99         p = Path(self.out_dir)
100         return p if p.is_absolute() else ROOT / p
101
102
103 def load(*, create_missing: bool = True, overrides: dict[str, Any] | None = None) -> Config:
104     """读取配置并合并覆盖值（通常来自命令行）。"""
105     defaults = _read_json(DEFAULTS_CONFIG, DEFAULTS_CONFIG_TEMPLATE, create=create_missing)
106     team = _read_json(Team_CONFIG, Team_CONFIG_TEMPLATE, create=create_missing)
107
108     merged: dict[str, Any] = {}
109     sources: list[str] = []
110     merged.update(defaults)
111     sources.append(str(DEFAULTS_CONFIG))
112     # 环境变量可覆盖（便于 CI / 多队员共用一台机器）
113     env_map = {
114         "DSH_B_TEAM_ID": "team_id",
115         "DSH_B_PROBLEM": "default_problem",
116         "DSH_B_PORT": "port",
117     }
118     for env_key, cfg_key in env_map.items():
119         if os.environ.get(env_key):
120             merged[cfg_key] = os.environ[env_key]
121             sources.append(f"env:{env_key}")
122     merged.update(team)
123     sources.append(str(Team_CONFIG))
124     if overrides:
125         merged.update({k: v for k, v in overrides.items() if v is not None})
126         sources.append("command line")
127
128     cfg = Config(**{k: v for k, v in merged.items() if k in Config.__dataclass_fields__})
129     cfg.sources = sources
130     cfg.port = int(cfg.port)
131     cfg.default_problem = int(cfg.default_problem)
132     cfg.real_limit_s = float(cfg.real_limit_s)
133     cfg.window_s = float(cfg.window_s)
134     if cfg.seed is not None:
135         cfg.seed = int(cfg.seed)
136     if cfg.count is not None:
137         cfg.count = int(cfg.count)
138     return cfg
139
140
141 def ensure_config_files() -> list[Path]:
142     """确保 config/ 下的模板文件存在，返回新建的文件列表。"""
143     created: list[Path] = []
144     for path, tpl in ((Team_CONFIG, Team_CONFIG_TEMPLATE),
145                       (DEFAULTS_CONFIG, DEFAULTS_CONFIG_TEMPLATE)):
146         if not path.exists():
147             path.parent.mkdir(parents=True, exist_ok=True)

```

```

148         path.write_text(json.dumps(tpl, ensure_ascii=False, indent=2),
149                           encoding="utf-8")
150         created.append(path)
151     return created
152
153
154 def team_is_configured() -> bool:
155     """team_id 是否已从占位值改成真实参赛队号。"""
156     if not TEAM_CONFIG.exists():
157         return False
158     try:
159         data = json.loads(_read_text(TEAM_CONFIG))
160     except Exception:
161         return False
162     tid = str(data.get("team_id", ""))
163     return bool(tid) and tid != TEAM_CONFIG_TEMPLATE["team_id"]

```

### vendor/mocksim/conformance.py

```

1 # -*- coding: utf-8 -*-
2 """Protocol conformance suite -- checks the *mock simulator* against 附件2.
3
4 Every assertion quotes the specification clause it enforces, so this file doubles
5 as an executable reading of the contract. It validates the harness itself:
6 if the mock disagrees with the written protocol, a robot tuned here would
7 misbehave on the real simulator.
8
9 Usage: python -m mocksim.conformance
10 """
11
12 from __future__ import annotations
13
14 import http.client
15 import json
16 import math
17 import threading
18 import time
19 import urllib.error
20 import urllib.request
21 from typing import Any
22
23 from . import arena as A
24 from .server import MockSimulator, serve
25
26 TEAM = "2026TEST0001"
27
28 #: 回环地址必须绕过系统代理, 否则 HTTP_PROXY 会把请求转出去 (502/407)。
29 OPENER = urllib.request.build_opener(urllib.request.ProxyHandler({}))
30
31 _ID_SEQ = 0
32
33
34 def _next_request_id() -> str:
35     global _ID_SEQ
36     _ID_SEQ += 1
37     return f"auto-{_ID_SEQ}"
38

```

```

39
40 class Runner:
41     def __init__(self) -> None:
42         self.passed = 0
43         self.failed: list[str] = []
44         self.notes: list[str] = []
45
46     def check(self, name: str, condition: bool, detail: str = "") -> None:
47         if condition:
48             self.passed += 1
49             print(f" [PASS] {name}")
50         else:
51             self.failed.append(f"{name} {detail}".strip())
52             print(f" [FAIL] {name} {detail}")
53
54     def eq(self, name: str, got: Any, want: Any) -> None:
55         self.check(name, got == want, f"(got {got!r}, want {want!r})")
56
57     def close(self, name: str, got: float, want: float, tol: float = 1e-6) -> None:
58         self.check(name, abs(got - want) <= tol, f"(got {got!r}, want {want!r})")
59
60
61 def raw_post(url: str, path: str, body: bytes, *, ctype: str = "application/json",
62             extra_headers: dict[str, str] | None = None) -> tuple[int, str]:
63     """返回 (http_status, body_text); 连接被直接关闭时返回 (-1, '')."""
64     headers = {"Content-Type": ctype}
65     if extra_headers:
66         headers.update(extra_headers)
67     req = urllib.request.Request(url + path, data=body, method="POST", headers=headers)
68     try:
69         with OPENER.open(req, timeout=5) as resp:
70             return resp.status, resp.read().decode("utf-8")
71     except urllib.error.HTTPError as exc:
72         return exc.code, exc.read().decode("utf-8", "replace")
73     except (urllib.error.URLError, ConnectionError, OSError, http.client.HTTPException):
74         # 接口未开放 / 测试已结束: 连接被直接关闭, 没有 JSON 体
75         return -1, ""
76
77
78 def jpost(url: str, path: str, payload: dict[str, Any]) -> tuple[int, dict[str, Any]]:
79     status, text = raw_post(url, path, json.dumps(payload).encode("utf-8"))
80     try:
81         return status, json.loads(text)
82     except json.JSONDecodeError:
83         return status, {}
84
85
86 def body(**over: Any) -> dict[str, Any]:
87     """请求体模板。默认给一个全新的 request_id: 规范要求每个新动作用新的 id,
88     重复使用会被幂等规则判为 409, 从而掩盖真正要测的校验分支。"""
89     base = {"arena_id": "default", "robot_id": TEAM,
90           "request_id": _next_request_id()}
91     base.update(over)
92     return base
93
94
95 def ride(**over: Any) -> dict[str, Any]:
96     """measure/clear 请求体。"""

```

```

97     b = body()
98     b.update({"position": {"x": 0, "y": 0}, "channel": 1})
99     b.update(over)
100     return b
101
102
103 def main() -> int:
104     r = Runner()
105     sim = MockSimulator(problem=3, mode="practice", seed=20260913, count=12,
106                         team_id=TEAM, port=0, svd_salt="conformance")
107     httpd = serve(sim)
108     port = httpd.server_address[1]
109     url = f"http://127.0.0.1:{port}"
110     threading.Thread(target=httpd.serve_forever, daemon=True).start()
111
112     try:
113         # =====
114         print("\n[1] 接口未开放 / 测试未开始 (附件2 §5.3: 连接可能直接关闭)")
115         st, _ = jpost(url, "/measure", ride())
116         r.eq("/measure before /enter closes the connection", st, -1)
117
118         # 从第 2 节起接口必须已经开放, 否则所有请求都会因接口未开放而失败。
119         sim.start()
120         st, resp = jpost(url, "/enter", body(request_id="boot"))
121         r.eq("interface opens and /enter succeeds", resp.get("accepted"), True)
122
123         # =====
124         print("\n[2] HTTP 方法 / 路径 (附件2 §5.3: 404, 405)")
125         st, text = raw_post(url, "/enter/", json.dumps(body()).encode("utf-8"))
126         r.eq("trailing slash -> 404", st, 404)
127         st, text = raw_post(url, "/enter?x=1", json.dumps(body()).encode("utf-8"))
128         r.eq("query string -> 404", st, 404)
129         st, text = raw_post(url, "/nope", json.dumps(body()).encode("utf-8"))
130         r.eq("unknown path -> 404", st, 404)
131         req = urllib.request.Request(url + "/enter", method="GET")
132         try:
133             with OPENER.open(req, timeout=5) as resp:
134                 st = resp.status
135         except urllib.error.HTTPError as exc:
136             st = exc.code
137         r.eq("GET on known path -> 405", st, 405)
138
139         # =====
140         print("\n[3] Content-Type / Content-Encoding (附件2 §5.1: 415)")
141         st, _ = raw_post(url, "/enter", b"{}", ctype="text/plain")
142         r.eq("wrong media type -> 415", st, 415)
143         st, _ = raw_post(url, "/enter", b"{}", ctype="application/json; charset=gbk")
144         r.eq("charset=gbk -> 415", st, 415)
145         st, _ = raw_post(url, "/enter", b"{}",
146                         ctype="application/json; charset=utf-8; foo=1")
147         r.eq("extra contentType param -> 415", st, 415)
148         st, _ = raw_post(url, "/enter", b"{}", extra_headers={"Content-Encoding": "gzip"})
149         r.eq("Content-Encoding: gzip -> 415", st, 415)
150
151         # =====
152         print("\n[4] JSON 语法 / 重复键 / 深度 (附件2 §5.1: 400)")
153         st, _ = raw_post(url, "/enter", b"{not json}")
154         r.eq("invalid JSON -> 400", st, 400)

```

```

155     st, _ = raw_post(url, "/enter", b'{"arena_id":"default","arena_id":"default"}')
156     r.eq("duplicate key -> 400", st, 400)
157     st, _ = raw_post(url, "/enter", "\uffff".encode("utf-8") + b"{}")
158     r.eq("UTF-8 BOM -> 400", st, 400)
159     deep = b'{"a":"' * 20 + b'1' + b'}' * 20
160     st, _ = raw_post(url, "/enter", deep)
161     r.eq("nesting > 16 -> 400", st, 400)
162     st, _ = raw_post(url, "/enter", b'["array"]')
163     r.eq("non-object top level -> 400", st, 400)
164
165     # =====
166     print("\n[5] /enter 成功路径 (附件2 §6.2)")
167     # 重新开一局, 使 /enter 处于"尚未进入"的状态
168     sim.new_case(seed=20260913, count=12)
169     sim.start()
170     st, resp = jpost(url, "/enter", body(request_id="enter-a"))
171     r.eq("/enter -> 200", st, 200)
172     r.eq("accepted true", resp.get("accepted"), True)
173     r.eq("max_virtual_duration_s default", resp.get("max_virtual_duration_s"), 360000)
174     r.eq("max_real_duration_s default", resp.get("max_real_duration_s"), 1200)
175     r.check("remaining_real_duration_s in 0..1200",
176            0 <= resp.get("remaining_real_duration_s", -1) <= 1200,
177            f"(got {resp.get('remaining_real_duration_s')})")
178     r.eq("virtual_time_s unchanged by /enter", resp.get("virtual_time_s"), 0)
179     r.check("real_timestamp_ms looks like epoch ms",
180            resp.get("real_timestamp_ms", 0) > 1_700_000_000_000)
181
182     # =====
183     print("\n[6] 未知字段 -> 200 accepted=false (附件2 §5.1/§5.3)")
184     st, resp = jpost(url, "/measure", ride(channel=3))
185     r.eq("typo field -> HTTP 200", st, 200)
186     r.eq("typo field -> accepted false", resp.get("accepted"), False)
187     r.eq("accepted=false -> virtual_time_s 0", resp.get("virtual_time_s"), 0)
188     r.check("accepted=false body has only the 3 base fields",
189            set(resp) <= {"accepted", "real_timestamp_ms", "virtual_time_s", "detail"},
190            f"(got {sorted(resp)})")
191     st, resp = jpost(url, "/measure", ride(position={"x": 0, "y": 0, "z": 0}))
192     r.eq("unknown position field -> accepted false", resp.get("accepted"), False)
193
194     # =====
195     print("\n[7] arena_id / robot_id 不匹配 -> 200 accepted=false (附件2 §5.1)")
196     st, resp = jpost(url, "/measure", ride(arena_id="other"))
197     r.eq("arena_id != default -> accepted false", resp.get("accepted"), False)
198     st, resp = jpost(url, "/measure", ride(robot_id="SOMEONEELSE"))
199     r.eq("robot_id mismatch -> accepted false", resp.get("accepted"), False)
200
201     # =====
202     print("\n[8] 字段类型 / 取值 -> 400 (附件2 §5.1, §7.1)")
203     # 每个新动作都必须用新的 request_id, 否则会先触发 409 幂等冲突
204     r.eq("channel 0 -> 400",
205          jpost(url, "/measure", ride(request_id="v-ch0", channel=0))[0], 400)
206     r.eq("channel 21 -> 400",
207          jpost(url, "/measure", ride(request_id="v-ch21", channel=21))[0], 400)
208     r.eq("channel 1.5 -> 400",
209          jpost(url, "/measure", ride(request_id="v-ch15", channel=1.5))[0], 400)
210     r.eq("channel 1.0 -> accepted",
211          jpost(url, "/measure", ride(request_id="v-ch10", channel=1.0))[1].get("accepted"),
212          True)

```

```

213 r.eq("missing channel -> 400",
214     jpost(url, "/measure", body(request_id="v-noch"))[0], 400)
215 r.eq("x over 2e6 -> 400",
216     jpost(url, "/measure", ride(request_id="v-xbig",
217                                 position={"x": 2_000_001, "y": 0}))[0], 400)
218 r.eq("x == 2e6 accepted",
219     jpost(url, "/measure", ride(request_id="v-xlim",
220                                 position={"x": 2_000_000, "y": 0}))[1].get("accepted"),
221     True)
222 # 这一步移动 2e6 米 = 4e5 秒虚拟时间, 超过 360000 秒上限, 会把本局结束
223 # (附件2 §4.5)。后续校验需要一局新的测试。
224 print("    (x == 2e6 消耗 4e5 s 虚拟时间并触发虚拟超时, 属预期行为)")
225 r.eq("virtual-time cap ends the run",
226     sim.live.end_reason if sim.live else None, "virtual_timeout")
227 sim.new_case(seed=20260914, count=12)
228 sim.start()
229
230 print("\n[8b] 标识符与坐标合法性 -> 400 (附件2 §5.1)")
231 st, _ = raw_post(url, "/measure",
232                 b'{"arena_id":"default","robot_id":"2026TEST0001",'
233                 b'"request_id":"r-nan","position":{"x":NaN,"y":0},"channel":1}')
234 r.eq("NaN coordinate -> 400", st, 400)
235 st, _ = raw_post(url, "/measure",
236                 b'{"arena_id":"default","robot_id":"2026TEST0001",'
237                 b'"request_id":"r-inf","position":{"x":1e999,"y":0},"channel":1}')
238 r.eq("Infinity coordinate -> 400", st, 400)
239 r.eq("empty robot_id -> 400",
240     jpost(url, "/enter", body(request_id="v-rid0", robot_id=""))[0], 400)
241 r.eq("robot_id with control char -> 400",
242     jpost(url, "/enter", body(request_id="v-rid1", robot_id="ab\u0001cd"))[0], 400)
243 r.eq("request_id of 129 bytes -> 400",
244     jpost(url, "/enter", body(request_id="x" * 129))[0], 400)
245 r.eq("request_id of 128 bytes is within the limit",
246     jpost(url, "/enter", body(request_id="y" * 128))[0], 200)
247 r.eq("int position accepted",
248     jpost(url, "/measure", ride(request_id="v-int",
249                                 position={"x": 1, "y": 2}))[1].get("accepted"),
250     True)
251
252 # =====
253 print("\n[9] 请求体大小 -> 413 (附件2 §5.1)")
254 big = json.dumps({"arena_id": "default", "robot_id": TEAM,
255                  "request_id": "big", "pad": "z" * 70000}).encode()
256 r.eq("body > 65536 bytes -> 413", raw_post(url, "/enter", big)[0], 413)
257
258 # =====
259 print("\n[10] 虚拟计时 (附件2 §10 完整计时示例)")
260 # 重开一局, 使用规范示例的动作序列
261 sim2 = MockSimulator(problem=3, mode="practice", seed=7, count=10, team_id=TEAM)
262 sim2.start()
263 s = sim2.live.session
264 # 用与规范相同的四个动作, 直接驱动状态机以便精确比对
265 calls = [("measure", 300, 400, 1, 105.0, 100.0, 0.0, 5.0),
266          ("measure", 300, 400, 2, 111.0, 0.0, 1.0, 5.0),
267          ("clear", 300, 0, 3, 194.0, 80.0, 0.0, 3.0),
268          ("measure", 300, 0, 2, 199.0, 0.0, 0.0, 5.0)]
269 s.enter()
270 for kind, x, y, ch, vt, mv, sw, act in calls:

```

```

271         o = s.measure(x, y, ch) if kind == "measure" else s.clear(x, y, ch)
272         r.close(f"{kind} ch{ch} virtual_time_s", o.virtual_time_s, vt)
273         r.close(f"{kind} ch{ch} move_s", o.move_s, mv)
274         r.close(f"{kind} ch{ch} switch_s", o.switch_s, sw)
275         r.close(f"{kind} ch{ch} action_s", o.action_s, act)
276
277     # =====
278     print("\n[11] 幂等规则 (附件2 §5.3)")
279     sim3 = MockSimulator(problem=3, mode="practice", seed=11, count=10, team_id=TEAM,
280                          port=0)
281     httpd3 = serve(sim3)
282     url3 = f"http://127.0.0.1:{httpd3.server_address[1]}"
283     threading.Thread(target=httpd3.serve_forever, daemon=True).start()
284     sim3.start()
285     jpost(url3, "/enter", body(request_id="e1"))
286     p = ride(request_id="m1", position={"x": 500, "y": 0}, channel=1)
287     st1, resp1 = jpost(url3, "/measure", p)
288     st2, resp2 = jpost(url3, "/measure", p)          # 完全相同的重试
289     r.eq("retry of identical action -> 200", st2, 200)
290     r.eq("retry replays the first response", resp2, resp1)
291     st3, resp3 = jpost(url3, "/measure", ride(request_id="m1",
292                                               position={"x": 600, "y": 0}, channel=1))
293     r.eq("same request_id, different content -> 409", st3, 409)
294     # 结构错误不占用 request_id
295     jpost(url3, "/measure", ride(request_id="m2", channel=99))
296     st4, resp4 = jpost(url3, "/measure", ride(request_id="m2",
297                                               position={"x": 800, "y": 0}, channel=2))
298     r.eq("400 does not consume request_id (reusable)", resp4.get("accepted"), True)
299     # accepted=false 也不占用
300     jpost(url3, "/measure", ride(request_id="m3", arena_id="bad"))
301     st5, resp5 = jpost(url3, "/measure", ride(request_id="m3",
302                                               position={"x": 900, "y": 0}, channel=3))
303     r.eq("accepted=false does not consume request_id", resp5.get("accepted"), True)
304     st6, resp6 = jpost(url3, "/exit", body(request_id="x1"))
305     r.eq("/exit -> exit_reason user_exit", resp6.get("exit_reason"), "user_exit")
306     st7, resp7 = jpost(url3, "/measure", ride(request_id="m9"))
307     r.eq("after /exit the interface is closed", st7, -1)
308     httpd3.shutdown()
309
310     # =====
311     print("\n[12] 物理与统计规则 (附录1/附录2)")
312     sim4 = MockSimulator(problem=4, mode="practice", seed=4242, count=10,
313                          team_id=TEAM)
314     sim4.start()
315     s4 = sim4.live.session
316     s4.enter()
317     j0 = sim4.case.jammers[0]
318     # 从干扰源正东侧 300 m 处检测: 真实方位角应为 180°(从检测点指向干扰源)
319     bx, by = j0.x + 300.0, j0.y
320     o = s4.measure(bx, by, j0.channel)
321     if o.payload.get("measure_result") == "direction":
322         true_deg = A.bearing_deg(bx, by, j0.x, j0.y)
323         got = o.payload["svd_deg"]
324         r.check("svd_deg within +/-1 deg of true bearing",
325               abs((got - true_deg + 180) % 360 - 180) <= 1.0 + 1e-9,
326               f"(true {true_deg:.4f}, got {got})")
327         r.eq("svd_deg has 2 decimals", round(got, 2), got)
328         r.check("svd_deg in [0,360)", 0.0 <= got < 360.0)

```

```

329     else:
330         r.notes.append("first jammer not detectable at +300 m east (directional); "
331                        "bearing check skipped")
332     # 同点重复测量误差恒定
333     o1 = s4.measure(bx, by, j0.channel)
334     s4b = A.Session(sim4.case)
335     s4b.enter()
336     o2 = s4b.measure(bx, by, j0.channel)
337     r.eq("same location -> identical svd_deg", o1.payload.get("svd_deg"),
338         o2.payload.get("svd_deg"))
339     # 覆盖角: 全向源在任意方向都可测到
340     omni = next((j for j in sim4.case.jammers if not j.directional), None)
341     if omni is not None:
342         angs = [0, 90, 180, 270]
343         hits = 0
344         for a in angs:
345             rad = math.radians(a)
346             s4c = A.Session(sim4.case)
347             s4c.enter()
348             res = s4c.measure(omni.x + 200 * math.cos(rad),
349                              omni.y + 200 * math.sin(rad), omni.channel)
350             if res.payload.get("measure_result") in ("direction", "near"):
351                 hits += 1
352         r.eq("omni jammer detectable from all 4 sides", hits, 4)
353     direc = next((j for j in sim4.case.jammers if j.directional), None)
354     if direc is not None:
355         # 定向源只朝 bearing_deg 一侧辐射: 站在它背后则收不到信号。
356         # 注意 covered() 判据是"检测点指向干扰源的方向与定向方向夹角 <= 90°",
357         # 因此"背后"= 检测点位于 jammer 沿定向方向的反方向一侧。
358         rad = math.radians(direc.bearing_deg)
359         px = direc.x + 200.0 * math.cos(rad)    # 正对波束
360         py = direc.y + 200.0 * math.sin(rad)
361         bx = direc.x - 200.0 * math.cos(rad)    # 波束背后
362         by = direc.y - 200.0 * math.sin(rad)
363         r.check("point in front of the beam is covered",
364               direc.covered(px, py), f"(bearing {direc.bearing_deg:.2f})")
365         r.check("point behind the beam is not covered",
366               not direc.covered(bx, by),
367               f"(bearing {direc.bearing_deg:.2f}, point ({bx:.1f},{by:.1f}))")
368         s4d = A.Session(sim4.case)
369         s4d.enter()
370         res = s4d.measure(bx, by, direc.channel)
371         r.eq("directional jammer behind its beam -> no_signal",
372             res.payload.get("measure_result"), "no_signal")
373         s4d2 = A.Session(sim4.case)
374         s4d2.enter()
375         res2 = s4d2.measure(px, py, direc.channel)
376         r.eq("directional jammer inside its beam -> direction",
377             res2.payload.get("measure_result"), "direction")
378     # 有效半径: 距离 > 1500 m 时任何源都收不到
379     s4e = A.Session(sim4.case)
380     s4e.enter()
381     res = s4e.measure(j0.x + 1600.0, j0.y, j0.channel)
382     r.eq("beyond 1500 m -> no_signal", res.payload.get("measure_result"), "no_signal")
383     # near 阈值 5 m
384     s4f = A.Session(sim4.case)
385     s4f.enter()
386     res = s4f.measure(j0.x + 4.0, j0.y, j0.channel)

```

```

387     if j0.directional and not j0.covered(j0.x + 4.0, j0.y):
388         r.notes.append("near-threshold check skipped (jammer faces away)")
389     else:
390         r.eq("distance <= 5 m -> near", res.payload.get("measure_result"), "near")
391     # 清除半径 20 m 与单次性
392     s4g = A.Session(sim4.case)
393     s4g.enter()
394     r.eq("clear at 20 m -> success",
395         s4g.clear(j0.x + 20.0, j0.y, j0.channel).payload["clear_result"], "success")
396     r.eq("clearing the same jammer twice -> no_target_in_range",
397         s4g.clear(j0.x + 20.0, j0.y, j0.channel).payload["clear_result"],
398         "no_target_in_range")
399     s4h = A.Session(sim4.case)
400     s4h.enter()
401     r.eq("clear at 20.01 m -> no_target_in_range",
402         s4h.clear(j0.x + 20.01, j0.y, j0.channel).payload["clear_result"],
403         "no_target_in_range")
404     # /clear 不改当前频道
405     s4i = A.Session(sim4.case)
406     s4i.enter()
407     s4i.measure(0, 0, 5)
408     before = s4i.current_channel
409     s4i.clear(0, 0, 9)
410     r.eq("/clear does not change the receiver channel", s4i.current_channel, before)
411     # 案例合法性
412     r.check("jammer count within 10..16",
413         A.JAMMER_COUNT_MIN <= sim4.case.total <= A.JAMMER_COUNT_MAX)
414     r.check("channels unique and within 1..20",
415         len({j.channel for j in sim4.case.jammers}) == sim4.case.total
416         and all(1 <= j.channel <= 20 for j in sim4.case.jammers))
417     r.check("effective radius within 1000..1500 m",
418         all(1000.0 <= j.eff_radius <= 1500.0 for j in sim4.case.jammers))
419     r.check("all jammers inside the 1800 m arena",
420         all(math.hypot(j.x, j.y) <= 1800.0 for j in sim4.case.jammers))
421     r.check("problem 4 case contains both omni and directional jammers",
422         sim4.case.omni_count >= 1 and sim4.case.directional_count >= 1)
423     p3 = A.make_case(problem=3, seed=5, count=10)
424     r.eq("problem 3 case is all-omni", p3.directional_count, 0)
425     # 误差在全局范围内统计上覆盖 [-1,1]
426     samples = [A.svd_error_deg(i * 13.7, i * 7.3, "s") for i in range(4000)]
427     r.check("svd error inside [-1,1]",
428         all(-1.0 <= v <= 1.0 for v in samples))
429     r.check("svd error spans most of [-1,1]",
430         min(samples) < -0.95 and max(samples) > 0.95,
431         f"(min {min(samples):.3f}, max {max(samples):.3f})")
432     mean = sum(samples) / len(samples)
433     r.check("svd error mean near 0", abs(mean) < 0.05, f"(mean {mean:.4f})")
434
435     # =====
436     print("\n[13] 现实时间约束 (附件2 §4.5, §6.2)")
437     time.sleep(0.3) # 让前面的服务器线程完全退出
438     sim5 = MockSimulator(problem=3, mode="practice", seed=99, count=10, team_id=TEAM,
439         port=0, real_limit_s=2.0, window_s=30.0)
440     httpd5 = serve(sim5)
441     url5 = f"http://127.0.0.1:{httpd5.server_address[1]}"
442     threading.Thread(target=httpd5.serve_forever, daemon=True).start()
443     sim5.start()
444     st, resp = jpost(url5, "/enter", body(request_id="real-enter"))

```

```

445         r.eq("/enter accepted under a short real limit", resp.get("accepted"), True)
446         r.check("remaining_real_duration_s <= 1200 and >= 0",
447                 0 <= resp.get("remaining_real_duration_s", -1) <= 1200,
448                 f"(got {resp.get('remaining_real_duration_s')})")
449         r.check("remaining respects the configured 2 s program limit",
450                 resp.get("remaining_real_duration_s", 99) <= 2,
451                 f"(got {resp.get('remaining_real_duration_s')})")
452         time.sleep(2.4)
453         sim5.tick()
454         st, resp = jpost(url5, "/measure", ride(position={"x": 10, "y": 0}, channel=1))
455         r.eq("after the real deadline the interface closes", st, -1)
456         r.eq("end reason recorded as program_timeout",
457             sim5.live.end_reason if sim5.live else None, "program_timeout")
458         r.eq("completion summary reports the reason",
459             sim5.result_summary().get("end_reason"), "program_timeout")
460         httpd5.shutdown()
461
462     finally:
463         httpd.shutdown()
464
465     print("\n" + "=" * 62)
466     print(f"conformance: {r.passed} passed, {len(r.failed)} failed")
467     for f in r.failed:
468         print(f"  FAILED: {f}")
469     for n in r.notes:
470         print(f"  note: {n}")
471     return 1 if r.failed else 0
472
473
474 if __name__ == "__main__":
475     raise SystemExit(main())

```

## vendor/mocksim/discovery.py

```

1 # -*- coding: utf-8 -*-
2 """策略插件发现与合规检查。
3
4 队员只需要把策略文件放进 ``strategies/`` 目录（或在 config 里指定路径），
5 本模块负责加载、实例化、以及做一次"是否偷看真值"的静态检查。
6 """
7
8 from __future__ import annotations
9
10 import importlib.util
11 import inspect
12 import re
13 import sys
14 from dataclasses import dataclass, field
15 from pathlib import Path
16 from typing import Any, Callable
17
18 from .config import ROOT
19
20 STRATEGY_DIR = ROOT / "strategies"
21
22 #: 策略只应该使用接口返回的信息。以下模式说明程序在偷看真值或在绕过接口，
23 #: 本地演练会因此失真，到官方模拟器上必然失效。

```

```

24 FORBIDDEN_PATTERNS: list[tuple[str, str]] = [
25     (r"/debug/", "访问模拟器调试接口 (官方模拟器没有该接口)",
26     (r"mocksim\.arena", "直接导入仿真内核读取真值"),
27     (r"mocksim\.server", "直接导入模拟器内部状态读取真值"),
28     (r"\bcase\.jammers\b", "直接读取案例真值"),
29     (r"\btruth_table\b", "直接读取案例真值"),
30 ]
31
32 #: 这些名字出现在策略文件里通常是"把真值硬编码进去"的迹象
33 SUSPICIOUS_HARDCODE = re.compile(
34     r"(?!([A-Za-z0-9_])(?:x|y|x_true|y_true|true_x|true_y)\s*=\s*[+-]?[d{3,}]",
35     re.IGNORECASE,
36 )
37
38
39 @dataclass
40 class StrategyInfo:
41     key: str
42     name: str
43     cls: type
44     path: Path
45     description: str = ""
46     warnings: list[str] = field(default_factory=list)
47
48     def factory(self) -> Callable[[], Any]:
49         return lambda: self.cls()
50
51
52 @dataclass
53 class CheckReport:
54     path: Path
55     warnings: list[str] = field(default_factory=list)
56     errors: list[str] = field(default_factory=list)
57
58     @property
59     def ok(self) -> bool:
60         return not self.errors
61
62
63 def _module_name_for(path: Path) -> str:
64     return "team_strategy_" + re.sub(r"\W+", "_", path.stem)
65
66
67 def _load_module(path: Path) -> Any:
68     name = _module_name_for(path)
69     if name in sys.modules:
70         del sys.modules[name]
71     spec = importlib.util.spec_from_file_location(name, path)
72     if spec is None or spec.loader is None:
73         raise ImportError(f"无法加载策略文件: {path}")
74     module = importlib.util.module_from_spec(spec)
75     sys.modules[name] = module
76     # 让策略文件可以直接 import 到包内的 client/harness
77     if str(ROOT) not in sys.path:
78         sys.path.insert(0, str(ROOT))
79     spec.loader.exec_module(module)
80     return module
81

```

```

82
83 def check_file(path: Path) -> CheckReport:
84     """静态检查: 是否偷看真值、是否有可疑硬编码、是否能被导入。"""
85     report = CheckReport(path=path)
86     try:
87         source = path.read_text(encoding="utf-8")
88     except (OSError, UnicodeDecodeError) as exc:
89         report.errors.append(f"无法读取文件: {exc}")
90     return report
91
92     for pattern, why in FORBIDDEN_PATTERNS:
93         for m in re.finditer(pattern, source):
94             line = source[:m.start()].count("\n") + 1
95             report.errors.append(f"第 {line} 行: {why} —— 策略只能使用接口返回的信息")
96     for m in SUSPICIOUS_HARDCODE.finditer(source):
97         line = source[:m.start()].count("\n") + 1
98         report.warnings.append(
99             f"第 {line} 行: 疑似硬编码坐标 `{m.group(0).strip()}`; "
100             f"如果这是写死的干扰源位置, 正式测试必然失效"
101         )
102
103     try:
104         module = _load_module(path)
105     except Exception as exc: # noqa: BLE001
106         report.errors.append(f"导入失败: {type(exc).__name__}: {exc}")
107     return report
108     if not find_strategies(module):
109         report.errors.append(
110             "没有找到任何策略类。策略类需要提供 run(self, client, "
111             "deadline_real_s=None, verbose=True) 方法"
112         )
113     return report
114
115
116 def find_strategies(module: Any) -> list[type]:
117     """在模块中寻找符合约定的策略类。"""
118     out: list[type] = []
119     for _name, obj in inspect.getmembers(module, inspect.isclass):
120         if obj.__module__ != module.__name__:
121             continue # 排除 import 进来的类
122         if obj.__name__.startswith("_"):
123             continue
124         run = getattr(obj, "run", None)
125         if callable(run) and "client" in inspect.signature(run).parameters:
126             out.append(obj)
127     return out
128
129
130 def discover(*, extra_paths: list[Path] | None = None) -> dict[str, StrategyInfo]:
131     """扫描 strategies/ 目录, 返回 {key: StrategyInfo}。"""
132     paths: list[Path] = []
133     if STRATEGY_DIR.exists():
134         paths.extend(sorted(p for p in STRATEGY_DIR.glob("*.py")
135                             if not p.name.startswith("_")))
136     for p in extra_paths or []:
137         p = Path(p)
138         if p.is_dir():
139             paths.extend(sorted(x for x in p.glob("*.py") if not x.name.startswith("_")))

```

```

140         elif p.exists():
141             paths.append(p)
142
143     found: dict[str, StrategyInfo] = {}
144     for path in paths:
145         try:
146             module = _load_module(path)
147         except Exception as exc: # noqa: BLE001
148             print(f"[warn] 跳过 {path.name}: {type(exc).__name__}: {exc}")
149             continue
150         # 允许策略文件用模块级 display_name 给整个文件起中文名
151         module_display = getattr(module, "display_name", None)
152         for cls in find_strategies(module):
153             key = cls.__name__
154             found[key] = StrategyInfo(
155                 key=key,
156                 name=getattr(cls, "display_name", None) or module_display or key,
157                 cls=cls,
158                 path=path,
159                 description=(inspect.getdoc(cls) or "").strip().split("\n")[0],
160             )
161     return found
162
163
164 def resolve(key: str, *, extra_paths: list[Path] | None = None) -> StrategyInfo:
165     """按名字取策略; 支持 ``文件名.py`` 或 ``文件名`` 兜底。"""
166     found = discover(extra_paths=extra_paths)
167     if key in found:
168         return found[key]
169     # 允许 "my_strategy.py" 或 "my_strategy" 这种写法: 取该文件里的第一个策略
170     for info in found.values():
171         if info.path.stem == key or info.path.name == key:
172             return info
173     available = ", ".join(sorted(found)) or "(strategies/ 目录为空)"
174     raise SystemExit(f"找不到策略 {key!r}。可用策略: {available}")

```

## vendor/mocksim/doctor.py

```

1 # -*- coding: utf-8 -*-
2 """自检 (doctor): 一条命令确认环境、配置、端口、策略、自测是否正常。
3
4 队员遇到问题先跑这个, 输出可以直接贴给队友定位。
5 """
6
7 from __future__ import annotations
8
9 import platform
10 import socket
11 import sys
12 import time
13 from pathlib import Path
14
15 from . import arena as A
16 from . import config as cfg
17 from . import discovery
18 from .conformance import main as conformance_main
19 from .harness import Harness

```

```

20 from .version import BANNER, __version__
21
22 OK = "[ OK ]"
23 WARN = "[WARN]"
24 BAD = "[FAIL]"
25
26
27 def _check_port(port: int) -> tuple[bool, str]:
28     """端口是否被占用 (占用 -> 无法启动本地模拟器)。"""
29     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
30         s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
31         try:
32             s.bind(("127.0.0.1", port))
33             return True, f"端口 {port} 可用"
34         except OSError as exc:
35             return False, f"端口 {port} 已被占用 ({exc}); 用 --port 换一个端口"
36
37
38 def _self_test(team_id: str, *, quick: bool = True) -> tuple[bool, str]:
39     """跑一局最短的自测, 确认模拟器 + 客户端 + 策略整条链路通。"""
40     h = Harness(problem=3, team_id=team_id, out_dir=cfg.ROOT / "runs" / "selfcheck",
41                 real_limit_s=1200.0, window_s=1500.0, verbose=False)
42     try:
43         rec = h.run_once(strategy="BaselineSweep", seed=20260913, count=10)
44     except Exception as exc: # noqa: BLE001
45         return False, f"自测异常: {type(exc).__name__}: {exc}"
46     if rec.error:
47         return False, f"自测失败: {rec.error}"
48     if rec.measure_count <= 0:
49         return False, "自测失败: 一次检测都没有发出, 链路不通"
50     return True, (f"自测通过: 案例 {rec.case_code} 清除 {rec.cleared_count} / "
51                 f"{rec.total_jammers} 平均 "
52                 f"{rec.avg_locate_clear_time_s and round(rec.avg_locate_clear_time_s, 1)}s "
53                 f"检测 {rec.measure_count} 次")
54
55
56 def run_doctor(*, skip_self_test: bool = False, conformance: bool = False,
57               port: int | None = None) -> int:
58     print(BANNER)
59     print(f"\nmocksim v{__version__} 根目录: {cfg.ROOT}\n")
60     failures = 0
61     warnings = 0
62
63     # --- 1. Python -----
64     print("【1】运行环境")
65     ver = sys.version_info
66     if ver >= (3, 10):
67         print(f"[OK] Python {platform.python_version()} ({platform.machine()})")
68     else:
69         print(f"[BAD] 需要 Python >= 3.10, 当前 {platform.python_version()}")
70         failures += 1
71     print(f"[OK] 标准库自检: math/hashlib/http.server 可用")
72     proxy = {k: v for k, v in __import__("os").environ.items()
73             if k.upper() in ("HTTP_PROXY", "HTTPS_PROXY", "ALL_PROXY")}
74     if proxy:
75         print(f"[WARN] 检测到系统代理 {proxy}")
76         print("本地客户端已自动绕过回环地址; 你们自己的程序如果用 requests,")
77         print("请设置 NO_PROXY=127.0.0.1,localhost 或 proxies={'http': None}")

```

```

78         warnings += 1
79     else:
80         print(f"{OK} 未检测到系统代理")
81
82     # --- 2. 配置 -----
83     print("\n【2】配置")
84     created = cfg.ensure_config_files()
85     for p in created:
86         print(f"{OK} 已生成配置模板 {p.relative_to(cfg.ROOT)}")
87     conf = cfg.load()
88     if cfg.team_is_configured():
89         print(f"{OK} team_id = {conf.team_id!r}")
90     else:
91         print(f"{WARN} team_id 仍是占位值 {conf.team_id!r}")
92         print(f"    请编辑 {cfg.TEAM_CONFIG.relative_to(cfg.ROOT)} 填你们的参赛队号")
93         print(f"    (本地演练不校验也能跑；正式测试必须用真实参赛队号)")
94         warnings += 1
95     print(f"{OK} 配置来源: {'', '.join(conf.sources)}")
96     print(f"    默认问题={conf.default_problem} 端口={conf.port} "
97           f"现实上限={conf.real_limit_s}s 窗口={conf.window_s}s")
98
99     # --- 3. 端口 -----
100    print("\n【3】端口")
101    want_port = port or conf.port
102    free, msg = _check_port(want_port)
103    print(f"{OK if free else WARN} {msg}")
104    if not free:
105        warnings += 1
106
107    # --- 4. 策略 -----
108    print("\n【4】策略插件")
109    found = discovery.discover()
110    if not found:
111        print(f"{BAD} strategies/ 目录里没有找到任何策略")
112        failures += 1
113    for key, info in sorted(found.items()):
114        print(f"{OK} {key:<24} {info.name} ({info.path.name})")
115        report = discovery.check_file(info.path)
116        for w in report.warnings:
117            print(f"{WARN} {info.path.name}: {w}")
118            warnings += 1
119        for e in report.errors:
120            print(f"{BAD} {info.path.name}: {e}")
121            failures += 1
122
123    # --- 5. 协议实现自检 -----
124    print("\n【5】模拟器实现自检")
125    if conformance:
126        code = conformance_main()
127        if code == 0:
128            print(f"{OK} 协议一致性断言全部通过")
129        else:
130            print(f"{BAD} 协议一致性断言有失败项")
131            failures += 1
132    else:
133        print("[SKIP] 协议一致性断言 (加 --conformance 运行, 约 10 秒)")
134
135    # --- 6. 端到端自测 -----

```

```

136     print("\n 【6】 端到端自测")
137     if skip_self_test:
138         print("[SKIP] 加 --full 可跑一局端到端自测")
139     else:
140         t0 = time.monotonic()
141         ok, msg = _self_test(conf.team_id)
142         print(f"{OK if ok else BAD} {msg} ({time.monotonic() - t0:.1f}s)")
143         if not ok:
144             failures += 1
145
146     # --- 结论 -----
147     print("\n" + "=" * 60)
148     if failures:
149         print(f"{BAD} 自检未通过: {failures} 项失败, {warnings} 项警告")
150         return 1
151     print(f"{OK} 自检通过 ({warnings} 项警告)")
152     print("\n下一步:")
153     print("    python -m mocksim.cli run --problem 3 --runs 6      # 批量演练")
154     print("    python -m mocksim.cli strategies                      # 看可用策略")
155     print("    python -m mocksim.cli serve --problem 3                # 起本地模拟器")
156     "接你们自己的程序")
157     return 0

```

## vendor/mocksim/harness.py

```

1 # -*- coding: utf-8 -*-
2 """演练框架：批量跑策略、统计、出表1、明文日志、真值诊断、策略对比。"""
3
4 from __future__ import annotations
5
6 import json
7 import statistics
8 import threading
9 import time
10 import traceback
11 from dataclasses import asdict, dataclass, field
12 from datetime import datetime
13 from pathlib import Path
14 from typing import Any, Callable
15
16 from . import arena as A
17 from .client import Rejected, RobotClient, TransportFailure
18 from .discovery import StrategyInfo, resolve
19 from .server import MockSimulator, serve
20 from .version import __version__
21
22 FORMAL_WINDOW_S = A.WINDOW_DURATION_S
23 FORMAL_PROGRAM_LIMIT_S = A.MAX_REAL_DURATION_S
24
25
26 @dataclass
27 class RunRecord:
28     index: int
29     strategy: str
30     case_code: str
31     problem: int
32     mode: str

```

```

33     seed: int | None
34     total_jammers: int | None
35     omni_count: int | None
36     directional_count: int | None
37     cleared_count: int
38     cleared_ratio: float
39     total_locate_clear_time_s: float
40     avg_locate_clear_time_s: float | None
41     program_runtime_s: float
42     measure_count: int
43     clear_count: int
44     clear_miss_count: int
45     end_reason: str | None
46     strategy_note: str = ""
47     error: str | None = None
48     log_path: str | None = None
49     truth: list[dict[str, Any]] = field(default_factory=list)
50
51     def as_row(self) -> dict[str, Any]:
52         return asdict(self)
53
54
55 class Harness:
56     """在进程内起 mock 模拟器，驱动策略跑一局或一批。"""
57
58     def __init__(self, *, problem: int = 3, team_id: str = "TEAM0000",
59                 out_dir: str | Path = "runs", svd_salt: str = "",
60                 real_limit_s: float = FORMAL_PROGRAM_LIMIT_S,
61                 window_s: float = FORMAL_WINDOW_S,
62                 verbose: bool = True) -> None:
63         self.problem = problem
64         self.team_id = team_id
65         self.out_dir = Path(out_dir)
66         self.out_dir.mkdir(parents=True, exist_ok=True)
67         self.svd_salt = svd_salt
68         self.real_limit_s = real_limit_s
69         self.window_s = window_s
70         self.verbose = verbose
71         self.records: list[RunRecord] = []
72
73     # -----
74     def _say(self, msg: str) -> None:
75         if self.verbose:
76             print(msg, flush=True)
77
78     # -----
79     def run_once(self, *, strategy: str | StrategyInfo = "BaselineSweep",
80                 seed: int | None = None, count: int | None = None,
81                 mode: str = "practice",
82                 strategy_kwargs: dict[str, Any] | None = None) -> RunRecord:
83         info = strategy if isinstance(strategy, StrategyInfo) else resolve(strategy)
84         sim = MockSimulator(problem=self.problem, mode=mode, seed=seed, count=count,
85                             team_id=self.team_id, port=0, svd_salt=self.svd_salt,
86                             real_limit_s=self.real_limit_s, window_s=self.window_s)
87         httpd = serve(sim)
88         port = httpd.server_address[1]
89         threading.Thread(target=httpd.serve_forever, daemon=True).start()
90         sim.start()

```

```

91
92     index = len(self.records) + 1
93     log_path = self.out_dir / f"{info.key}-{mode}-{index:03d}-{sim.case.case_code}.jsonl"
94     fh = log_path.open("w", encoding="utf-8")
95     stats: dict[str, int] = {"requests": 0, "retries": 0, "http_errors": 0,
96                             "rejected": 0, "transport_failures": 0}
97
98     def sink(entry: dict[str, Any]) -> None:
99         stats["requests"] = stats.get("requests", 0) + 1
100         fh.write(json.dumps(entry, ensure_ascii=False) + "\n")
101         fh.flush()
102
103     client = RobotClient(base_url=f"http://127.0.0.1:{port}", robot_id=self.team_id,
104                          log_sink=sink)
105     note = ""
106     error: str | None = None
107     started = time.monotonic()
108     try:
109         enter = client.enter()
110         if enter.get("accepted") is not True:
111             raise Rejected(enter)
112         remaining = int(enter["remaining_real_duration_s"])
113         budget = max(0.0, remaining - 10.0)      # 给 /exit 与网络留余量
114         try:
115             instance = info.cls(**(strategy_kwargs or {}))
116         except TypeError as exc:
117             raise SystemExit(
118                 f"策略 {info.key} 的构造函数不接受这些参数 "
119                 f"({strategy_kwargs}): {exc}") from exc
120         try:
121             result = instance.run(client, deadline_real_s=budget, verbose=False)
122             note = getattr(result, "note", "") or ""
123         except (TransportFailure, Rejected) as exc:
124             note = f"strategy aborted: {type(exc).__name__}: {exc}"
125         try:
126             client.exit()
127         except Exception: # noqa: BLE001 - 接口可能已关闭
128             pass
129     except BaseException as exc: # noqa: BLE001
130         error = f"{type(exc).__name__}: {exc}"
131         self._say(f"    [error] {error}")
132         traceback.print_exc()
133     finally:
134         fh.close()
135         sim.tick()
136         httpd.shutdown()
137         httpd.server_close()
138
139     stats["retries"] = client.stats.retries
140     stats["http_errors"] = client.stats.http_errors
141     stats["rejected"] = client.stats.rejected
142     stats["transport_failures"] = client.stats.transport_failures
143     _ = started
144
145     total = sim.case.total
146     cleared = len(sim.live.session.cleared) if sim.live else 0
147     vt = sim.live.session.virtual_time_s if sim.live else 0.0
148     rec = RunRecord(

```

```

149         index=index,
150         strategy=info.key,
151         case_code=sim.case.case_code,
152         problem=self.problem,
153         mode=mode,
154         seed=sim.case.seed,
155         total_jammers=total if mode == "practice" else None,
156         omni_count=sim.case.omni_count if mode == "practice" else None,
157         directional_count=sim.case.directional_count if mode == "practice" else None,
158         cleared_count=cleared,
159         cleared_ratio=(cleared / total) if total else 0.0,
160         total_locate_clear_time_s=vt,
161         avg_locate_clear_time_s=(vt / cleared) if cleared else None,
162         program_runtime_s=sim.live.program_runtime_s() if sim.live else 0.0,
163         measure_count=sim.live.session.measure_count if sim.live else 0,
164         clear_count=sim.live.session.clear_count if sim.live else 0,
165         clear_miss_count=((sim.live.session.clear_count - sim.live.session.clear_success)
166                          if sim.live else 0),
167         end_reason=sim.live.end_reason if sim.live else None,
168         strategy_note=note,
169         error=error,
170         log_path=str(log_path),
171         truth=sim.case.truth_table() if mode == "practice" else [],
172     )
173     rec.strategy = info.key
174     self.records.append(rec)
175     avg_txt = (f"{rec.avg_locate_clear_time_s:.1f}"
176              if rec.avg_locate_clear_time_s else "--")
177     self._say(
178         f" [{index:>3}] {rec.case_code} seed={rec.seed} "
179         f"清除 {cleared}/{total} 平均 {avg_txt}s "
180         f"虚拟总耗时 {rec.total_locate_clear_time_s:.0f}s "
181         f"检测{rec.measure_count} 清除{rec.clear_count} "
182         f"结束={rec.end_reason}"
183         + (f" 错误={rec.error}" if rec.error else "")
184     )
185     return rec
186
187 # -----
188 def run_batch(self, n: int, *, strategy: str | StrategyInfo = "BaselineSweep",
189              mode: str = "practice", seeds: list[int] | None = None,
190              counts: list[int] | None = None,
191              strategy_kwargs: dict[str, Any] | None = None) -> list[RunRecord]:
192     info = strategy if isinstance(strategy, StrategyInfo) else resolve(strategy)
193     self._say(f"\n策略 {info.key} | 问题 {self.problem} | {mode} | {n} 局")
194     out = []
195     for i in range(n):
196         seed = seeds[i] if seeds and i < len(seeds) else None
197         count = counts[i] if counts and i < len(counts) else None
198         out.append(self.run_once(strategy=info, seed=seed, count=count, mode=mode,
199                                strategy_kwargs=strategy_kwargs))
200     return out
201
202 # -----
203 def aggregate(self) -> dict[str, Any]:
204     recs = self.records
205     if not recs:
206         return {}

```

```

207     known = [r for r in recs if r.total_jammers]
208     ratios = [r.cleared_ratio for r in known]
209     avgs = [r.avg_locate_clear_time_s for r in recs if r.avg_locate_clear_time_s]
210     return {
211         "runs": len(recs),
212         "problems": sorted({r.problem for r in recs}),
213         "strategies": sorted({r.strategy for r in recs}),
214         "mean_cleared_ratio": statistics.fmean(ratios) if ratios else None,
215         "worst_cleared_ratio": min(ratios) if ratios else None,
216         "mean_avg_locate_clear_time_s": statistics.fmean(avgs) if avgs else None,
217         "mean_program_runtime_s": statistics.fmean([r.program_runtime_s for r in recs]),
218         "full_clear_runs": sum(1 for r in known if r.cleared_count == r.total_jammers),
219         "known_runs": len(known),
220         "mean_measure_count": statistics.fmean([r.measure_count for r in recs]),
221         "mean_clear_count": statistics.fmean([r.clear_count for r in recs]),
222         "mean_clear_miss_count": statistics.fmean([r.clear_miss_count for r in recs]),
223         "runs_with_error": sum(1 for r in recs if r.error),
224         "end_reasons": _tally(r.end_reason or "unknown" for r in recs),
225     }
226
227 # -----
228 def export_results_table(self, path: str | Path | None = None, *,
229     tests: int = 3, mode: str = "formal") -> Path:
230     """导出题目表1 格式的结果表 (Markdown + CSV)。"""
231     path = Path(path) if path else self.out_dir / "table1-results.md"
232     rows = [r for r in self.records if r.mode == mode][:tests]
233     if not rows:
234         rows = self.records[-tests:]
235     lines = [
236         f"| 测试 | 测试案例编码 | 清除干扰源个数 | 平均定位清除时间(s) | 程序运行时间(s) |",
237         "|---|---|---|---|---|",
238     ]
239     csv_lines = ["test,case_code,cleared_count,avg_locate_clear_time_s,program_runtime_s"]
240     for i, r in enumerate(rows, 1):
241         avg_txt = f"{r.avg_locate_clear_time_s:.2f}" if r.avg_locate_clear_time_s else ""
242         avg_csv = (f"{r.avg_locate_clear_time_s:.6f}"
243             if r.avg_locate_clear_time_s else "")
244         lines.append(f"| 测试{i} | {r.case_code} | {r.cleared_count} | "
245             f"{avg_txt} | {r.program_runtime_s:.2f} |")
246         csv_lines.append(f"{i},{r.case_code},{r.cleared_count},{avg_csv},"
247             f"{r.program_runtime_s:.6f}")
248     path.parent.mkdir(parents=True, exist_ok=True)
249     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
250     path.with_suffix(".csv").write_text("\n".join(csv_lines) + "\n", encoding="utf-8")
251     return path
252
253 def export_json(self, path: str | Path | None = None) -> Path:
254     path = Path(path) if path else self.out_dir / "summary.json"
255     payload = {
256         "environment": "26B local mock simulator",
257         "version": __version__,
258         "team_id": self.team_id,
259         "generated_at": datetime.now().isoformat(timespec="seconds"),
260         "problem": self.problem,
261         "limits": {"real_limit_s": self.real_limit_s, "window_s": self.window_s},
262         "aggregate": self.aggregate(),
263         "runs": [r.as_row() for r in self.records],
264     }

```

```

265     path.parent.mkdir(parents=True, exist_ok=True)
266     path.write_text(json.dumps(payload, ensure_ascii=False, indent=2),
267                     encoding="utf-8")
268     return path
269
270 def diagnosis(self) -> str:
271     """对照真值诊断：哪些频道没清掉、真值在哪、交会环节是否够用。"""
272     out: list[str] = []
273     for r in self.records:
274         if not r.truth:
275             continue
276         miss = (r.total_jammers or 0) - r.cleared_count
277         flag = "OK " if miss == 0 else "MISS"
278         out.append(
279             f"[{flag}] {r.case_code} seed={r.seed} 干扰源 {r.total_jammers} "
280             f"(全向 {r.omni_count}/定向 {r.directional_count}) 清除 {r.cleared_count} "
281             f"平均 {r.avg_locate_clear_time_s} and round(r.avg_locate_clear_time_s, 1)}s "
282             f"检测 {r.measure_count} 清除 {r.clear_count}(落空 {r.clear_miss_count}) "
283             f"结束 {r.end_reason}"
284         )
285         if r.error:
286             out.append(f"      错误: {r.error}")
287         if miss and r.strategy_note:
288             out.append(f"      {r.strategy_note}")
289         if miss:
290             for t in r.truth:
291                 out.append(f"          ch[{t['channel']}:>2] ({t['x']:>9.1f},{t['y']:>9.1f}) "
292                             f"r={t['effective_radius_m']:.0f} {t['type']}"
293                             + (f" bearing={t['bearing_deg']:.1f}"
294                                if t["bearing_deg"] is not None else ""))
295     return "\n".join(out)
296
297
298 def _tally(values: Any) -> dict[str, int]:
299     out: dict[str, int] = {}
300     for v in values:
301         out[v] = out.get(v, 0) + 1
302     return out
303
304
305 # -----
306 # 策略对比：同一批 seed 下比较两个策略，便于 A/B 调参
307 # -----
308
309
310 def compare(seeds: list[int], *, problem: int = 3, team_id: str = "TEAM0000",
311            left: str = "BaselineSweep", right: str | None = None,
312            out_dir: str | Path = "runs/compare", verbose: bool = True,
313            left_kwargs: dict[str, Any] | None = None,
314            right_kwargs: dict[str, Any] | None = None) -> str:
315     results: dict[str, list[RunRecord]] = {}
316     for key, kwargs in ((left, left_kwargs), (right, right_kwargs)):
317         if key is None:
318             continue
319         h = Harness(problem=problem, team_id=team_id,
320                    out_dir=Path(out_dir) / key, verbose=verbose)
321         results[key] = h.run_batch(len(seeds), strategy=key, seeds=seeds,
322                                   strategy_kwargs=kwargs)

```

```

323
324 lines = ["| seed | 案例编码 | " + " | ".join(
325     f"{k} 清除/总数 | {k} 平均时间(s)" for k in results) + " |",
326     "|---|---|" + "---|" * (2 * len(results))]
327 for i, seed in enumerate(seeds):
328     row = [str(seed)]
329     code = ""
330     for key, recs in results.items():
331         if i < len(recs):
332             r = recs[i]
333             code = r.case_code
334             avg = f"{r.avg_locate_clear_time_s:.1f}" if r.avg_locate_clear_time_s else "--"
335             row.append(f"{r.cleared_count}/{r.total_jammers}")
336             row.append(avg)
337         else:
338             row.extend(["—", "—"])
339     lines.append("| " + " | ".join([row[0], code] + row[1:]) + " |")
340
341 lines.append("")
342 for key, recs in results.items():
343     known = [r for r in recs if r.total_jammers]
344     ratios = [r.cleared_ratio for r in known] or [0.0]
345     avgs = [r.avg_locate_clear_time_s for r in recs if r.avg_locate_clear_time_s]
346     full = sum(1 for r in known if r.cleared_count == r.total_jammers)
347     if avgs:
348         lines.append(
349             f"- **{key}**: 平均清除比例 {statistics.fmean(ratios):.3f}, "
350             f"最差 {min(ratios):.3f}, 满分 {full}/{len(known)}, "
351             f"平均定位清除时间 {statistics.fmean(avgs):.0f}s"
352         )
353     else:
354         lines.append(f"- **{key}**: 无成功清除记录 (平均清除比例 "
355             f"{statistics.fmean(ratios):.3f})")
356 report = "\n".join(lines)
357 out = Path(out_dir) / "compare.md"
358 out.parent.mkdir(parents=True, exist_ok=True)
359 out.write_text(report + "\n", encoding="utf-8")
360 return report
361
362
363 def euclidean(a: tuple[float, float], b: tuple[float, float]) -> float:
364     import math
365     return math.hypot(a[0] - b[0], a[1] - b[1])
366
367
368 # -----
369 # 命令行: 统一入口在 mocksim/cli.py, 这里只转发, 避免出现两套参数
370 # -----
371
372
373 def main(argv: list[str] | None = None) -> int:
374     print("提示: 演练入口已统一到 mocksim/cli (写法见 QUICKSTART.md):")
375     print(" python -m mocksim.cli run --problem 3 --runs 6\n")
376     from .cli import main as cli_main
377     args = list(argv if argv is not None else [])
378     if not args or args[0].startswith("-"):
379         args = ["run", *args]
380     return cli_main(args)

```

```

381
382
383 if __name__ == "__main__":
384     raise SystemExit(main())

```

## vendor/mocksim/server.py

```

1 # -*- coding: utf-8 -*-
2 """mock simulator HTTP layer: /enter /measure /clear /exit per 附件2.
3
4 Implements, from the published specification only:
5
6 * request validation -> HTTP 400/404/405/409/413/415/429 with JSON bodies
7 * unknown declared-path fields -> HTTP 200 + accepted=false
8 * per-session request_id idempotency (replay / 409 on differing content)
9 * virtual clock arithmetic (microsecond based, <=6 decimals in responses)
10 * real-time watchdog: 25-minute window and 20-minute program run limit
11
12 Run: python -m mocksim.server --problem 3 --practice
13 """
14
15 from __future__ import annotations
16
17 import argparse
18 import hashlib
19 import json
20 import socket
21 import threading
22 import time
23 from dataclasses import dataclass, field
24 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
25 from typing import Any
26
27 from . import arena as A
28
29 # 本机回环地址必须绕过系统代理 (Windows 上常见的 HTTP_PROXY/HTTPS_PROXY 会把
30 # 发往 127.0.0.1:2026 的请求转发给代理, 得到 502/407, 与模拟器本身无关).
31 NO_PROXY = ("127.0.0.1", "localhost", "::1", "[::1]")
32
33 # -----
34 # JSON parsing with duplicate-key and depth rejection
35 # -----
36
37
38 class BadJson(Exception):
39     def __init__(self, message: str) -> None:
40         super().__init__(message)
41         self.message = message
42
43
44 def _no_duplicates(pairs: list[tuple[str, Any]]) -> dict[str, Any]:
45     out: dict[str, Any] = {}
46     for k, v in pairs:
47         if k in out:
48             raise BadJson(f"duplicate key: {k}")
49         out[k] = v
50     return out

```

```

51
52
53 def _depth(obj: Any, level: int = 1) -> int:
54     if isinstance(obj, dict):
55         return max([level] + [_depth(v, level + 1) for v in obj.values()])
56     if isinstance(obj, list):
57         return max([level] + [_depth(v, level + 1) for v in obj])
58     return level
59
60
61 def load_strict_json(raw: bytes) -> dict[str, Any]:
62     """无 BOM 的 UTF-8 JSON 对象，不允许重复键，嵌套不超过 16 层."""
63     if raw[:3] == b"\xef\xbb\xbf":
64         raise BadJson("UTF-8 BOM is not allowed")
65     try:
66         text = raw.decode("utf-8")
67     except UnicodeDecodeError as exc:
68         raise BadJson(f"body is not valid UTF-8: {exc}") from exc
69     try:
70         obj = json.loads(text, object_pairs_hook=_no_duplicates)
71     except BadJson:
72         raise
73     except json.JSONDecodeError as exc:
74         raise BadJson(f"invalid JSON: {exc.msg} at pos {exc.pos}") from exc
75     if not isinstance(obj, dict):
76         raise BadJson("top level value must be a JSON object")
77     if _depth(obj) > A.MAX_JSON_DEPTH:
78         raise BadJson(f"JSON nesting exceeds {A.MAX_JSON_DEPTH} levels")
79     return obj
80
81
82 # -----
83 # Field validation helpers
84 # -----
85
86
87 class BadField(Exception):
88     def __init__(self, message: str) -> None:
89         super().__init__(message)
90         self.message = message
91
92
93 CONTROL_OR_FORMAT = set(range(0x00, 0x21)) | {0x7F} | set(range(0x80, 0xA0))
94
95
96 def _check_id(value: Any, name: str, max_bytes: int) -> str:
97     if not isinstance(value, str):
98         raise BadField(f"{name} must be a string")
99     raw = value.encode("utf-8")
100     if not (A.ID_BYTE_MIN <= len(raw) <= max_bytes):
101         raise BadField(f"{name} UTF-8 length must be 1..{max_bytes} bytes")
102     for ch in value:
103         if ord(ch) in CONTROL_OR_FORMAT or ch.isspace():
104             raise BadField(f"{name} must not contain control/space/format characters")
105     return value
106
107
108 def _check_channel(value: Any) -> int:

```

```

109     if isinstance(value, bool) or not isinstance(value, (int, float)):
110         raise BadField("channel must be an integer in 1..20")
111     if isinstance(value, float):
112         if not value.is_integer():
113             raise BadField("channel must be an integer in 1..20")
114         value = int(value)
115     if not (A.CHANNEL_MIN <= value <= A.CHANNEL_MAX):
116         raise BadField("channel must be an integer in 1..20")
117     return int(value)
118
119
120 def _check_position(value: Any) -> tuple[float, float]:
121     if not isinstance(value, dict):
122         raise BadField("position must be an object")
123     unknown = set(value) - {"x", "y"}
124     if unknown:
125         raise BadField(f"unknown position field(s): {sorted(unknown)}")
126     for key in ("x", "y"):
127         if key not in value:
128             raise BadField(f"position.{key} is required")
129     coords = []
130     for key in ("x", "y"):
131         v = value[key]
132         if isinstance(v, bool) or not isinstance(v, (int, float)):
133             raise BadField(f"position.{key} must be a finite number")
134         fv = float(v)
135         if fv != fv or fv in (float("inf"), float("-inf")):
136             raise BadField(f"position.{key} must be finite (NaN/Inf rejected)")
137         if abs(fv) > A.COORD_LIMIT_M:
138             raise BadField(f"position.{key} absolute value must be <= {A.COORD_LIMIT_M}")
139         coords.append(fv)
140     return coords[0], coords[1]
141
142
143 # Declared field sets per action (附件2 §6..§9). Any other key -> accepted=false.
144 FIELDS = {
145     "/enter": {"enter": {"arena_id", "robot_id", "request_id"}},
146     "/measure": {"measure": {"arena_id", "robot_id", "request_id", "position", "channel"}},
147     "/clear": {"clear": {"arena_id", "robot_id", "request_id", "position", "channel"}},
148     "/exit": {"exit": {"arena_id", "robot_id", "request_id"}},
149 }
150
151
152 # -----
153 # Test lifecycle
154 # -----
155
156
157 @dataclass
158 class IdemRecord:
159     body_hash: str
160     status: int
161     response: dict[str, Any]
162
163
164 @dataclass
165 class LiveTest:
166     """一局正在进行（接口已开放）的测试。"""

```

```

167
168 session: A.Session
169 mode: str                                # practice | formal
170 window_started: float
171 real_limit_s: float
172 virtual_limit_s: float
173 window_duration_s: float = A.WINDOW_DURATION_S
174
175 def __post_init__(self) -> None:
176     self.entry_wall: float | None = None
177     self.enter_wall: float | None = None
178     self.end_wall: float | None = None
179     self.end_reason: str | None = None
180     self.entered_robot: str | None = None
181     self.idem: dict[str, IdemRecord] = {}
182     self.busy = False
183     self.log: list[dict[str, Any]] = []
184     self.request_seq = 0
185     self.lock = threading.RLock()
186
187 # -- deadlines -----
188 @property
189 def window_deadline(self) -> float:
190     return self.window_started + self.window_duration_s
191
192 def program_deadline(self) -> float | None:
193     if self.enter_wall is None:
194         return None
195     return self.enter_wall + self.real_limit_s
196
197 def remaining_real_s(self) -> int:
198     now = time.monotonic()
199     deadlines = [self.window_deadline]
200     pd = self.program_deadline()
201     if pd is not None:
202         deadlines.append(pd)
203     return max(0, int(min(deadlines) - now))
204
205 def real_expired(self) -> bool:
206     now = time.monotonic()
207     if now >= self.window_deadline:
208         return True
209     pd = self.program_deadline()
210     return pd is not None and now >= pd
211
212 def program_runtime_s(self) -> float:
213     if self.enter_wall is None:
214         return 0.0
215     end = self.end_wall if self.end_wall is not None else time.monotonic()
216     return round(end - self.enter_wall, 6)
217
218 def virtual_expired(self) -> bool:
219     return self.session.virtual_time_s >= self.virtual_limit_s
220
221 def finished(self) -> bool:
222     return self.end_reason is not None
223
224 def finish(self, reason: str) -> None:

```

```

225         if self.end_reason is None:
226             self.end_reason = reason
227             self.end_wall = time.monotonic()
228
229     # -- logging -----
230     def record(self, kind: str, request_id: str | None, status: int,
231               payload: dict[str, Any] | None, response: dict[str, Any],
232               extra: dict[str, Any] | None = None) -> None:
233         self.request_seq += 1
234         entry = {
235             "seq": self.request_seq,
236             "wall_ms": int(time.time() * 1000),
237             "kind": kind,
238             "request_id": request_id,
239             "http_status": status,
240             "request": payload,
241             "response": response,
242         }
243         if extra:
244             entry.update(extra)
245         self.log.append(entry)
246
247
248 # -----
249 # Server
250 # -----
251
252
253 class MockSimulator:
254     """模拟器状态机 + 请求校验。可独立于 HTTP 层被测试套件直接驱动。"""
255
256     def __init__(self, *, problem: int = 3, mode: str = "practice",
257                 seed: int | None = None, count: int | None = None,
258                 team_id: str = "TEAM0000", port: int = 2026,
259                 svd_salt: str = "", virtual_limit_s: float = A.MAX_VIRTUAL_DURATION_S,
260                 real_limit_s: float = A.MAX_REAL_DURATION_S,
261                 window_s: float = A.WINDOW_DURATION_S,
262                 expose_truth: bool = True, clock_scale: float = 1.0) -> None:
263         self.problem = problem
264         self.mode = mode
265         self.team_id = team_id
266         self.port = port
267         self.seed = seed
268         self.count = count
269         self.svd_salt = svd_salt
270         self.virtual_limit_s = virtual_limit_s
271         self.real_limit_s = real_limit_s
272         self.window_s = window_s
273         self.expose_truth = expose_truth and mode == "practice"
274         self.clock_scale = clock_scale
275
276         self.lock = threading.RLock()
277         self.case = A.make_case(problem=problem, seed=seed, count=count)
278         self.status = "idle" # idle | running | finished
279         self.live: LiveTest | None = None
280         self.last_finished: LiveTest | None = None
281         self.finished_tests: list[dict[str, Any]] = []
282         self.started_wall: float = 0.0

```

```

283         self.idem_limit = 200000
284         self.verbose = False
285         self.enable_debug = False
286
287     # -- lifecycle -----
288     def start(self) -> None:
289         with self.lock:
290             if self.status == "running":
291                 raise RuntimeError("a test is already running")
292             session = A.Session(self.case, svd_salt=self.svd_salt)
293             self.live = LiveTest(session=session, mode=self.mode,
294                                 window_started=time.monotonic(),
295                                 real_limit_s=self.real_limit_s,
296                                 virtual_limit_s=self.virtual_limit_s)
297             self.live.window_duration_s = self.window_s
298             self.status = "running"
299             self.started_wall = time.monotonic()
300
301     def new_case(self, *, seed: int | None = None, count: int | None = None,
302                 problem: int | None = None) -> None:
303         with self.lock:
304             if problem is not None:
305                 self.problem = problem
306             if seed is not None:
307                 self.seed = seed
308             if count is not None:
309                 self.count = count
310             self.case = A.make_case(problem=self.problem, seed=self.seed,
311                                   count=self.count)
312             self.status = "idle"
313             self.live = None
314
315     def result_summary(self) -> dict[str, Any]:
316         with self.lock:
317             live = self.live
318             if live is None:
319                 return {"status": self.status}
320             session = live.session
321             out: dict[str, Any] = {
322                 "status": "finished" if live.finished() else "running",
323                 "case_code": self.case.case_code,
324                 "problem": self.problem,
325                 "mode": self.mode,
326                 "end_reason": live.end_reason,
327                 **session.statistics(live.program_runtime_s()),
328             }
329             if self.expose_truth:
330                 out["jammer_total"] = self.case.total
331                 out["omni_count"] = self.case.omni_count
332                 out["directional_count"] = self.case.directional_count
333             return out
334
335     # -- generic guards -----
336     def _open(self) -> None:
337         """接口未开放时连接可能被直接关闭 -> 由 HTTP 层抛 ConnectionClosed。"""
338         if self.status != "running" or self.live is None:
339             raise ConnectionClosed("robot interface is not open")
340         if self.live.finished():

```

```

341         raise ConnectionClosed("test already finished")
342
343     def _error_body(self, detail: str) -> dict[str, Any]:
344         vtime = 0.0
345         if self.live is not None:
346             vtime = self.live.session.virtual_time_s
347         return {
348             "accepted": False,
349             "real_timestamp_ms": int(time.time() * 1000),
350             "virtual_time_s": A.quantise_time(vtime),
351             "detail": detail,
352         }
353
354     def _guard_common(self, action: str, payload: dict[str, Any],
355                      base: dict[str, Any]) -> int | None:
356         """通用校验。返回 HTTP 状态码表示应拒绝(200 accepted=false); None 表示通过。
357
358         抛 BadField -> 400。
359         """
360         declared = FIELDS[action][action.strip("/")]
361         unknown = set(payload) - declared
362         if unknown:
363             base["accepted"] = False
364             base["detail"] = f"unknown field(s): {sorted(unknown)}"
365             return 200
366         for key in ("arena_id", "robot_id", "request_id"):
367             if key not in payload:
368                 raise BadField(f"{key} is required")
369             if not isinstance(payload[key], str):
370                 raise BadField(f"{key} must be a string")
371             _check_id(payload[key], key, A.ID_BYTE_MAX)
372             _check_id(payload[key], key, A.REQ_ID_BYTE_MAX)
373             if payload[key] != "default":
374                 base["detail"] = f"{key} must be 'default'"
375             return 200
376         if payload["robot_id"] != self.team_id:
377             base["detail"] = "robot_id does not match the logged-in team id"
378             return 200
379         return None
380
381     # -- dispatch -----
382     def handle(self, action: str, payload: dict[str, Any]) -> tuple[int, dict[str, Any]]:
383         if action not in ("enter", "measure", "clear", "exit"):
384             return 404, self._error_body("unknown path")
385         with self.lock:
386             return self._handle_locked(action, payload)
387
388     def _error_body(self, detail: str) -> dict[str, Any]:
389         vtime = 0.0
390         if self.live is not None:
391             vtime = self.live.session.virtual_time_s
392         return {
393             "accepted": False,
394             "real_timestamp_ms": int(time.time() * 1000),
395             "virtual_time_s": A.quantise_time(vtime),
396             "detail": detail,
397         }
398

```

```

399 def _handle_locked(self, action: str, payload: dict[str, Any]) -> tuple[int, dict[str, Any
    ]]:
400     self._open()
401     assert self.live is not None
402     live = self.live
403     path = "/" + action
404
405     # 1) field validation (400 / accepted=false for unknown fields)
406     base: dict[str, Any] = {
407         "accepted": False,
408         "real_timestamp_ms": int(time.time() * 1000),
409         "virtual_time_s": A.quantise_time(live.session.virtual_time_s),
410     }
411     guard = self._guard_common(path, payload, base)
412     if guard is not None:
413         live.record(action, payload.get("request_id"), guard, payload, base)
414         return guard, base
415
416     # 2) idempotency
417     request_id = payload["request_id"]
418     canon = json.dumps({"action": action, "body": payload},
419                       sort_keys=True, ensure_ascii=False, separators=(",", ":"))
420     body_hash = hashlib.blake2b(canon.encode("utf-8"), digest_size=16).hexdigest()
421     prev = live.idem.get(request_id)
422     if prev is not None:
423         if prev.body_hash != body_hash:
424             body = self._error_body("request_id reused with different action/content")
425             body["accepted"] = False
426             return 409, body
427         return prev.status, prev.response
428     if len(live.idem) >= self.idem_limit:
429         return 429, self._error_body("idempotency record limit reached")
430
431     # 3) action specific validation
432     if action in ("measure", "clear"):
433         if "position" not in payload:
434             raise BadField("position is required")
435         if "channel" not in payload:
436             raise BadField("channel is required")
437         x, y = _check_position(payload["position"])
438         channel = _check_channel(payload["channel"])
439
440     # 4) state machine
441     if action == "enter":
442         if live.session.entered:
443             base["detail"] = "duplicate /enter"
444             self._commit(live, action, request_id, body_hash, 200, base, payload)
445             return 200, base
446         if live.entered_robot is not None:
447             base["detail"] = "another robot already entered"
448             self._commit(live, action, request_id, body_hash, 200, base, payload)
449             return 200, base
450         live.enter_wall = time.monotonic()
451         live.entered_robot = payload["robot_id"]
452         outcome = live.session.enter()
453         response = {
454             "accepted": True,
455             "real_timestamp_ms": int(time.time() * 1000),
456             "virtual_time_s": A.quantise_time(outcome.virtual_time_s),

```

```

456         "max_virtual_duration_s": self.virtual_limit_s,
457         "max_real_duration_s": self.real_limit_s,
458         "remaining_real_duration_s": live.remaining_real_s(),
459     }
460     self._commit(live, action, request_id, body_hash, 200, response, payload,
461                 extra={"move_s": 0.0, "switch_s": 0.0, "action_s": 0.0})
462     return 200, response
463
464     if action == "exit":
465         if not live.session.entered:
466             base["detail"] = "/enter must succeed before /exit"
467             self._commit(live, action, request_id, body_hash, 200, base, payload)
468             return 200, base
469         outcome = live.session.exit()
470         response = {
471             "accepted": True,
472             "real_timestamp_ms": int(time.time() * 1000),
473             "virtual_time_s": A.quantise_time(outcome.virtual_time_s),
474             "exit_reason": "user_exit",
475         }
476         live.finish("user_exit")
477         self._commit(live, action, request_id, body_hash, 200, response, payload)
478         return 200, response
479
480     # measure / clear need an open session
481     if not live.session.entered:
482         base["detail"] = "/enter must succeed before this action"
483         self._commit(live, action, request_id, body_hash, 200, base, payload)
484         return 200, base
485
486     if action == "measure":
487         outcome = live.session.measure(x, y, channel)
488     else:
489         outcome = live.session.clear(x, y, channel)
490     response = {
491         "accepted": True,
492         "real_timestamp_ms": int(time.time() * 1000),
493         "virtual_time_s": A.quantise_time(outcome.virtual_time_s),
494         **outcome.payload,
495     }
496     self._commit(live, action, request_id, body_hash, 200, response, payload,
497                 extra={"move_s": outcome.move_s, "switch_s": outcome.switch_s,
498                       "action_s": outcome.action_s,
499                       "elapsed_s": A.quantise_time(outcome.total_s),
500                       "position": {"x": x, "y": y}, "channel": channel})
501     if live.session.virtual_expired:
502         live.finish("virtual_timeout")
503     if live.session.remaining_count == 0 and not live.session.exited:
504         pass # 由机器狗主动 /exit; 模拟器不代为结束
505     return 200, response
506
507 def _commit(self, live: LiveTest, action: str, request_id: str, body_hash: str,
508            status: int, response: dict[str, Any], payload: dict[str, Any],
509            extra: dict[str, Any] | None = None) -> None:
510     if response.get("accepted") is False and status == 200:
511         # 结构错误/未知字段/不匹配不占用 request_id (附件2 §5.3)
512         live.record(action, request_id, status, payload, response, extra)
513     return

```

```

514         live.idem[request_id] = IdemRecord(body_hash=body_hash, status=status,
515                                             response=response)
516         live.record(action, request_id, status, payload, response, extra)
517
518     # -- watchdog -----
519     def tick(self) -> None:
520         with self.lock:
521             live = self.live
522             if live is None or live.finished():
523                 return
524             if live.real_expired():
525                 live.finish("window_timeout" if not live.session.entered
526                             else "program_timeout")
527             elif live.virtual_expired():
528                 live.finish("virtual_timeout")
529
530
531 class ConnectionClosed(Exception):
532     """接口未开放/测试已结束: 直接关闭连接, 没有 JSON 响应 (附件2 §5.3)."""
533
534
535 # -----
536 # HTTP handler
537 # -----
538
539
540 class Handler(BaseHTTPRequestHandler):
541     server_version = "MockJammers/1.0"
542     protocol_version = "HTTP/1.1"
543     sim: MockSimulator # injected
544
545     def log_message(self, fmt: str, *args: Any) -> None: # noqa: A003
546         if self.sim.verbose:
547             print(f"[http] {self.address_string()} {fmt % args}")
548
549     # -- helpers -----
550     def _send_json(self, status: int, body: dict[str, Any]) -> None:
551         raw = json.dumps(body, ensure_ascii=False, separators=(",", ":")).encode("utf-8")
552         self.send_response(status)
553         self.send_header("Content-Type", "application/json")
554         self.send_header("Content-Length", str(len(raw)))
555         self.send_header("Connection", "keep-alive")
556         self.end_headers()
557         self.wfile.write(raw)
558
559     def _drop(self) -> None:
560         """直接关闭 TCP 连接 (接口未开放 / 测试已结束).
561
562         附件2 §5.3: "连接可能被直接关闭, 此时没有JSON体".
563         因此不发送任何 HTTP 响应, 客户端应把它当作"连不上"而不是错误码。
564         """
565         self.close_connection = True
566         try:
567             self.connection.shutdown(socket.SHUT_RDWR)
568         except Exception:
569             pass
570         try:
571             self.connection.close()

```

```

572         except Exception:
573             pass
574
575     def do_GET(self) -> None: # noqa: N802
576         if self.sim.enable_debug and self.path.startswith("/debug"):
577             return self._debug_get()
578         self._send_json(405, self.sim._error_body("only POST is allowed"))
579
580     def do_POST(self) -> None: # noqa: N802
581         sim = self.sim
582         # path must be exact: no trailing slash, no query string
583         if self.path not in ("/enter", "/measure", "/clear", "/exit"):
584             self._send_json(404, sim._error_body(f"unknown or inexact path: {self.path}"))
585             return
586         ctype = self.headers.get("Content-Type", "")
587         parts = [p.strip() for p in ctype.split(";")] if ctype else []
588         media = parts[0].lower() if parts else ""
589         params = {p.split("=")[0].strip().lower() for p in parts[1:] if p}
590         if media != "application/json" or params - {"charset"}:
591             self._send_json(415, sim._error_body("unsupported Content-Type"))
592             return
593         if params == {"charset"}:
594             charset = [p for p in parts[1:] if p.lower().startswith("charset")][0]
595             if charset.split("=", 1)[1].strip().strip("'").lower() != "utf-8":
596                 self._send_json(415, sim._error_body("only charset=utf-8 is supported"))
597             return
598         enc = self.headers.get("Content-Encoding", "identity").strip().lower()
599         if enc not in ("", "identity"):
600             self._send_json(415, sim._error_body("Content-Encoding must be identity or absent"))
601
602         return
603
604     try:
605         length = int(self.headers.get("Content-Length") or 0)
606     except ValueError:
607         self._send_json(400, sim._error_body("bad Content-Length"))
608         return
609     if length > A.BODY_LIMIT_BYTES:
610         self._send_json(413, sim._error_body("body exceeds 65536 bytes"))
611         return
612     raw = self.rfile.read(length) if length else b""
613     try:
614         payload = load_strict_json(raw)
615     except BadJson as exc:
616         self._send_json(400, sim._error_body(exc.message))
617         return
618     action = self.path[1:]
619     try:
620         status, body = sim.handle(action, payload)
621     except BadField as exc:
622         self._send_json(400, sim._error_body(exc.message))
623         return
624     except ConnectionClosed:
625         self._drop()
626         return
627     except Exception as exc: # pragma: no cover - defensive
628         import traceback
629         traceback.print_exc()
630         self._send_json(500, sim._error_body(f"internal error: {exc!r}"))

```

```

629         return
630     self._send_json(status, body)
631
632     # -- debug (only with --expose-debug) -----
633     def _debug_get(self) -> None:
634         sim = self.sim
635         if self.path.startswith("/debug/truth"):
636             self._send_json(200, {
637                 "case_code": sim.case.case_code,
638                 "problem": sim.problem,
639                 "total": sim.case.total,
640                 "omni_count": sim.case.omni_count,
641                 "directional_count": sim.case.directional_count,
642                 "jammers": sim.case.truth_table(),
643             })
644         elif self.path.startswith("/debug/status"):
645             self._send_json(200, sim.result_summary())
646         elif self.path.startswith("/debug/log"):
647             live = sim.live
648             self._send_json(200, {"log": live.log if live else []})
649         else:
650             self._send_json(404, {"detail": "unknown debug path"})
651
652
653     def serve(sim: MockSimulator, *, host: str = "127.0.0.1",
654             expose_debug: bool = False, verbose: bool = False) -> ThreadingHTTPServer:
655         sim.verbose = verbose
656         sim.enable_debug = expose_debug
657         handler = type("BoundHandler", (Handler,), {"sim": sim})
658         try:
659             httpd = ThreadingHTTPServer((host, sim.port), handler)
660         except OSError as exc:
661             raise SystemExit(
662                 f"无法监听 {host}:{sim.port} ({exc}).\n"
663                 f"端口可能已被占用 (例如官方模拟器或上一次未退出的实例), "
664                 f"请改用 --port 指定其他端口, 或先释放该端口。"
665             ) from exc
666         httpd.daemon_threads = True
667         stop = threading.Event()
668         httpd._mock_stop = stop # type: ignore[attr-defined]
669
670         def watchdog() -> None:
671             while not stop.wait(0.02):
672                 sim.tick()
673                 if sim.live is not None and sim.live.finished() and sim.status == "running":
674                     sim.status = "finished"
675                     sim.last_finished = sim.live
676                     sim.finished_tests.append(sim.result_summary())
677
678         thread = threading.Thread(target=watchdog, daemon=True, name="mocksim-watchdog")
679         thread.start()
680         httpd._mock_watchdog = thread # type: ignore[attr-defined]
681
682         original_close = httpd.server_close
683
684         def server_close() -> None:
685             stop.set()
686             thread.join(timeout=1.0)

```

```

687         original_close()
688
689     httpd.server_close = server_close # type: ignore[method-assign]
690     return httpd
691
692
693 def main(argv: list[str] | None = None) -> int:
694     ap = argparse.ArgumentParser(description="Local mock jammers simulator (CUMCM 2026 B)")
695     ap.add_argument("--problem", type=int, default=3, choices=(3, 4))
696     ap.add_argument("--mode", default="practice", choices=("practice", "formal"))
697     ap.add_argument("--port", type=int, default=2026)
698     ap.add_argument("--seed", type=int, default=None, help="fix the case seed for reproducibility")
699     ap.add_argument("--count", type=int, default=None, help="jammer count 10..16")
700     ap.add_argument("--team", default="TEAM0000", help="robot_id the client must use")
701     ap.add_argument("--virtual-limit", type=float, default=A.MAX_VIRTUAL_DURATION_S)
702     ap.add_argument("--real-limit", type=float, default=A.MAX_REAL_DURATION_S)
703     ap.add_argument("--window", type=float, default=A.WINDOW_DURATION_S)
704     ap.add_argument("--expose-debug", action="store_true",
705                     help="enable GET /debug/truth|status|log on the same port")
706     ap.add_argument("--verbose", action="store_true")
707     args = ap.parse_args(argv)
708
709     sim = MockSimulator(problem=args.problem, mode=args.mode, seed=args.seed,
710                         count=args.count, team_id=args.team, port=args.port,
711                         virtual_limit_s=args.virtual_limit, real_limit_s=args.real_limit,
712                         window_s=args.window)
713     httpd = serve(sim, expose_debug=args.expose_debug, verbose=args.verbose)
714     sim.start()
715     print(f"mock simulator listening on http://127.0.0.1:{args.port}")
716     print(f"  problem={args.problem} mode={args.mode} case={sim.case.case_code} "
717           f"jammers={sim.case.total}")
718     print(f"  robot_id must be {args.team!r}; /enter now open")
719     if args.expose_debug:
720         print(f"  debug: http://127.0.0.1:{args.port}/debug/truth")
721     try:
722         httpd.serve_forever()
723     except KeyboardInterrupt:
724         pass
725     finally:
726         httpd.server_close()
727         print("\n--- final summary ---")
728         print(json.dumps(sim.result_summary(), ensure_ascii=False, indent=2))
729     return 0
730
731
732 if __name__ == "__main__":
733     raise SystemExit(main())

```

## vendor/mocksim/version.py

```

1 # -*- coding: utf-8 -*-
2 """版本与发布信息。"""
3
4 from __future__ import annotations
5
6 __version__ = "1.0.0"

```

```

7 RELEASE_NAME = "26B 本地测试环境"
8 BUILD_DATE = "2026-09-10"
9
10 #: 本研究环境对标的问题
11 SUPPORTED_PROBLEMS = (3, 4)
12
13 #: 官方模拟器默认接口地址（与本地环境一致，便于直接切换）
14 OFFICIAL_BASE_URL = "http://127.0.0.1:2026"
15 OFFICIAL_PORT = 2026
16
17 #: 官方模拟器关键截止时间（北京时间，题目正文）
18 FORMAL_TEST_DEADLINE = "2026-09-13 17:30"
19 FORMAL_TEST_ADVICE = "2026-09-13 15:30"
20
21 BANNER = rf"""
22 =====
23 {RELEASE_NAME} v{_version_}
24 2026 高教社杯 B 题 · 无线电干扰源的快速自动定位与清除
25 内部演练用 · 不消耗官方正式测试机会
26 =====
27 """.strip("\n")

```